Proceedings of the First International Workshop on Trends in Knowledge Representation and Reasoning (TKR'25)

Preface

The First International Workshop on Trends in Knowledge Representation and Reasoning (TKR'25) aimed at providing a forum for the general area of Knowledge Representation and Reasoning (KR), which is a well-established and active area of research within Artificial Intelligence. KR is about the declarative representation of knowledge and develops methods for automated reasoning under vagueness, uncertainty, incompleteness, and inconsistency.

We welcomed contributions from all areas of KR and two types of submissions: full papers must be original and constitute significant contributions to the field and Extended abstracts of recently published works or teasers for ongoing work. All submissions have be evaluated through peer-reviewing based on originality, significance, technical soundness, and clarity of exposition. The reviewing process was single blind. We specifically welcomed extended abstracts of papers published at IJCAI'25, for which the workshop can serve as medium for extended presentations.

TKR2025 was hosted as an IJCAI 2025 workshop and took place in August 2025 in Montreal, Canada.

The workshop received 22 submissions (8 full papers and 14 extended abstracts) and 13 of them (4 full papers and 9 extended abstracts) had been accepted for this volume. We were also pleased to welcome Meghyn Bienvenu as a keynote speaker.

July 2025

Matthias Thimm
Johannes K. Fichte
Jean-Christoph Jung
(Chairs)

Organization

Chairs

Matthias Thimm	University of Hagen, Germany
Johannes K. Fichte	Linköping University, Sweden
Jean Christoph Jung	TU Dortmund University, Germany

Program Committee

Ofer Arieli
Franz Baader
Salem Benferhat
Lars Bengel
Richard Booth
Giovanni Casini
Federico Cerutti
Giuseppe De Giacomo
Wolfgang Dvořák
Thomas Eiter
Eduardo Fermé
Andreas Herzig
Jesse Heyninck
Laura Giordano
Gabriele Kern-Isberner
Gerhard Lakemeyer
João Leite
Carsten Lutz
Jean-Guy Mailly
Maria Vanina Martinez
Thomas Meyer
Juan Carlos Nieves
Magdalena Ortiz
Rafael Peñaloza
Tjitze Rienstra
Sebastian Rudolph
Vladislav Ryzhikov
Kai Sauerwald
Michael Thielscher
Johannes P. Wallner
Renata Wassermann

Papers in this Volume

1. *Ali Khazraee, Abdolreza Mirzaei, Majid Farhadi, Parmis Naddaf, Kiarash Zahirnia, Mohammad Salameh, Kevin Cannons, Richard Mar, Erfaneh Mahmoudzadeh, Oliver Schulte*: Enhancing Graph Learning With First-Order Logic Rules

2. *Bita Banihashemi, Megh Patel, Yves Lesperance*: Using Large Language Models for Abstraction of Planning Domains

3. *Lars Bengel, Matthias Thimm*: Initial Models and Serialisability in Abstract Dialectical Frameworks (Extended Abstract)

4. *Mikhail Soutchanski, Yongmei Liu*: Planning with Dynamically Changing Domains

5. *Lydia Blümel, Anna Rapberger, Matthias Thimm, Francesca Toni*: On Independence and SCC-Recursiveness in Assumption-Based Argumentation (Extended Abstract)

6. *Dillon Z. Chen, Till Hofmann, Toryn Q. Klassen, Sheila A. McIlraith*: MOOSE: Satisficing and Optimal Generalised Planning via Goal Regression

7. *Victor Lagerkvist, Mohamed Maizia, Johannes Schmidt*: A Fine-Grained Complexity View on Propositional Abduction — Algorithms and Lower Bounds (Extended Abstract)

8. *Jeroen Spaans, Jesse Heyninck*: A Unifying Framework for Semiring-Based Constraint Logic Programming With Negation (Extended Abstract)

9. *Yasir Mahmood, Jonathan Persson, Victor Lagerkvist, Nicolas Fröhlich, Arne Meier, Markus Hecher, Johannes K. Fichte*: Facets in Argumentation: A Formal Approach to Argument Significance (Extended Abstract)

10. *Meghyn Bienvenu, Camille Bourgaux, Atefe Khodadaitaghanaki*: Inconsistency Handling in DatalogMTL (Extended Abstract)

11. *Gaia Belardinelli, Thomas Bolander, Sebastian Watzl*: A Logic for General Attention Using Edge-Conditioned Event Models (Extended Abstract)

12. *Lars-Phillip Spiegel, Jonas Haldimann, Jesse Heyninck, Gabriele Kern-Isberner, Christoph Beierle*: Generalized Safe Conditional Syntax Splitting of Belief Bases (Extended Abstract)

13. *Thomas Eiter, Davide Soldà*: On Temporal ASP with Eager Unfoldable Operators (Extended Abstract)

Enhancing Graph Learning With First-Order Logic Rules

Ali Khazraee[1] , **Abdolreza Mirzaei**[1,2] , **Majid Farhadi**[3] , **Parmis Naddaf**[1] , **Kiarash Zahirnia**[1] , **Mohammad Salameh**[4] , **Kevin Cannons**[4] , **Richard Mar**[1] , **Erfaneh Mahmoudzadeh**[1] , **Oliver Schulte**[1]

[1]Simon Fraser University, [2]Isfahan University of Technology, [3]University of Alberta, [4]Huawei Technologies

khali7494@gmail.com, abdolreza_mirzaei@sfu.ca, farhadi@ualberta.ca, parmis_naddaf@sfu.ca, kiarash_zahirnia@sfu.ca, mohammad.salameh@huawei.com, kevin.cannons@huawei.com, richard_mar@sfu.ca, erfaneh_mahmoudzadeh_ahmadi_nejad@sfu.ca, oschulte@cs.sfu.ca

Abstract

Existing graph generative models produce graphs that are often quite realistic, but sometimes miss domain-specific patterns. Enhancing graph learning with domain knowledge is one of the current frontiers for neural models of graph data. In this paper, we propose a new approach to enhancing deep graph generative models with knowledge that is represented by first-order logic rules. First-order logic provides an expressive formalism for representing interpretable domain knowledge about relational structures. Our conceptual contribution is a new first-order semantic loss function for training a graph generative model on relational data: maximize the model likelihood subject to a *rule moment matching constraint*, namely that the expected instance count of each rule matches its observed instance count. Our algorithmic contribution is a novel method for computing the expected instance count of a first-order rule for a Variational Graph Autoencoder (VGAE), based on matrix multiplication. Empirical evaluation on seven benchmark datasets, both homogeneous and heterogeneous, shows that rule moment matching improves the quality of generated graphs substantially (by orders of magnitude on standard graph quality metrics), and improves predictive accuracy on the downstream task of node classification.

1 Introduction

Generative models for graphs based on graph neural networks (GNNs) have achieved great success in modeling complex graphs [Hamilton, 2020]. One of the current research frontiers is enhancing graph learning with domain knowledge [Tian *et al.*, 2024]. Different enhancement methodologies are appropriate for different types of knowledge. In this paper, we consider leveraging knowledge in the form of a *first-order logic knowledge base* [Russell and Norvig, 2010], comprising a set of if-then rules. An example rule would be "If person X works in city Y, then X lives in city Y (with probability p)".

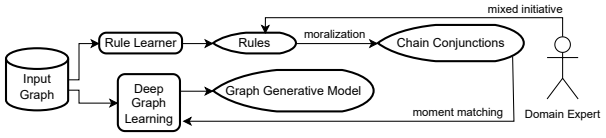


Figure 1: System Overview for Rule-Enhanced Graph Generation

Advantages. Logical rules have several advantages for enhancing graph learning. (1) *Expressiveness*: First-order rules are one of the most common formalisms for representing domain knowledge in AI and database systems [Russell and Norvig, 2010]. (2) *Interpretability*: If-then rules are easily understood by users and domain experts. In this work we derive rules from a first-order Bayesian network (FOBN) [Kimmig *et al.*, 2014; Wang *et al.*, 2008], which provides a graphical interpretation of associations among random variables. Under assumptions described in the causal discovery literature [Pearl, 2000; Spirtes *et al.*, 2000], the network links can be interpreted as causal relationships. (3) *Learnability*: The field of statistical-relational learning (SRL) has developed methods for learning a FOBN from a heterogeneous training graph, known as *structure learning*. (4) *User Control*: Users can control the behavior of the final graph generation system in a mixed-initiative approach, by specifying and/or rejecting rules. (5) *Graph Realism and Data Efficiency*: Matching first-order rules leads to generating more realistic graphs, while requiring less training data.

Approach. Figure 1 shows our system components. We show how fundamental ideas from SRL [Raedt *et al.*, 2016] can be combined with deep graph generative models (GGMs). A fundamental concept of SRL is *rule moment matching* [Domingos and Lowd, 2019; Russell, 2015; Kuzelka *et al.*, 2018]. The general idea is that a rule can be viewed as specifying a *motif* or subgraph pattern with an **instance count** in a given graph. Rule (moment) matching requires that *the expected rule instance count for a model should match the observed rule count in a training graph.* Our novel GGM training objective is to *maximize the GGM likelihood subject to rule matching*.

Our algorithmic contribution is a *differentiable new matrix multiplication method* for computing observed and expected

rule instance counts. We show that for every rule (satisfying a minor syntactic constraint), there is a corresponding sequence of adjacency matrices, such that i) the observed rule count is obtained by multiplying the data adjacency matrices, and ii) the expected rule count for a VGAE model is obtained by multiplying expected adjacency matrices.

Evaluation. Our methodology uses an A-B design where we compare training a recent state-of-the-art variational graph auto-encoder [Mahmoudzadeh *et al.*, 2024] with and without rule moment matching, on seven benchmark datasets. Our approach is in principle compatible with any FO rule learner; we deploy the Factorbase system, which uses BN graph structure learning to find a comprehensive set of first-order rules [Qian and Schulte, 2015]. We find that rule-enhanced VGAEs score better than standard VGAEs on several metrics: (1) They generate *more realistic graphs*, by orders of magnitude, as measured by SOTA graph quality metrics (F1 MMD) [Thompson *et al.*, 2022; O'Bray *et al.*, 2022]. (2) On the downstream task of *node classification*, the rule-enhanced VGAE node embeddings improve accuracy compared to standard VGAE. (3) Learning curves for node classification show that injecting domain knowledge with first-order rules often leads to more data efficient learning.

Contributions Our main contributions can be summarized as follows.

- A new semantic loss objective function for enhancing generative graph learning with domain knowledge represented by first-order logical rules.
- A new matrix multiplication algorithm for counting i) the number of rule instances in a graph and ii) the expected number of instances in an expected graph.
- A proof that the matrix multiplication algorithm can be used to estimate the expected number of rule instances for a Variational Graph Autoencoder model.
- Leveraging the matrix multiplication methods to compute the new semantic loss objective: Maximize the data likelihood of a graph generative model, constrained so that the observed number of rule instances matches the expected number of rule instances.

2 Related Work

Our work falls under the heading of *neuro-symbolic AI*, a cutting-edge field of AI that aims to combine symbolic formalisms, such as first-order logic, with neural network learning; see Figure 2. For surveys of neuro-symbolic AI, please see [Raedt *et al.*, 2020; Garcez and Lamb, 2023], and Kautz's 2022 Engelmore lecture. Within Kautz's taxonomy, our approach belongs to the *semantic loss* frameworks (type 5) where symbolic knowledge is represented as a regularization term added to a standard deep learning objective function [Kautz, 2022; Xu *et al.*, 2018; Marra *et al.*, 2019]. The trained system is a standard NN model that does not utilize rules at test time. In contrast, reasoning approaches typically perform symbolic reasoning [Raedt *et al.*, 2020; Qu *et al.*, 2021] based on rules.

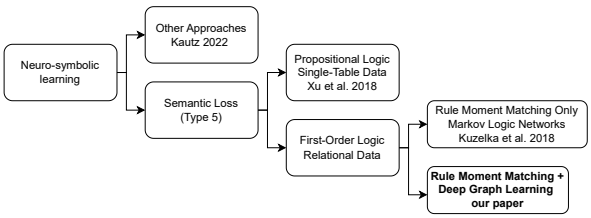


Figure 2: Within neuro-symbolic AI, we develop a new first-order semantic loss approach.

Compared to previous semantic loss approaches [Xu *et al.*, 2018], our main novelty is that *we incorporate knowledge expressed in first-order logic, rather than the less powerful formalism of propositional logic*. For example, a propositional rule would be "if a movie is a horror movie, it is not likely to be a romance". A first-order rule could be "if a user rates a horror movie, the user is most likely to be a man". Since first-order rules incorporate relationships, but propositional rules do not, *first-order logic can leverage the full power of relational data.* Previous FOL approaches (e.g., Marra *et al.* [2019]), are based on fuzzy logic, and the satisfaction function is used as a training objective on its own, rather than as a regularizer for a standard graph generative likelihood.

Deep graph generative models. The closest predecessor to our work is the constrained VGAE model of Ma *et al.* [2018] where a VGAE likelihood is maximized subject to a constraint of the form $g(\boldsymbol{\theta}) = 0$. While this general form covers rule matching, the work of Ma *et al.* does not incorporate first-order logic for specifying graph patterns, nor does it address computing pattern counts.

In principle the rule-enhanced likelihood objective can be used for maximum likelihood training with any deep graph generative model. We selected VGAEs as our base model for several reasons. (1) They are a well-established and widely used GGM. Mahmoudzadeh *et al.* [2024] show that their VGAE+ model is a strong multitask model that provides accurate predictions for a wide range of knowledge graph queries, based on inference from a single model. (2) They support learning from a single large graph, rather than from a set of graphs [Faez *et al.*, 2021]. Rule learners also utilize the single-graph setting [Qian and Schulte, 2015; Meilicke *et al.*, 2024], so the VGAE input data are compatible with the rule learner input data. (3) As we show in this paper, the conditional link independence assumptions of VGAEs facilitates the computation of expected rule instance counts. We believe that extending rule moment matching to other generative models is a fruitful topic for future research.

Maximum Entropy Moment Matching. Kuzelka *et al.* [2018] show that a distribution P over graphs maximizes entropy subject to rule moment matching if and only if P is defined by a log-linear model known as a *Markov Logic Network* (MLN) with maximum likelihood weights $\boldsymbol{w}$. Both the maximum entropy objective and our constrained likelihood objective capture the global graph statistics represented by rule counts. However, the VGAE likelihood can in addition capture local graph patterns. For example, matching the number of observed triangles in a graph is unlikely to capture

community structure, or which nodes have special properties such as centrality.

3 Background on First-Order Logic

We review background on logic and rule learning.

3.1 Attributed Heterogenous Graphs

An attributed graph is a pair $\mathcal{G} = (V, E)$ where V is a set of nodes of size $|V| = n$ and $E \subseteq V \times V$ is a set of edges. Node features are summarized in $n \times f$ matrix $\mathbf{X}$ and node labels in a $n \times L$ matrix $\mathbf{L}$ where the u-th row of $\mathbf{L}$ is a one-hot encoding of the label of node u. Different edge types are represented by a set of adjacency matrices $\mathbf{A} = \{\mathbf{A}_1, \ldots, \mathbf{A}_T\}$. The notation $\mathbf{A}_r[u, v] = 1$ indicates that there is a link $u \rightarrow_r v$ of type r from node u to node v. Figure 3 shows part of the information in an attributed graph using the tabular SQL format.

User

User Id	Age	Gender
3	0 (0.34)	M (0.55)
5	1 (0.43)	F (0.34)
7	2 (0.90)	M (0.84)
...	...	...

Rating

User Id	Movie Id	Rating
3	The Dictator	1 (0.75)
5	Thor	4 (0.36)
5	The Dictator	3 (0.84)
7	BraveHeart	5 (0.98)
...	...	...

Movie

Movie Id	Action	Drama	Horror
The Dictator	0 (0.38)	0 (0.73)	0 (0.85)
Thor	1 (0.49)	0 (0.66)	0 (0.4)
BraveHeart	1 (0.91)	1 (0.41)	1 (0.7)
...	...	...	...

Figure 3: Excerpt from a relational dataset. (a) An attributed graph represented in table format. (b) The probabilities assigned to each data entry specify a probabilistic graph (see below).

3.2 First-Order Logic

We follow previous work in SRL [Schulte and Gholami, 2017; Kimmig *et al.*, 2014]. A **population** is a set of individuals of the same type (e.g., a set of $Users$, a set of $Movies$). Individuals are denoted by constants (e.g., $user_3$ and $thor$). An attributed graph specifies a set of individuals (nodes) for each type. A **node variable** ranges over a population, and is denoted in upper case such as $User, Movie, U, V$. A unary **functor** maps an individual to a value, and corresponds to a node attribute/label. A binary functor maps an ordered pair of individuals to a value, and corresponds to an edge/edge type. Functors are denoted f, f' etc.

A **first-order** (FOT) term is of the form $f(\boldsymbol{U})$ where each population variable U_i is of the appropriate type. FOT examples are $age(User)$ and $rating(User, Movie)$. A FOT can be instantiated with individual constants, much like an index in a plate model [Kimmig *et al.*, 2014]. A **grounding** $\boldsymbol{U} = \boldsymbol{u}$ for a list of FOTs simultaneously replaces each population variable in the list by a constant. (We assume that different population variables are replaced by different constants.) Ground term examples are $age(User = user_5)$, and $rating(User = user_5, Movie = thor)$. A ground term, in Python-style, assigns individuals as argument to node variables, then applies the functor to return a value.

An FO **literal** is of the form $\ell \equiv f(\boldsymbol{U}) = v$. A **conjunction** is a list of literals $\phi = \ell_1, \ldots, \ell_s$. We write $\phi(\boldsymbol{U})$ for an FO conjunction and $\phi(\boldsymbol{U} = \boldsymbol{u})$ for a ground conjunction. A graph $\mathcal{G}$ **satisfies** a ground literal if the graph assigns value v to the ground term $f(\boldsymbol{U} = \boldsymbol{u})$, and satisfies the conjunction ϕ if it satisfies each ground literal in the conjunction. The

Conjunction ϕ	$n_\phi(\mathcal{G})$
$Age(User) = 0$	376
$Rating(User, Movie) = 1$	4701
$Age(User) = 0, Rating(User, Movie) = 1$	2524

Table 1: Conjunction Instance counts in the MovieLens database $\mathcal{G}$

instance count $n_\phi(\mathcal{G})$ in a graph $\mathcal{G}$ returns the number of ϕ-groundings satisfied by graph $\mathcal{G}$.

A **probabilistic graph** $\tilde{\mathcal{G}}$ assigns a probability $p_{\tilde{\mathcal{G}}}(\ell(\boldsymbol{U} = \boldsymbol{u}))$ to each ground literal. The **probabilistic instance count** of a conjunction [Kuzelka, 2023] is the probability product, summed over all conjunction groundings:

$$n_\phi(\tilde{\mathcal{G}}) = \sum_{\boldsymbol{U}=\boldsymbol{u}} \prod_{i=1}^{s} p_{\tilde{\mathcal{G}}}(\ell_i(\boldsymbol{U} = \boldsymbol{u})) \text{ for } \phi = \ell_1, \ldots, \ell_s \quad (1)$$

Examples. $Age(User) = 1$,$rating(User, Movie) = 4$ is an FO conjunction. Its grounding $age(User = user_5) = 1, rating(User = user_5, Movie = thor) = 4$ is satisfied by the data of Figure 3(a). In the probabilistic graph Figure 3(b), the probability of this conjunction is $0.43 \times 0.36 = 0.1548$.

Table 1 illustrates FO instance counts using the MovieLens dataset [Qian and Schulte, 2015]. Movielens contains 376 users at age level 0. The number of user-movie pairs with a rating of 1 is 4701. The number of such pairs with the user at age level 0 is 2524. *An FO conjunction specifies a graph motif*, and the instance count is the motif count [Ma *et al.*, 2019] (see Figure 8 for illustration).

3.3 First-Order Logic Rules

A rule is of the form $\boldsymbol{B} \rightarrow H$ where the body $\boldsymbol{B}$ is an FO conjunction and the head H is a single FO literal. An example rule is

$$\overbrace{Age(User) = 1}^{\text{body } \boldsymbol{B}} \rightarrow \overbrace{Rating(User, Movie) = 4}^{\text{head } H}.$$

This rule expresses the knowledge that the age level of a user influences their ratings. A question researched in SRL is how to convert a set of rules to a set of graph features/statistics that support *generative* graph modelling. The recommended answer is to convert each rule to a conjunction $\phi = (\boldsymbol{B}, H)$ [Domingos and Richardson, 2007, 12.5.3], which is known as *moralization*. A basic result in graphical model theory states that moralization produces an undirected model from a directed one that is equivalent in the sense of representing the same joint distribution [Evans, 2022]. Empirical work on relational data has validated moralization for obtaining useful conjunctive features for generative relational models [Kazemi *et al.*, 2014; Khosravi *et al.*, 2012]; see Appendix A.2 for illustration and details.

4 Rule-Enhanced Graph Generation

This section considers how to enhance training a parametrized graph generative model (GGM) $P_{\boldsymbol{\theta}}$ on a training graph D, with a list of conjunctions $\phi_1, \ldots, \phi_k$.

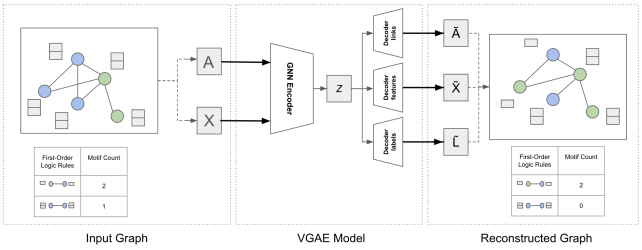


Figure 4: Encoder-Decoder Training Architecture for the VGAE+ Model. Full details are in Appendix A.4.

Our *semantic loss objective* maximizes the data likelihood $P_{\boldsymbol{\theta}}(D)$, subject to the rule moment matching constraint that $E_{\boldsymbol{\theta}}[n_i] = n_i(D)$, where $n_i(D)$ is the **data instance count** of conjunction ϕ_i, and $E_{\boldsymbol{\theta}}[n_i] \equiv \sum_{\mathcal{G}} P_{\boldsymbol{\theta}}(\mathcal{G}) n_i(\mathcal{G})$ is the **expected instance count** for the GGM. For a mixture model GGM, we derive the following Lagrangian evidence lower bound (ELBO) as a training objective.

Proposition 1. *Suppose that* $P_{\boldsymbol{\theta}}(\mathcal{G}) = \int p(\mathcal{G}|\boldsymbol{z})p(\boldsymbol{z})d\boldsymbol{z}$ *is a mixture model. Then*

$$\ln P_{\boldsymbol{\theta}}(D) - \lambda/k \sum_{i=1}^{k} \rho(n_i(D), E_{\boldsymbol{\theta}}[n_i]) \geq \quad (2)$$

$$E_{\boldsymbol{z}\sim q_\phi(\boldsymbol{z}|D)}[\ln P_{\boldsymbol{\theta}}(D|\boldsymbol{z})] - KL\big(q_\phi(\boldsymbol{z}|D)||p(\boldsymbol{z})\big) \quad (3)$$

$$-\lambda/k \sum_{i=1}^{k} \rho(n_i(D), E_{\boldsymbol{\theta}}[n_i|\boldsymbol{z}]), \quad (4)$$

where $\rho(count_1, count_2) \geq 0$ *is a differentiable distance metric that maps two counts to a non-negative number.*

Proposition 1 says that the constrained likelihood (2) can be approximated by our new **rule-matching variational ELBO objective** [(3)-(4)]. Equation (2) is the standard ELBO for a graph generative mixture model [Kipf and Welling, 2016]. Equation (4) is the rule-matching semantic loss component. Since an FO conjunction specifies a graph motif, we refer to Equation (4) as a **motif loss** for brevity.

4.1 Implementing the Rule-Matching ELBO

Our novel VGAE+R architecture extends the recent VGAE+ architecture [Mahmoudzadeh *et al.*, 2024] to match rules.

Encoder-Decoder Architecture. Figure 4 shows the VGAE+R architecture. The **encoder** model $q_\phi(\boldsymbol{z}|D)$ can be any GNN that maps a heterogeneous graph to node embeddings, such as RGCN. The VGAE+R **decoder** independently maps node embeddings to different graph components with three different decoders [Mahmoudzadeh *et al.*, 2024]:

$$\ln P_{\boldsymbol{\theta}}(D|\boldsymbol{z}) = [\alpha \ln p_\eta(\mathbf{A}|\boldsymbol{z}) + \beta \ln p_\psi(\boldsymbol{X}|\boldsymbol{z}) + \gamma \ln p_\phi(\boldsymbol{L}|\boldsymbol{z})$$

where $p_\eta : \mathbb{R}^d \times \mathbb{R}^d \to [0,1]$ is a trainable **link decoder**, p_ψ is a trainable **feature decoder**, and p_ϕ is a trainable **label decoder** (see Figure 4). The hyperparameters α, β and γ weight the importance of different reconstruction tasks.

For the *distance metric* ρ to compare an expected count to an observed count we use

$$\rho(n_i(D), E_{\boldsymbol{\theta}}[n_i|\boldsymbol{z}]) = |\ln n_i(D) - \ln E_{\boldsymbol{\theta}}[n_i|\boldsymbol{z}]|.$$

The magnitude of conjunction counts grows exponentially with the number of node variables in the conjunction. Comparing expected counts on a log-scale decreases the impact of the number of variables and improves numeric stability.

Computing Expected Instance Counts. The next proposition shows that the expected conjunction count over all graphs can be found as a *probabilistic instance count applied to a single graph*, the expected graph. Given a set of node embeddings $\boldsymbol{z}$, the **expected graph** $\tilde{\mathcal{G}}_{\boldsymbol{z}}$ is a probabilistic graph that assigns a probability to each ground literal by applying the decoder to the relevant links/node features/edge types. For examples see Figure 3 and Figure 12.

Proposition 2. *The expected conjunction count given a set of node embeddings can be computed as the conjunction count in the expected graph:* $E_{\boldsymbol{\theta}}[n_i|\boldsymbol{z}] = n_i(\tilde{\mathcal{G}}_{\boldsymbol{z}})$.

The proof is in the supplement. The upshot is that rule moment matching can be implemented by performing instance counting in a single graph, which we address next.

5 Matrix Multiplication for Instance Counting

This section presents a novel matrix multiplication method for instance counting, that is differentiable and applies to both discrete and probabilistic graphs.

To illustrate the basic idea, consider the instance count of the conjunction $R(U_1, V_1), R(V_1, V_2), R(V_2, U_1)$, which finds the number of triangles in an undirected graph represented by an adjacency matrix $\mathbf{A}$. It is well-known that the triangle count is given by $\sum_{u=1}^{n} \mathbf{A}^3_{u,u}$, the trace of the third power of the adjacency matrix. We generalize this approach to a large class of logical formulas: given a chain conjunction, our algorithm computes a sequence of matrix multiplications such that the conjunction's instance count can be found by executing the matrix multiplications. For simplicity, we specify our algorithm for binary literals, then discuss extensions.

A **chain conjunction** of binary literals is of the form $\phi = \ell_1(U_1, V_1), \ldots, \ell_P(U_P, V_P)$ where $V_i = U_{i+1}$ for every i. Algorithm 1 maps each chain conjunction to a sequence of adjacency matrix multiplications.

Example. Consider the chain conjunction

$$AdvisedBy(Student, Professor),$$
$$Teaches(Professor, Course),$$
$$TakesCourse(Course, Student)$$

For the example graph of Table 2, Figure 5 shows the corresponding sequence of matrix multiplications.

Extensions. Unary literals can be included by omitting nodes from the input graph that do not satisfy them. Probabilistic instance counts can be obtained by using soft matrices $\tilde{A}, \tilde{X}, \tilde{L}$; see Appendix A.7 for details.

Correctness. The next proposition shows that the instance count for the chain conjunction can be obtained through summing over the entries in the constructed matrix product.

Algorithm 1 Matrix Multiplication for Instance Counting

Input: Chain conjunction $\phi = \{\ell_1(U_1, V_1), \ldots, \ell_P(U_P, V_P)\}$
Output: Instance count $n_\phi(\mathcal{G})$ or expected count $n_\phi(\tilde{\mathcal{G}}_{\boldsymbol{z}})$
{Initialize adjacency matrices $\mathbf{A}_{\ell_k}$ for binary literals $\ell_k, k = 1, \ldots, P$ }
for $k = 1$ to P **do**
  if positive literal $\ell_k = R(U_k, V_k)$ = 1 **then**
    $\mathbf{A}_{\ell_k} \leftarrow \mathbf{A}_r$
  else if $\ell_k = R(U_k, V_k)$ = 0 **then**
    $\mathbf{A}_{\ell_k} \leftarrow \neg\mathbf{A}_r$ where $\neg\mathbf{A}_r$ is the complement of $\mathbf{A}_r$
  end if
end for
$O_1 \leftarrow \mathbf{A}_{\ell_1}$
for $k = 1$ to $P - 1$ **do**
  $O_{k+1} \leftarrow O_k \cdot \mathbf{A}_{\ell_{k+1}}$
  if $V_{k+1} = U_1$ **then**
    Zero out the non-diagonal entries of O_{k+1}
  end if
end for
Return:
$n_\phi(\mathcal{G}) = \sum(O_P(\phi))$ for input graph $\mathcal{G}$
$n_\phi(\tilde{\mathcal{G}}_{\boldsymbol{z}}) = \sum(O_P(\phi))$ for expected graph $\tilde{\mathcal{G}}_{\boldsymbol{z}}$

Advised by	
Student	**Professor**
Jack	Jane
Joey	Tom

Teaches	
Professor	**Course**
Jane	DL
Tom	DL
Tom	NLP

Takes Course	
Course	**Student**
DL	Jack
DL	Joey
DL	Lucas
DL	Ellie
NLP	Lucas
NLP	Ellie

Table 2: An example graph represented as tables containing unary and binary relations.

Proposition 3. *Let ϕ be a centered chain conjunction of length k, i.e., the first node variable is the only one that appears twice non-consecutively.*

1. *For an input graph $\mathcal{G}$, the (u, v)-th entry of O_k counts the number of groundings of ϕ in $\mathcal{G}$ where $U_1 = u$ and $V_P = v$. Therefore $n_\phi(\mathcal{G}) = \sum(O_k(\phi))$.*
2. *For an expected graph $\tilde{\mathcal{G}}_{\boldsymbol{z}}$, the (u, v)-th entry of O_k counts the expected number of groundings of ϕ where $U_1 = u$ and $V_P = v$. Therefore $n_\phi(\tilde{\mathcal{G}}_{\boldsymbol{z}}) = \sum(O_k(\phi))$.*

In our experiments, we found that all learned rules were centered. The Appendix extends matrix multiplication to non-centered chains.

6 Evaluation

We detail our methodology and then discuss our empirical results. We will make our code available on-line.

6.1 Experimental Design.

We describe our benchmark datasets, comparison methods, and how evaluation metrics are computed.

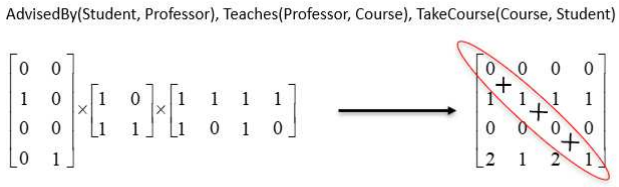


Figure 5: The matrix multiplication sequence for our example conjunction and table 2 graph. The final result is 2, which is the number of satisfying groundings in the input graph.

Datasets
We use datasets from previous studies of GGMs [Mahmoudzadeh *et al.*, 2024; Yun *et al.*, 2019; Hao *et al.*, 2020]. Cora, ACM, and CiteSeer are citation networks, IMDb is a movie dataset, and UW represents an academic department. Appendix A.1 presents dataset and preprocessing details.

Evaluation Metrics
We compare rule-enhanced VGAE+R training with plain VGAE+ training, using three main metrics. In the following, we refer to a complete dataset as the **input graph**. Our evaluation measures *graph realism*—the quality of generated graphs—and the downstream task of node classification.

Count Distance Given the training graph D, we sample one node embedding matrix $\boldsymbol{z}$ from the encoder posterior $q_\phi(\boldsymbol{z}|D)$ and then apply the decoder model eq. (5) to $\boldsymbol{z}$ to obtain the expected graph $\tilde{D}_{\boldsymbol{z}}$. We report the mean squared distance $(1/k \sum_{i=1}^{k} [n_i(D) - n_i(\tilde{D}_{\boldsymbol{z}})]^2)^{1/2}$—between the observed motif counts and the expected motif counts in the reconstructed graph—as the **count distance** (CD), where k is the number of rules.

Graph Realism measures how similar graphs generated by the model are to observed graphs. How to quantitatively assess generated graphs has been studied in recent papers. We adapt the SOTA approach that compares graph embeddings of the training graph to embeddings of generated graphs using Maximum Mean Distance (MMD) [O'Bray *et al.*, 2022; Thompson *et al.*, 2022; Shirzad *et al.*, 2022]; see Appendix A.8 for details. Our measurements utilize the MMD metric with a linear kernel, as recommended by Thompson *et al.* [2022]. Unlike Count Distance, the MMD metric is independent of the training objective.

Node Classification To compute a node classification score, we randomly divide the nodes in the input graph into training and test nodes (80%/20%). The training graph is the input graph but with the test node labels removed. At test time, we run the encoder on the input graph to obtain node embeddings for all nodes, then apply the decoder to predict node labels for the test nodes.

6.2 Experimental Results

Count Distance and Graph Realism are the most important metrics for us since they directly pertain to graph generation quality. Our graph generation baseline is the VGAE+ model trained without rule matching (i.e., $\lambda = 0$). To obtain rules

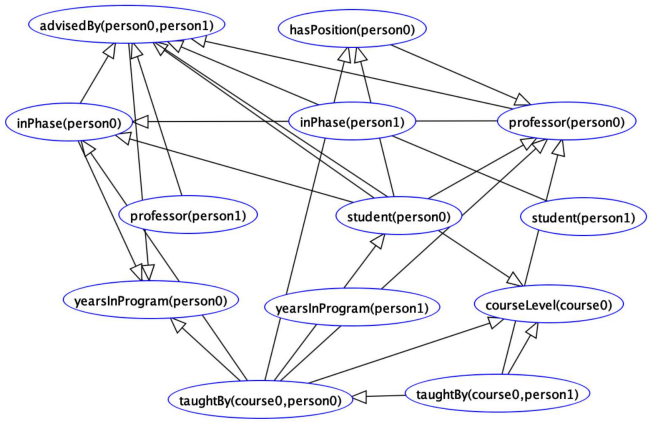


Figure 6: The BN graph for the UW dataset used to train the VGAE+R model.

Table 3: Count Distance (MSE↓) and Improvements in %.

Dataset	VGAE+	VGAE+R	Improvement (%)
Cora	112018.5	**36608.7**	67.33
CiteSeer	47963.6	**20614.9**	57.00
IMDb	981433.1	**108584.2**	88.94
ACM	101733.1	**34396.8**	66.20
Computers	7809684.3	**229343.2**	97.06
Photo	94124.1	**52918.8**	43.76
UW	616966.8	**545615.0**	11.56

for training a VGAE+R model, we used the Factorbase system [Qian and Schulte, 2015] with default settings. Factorbase outputs a FOBN whose structure defines a set of rules $\boldsymbol{B} \rightarrow H$. See Appendix A.3 for details on FO rule learning. Figure 6 shows the BN graph for the UW dataset. It captures several patterns that express university domain knowledge, such as the following. (1) Whether a person teaches a course correlates with whether they have a position. (2) Course teachers are more likely to be professors. (3) A person's program phase predicts their years in the program.

Count Distance and Graph Realism

Table 3 shows the difference between expected and observed rule instance counts. Both methods show large absolute distances because a VGAE model tends to produce overly dense graphs [Orbanz and Roy, 2014]. However in comparison, we observe a *very large improvement in the match between expected and observed counts*, at least 36% on all datasets, except for the small graph UW with an improvement of 11.56%. This shows that VGAE training without the moment matching constraint is far from matching rule counts.

On the graph realism metric shown in Table 4, we again find large absolute distances with the training set, and *very large improvements through rule learning, by an order of magnitude*. Overall we conclude that unconstrained VGAE training does not match the instantiation counts of the learned rules, and that moment matching has a large impact on the realism of the graphs generated.

Table 4: Graph Realism (MMD↓) with Log Improvement.

Dataset	VGAE+	VGAE+R	Log Improv.
Cora	5.0×10^{18}	$\mathbf{2.4 \times 10^{17}}$	1.32
CiteSeer	1.5×10^{18}	$\mathbf{9.1 \times 10^{16}}$	1.23
IMDb	1.7×10^{23}	$\mathbf{1.9 \times 10^{18}}$	5.89
ACM	2.7×10^{20}	$\mathbf{7.3 \times 10^{18}}$	1.57
Computers	1.4×10^{25}	$\mathbf{2.1 \times 10^{24}}$	0.84
Photo	5.1×10^{23}	$\mathbf{7.4 \times 10^{21}}$	1.85
UW	8.8×10^{10}	$\mathbf{6.4 \times 10^{9}}$	1.13

Node Classification

Node classification is an extensively studied task in graph learning. Since SOTA performance is nearly saturated, we do not claim that VGAE+R leads to uniformly best node classification. Instead we investigate two hypotheses:

1. Rule enhancement can improve GGM-based classification when the rules capture relevant domain knowledge.
2. The VGAE+R model is competitive with current baselines.

The baselines apply to homogeneous graphs only, so we homogenized ACM and IMDb [Mahmoudzadeh *et al.*, 2024] to create ACM* and IMDB* and omitted UW. Table 5 shows an improvement on 4 out 6 datasets for rule enhancement, substantive for two of them (Cora and UW). The biggest improvement is on Cora, where the rules increase the AUC score by 10%. Rule enhancement decreases classification performance only slightly even when the rules are not very relevant for the class lable.

Dataset	Model	AUC	F1 Score
Cora	VGAE+R	**0.965 ± 0.013**	**0.887 ± 0.016**
	VGAE+	0.865 ± 0.043	0.699 ± 0.103
UW	VGAE+R	**0.794 ± 0.000**	**0.6323 ± 0.008**
	VGAE+	0.683 ± 0.000	0.6154 ± 0.008
CiteSeer	VGAE+R	**0.903 ± 0.008**	**0.794 ± 0.042**
	VGAE+	0.891 ± 0.013	0.733 ± 0.058
Computers	VGAE+R	0.915 ± 0.022	0.827 ± 0.047
	VGAE+	**0.920 ± 0.004**	**0.837 ± 0.005**
Photo	VGAE+R	**0.991 ± 0.003**	**0.972 ± 0.002**
	VGAE+	0.980 ± 0.021	0.946 ± 0.052
ACM*	VGAE+R	0.761 ± 0.077	**0.525 ± 0.014**
	VGAE+	**0.775 ± 0.074**	0.523 ± 0.009
IMDb*	VGAE+R	**0.829 ± 0.006**	**0.697 ± 0.008**
	VGAE+	0.828 ± 0.011	0.687 ± 0.014

Table 5: Node classification results for graph generation with and without rule enhancement.

Table 6 compares the rule-enhanced VGAE+R with recent node classification baselines (see Appendix A.9). We focused on baselines that are similar to generative models in that they aim to support multiple prediction tasks. The baselines apply to homogeneous graphs only, so we homogenized ACM and IMDb Mahmoudzadeh *et al.* [2024] and omitted UW.

Dataset	Model	AUC	F1-score
Cora	VGAE+R	**0.965 ± 0.013**	**0.887 ± 0.016**
	GiGaMAE	0.920	0.856
	G^2P_{xy}	0.921	0.781
	MVGRL	0.888	0.886
CiteSeer	VGAE+R	**0.903 ± 0.008**	0.794 ± 0.042
	GiGaMAE	0.842	**0.798**
	G^2P_{xy}	0.850	0.781
	MVGRL	0.807	0.710
Computers	VGAE+R	0.915 ± 0.022	**0.827 ± 0.047**
	GiGaMAE	0.941	0.770
	G^2P_{xy}	0.680	0.578
	MVGRL	**0.983**	0.816
Photo	VGAE+R	**0.991 ± 0.003**	**0.972 ± 0.002**
	GiGaMAE	0.963	0.569
	G^2P_{xy}	0.773	0.513
	MVGRL	0.963	0.960
ACM*	VGAE+R	0.761 ± 0.084	0.525 ± 0.014
	GiGaMAE	**0.823**	0.440
	G^2P_{xy}	0.742	0.661
	MVGRL	0.708	**0.803**
IMDb*	VGAE+R	0.829 ± 0.006	**0.697 ± 0.008**
	GiGaMAE	**0.890**	0.457
	G^2P_{xy}	0.654	0.541
	MVGRL	0.788	0.653

Table 6: Node classification results comparing rule-enhanced graph generation against baselines.

In Table 6, our VGAE+R model shows the best node classification performance on 3 out of 6 datasets. The biggest improvement is on CiteSeer where our baselines are far from SOTA performance. GiGaMAE is a strong baseline that achieves the best result on two datasets. Our conclusion is that *rule-enhanced graph generation supports node classification that is competitive with recent baselines*.

Learning Curve After learning an informative set of rules on the entire training graph, we sample 20% of node labels as test labels and reserve the other 80% as training node labels. Then we sample $x\%$ of the training node labels for training the VGAE with and without rules. We report the predictive accuracy on the test labels, for $x = 25\%, 50\%, 75\%, 100\%$ of training labels. The idea is to simulate the impact of a domain expert providing the model with a strong set of rules. Figure 7 shows that *rule matching improves data efficiency substantially on the CiteSeer dataset*, where rule learning achieves a higher accuracy with 50% of node labels than baseline VGAE learning with 100%. We see similar efficiency improvements on Cora and Photo. The learning curves with and without rules are similar for the datasets Computers, ACM, and IMDb because the rules do not affect node classification as much as they affect graph generation. UW is too small to obtain a meaningful learning curve.

7 Conclusion and Future Work

We proposed a new semantic loss objective function for training a deep graph generative model (GGM) to incorporate domain knowledge expressed by logical rules: Maximize the data likelihood subject to a moment matching constraint, which requires the expected rule instance counts under a model to match the observed rule instance count. Our main algorithmic contribution is a new differentiable matrix multiplication method for computing both observed and expected counts. In empirical evaluation, rule matching improves the quality of the graphs generated by a Variational Graph Auto-Encoder (VGAE) model by an order of magnitude or more, both with respect to rule counts and with respect to a standard metric of graph realism. Applying the trained GGM to the downstream task of node classification, rule matching improved classification accuracy on all but one of our benchmark datasets. The domain knowledge incorporated in the model is often effective in improving predictions from small datasets, as shown in learning curves.

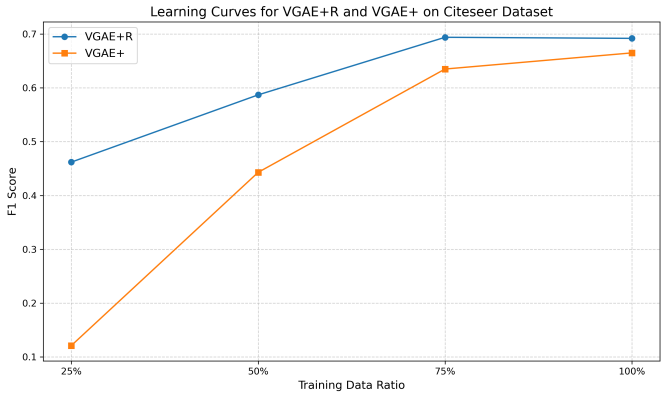


Figure 7: Learning curve for the CiteSeer dataset.

The main limitation of our current approach is scaling the matrix multiplication algorithm for expected counts. A possible approach would be approximation algorithms from the related problem of weighted model counting [van Bremen and Kuzelka, 2020].

Our novel approach to rule-enhanced graph learning opens several avenues for future research, including (1) Extracting different rule statistics for moment matching (e.g. conditional probabilities from if-then rules). (2) Evaluating methods other than moment-matching for measuring how well a set of FO rules fits a graph (e.g.,Marra *et al.* [2019]; Badreddine *et al.* [2022]). (3) Enhancing different types of GGMs with rule matching (e.g., diffusion and auto-regressive models).

In sum, rule moment matching presents a novel semantic loss approach to neuro-symbolic AI that combines logical rules with deep graph learning. Our experiments show great potential for enhancing deep graph generative models with rule-based knowledge.

References

Samy Badreddine, Artur d'Avila Garcez, Luciano Serafini, and Michael Spranger. Logic tensor networks. *Artificial Intelligence*, 303:103649, 2022.

Pedro Domingos and Daniel Lowd. Unifying logical and

statistical AI with Markov logic. *Communications of the ACM*, 62(7):74–83, 2019.

Pedro Domingos and Matthew Richardson. Markov logic: A unifying framework for statistical relational learning. In *Introduction to Statistical Relational Learning*. MIT Press, 2007.

Robin J. Evans. *Graphical Models: DAGs, UGs, and Beyond*. Cambridge University Press, Cambridge, 2022.

Faezeh Faez, Yassaman Ommi, Mahdieh Soleymani Baghshah, and Hamid R Rabiee. Deep graph generators: A survey. *IEEE Access*, 9:106675–106702, 2021.

Artur d'Avila Garcez and Luis C Lamb. Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review*, pages 1–20, 2023.

William L Hamilton. *Graph representation learning*, volume 14. Morgan & Claypool Publishers, 2020.

Yu Hao, Xin Cao, Yixiang Fang, Xike Xie, and Sibo Wang. Inductive link prediction for nodes having only attribute information. *arXiv preprint arXiv:2007.08053*, 2020.

Kaveh Hassani and Amir Hosein Khasahmadi. Contrastive multi-view representation learning on graphs. In *International conference on machine learning*, pages 4116–4126. PMLR, 2020.

Manfred Jaeger and Oliver Schulte. A complete characterization of projectivity for statistical relational models. In *International Joint Conferences on Artificial Intelligence IJCAI-20*, pages 4283–4290, 2020.

Henry Kautz. The third AI summer: AAAI Robert S. Engelmore Memorial Lecture. *AI magazine*, 43(1):105–125, 2022.

Seyed Mehran Kazemi, David Buchman, Kristian Kersting, Sriraam Natarajan, and David Poole. Relational logistic regression. In *International Conference on the Principles of Knowledge Representation and Reasoning, KRR 2014*, 2014.

Hassan Khosravi, Oliver Schulte, Jianfeng Hu, and Tianxing Gao. Learning compact Markov logic networks with decision trees. *Machine Learning*, 89(3):257–277, 2012.

Angelika Kimmig, Lilyana Mihalkova, and Lise Getoor. Lifted graphical models: a survey. *Machine Learning*, 99(1):1–45, 2014.

Thomas N Kipf and Max Welling. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308*, 2016.

Ondrej Kuzelka, Yuyi Wang, Jesse Davis, and Steven Schockaert. Relational marginal problems: Theory and estimation. In *AAAI Conference on Artificial Intelligence (AAAI-18)*, pages 1–8, 2018.

Ondrej Kuzelka. Counting and sampling models in first-order logic. In *International Joint Conference on Artificial Intelligence, IJCAI*, pages 7020–7025, 2023.

Tengfei Ma, Jie Chen, and Cao Xiao. Constrained generation of semantically valid graphs via regularizing variational autoencoders. In *Advances in Neural Information Processing Systems, NeurIPS 2018*, pages 7113–7124, 2018.

Chenhao Ma, Reynold Cheng, Laks VS Lakshmanan, Tobias Grubenmann, Yixiang Fang, and Xiaodong Li. Linc: a motif counting algorithm for uncertain graphs. *VLDB Endowment*, 13(2):155–168, 2019.

Erfaneh Mahmoudzadeh, Parmis Naddaf, Kiarash Zahirnia, and Oliver Schulte. Deep generative models for subgraph prediction. In *European Conference on Artificial Intelligence, ECAI 2024*, volume 392, pages 3128–3136, 2024.

Giuseppe Marra, Francesco Giannini, Michelangelo Diligenti, and Marco Gori. Lyrics: A general interface layer to integrate logic inference and deep learning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases, ECML PKDD*, pages 283–298, 2019.

Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. Image-based recommendations on styles and substitutes. *ACM*, 2015.

Christian Meilicke, Melisachew Wudage Chekol, Patrick Betz, Manuel Fink, and Heiner Stuckeschmidt. Anytime bottom-up rule learning for large-scale knowledge graph completion. *The VLDB Journal*, 33(1):131–161, 2024.

Leslie O'Bray, Max Horn, Bastian Rieck, and Karsten Borgwardt. Evaluation metrics for graph generative models: Problems, pitfalls, and practical solutions. In *International Conference on Learning Representations, ICLR*, 2022.

Peter Orbanz and Daniel M Roy. Bayesian models of graphs, arrays and other exchangeable random structures. *IEEE transactions on pattern analysis and machine intelligence*, 37(2):437–461, 2014.

J. Pearl. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988.

J. Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge university press, 2000.

Zhensong Qian and Oliver Schulte. Factorbase: Multi-relational model learning with sql all the way. In *Data Science and Advanced Analytics (DSAA)*, pages 1–10, 2015.

Meng Qu, Junkun Chen, Louis-Pascal Xhonneux, Yoshua Bengio, and Jian Tang. RNNLogic: Learning logic rules for reasoning on knowledge graphs. In *International Conference on Learning Representations, ICLR*, 2021.

Luc De Raedt, Kristian Kersting, Sriraam Natarajan, and David Poole. Statistical relational artificial intelligence: Logic, probability, and computation. *Synthesis lectures on artificial intelligence and machine learning*, 10(2):1–189, 2016.

Luc de Raedt, Sebastijan Dumančić, Robin Manhaeve, and Giuseppe Marra. From statistical relational to neuro-symbolic artificial intelligence. In *International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 4943–4950, 2020.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 2010.

Stuart Russell. Unifying logic and probability. *Communications of the ACM*, 58(7):88–97, 2015.

Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *International conference on the semantic web (ESWC 2018)*, pages 593–607, 2018.

Oliver Schulte and Sajjad Gholami. Locally consistent Bayesian network scores for multi-relational data. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2693–2700, 2017.

Oliver Schulte and Zhensong Qian. FACTORBASE: multi-relational structure learning with SQL all the way. *International Journal of Data Science and Analytics*, 7(4):1–21, 2018.

Yucheng Shi, Yushun Dong, Qiaoyu Tan, Jundong Li, and Ninghao Liu. Gigamae: Generalizable graph masked autoencoder via collaborative latent space reconstruction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 2259–2269, 2023.

Hamed Shirzad, Kaveh Hassani, and Danica J Sutherland. Evaluating graph generative models with contrastively learned features. *Advances in Neural Information Processing Systems, NeurIPS*, 2022.

P. Spirtes, C. Glymour, and R. Scheines. *Causation, Prediction, and Search*. MIT Press, 2000.

Rylee Thompson, Boris Knyazev, Elahe Ghalebi, Jungtaek Kim, and Graham W Taylor. On evaluation metrics for graph generative models. In *International Conference on Learning Representations, ICLR*, 2022.

Yijun Tian, Shichao Pei, Xiangliang Zhang, Wei Wang, Hanghang Tong, and Nitesh V. Chawla, editors. *Knowledge-enhanced Graph Learning*. Workshop at AAAI, 2024.

Timothy van Bremen and Ondrej Kuzelka. Approximate weighted first-order model counting: Exploiting fast approximate model counters and symmetry. In *International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 4252–4258, 7 2020.

Daisy Zhe Wang, Eirinaios Michelakis, Minos Garofalakis, and Joseph M Hellerstein. Bayesstore: managing large, uncertain data repositories with probabilistic graphical models. In *VLDB*, pages 340–351, 2008.

Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, and Guy Van den Broeck. A semantic loss function for deep learning with symbolic knowledge. In *International Conference on Machine Learning, ICML*, volume 80, pages 5502–5511, 2018.

Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J Kim. Graph transformer networks. *Neural Information Processing Systems, NeurIPS*, 32, 2019.

Qin Zhang, Zelin Shi, Xiaolin Zhang, Xiaojun Chen, Philippe Fournier-Viger, and Shirui Pan. G2pxy: generative open-set node classification on graphs with proxy unknowns. In *International Joint Conference on Artificial Intelligence, IJCAI-23*, pages 4576–4583, 2023.

A Technical Appendix

A.1 Dataset Information

To evaluate all the methods we utilize 5 datasets used in previous studies of generative models Kipf and Welling [2016]; Yun *et al.* [2019]; Hao *et al.* [2020].

- **Cora** [Kipf and Welling, 2016] is a citation dataset that consists of nodes that represent machine-learning papers divided into seven classes (the subjects of the papers) and links that represent citation between them. The target for node classification is the subject of the papers. This dataset has 5,429 links, 2,708 nodes with an average node degree of 3.8.
- **ACM** [Yun *et al.*, 2019] is a citation dataset. It has three types of nodes (paper, author, and venue) and four types of links. The target for node classification is predicting one of three labels corresponding to the conferences where the papers were published. This dataset has 18,929 links, 8,993 nodes with an average node degree of 2.209.
- **IMDb** [Yun *et al.*, 2019] is a movie dataset with three types of nodes (movies, actors, and directors) and it uses the genre of movies as their labels. The target for node classification is predicting one of three genres of movies. This dataset has 19,120 links, 12,772 nodes with an average node degree of 2.9.
- **CiteSeer** [Kipf and Welling, 2016] is also a citation dataset that consists of nodes that represent machine-learning papers divided into six classes (the topics of the papers) and links that represent citation between them. The target for node classification is the topic of the publications. This dataset has 4,732 links, 3,327 nodes with an average node degree of 2.7.
- The **UW** dataset models academic relationships at a university, where persons can be both students and professors and key relations include AdvisedBy (a student advised by a professor) and TaughtBy (a student taking a course taught by a professor). The dataset contains 278 person entities and 132 course entities. The classification target is the phase of study of the student, which can be one of three labels: *pre-quals*, *post-quals*, or *post-generals*. If the person is not a student, the label is 0.
- **Photo & Computers** are datasets from the Amazon co-purchase graph [McAuley *et al.*, 2015]. In these datasets, nodes represent goods, links indicate that two goods are frequently bought together, node features are bag-of-words encoded product reviews, and class labels are given by the product category [Hao *et al.*, 2020].

Data Preprocessing Following previous work [Kipf and Welling, 2016], for GNN message passing we add self-loops and make all links undirected (i.e., if the training data contains an adjacency, $v \to u$, it also contains $u \to v$.) Rule learning is applied to the original data.

A.2 First-order Rules, Conjunctions, and Motifs

Figure 8 illustrates how moralization converts a rule to a conjunction, and how a conjunction represents a motif. In terms of graphical model structure, moralizing converts a Bayesian network to an undirected network (known as a Markov random field), by connecting all parents of a child to each other, and turning all directed links into undirected links. A well-known result in graphical model theory states that if $P_B(\boldsymbol{X} = \boldsymbol{x})$ is the joint distribution over a set of random variables $\boldsymbol{X}$ represented by a Bayesian network, and M is the Markov random field obtained from B through randomization, then M can be parametrized to represent P_B. Further details can be found in texts on graphical models, such as Evans [2022].

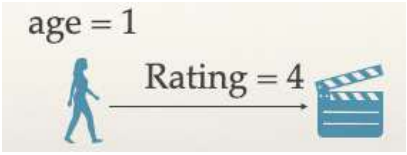


Figure 8: The FO conjunction $Age(User) = 1, Rating(User, Movie) = 4$ can be viewed as a two-node motif. The conjunction is obtained from the rule $Age(User) = 1 \to Rating(User, Movie) = 4$ by moralization.

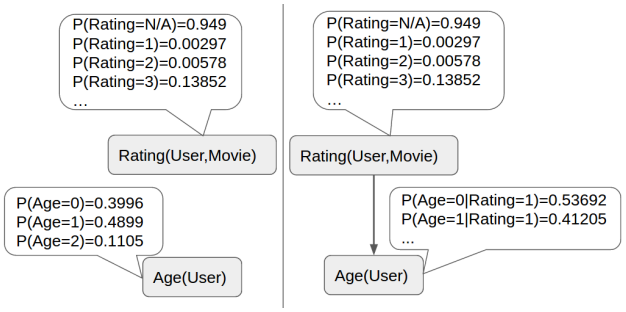


Figure 9: Example First-Order Bayesian Networks: left = B_1 with graph $\mathcal{G}_1$, right = B_1^+ with graph $\mathcal{G}_1^+$.

A.3 Rule Learning and First-Order Bayesian Networks

A **Bayesian network (BN) structure** is a directed acyclic graph G (DAG) whose nodes comprise a set of random variables Pearl [1988]. The causal interpretation is that the parents of a node represent its direct causes Pearl [2000]. A **Bayesian network** B is a structure G together with a set of parameter values of the form $P(child_value|parent_values)$, which specify the distribution of a child node given an assignment of values to its parent node. A first-order BN Wang *et al.* [2008]; Kimmig *et al.* [2014] (FOBN) is a BN whose nodes are first-order terms. The BN parameters specify the distribution of a child node given an assignment of values to its parent node. Figure 9 shows two parametrized FOBNs. The right FOBN connects the rating of a movie to the age of the rater. In the left FOBN, ratings are independent of ages. The process of structure learning searches for statistically significant connections between first-order terms to introduce or remove edges [Schulte and Qian, 2018; Schulte and Gholami, 2017].

A single rule $\boldsymbol{B} \to H$ is defined by a combination $parent_values \to child_value$, for each child node value and possible combination of parents values [Khosravi *et al.*,

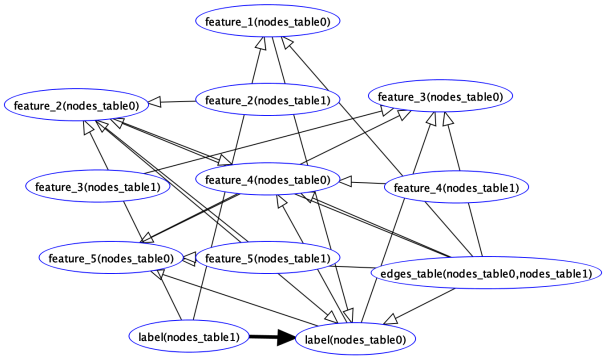


Figure 10: The BN graph for the Cora dataset. We have highlighted the homophily edge that shows that if paper 0 cites paper 1, then the label of paper 0 correlates with the label of paper 1.

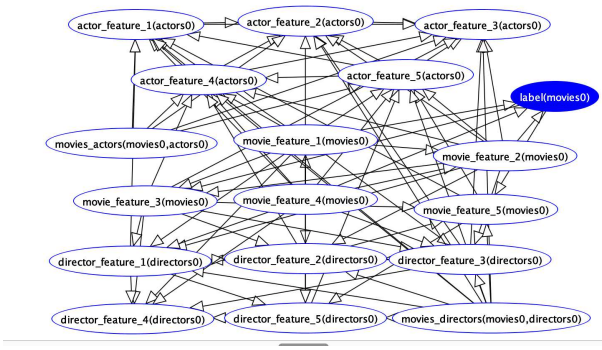


Figure 11: The BN graph for the IMDb dataaset.

2012]. For example, the BN graph of Figure 9(right) defines the rule

$$Rating(User, Movie) = 3 \rightarrow Age(User) = 1).$$

Given 6 possible rating values and 5 age levels, the link $Rating(User, Movie) \rightarrow Age(User))$ defines $6 \times 5 = 30$ rules.

Factorbase is the most scalable system for learning a FO generative graphical model with state-of-the-art predictive performance. As the Factorbase system can output many rules, we use
$2n(\boldsymbol{B}, H) \ln(n(H|\boldsymbol{B})/n(H)) - \ln n(H)$ as a *rule quality metric* for pruning. The conditional count is given by $n(H|\boldsymbol{B}) = n(H, \boldsymbol{B})/n(\boldsymbol{B})$. This metric computes the increase in the log-probability of the head given the body, relative to the prior probability of the head, together with a BIC-type correction for sample size Schulte and Gholami [2017]. The quality metric can be interpreted as a positive local BIC model selection score. We keep all rules whose quality metric is above 0. Rule pruning reduces the number of rules for scalability, and also simulates the impact of a domain expert selecting the most important rules.

We show the BN graphs learned by Factorbase for three datasets. We find that many of the learned rules capture intuitively plausible domain constraints, such as homophily: if one paper cites another, then they are likely from the same research area. For example in the graph for the Cora dataset, the parents of the target label of a paper (node) are paper features 1 and 2, the citation relationship between a paper and the target paper (denoted "edges_table"), and the label of the citing paper. The graph defines a rule for each combination of the 4 parent nodes, and each of the 7 possible child node values (paper classes).

ACM and CiteSeer are citation networks like Cora with a similar graph, which we omit.

Figure 11 shows the BN graph for the IMDb dataset. The graph captures many correlations between features of movies and actors. An interesting aspect of the graph is that for the target label (genre of the movie), its only parents are other features of the movie to be classified (features 1,2,4). This suggests that relational features, such as the features of actors who appear in the movie, should not add information beyond movie features. We verified this directly by building a classifier based on movie features only, which performed better than the GNN that uses relational features.

A.4 Details on the VGAE+R Model

The decoders generate links, node features, and node labels independently as follows.

$$p_\eta(\mathbf{A}|\boldsymbol{z}) = \prod_{r=1}^{T} \prod_{u,v} p_{\eta_r}(\mathbf{A}_r[u,v]|\boldsymbol{z}[u], \boldsymbol{z}[v]) \quad (5)$$

$$p_\psi(\boldsymbol{X}|\boldsymbol{z}) = \prod_u p_\psi(\boldsymbol{X}[u]|\boldsymbol{z}[u]) \quad (6)$$

$$p_\phi(\boldsymbol{L}|\boldsymbol{z}) = \prod_u p_\phi(\boldsymbol{L}[u]|\boldsymbol{z}[u]), \quad (7)$$

Jaeger and Schulte [2020] provide a strong theoretical foundation for the independent decoder model.

Hyperparameters For the encoder, we used an RGCN [Schlichtkrull *et al.*, 2018], with the Pytorch Geometric implementation. The node embedding dimension was 64, the number of RGCN layers 2, with separate layers for estimating the Gaussian posterior mean and the Gaussian posterior standard deviation for each node. For the VGAE+ model with $\lambda = 0$, we follow Mahmoudzadeh *et al.* [2024] and use Bayesian optimization to fix the ELBO hyperparameters p_η, p_ψ, p_ϕ. For the VGAE+R model, after fixing the ELBO hyperparameters, we set the motif loss weight λ empirically based on a validation set. For the learning curves, we trained the model for 100 epochs at each sample size. For the other graph generative model results, we trained until convergence. The number of training epochs were as follows. 700 for Cora and CiteSeer, 1000 for IMDb, Computers, ACM, Photo, and 100 for UW.

A.5 Expanded First-Order Logic Definitions

We add to the definitions in the main body to introduce concepts we need in our proofs and in the full description of matrix multiplication algorithm. We make this section self-contained for ease of reference. A positive **relationship literal** is of the form $R(U, V)$. A **negative** relationship literal is of the form $\neg R(U, V)$. A generic relationship literal (positive

or negative) is denoted as $\ell(U, V)$. A **ground** relationship literal is of the form $\ell(U = u, V = v)$ where u and v are two node indices. Similar to specifying arguments for variables in a programming language, grounding specifies arguments for node variables. Positive and negative unary literals are defined similarly.

For a positive ground relationship literal $\ell(U = u, V = v) = R(U = u, V = v)$, the indicator $I_{\mathcal{G}}(\ell(U = u, V = v)) = 1$ if the two nodes u and v are linked by edge type R in graph G. For a negative ground relationship literal, $\ell(U = u, V = v) = \neg R(U = u, V = v)$, we have $I_{\mathcal{G}}(\neg R(U = u, V = v)) = 1 - I_{\mathcal{G}}(R(U = u, V = v))$.

A conjunction is a list of literals. Intuitively, a conjunction is a template for a motif or frequently occurring subgraph. The indicator function specifies which nodes satisfy the literal/conjunction. Our formal definitions are as follows.

A **conjunction** ϕ comprises three elements:

1. A list $\ell_1(U_1, V_1), \ldots, \ell_P(U_P, V_P)$ comprising P relationship literals where U_i and V_i are node variables.
2. A list $\ell_1(W_1), \ldots, \ell_Q(W_Q)$ comprising Q unary literals, where W_i is a node variable.
3. A set of equality constraints EQ of the form $D_k = E_k$ for any two node variables D_k and E_k that appear in the list of relationship literals or unary literals. Formally, EQ is a set of unordered pairs of node variables.

This definition is equivalent to the definition in the main paper: all node variables are assumed to occur exactly once in a conjunction, and the equality constraints specify which node variables must be mapped to the same node indices. Representing equality constraints in an explicit list facilitates the statement of our matrix multiplication algorithm.

A conjunction with P relationship literals $\ell_1(U_1, V_1), \ldots, \ell_P(U_P, V_P)$ is a **chain conjunction** if there is a permutation π of the literals such that the equality constraints comprise $V_{\pi(i-1)} = U_{\pi(i)}$ for $i = 2, \ldots, P$. A conjunction ϕ has $2P + Q$ parameters (i.e., node variables). Specifying a list of $2P + Q$ node indices as arguments to the conjunction returns a **grounded conjunction.** A grounded chain conjunction corresponds to a path in the graph where each consecutive pair of nodes is connected by an edge.

The indicator function for a grounded conjunction is given by:

$$
\begin{aligned}
&I_{\mathcal{G}}(\ell_1(U_1 = u_1, V_1 = v_1), \ldots, \ell_P(U_P = u_P, V_P = v_P), \\
&\ell_1(W_1 = w_1), \ldots, \ell_Q(W_Q = w_Q)) \\
&= \prod_{i=1}^{P} I_{\mathcal{G}}(\ell_i(U_i = u_i, V_i = v_i)) \prod_{j=1}^{Q} I_{\mathcal{G}}(\ell_j(W_j = w_j))
\end{aligned}
$$

Conjunction Counts. For compactness, write a grounding as $\langle \boldsymbol{U} = \boldsymbol{u}, \boldsymbol{V} = \boldsymbol{v}, \boldsymbol{W} = \boldsymbol{w} \rangle$ so the indicator function returns a 0/1 value for $I_{\mathcal{G}}(\langle \boldsymbol{U} = \boldsymbol{u}, \boldsymbol{V} = \boldsymbol{v}, \boldsymbol{W} = \boldsymbol{w} \rangle, EQ)$. A grounding $\langle \boldsymbol{U} = \boldsymbol{u}, \boldsymbol{V} = \boldsymbol{v}, \boldsymbol{W} = \boldsymbol{w} \rangle$ is **valid** for a set of equality constraints EQ if for any two assignments $D = d$ and $E = e$ we have $d = e$ if and only if $(D = E) \in EQ$. Thus node variables constrained to be equal must be assigned the same node, and otherwise must be assigned different nodes. We write $Valid_{EQ}$ for the set of valid groundings. The **instance count** for a conjunction ϕ in a graph $\mathcal{G}$ returns the number of valid groundings that satisfy the conjunction:

$$
n_{\phi}(\mathcal{G}) = \sum_{\langle \boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w} \rangle \in Valid_{EQ}} I_{\mathcal{G}}(\langle \boldsymbol{U} = \boldsymbol{u}, \boldsymbol{V} = \boldsymbol{v}, \boldsymbol{W} = \boldsymbol{w} \rangle)
$$

Here, we evaluate the indicator function for each combination of $\langle \boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w} \rangle$, and sum up the values for all combinations to obtain the desired count of satisfying groundings.

A.6 Proof of Proposition 2

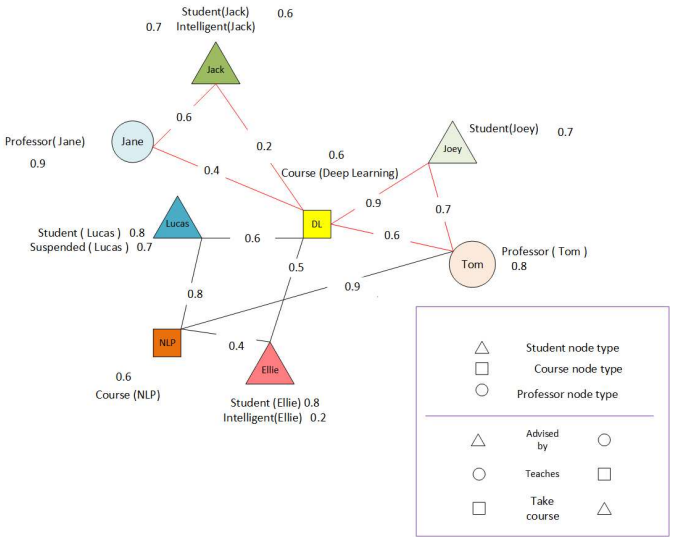


Figure 12: An example expected graph corresponding to Table 2.

The output of the VGAE+ model is an expected graph like Figure 12. The following proposition states that the expected counts can be computed based on a expected graph.

Proposition. The expected conjunction count given a set of node embeddings can be computed as the conjunction count in the expected graph: $E_{\mathcal{G} \sim P(\mathcal{G}|\boldsymbol{z})}[n_{\phi}(\mathcal{G})] = n_{\phi}(\tilde{\mathcal{G}}_{\boldsymbol{z}})$

Proof. Without loss of generality, assume the conjunction is of the form $\ell_1(U_1 = u_1, V_1 = v_1), \ldots, \ell_P(U_P = u_P, V_P = v_P), \ell_1(W_1 = w_1), \ldots, \ell_Q(W_Q = w_Q)$ with equality constraints EQ. Define the random variables $l_i^{uv}, i = 1, \ldots, P$ to return the indicator value $I_{\mathcal{G}}(\ell_i(U_i = u, V_i = v)$ and $l_j^{w}, j = 1, \ldots, Q$ to return the indicator value $I_{\mathcal{G}}(\ell_j(W_j = w))$. Then

$$
\begin{aligned}
&E_{\mathcal{G}\sim P(\mathcal{G}|\boldsymbol{z})}[n_\phi(\mathcal{G})] \\
&= E[\sum_{\langle \boldsymbol{u},\boldsymbol{v},\boldsymbol{w}\rangle \in Valid_{EQ}} \prod_{i=1}^{P} I_{\mathcal{G}}(\ell_i(U_i=u_i,V_i=v_i)) \\
&\prod_{j=1}^{Q} I_{\mathcal{G}}(\ell_j(W_j=w_j))] \\
&= E[\sum_{\langle \boldsymbol{u},\boldsymbol{v},\boldsymbol{w}\rangle \in Valid_{EQ}} \prod_{i=1}^{P} l_i^{\boldsymbol{u}_i\boldsymbol{v}_i} \prod_{j=1}^{Q} l_j^{\boldsymbol{w}_j}] \\
&= \sum_{\langle \boldsymbol{u},\boldsymbol{v},\boldsymbol{w}\rangle \in Valid_{EQ}} E[\prod_{i=1}^{P} l_i^{\boldsymbol{u}_i\boldsymbol{v}_i} \prod_{j=1}^{Q} l_j^{\boldsymbol{w}_j}] \qquad (8) \\
&= \sum_{\langle \boldsymbol{u},\boldsymbol{v},\boldsymbol{w}\rangle \in Valid_{EQ}} \prod_{i=1}^{P} E[l_i^{\boldsymbol{u}_i\boldsymbol{v}_i}] \prod_{j=1}^{Q} E[l_j^{\boldsymbol{w}_j}] \\
&= \sum_{\langle \boldsymbol{u},\boldsymbol{v},\boldsymbol{w}\rangle \in Valid_{EQ}} \prod_{i=1}^{P} p_{\tilde{\mathcal{G}}_{\boldsymbol{z}}}(\ell_i(U_i=\boldsymbol{u}_i,V_i=\boldsymbol{v}_i)) \\
&\prod_{j=1}^{Q} p_{\tilde{\mathcal{G}}_{\boldsymbol{z}}}(\ell_j(W_j=\boldsymbol{w}_j)) = n_\phi(\tilde{\mathcal{G}}_{\boldsymbol{z}})
\end{aligned}
$$

Equation (8) follows because the expectation of a product of independent random variables is the product of their expectations. The random variables $l_i^{\boldsymbol{u}_i\boldsymbol{v}_i}$ and $l_j^{\boldsymbol{w}_j}$ are independent because the (in)equality constraints ensure that in a valid grounding, no two different literals are ground to the same ground literal. And conditional on the node embeddings $\boldsymbol{z}$, any two different ground literals are independent. □

A.7 Matrix Multiplication Method

Example for Expected Instance Count

Figure 13 illustrates the matrix multiplication method for the expected graph in our example.



Figure 13: The matrix multiplication sequence for our example conjunction and Figure 12 graph.

Full Specification of Matrix Multiplication Method

Intuitively, a chain conjunction is a template for a motif or frequently occurring subgraph. The indicator function specifies which nodes satisfy the literal/conjunction. Our formal definitions are as follows.

The input to our counting algorithm is a chain conjunction $\{\ell_1(U_1,V_1),\ldots,\ell_P(U_P,V_P),\ell_1(W_1),\ldots,\ell_Q(W_Q),EQ\}$ and an input graph $\mathcal{G}$. The first step is to process the unary literals by masking adjacency matrix entries of nodes that do not satisfy all unary literals. The second step is to define inductively a sequence of matrices such that the instance count of the conjunction can be computed as the entry sum of the matrix product. We use $A \circ B$ to denote the element-wise matrix (Hadamard) product and I for the identity matrix of the appropriate dimension. A positive relationship literal $R(U,V)$ is **associated with** $\mathbf{A}_r$, the adjacency matrix for relation R. A negative relationship literal $\neg R(U,V)$ is associated with $\neg\mathbf{A}_r$ where $\neg\mathbf{A}_r[u,v] = 1 - \mathbf{A}_r[u,v]$ for all node indices u,v.

Step 1: Unary Literals Consider binary relationship literal $\ell_i(U,V)$ with associated $m \times n$ adjacency matrix $A_i(\mathcal{G})$. We search for every unary literal $\ell(W)$ where $(U=W) \in EQ$. For each such literal, we create a binary vector T of size m such that $T[w] := I_{\mathcal{G}}(\ell(W=w))$. Thus the entry $T[w]$ masks all the nodes that do not satisfy the unary literal. We apply the mask to the w row of matrix A_i, setting $\bar{\mathbf{A}}_i[w,:] = A_i[w,:] \circ T[w]$. where $A(w,:)$ represents the entire w row of matrix A. If the w entry of T is zero, the entire row of $\bar{\mathbf{A}}_i$ is set to 0. If the w entry of T is one, the entire row of A_i is copied to $\bar{\mathbf{A}}_i$.

Similarly, if a unary literal $\ell(W)$ exists where $(V=W) \in EQ$, we mask the corresponding column entries in the adjacency matrix A_i, and repeat the masking process for all such unary literals. We refer to the adjacency matrix that incorporates the unary functor constraints as the *masked adjacency matrix* $\bar{\mathbf{A}}_i$.

Step 2: Binary Literals A chain conjunction is centered if all equality constraints for the binary literals (other than the chain constraints) involve the first node variable, that is they are of the form $U_1 = E_k$. For a centered chain, we define a sequence of matrix multiplications as follows.

1. For a single literal conjunct $\phi = \ell(U,V)$ with associated masked matrix $\bar{\mathbf{A}}$, let
$$O_1(\phi) = \begin{cases} \bar{\mathbf{A}}, & \text{if } U=V \notin EQ \\ \bar{\mathbf{A}} \circ I, & \text{if } U=V \in EQ \end{cases}$$
$\bar{\mathbf{A}} \circ I$ agrees with $\bar{\mathbf{A}}$ on the diagonal and is 0 off-diagonal.
2. Inductively, consider a conjunction ϕ of length k+1 in the form of $\phi = \phi', \ell_{k+1}(U_{k+1},V_{k+1})$ where $\phi' = \ell_1(U_1,V_1),\ldots,\ell_k(U_k,V_k)$ is a conjunction of length k. Let
$$O_{k+1}(\phi) = \begin{cases} O_k(\phi')\bar{\mathbf{A}}_{k+1}, & \text{if } U_1 = V_{k+1} \notin EQ \\ (O_k(\phi')\bar{\mathbf{A}}_{k+1}) \circ I, & \text{if } U_1 = V_{k+1} \in EQ \end{cases} \qquad (9)$$

Correctness Proof

We next formulate the proposition that for every chain conjunction, there is a corresponding sequence of matrix multiplication operations such that: for every input graph $\mathcal{G}$, applying the operation sequence to the graph edge label tensor returns the instance count. In this formulation, we use these facts: 1) Every ground positive (negative) relationship literal corresponds to a link present (absent) in the graph. 2) A grounded chain conjunction corresponds to a path in the graph where each consecutive pair of nodes is connected by a present/absent link.

Proposition. chain conjunction of length k.

1. For an input graph $\mathcal{G}$, the (u, v)-th entry of O_k counts the number of groundings of ϕ in $\mathcal{G}$ where $U_1 = u$ and $V_P = v$. Therefore $n_\phi(\mathcal{G}) = \sum(O_k(\phi))$.
2. For an expected graph $\tilde{\mathcal{G}}_{\boldsymbol{z}}$, the (u, v)-th entry of O_k counts the expected number of groundings of ϕ where $U_1 = u$ and $V_P = v$. Therefore $n_\phi(\tilde{\mathcal{G}}_{\boldsymbol{z}}) = \sum(O_k(\phi))$.

Proof. We give the proof for clause 1, counting observed counts in an input graph $\mathcal{G}$. The argument for expected counts computed from an expected graph is exactly parallel.

Base case, $k = 1$. If $\phi = \{\ell(U, V)\}$, then the conjunction count is the number of pairs (u, v) such that (i) both groundings $U = u$ and $V = v$ satisfy all unary literals, and (ii) $I_{\mathcal{G}}(\ell(U = u, V = V)) = 1$. All and only such pairs have the entry $\bar{\mathbf{A}}[u, v] = 1$ in the masked adjacency matrix associated with $\ell(U, V)$.

Case 1: $(U = V) \notin EQ$. Then the number of satisfying groundings is simply given by $\sum(\bar{\mathbf{A}})$.

Case 2: $(U = V) \in EQ$. Then the satisfying groundings are of the form $U = u, V = u$, so their count is given by the matrix trace of $\bar{\mathbf{A}}$, or equivalently $\sum(\bar{\mathbf{A}} \circ I)$. This establishes the base case.

Inductive Step: Assume the proposition holds for k and consider the matrix O_{k+1} computed by Equation (9). By the inductive hypothesis, the (u, v)-th entry of O_k counts the number of instantiations of length k between vertices u and v that satisfy ϕ'. Now, the number of instantiations of length $k + 1$ between u and w equals the number of instantiations of length k from vertex u to each vertex v that has ℓ_{k+1} relation with w. The non-zero entries of column w of masked matrix $\bar{\mathbf{A}}_{k+1}$ represent vs related by ℓ_{k+1} to w. So, (u, w)-th entry of $O_k\bar{\mathbf{A}}_{k+1}$ gives the number of instantiations between u and w satisfying the centered conjunction and all equality constraints except possibly $U_1 = V_{k+1}$. Therefore for the case where $U_1 = V_{k+1} \notin EQ$, the matrix O_{k+1} satisfies the inductive hypothesis. For the case where $U_1 = V_{k+1} \in EQ$, we observe that the number of instantiations of length $k + 1$ from node u to u equals the (u, u) diagonal entry of $O_k\bar{\mathbf{A}}_{k+1}$ or equivalently, $(O_k\bar{\mathbf{A}}_{k+1}) \circ I$. Thus the total number of satisfying groundings is given by $\sum(O_{k+1}(\phi))$ in either case, which establishes the inductive hypothesis. □

Extensions

- Our counting method is based on a **sorting algorithm**. A sorting algorithm is a procedure that takes a set of relationship literals

  $$\ell_1(U_1, V_1), \ldots, \ell_P(U_P, V_P)$$

  and determines if there exists a permutation π such that the literals can be arranged to form a *chain conjunction*. Specifically, it seeks to satisfy the equality constraints given by

  $$V_{\pi(i-1)} = U_{\pi(i)} \quad \text{for } i = 2, \ldots, P.$$

  Example

  Consider the following conjunction of relationship literals:

  $$\begin{aligned} &AdvisedBy(Student, Professor),\\ &TaughtBy(Course, Professor),\\ &Registered(Student, Course) \end{aligned}$$

  This set of relationships is not a chain conjunction because there is no permutation of the literals that satisfies the necessary chain equality constraints. However, we can transform this into a chain conjunction using the reverse relations:

  $$\begin{aligned} &AdvisedBy(Student, Professor),\\ &Teaches(Professor, Course),\\ &TakeCourse(Course, Student) \end{aligned}$$

  In this case, the literals can be rearranged to form a chain conjunction, satisfying the equality constraints. Here $Teaches$ is the reverse of $TaughtBy$ and $TakeCourse$ is the reverse of $Registered$.

- Our algorithm and proof can be extended to the case of *nested conjunctions*, which have no crossing equalities. Say that two variable equalities $U_{k_1} = V_{k_2}$ and $U_{k_3} = V_{k_4}$ **cross** if $k_1 < k_3 < k_2$ and $k_2 < k_4$. A nested chain is composed of centered chains, so our matrix multiplication algorithm can be used to recursively compute instance counts.

A.8 Graph Realism Metric for evaluating Graph Generation

How to quantitatively assess generated graphs has been studied in recent papers O'Bray *et al.* [2022]; Thompson *et al.* [2022]; Shirzad *et al.* [2022]. The general approach proceeds in two stages:

1. For a graph $\mathcal{G}$, extract a real-valued **descriptor** vector $\phi(\mathcal{G})$.
2. Measure the similarity $\mu(\mathcal{G}, \hat{\mathcal{G}})$ of an observed graph $\mathcal{G}$ and a generated graph $\hat{\mathcal{G}}$ by applying a distance/kernel on their real-valued descriptors vectors $\phi(\mathcal{G})$ and $\phi(\hat{\mathcal{G}})$.

The similarity of a set of observed graphs and a set of generated graphs can be quantified as the similarity of their descriptor sets using Maximum Mean Distance (MMD). The SOTA descriptor function utilizes a *reference embedding network* GNN $\mathcal{E}$. The embedder $\mathcal{E}$ is obtained from random weights and is therefore independent of any of the models under evaluation.

We adapt the SOTA GNN-based approach to the setting of learning from a single training graph D as follows. We compare the training graph to generated expected graphs $\tilde{\mathcal{G}}_1, \ldots \tilde{\mathcal{G}}_m$. An expected graph $\tilde{\mathcal{G}}_i$ is generated by sampling a node embedding $\boldsymbol{z}_i$ from the prior distribution $p(\boldsymbol{z})$, then applying the decoder model eq. (5) to $\boldsymbol{z}_i$. Following Thompson *et al.* [2022], we apply an embedder $\mathcal{E}$ with random weights to the training graph resp. generated expected graphs to obtain embeddings e resp. $\hat{e}_1, \ldots, \hat{e}_m$. The random GNN option does not require multiple training graphs. The message-passing mechanism of GNNs naturally extends to weighted

graphs, so we can apply the GNN embedder to expected graphs directly. To quantify the similarity of the generated embeddings $\hat{e}_1, \ldots, \hat{e}_m$ to the training graph embedding e, we utilize the MMD metric with a linear kernel, which is recommended by Thompson *et al.* [2022]. We used their code to compute the MMD metric results.

A.9 Node Classification Baseline Methods

Generalizable Graph Masked AutoEncoder (GiGaMAE) [Shi *et al.*, 2023] introduces a novel graph encoder based on aligning different node embeddings that respectively encode structural and feature information. **MVGRL** is an inductive self-supervised approach for learning representations of nodes and graphs by contrasting different structural views of graphs [Hassani and Khasahmadi, 2020]. $\mathcal{G}^2\mathbf{Pxy}$ [Zhang *et al.*, 2023] generates proxy nodes to support classification. For all methods we used the authors' code to train a node classifier and generate class labels.

A.10 Impact of Rules on Training.

In this section we show that rule matching has a big impact on training, by in effect initializing the GGM model in a part of weight space that encodes the rule knowledge. After the VGAE+R model has encoded the background knowledge in its weights, it learns to optimize the other components, including the node label loss. This shows that rule matching is a strong regularizer that takes the network to a very different part of weight space compared to the baseline VGAE+ loss, which supports better generalization.

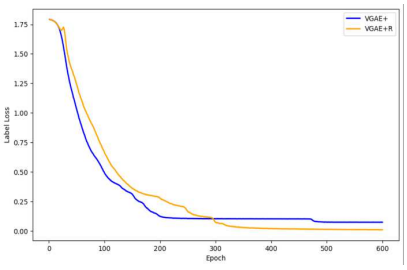


Figure 14: Label loss component for CiteSeer dataset during model training

Figure 14 shows the node label loss component of decoder training Equation (2) for the CiteSeer dataset. Rule matching adds a difficult new component to the VGAE+ objective, which initially causes a spike in the label loss component. The remainder of the supplement shows more loss curves with a similar pattern.

Total Loss Curves

In this section, we present the total loss curves for both the VGAE+ and VGAE+R models across multiple datasets. While it is not possible to directly compare the loss values between the two models, the downward trend in loss during training for both models indicates successful convergence. Figures 15, 16, 17, 18, and 19 illustrate the total loss for each dataset, confirming the models' progress throughout the training process.

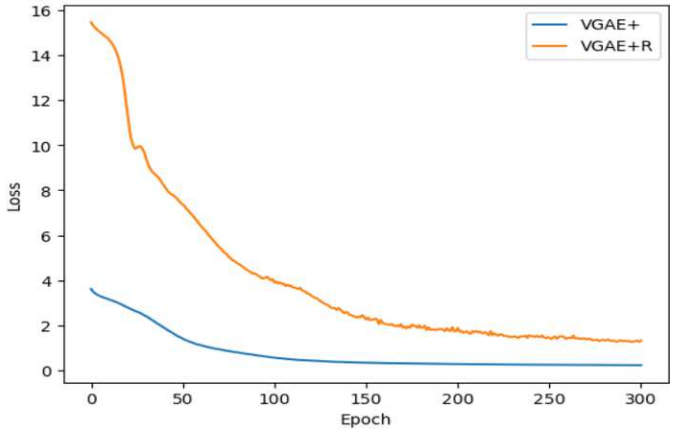


Figure 15: Total loss for Cora dataset during model training

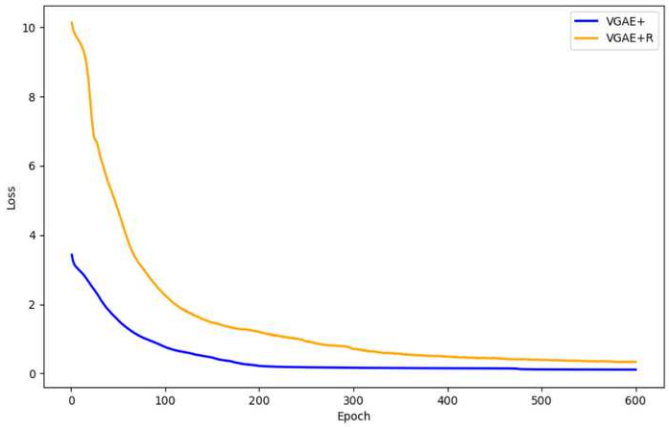


Figure 16: Total Loss for CiteSeer dataset during model training

A.11 Label Loss Curves

In this section, we compare the label loss component for each model across different datasets. As shown in Figures 20, 21, 22, 23, and 24, the label loss for VGAE+R is generally lower than that for VGAE+ across most datasets, suggesting that VGAE+R may yield better accuracy in node classification.

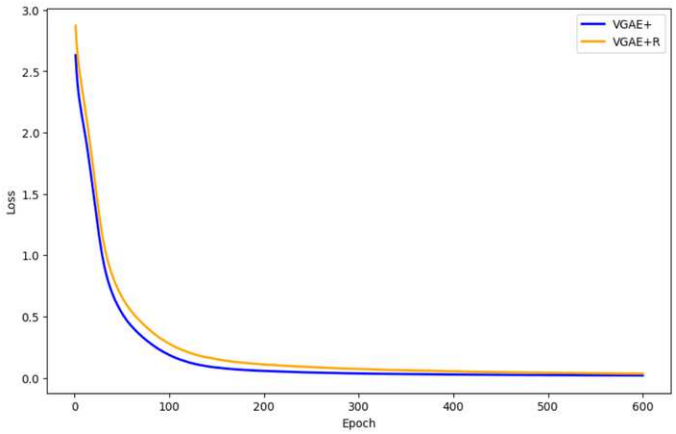


Figure 17: Total Loss for ACM dataset during model training

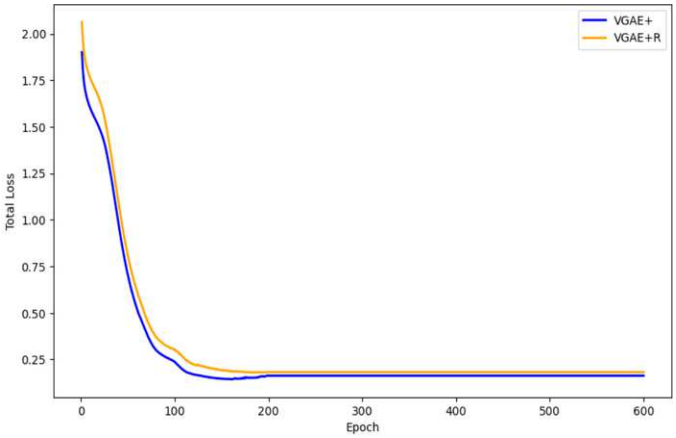


Figure 18: Total Loss for IMDb dataset during model training

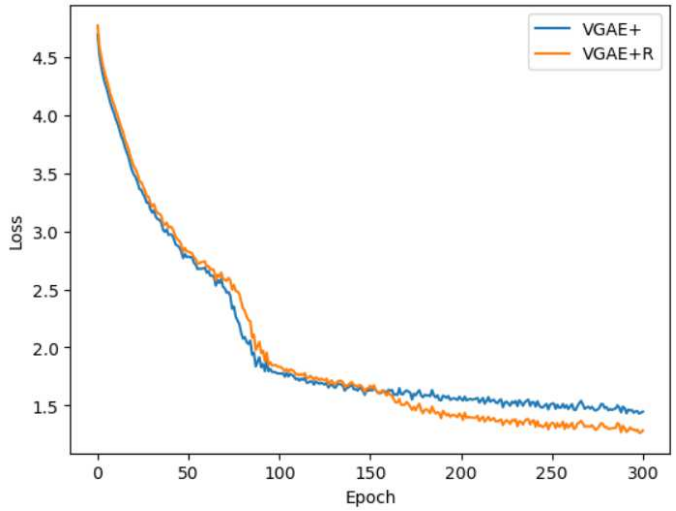


Figure 19: Total Loss for UW dataset during model training

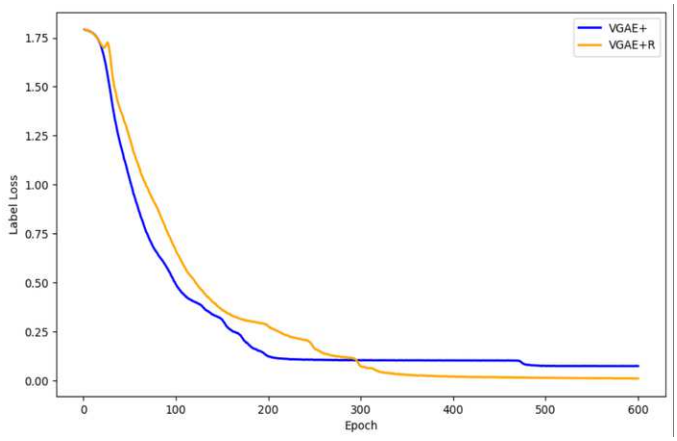


Figure 20: Label loss component for CiteSeer dataset during model training

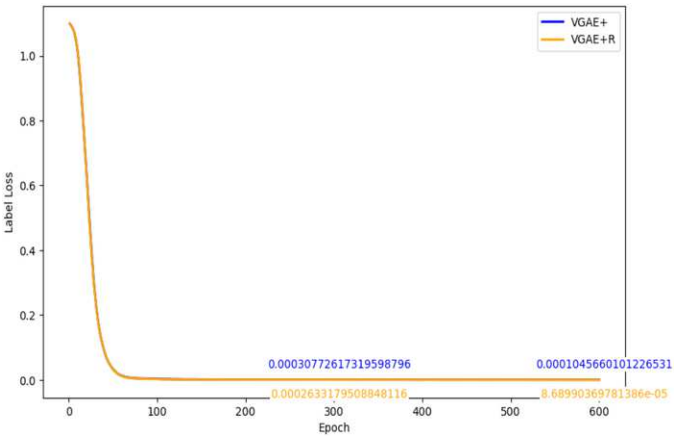


Figure 21: Label Loss component for ACM dataset during model training

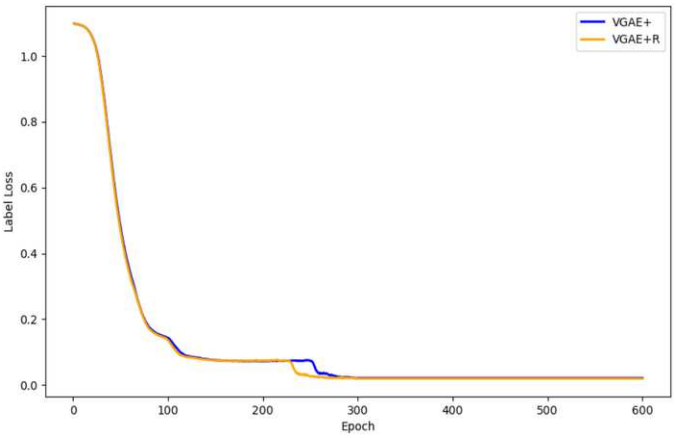


Figure 22: Label Loss component for IMDb dataset during model training

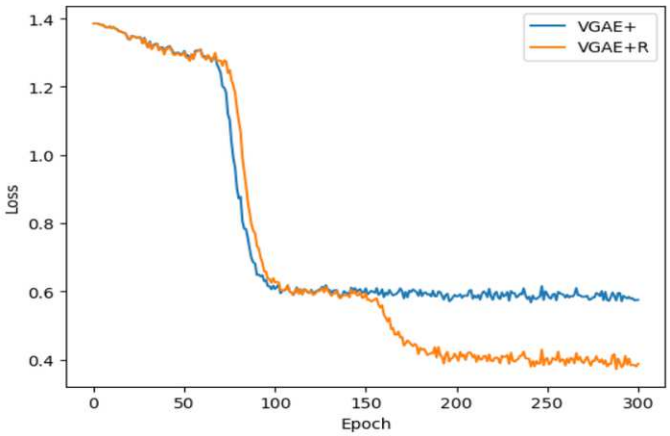


Figure 23: Label Loss component for UW dataset for person node type during model training

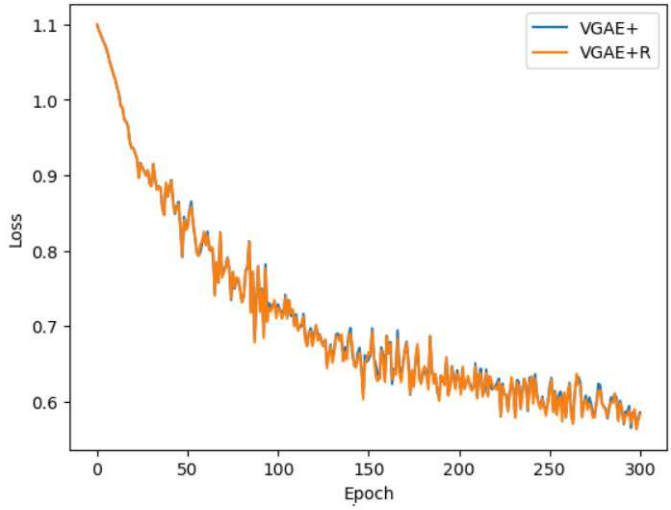


Figure 24: Label Loss component for UW dataset for course node type during model training

Using Large Language Models for Abstraction of Planning Domains

Bita Banihashemi[1] , **Megh Patel**[2] , **Yves Lespérance**[2]
[1]IGDORE
[2]York University
bita.banihashemi@igdore.org, megh2k@my.yorku.ca, lesperan@yorku.ca

Abstract

Generating an abstraction of a dynamic domain that aligns with a given purpose remains a significant challenge given that the choice of such an abstraction can impact an agent's ability to plan, reason, and provide explanations effectively. We model the agent's concrete behaviors in PDDL and investigate the use of in-context learning with large language models (LLMs) for the generation of abstract PDDL domains and problem instances, given an abstraction objective specified in natural language. The benchmark examples we use are new and have not been part of the data any LLMs have been trained on. We consider three categories of abstractions: abstraction of choice of alternative concrete actions, abstraction of sequences of concrete actions, and abstraction of action/predicate parameters, as well as combinations of these. The generated abstract PDDL domains and problem instances are then checked by symbolic validation tools as well as human experts. Our experiments show that GPT-4o can generally synthesize useful planning domain abstractions in simple settings, although they are better at abstracting over actions than over the associated fluents.

1 Introduction

The ability to generate abstractions that ignore irrelevant details is a crucial human cognitive ability that supports reasoning and communication. This has inspired significant research in AI, where for instance, abstraction has been exploited to improve the efficiency of planning (e.g., [Chen and Bercher, 2021]), provide explanations of agents' behavior (e.g., [Seegebarth *et al.*, 2012]), and in reinforcement learning (e.g., [Sutton *et al.*, 1999]).

[Banihashemi *et al.*, 2017; Banihashemi *et al.*, 2024] (BDL17) developed a general framework for *agent abstraction* in *dynamic domains* which is based on the situation calculus [McCarthy and Hayes, 1969; Reiter, 2001] and the ConGolog agent programming language [De Giacomo *et al.*, 2000]. The account formalizes notions of sound/complete abstractions between a high-level (HL) action theory and a low-level (LL) action theory representing the agent's possible behaviors at different levels of detail. These notions are based on the existence of a bisimulation relation between their respective models relative to a *refinement mapping* that maps high-level fluents to low-level state formulas and high-level actions to ConGolog programs over the low-level theory that implement them. It is shown that sound/complete abstractions have many useful properties that allow one to reason at the high level and refine the results at the low level, and they can also be used for monitoring and explanation. There is also related work on hierarchical planning, for instance, hierarchical task network (HTN) planning [Erol *et al.*, 1996] and planning with domain-specific control knowledge specified in linear temporal logic (LTL) [Bacchus and Kabanza, 2000].

(BDL17) specifies under what conditions a high-level action theory is a sound/complete abstraction of a low-level one under a given refinement mapping, but it does not say how one obtains such an abstraction. In general, there would be several different abstractions of a domain, each of which serves a particular purpose and could be used in a different application. Generating an abstraction of a domain that aligns with a given purpose remains a significant challenge. So far, there has only been limited work on solvers that can synthesize abstractions for a given mapping and verify their soundness and/or completeness (e.g., [Luo *et al.*, 2020; Luo, 2023; Fang *et al.*, 2025; Dong *et al.*, 2025]). But relying on human experts to generate such abstractions by hand is costly, time consuming, and does not scale.

Much recent work has investigated the use of large language models (LLMs) [Zhao *et al.*, 2023] for reasoning about action and the generation of planning specifications. While some work shows limitations of LLMs for reasoning about actions and planning (e.g., [Valmeekam *et al.*, 2024; He *et al.*, 2023]), other work shows good potential for the generation of Planning Domain Definition Language (PDDL) [McDermott *et al.*, 1998] specifications (e.g., [Oswald *et al.*, 2024; Guan *et al.*, 2023]).

It is clear that LLMs embed comprehensive world knowledge and that they can generate PDDL specifications. Thus it seems reasonable that they could be very helpful in generating useful abstractions of planning domains and problems and encode them in a formal language such as PDDL [Fox and Long, 2011].

In this paper, we investigate the ability of GPT-4o [Hurst *et al.*, 2024] to generate such abstractions. We define the *Plan-*

ning Domain Abstraction Generation (PDAG) task as: given (1) a concrete planning domain in PDDL (2) a concrete planning problem instance in PDDL, (3) a brief natural language description of the domain, and (4) a purpose of abstraction in natural language, use in-context learning with LLMs to generate (a) an abstract planning domain in PDDL and (b) an abstract planning problem instance in PDDL that meet the purpose of abstraction.

We consider three basic categories of abstractions: (1) abstraction of alternative concrete actions, e.g. $book_hotel$ and $book_airbnb$ could be abstracted to $book_accommodation$, (2) abstraction of sequences of concrete actions, e.g., the sequence $enter_username$ and $enter_password$ could be abstracted to $login$, and (3) abstraction of action/predicate parameters, e.g., abstracting over the $room_view$ parameter of $book_room$ and dropping associated fluents, types, and actions. We also examine a fourth category which is a combination of (1) and (2), i.e., abstraction of alternative sequences of concrete actions. We have developed a number of benchmark examples for the PDAG task for each of these categories. We ran experiments to evaluate the capability of GPT-4o to solve the PDAG task for these benchmarks, which are new and have not been seen by the LLM before. We use zero-shot and one-shot prompting [Brown *et al.*, 2020; Dong *et al.*, 2024; Li, 2023], augmented with chain-of-thought reasoning [Wei *et al.*, 2022] and role-play [Kong *et al.*, 2024]. To evaluate the generated abstract planning domains and problems, we use the plan validator tool VAL [Howey *et al.*, 2004], in addition to evaluation by human experts. We also use the Fast Downward planner [Helmert, 2006] to generate plans and detect errors. The PDAG task and our benchmarks for it are inspired by the (BDL17) framework, but we do not require the LLM to generate the refinement mapping. Nor do we formally verify that the abstract PDDL domain and problem returned by the LLM is a solution to the PDAG task. We leave these topics for future work.

Our experiments show that GPT-4o can generate almost error-free results for abstraction of action/predicate parameters, but that it makes more mistakes as the difficulty of abstraction increases with abstraction of alternative concrete actions, abstraction of sequences of concrete actions, and abstraction of alternative sequences of concrete actions.

2 Background

Planning Domain Definition Language (PDDL) [McDermott *et al.*, 1998] is a widely used language for representing classical planning [Ghallab *et al.*, 2004] problems. In this paper, we use the STRIPS fragment of PDDL with typing. PDDL separates the definition of a planning problem into two parts: the *domain* definition and a *problem* definition. The *domain* definition provides a lifted representation of the relevant aspects and underlying rules of the world, and includes *types*, *predicates* and *actions*. The *problem* definition models a particular problem instance in the domain which specifies the *initial state*, *goal* condition, and the *object* names.

In-Context Learning refers to the ability of LLMs to generalize to novel tasks by interpreting examples or task descriptions provided within the input prompt, without requiring explicit task-specific training or parameter fine-tuning [Brown *et al.*, 2020; Dong *et al.*, 2024; Li, 2023]. In *one-shot* learning, the LLM is provided with a single task demonstration, along with a natural language description of the task. The example task is given together with its desired completion, and then a final task instance, i.e, user query, is provided to the LLM, and the LLM is expected to generate the completion for it. *Zero-shot* learning is similar to one-shot learning, except that no task demonstrations are provided. Here, we use zero-shot and one-shot prompts that are augmented with chain-of-thought (CoT) [Wei *et al.*, 2022] and role-play [Kong *et al.*, 2024] prompting techniques. CoT facilitates step-by-step reasoning by encouraging the model to generate intermediate reasoning steps before arriving at a final answer, thereby improving performance on commonsense and symbolic reasoning tasks. Role-Play prompting enhances task understanding by guiding the model to assume specific personas, which helps contextualize responses more effectively.

3 Planning Domain Abstraction Generation

We define the *Planning Domain Abstraction Generation* (PDAG) task as follows: given (1) a concrete planning domain in PDDL D_c, (2) a concrete planning problem instance in PDDL P_c, (3) a brief natural language description of the domain dd_c, and (4) a purpose of abstraction in natural language pa, use in-context learning with LLMs to generate (a) an abstract planning domain in PDDL D_a and (b) an abstract planning problem instance in PDDL P_a that meet the purpose of abstraction.

We can formalize the PDAG task based on the notion of sound and complete abstraction of situation calculus basic action theories (BATs) relative to a refinement mapping m of (BDL17). For this, the purpose of abstraction must be expressed as a set of constraints C_m on the refinement mapping m (these constraints restrict which fluents and actions may be included in the abstract theory and how they can be mapped to the concrete theory; see the next section for an example). First, note that for any domain D and problem P in the ADL [Pednault, 1989] fragment of PDDL, we can obtain an equivalent BAT $t(D, P) = \mathcal{D}$ as shown in [Claßen *et al.*, 2007]. Note that this BAT must have a finite object domain and that the initial situation description must be complete, i.e., it completely specifies which tuples of objects are in the extension of every fluent initially. Using this, we say that an abstract PDDL domain D_a and problem P_a is a *solution to the PDAG task* (D_c, P_c, C_m), where we have the concrete PDDL domain D_c and problem P_c and constraints on the mapping C_m, if there exists a refinement mapping m such that m satisfies C_m and $t(D_a, P_a)$ is a sound and complete abstraction of $t(D_c, P_c)$ relative to m. [Banihashemi *et al.*, 2024] identifies a set of necessary and sufficient properties for an abstract BAT $\mathcal{D}_a$ to be a sound and complete abstraction of a concrete BAT $\mathcal{D}_c$ relative to mapping m. Since the action theories involved have a finite object domain and a complete initial state specification, given a mapping m, it should be possible to check if these properties are satisfied using model checking techniques. But this requires identifying a suitable mapping m and one also must show that m satisfies the constraints C_m.

4 Abstraction of PDDL Domains and Benchmark Problems

In our experiments, we use a collection of new examples that we have developed and hence, have not been part of the training data of GPT-4o. Each example includes 4 components: a low-level PDDL domain specification, a low-level PDDL problem instance, a brief natural language description of the domain, and a brief natural language description of purpose of abstraction. We consider the following categories of abstraction tasks: abstraction of alternative concrete actions, abstraction of a sequence of concrete actions, abstraction of action/predicate parameters, and a combination of the first two, i.e., abstraction of alternative sequences of concrete actions.

Abstraction of Alternative Concrete Actions. In scenarios where the low-level domain offers multiple alternative actions to achieve a subgoal, such actions can be abstracted into a single high-level action. This is typically accompanied by the abstraction of associated predicates and types. Note that depending on the purpose of abstraction, the concrete domain, and the planning goal, some types, predicates, or actions may not need to be abstracted.

Example 1. Consider the following low-level domain which models travel arrangements. In order to go to the destination, one may book a flight or train, if a seat is available, and also book a hotel or airbnb, if a room is available.

```
(define (domain travelArrange01_LL)
(:requirements :strips :typing)
(:types
    hotel airbnb room flight trainRide seat - object)
(:predicates
   (booked_hotel  ?r -room  ?h - hotel)
   (booked_airbnb  ?r - room ?ab - airbnb)
   (available_room_hotel  ?r  - room ?h - hotel)
   (available_room_airbnb  ?r - room ?ab - airbnb)
   (bookedHotelOrAirbnb)
   (available_seat_flight ?s - seat ?f - flight)
   (available_seat_trainRide ?s -seat ?t - trainRide)
   (booked_flight ?s - seat ?f - flight)
   (booked_trainRide ?s - seat ?t - trainRide)
   (bookedFlightOrTrainRide))
(:action book_hotel
    :parameters (?h - hotel ?r - room)
    :precondition (available_room_hotel ?r ?h)
    :effect (and (booked_hotel ?r ?h)
                 (not (available_room_hotel ?r ?h))
                 (bookedHotelOrAirbnb)))
(:action book_airbnb
    :parameters (?ab - airbnb ?r - room)
    :precondition (available_room_airbnb ?r ?ab)
    :effect (and (booked_airbnb ?r ?ab)
                 (not (available_room_airbnb ?r ?ab))
                 (bookedHotelOrAirbnb)))
(:action book_flight
    :parameters (?f - flight ?s - seat)
    :precondition (available_seat_flight ?s ?f)
    :effect (and (booked_flight ?s ?f)
                 (not (available_seat_flight ?s ?f))
                 (bookedFlightOrTrainRide)))
(:action book_trainRide
    :parameters (?t - trainRide ?s - seat)
    :precondition (available_seat_trainRide ?s ?t)
    :effect (and (booked_trainRide ?s ?t)
                 (not (available_seat_trainRide ?s ?t))
                 (bookedFlightOrTrainRide))))
```

The goal in the associated problem instance is defined as follows (see [Banihashemi *et al.*, 2025] for the complete listings of concrete and abstract domains and problem instances of examples 1 to 4):

```
(:goal (and (bookedFlightOrTrainRide)
            (bookedHotelOrAirbnb)))
```

The purpose of abstraction may be expressed in natural language as "provide an abstraction of the concrete domain such that the high-level domain abstracts over booking various kinds of transportation and accommodation". This purpose of abstraction might be specified formally as a set of constraints on the mapping m, which should be entailed by the concrete action theory $\mathcal{D}_c$ obtained from the concrete PDDL domain and problem. For the booking of accommodation, we could have:

$$\exists r, h.bookedHotel(r, h, s) \vee \exists r, ab.bookedAirbnb(r, ab, s) \supset m(doneBookingAccomodation)[s]$$
$$Do(m(bookAccomodation(a, r)), s, s') \supset m(doneBookingAccomodation)[s']$$

This says that booking a hotel or an airbnb is sufficient to achieve the concrete condition into which $doneBookingAccomodation$ is mapped and that executing the program into which the abstract action $bookAccomodation(a, r)$ is mapped achieves $doneBookingAccomodation$ ($Do(\delta, s, s')$ means that there is an execution of the program δ starting in situation s and ending in situation s'). We could also have a constraint that ensures that there is no abstract action a_h that is mapped into the concrete action $bookHotel(h, r)$, i.e. $m(a_h(h, r)) = bookHotel(h, r)$, and thus this action cannot appear at the abstract level, and similarly for $bookAirbnb$. The constraints for the booking of transportation are similar.

In such a case, we would expect the LLM to generate the following high-level domain:

```
(define (domain travelArrange01_HL)
(:requirements :strips :typing)
(:types accommodation room transportation seat
 - object)
(:predicates
   (booked_accommodation  ?r - room ?a - accommodation)
   (available_room  ?r - room ?a - accommodation)
   (doneBookingAccommodation)
   (available_seat ?s - seat ?tp - transportation)
   (booked_transportation ?s - seat ?tp - transportation)
   (doneBookingTransportation))
(:action book_accommodation
    :parameters (?a - accommodation ?r - room)
    :precondition (available_room ?r ?a)
    :effect (and (booked_accommodation ?r ?a)
                 (not (available_room ?r ?a))
                 (doneBookingAccommodation)))
(:action book_transportation
   :parameters (?tp - transportation ?s - seat)
   :precondition (available_seat ?s ?tp)
   :effect (and (booked_transportation ?s ?tp)
                (not (available_seat ?s ?tp))
                (doneBookingTransportation))))
```

In the above, the types $hotel$ and $airbnb$ have been abstracted to type $accommodation$, actions $book_hotel$ and $book_airbnb$ to abstract action $book_accommodation$, and predicates $booked_hotel$ and $booked_airbnb$ to high-level predicate $booked_accommodation$. The type $room$ on the other hand, has not been abstracted.

The problem instance is then abstracted by using the types, predicates, and actions in the high-level domain. Objects also need to be assigned to the abstract types when necessary. ■

This category includes 8 benchmark examples, with domains modeling diverse scenarios such as apparel selection,

daily activities, household chores, and software development. The number of actions defined within these domains ranges from 3 to 6. The purpose of abstraction varies according to the domain. For example, in *SoftwareDev01*, where the low-level domain includes actions to resolve bugs off-by-one, operator precedence, wrong indentation, and unmatched parentheses, the purpose of abstraction instructs the LLM to provide an abstraction such that the high-level domain abstracts over resolving similar types of software program bugs, i.e., logical bugs and syntax bugs. In *DailyPlan01* on the other hand, where the concrete domain includes actions to read a book, watch a tutorial, cycle, hike, order food, and cook, the purpose of abstraction instructs the LLM to provide an abstraction such that the high-level domain abstracts over similar classes of daily activities.

Abstraction of Sequences of Concrete Actions. In this category, sequences of two or more low-level actions are abstracted to one high-level action. This typically also involves abstraction of conjunction of several low-level predicates into a single high-level predicate. As the previous category, some predicates or actions need not be abstracted.

Example 2. Consider the following concrete domain which models editing a file in an online editor hosted in a cloud. A user needs to first enter a valid username and then a valid matching password to login to his cloud account. Then he can open the file in an editor provided the file is originally closed and he has permission to edit it and after that, make changes to the file content.

```
(define (domain cloudApps01_LL)
...
 (:action enter_UserName
      :parameters (?u - userName)
      :precondition (valid_userName ?u)
      :effect (authenticated_userName ?u))
 (:action enter_passWord
      :parameters (?u - userName ?p - passWord)
      :precondition (and (valid_passWord ?p)
                         (authenticUserPassword ?u ?p)
                         (authenticated_userName ?u))
      :effect (authenticated_passWord ?p))
 (:action openFileInEditor
      :parameters (?f - file ?p - passWord ?u - userName)
      :precondition (and (closed_file ?f)
                         (hasEditPermission ?u ?f)
                         (authenticated_passWord ?p))
      :effect (and (openedFileInEditor ?f)
                   (not(closed_file ?f)) ))
 (:action changeFileConent
      :parameters (?f - file)
      :precondition (openedFileInEditor ?f)
      :effect (changedFileContent ?f)))
```

Now suppose that the purpose of abstraction states "provide an abstraction of the concrete domain such that the high-level domain abstracts over detailed steps of logging in and editing a file on cloud". The following high-level domain satisfies the purpose of abstraction:

```
(define (domain cloudApps01_HL)
...
(:action login
     :parameters (?u - userName ?p - passWord)
     :precondition (valid_credentials ?u ?p)
     :effect (logged_in ?u ?p))
(:action edit_file
     :parameters (?f - file ?p - passWord ?u - userName)
     :precondition (and (closed_file ?f)
                        (hasEditPermission ?u ?f)
                        (logged_in ?u ?p) )
     :effect (and (edited_file ?f)
                  (not(closed_file ?f)) ) ) )
```

Here, the sequence of actions $enter_userName$ and $enter_passWord$ is abstracted to $login$, and the predicate $logged_in$ (which abstracts over the conjunction of predicates $authenticated_userName$ and $authenticated_passWord$) is a precondition for the abstract action $edit_file$. Predicates $hasEditPermission$ and $closed_file$ are retained in the abstract domain. The problem instance is then abstracted by using the types, predicates, and actions in the high-level domain. ■

This category includes 11 examples, with domains modeling scenarios such as car manufacturing, cooking, painting, order delivery, laptop purchase, and rescue robot. The number of actions defined in these domains ranges from 3 to 6.

Abstraction of Action/Predicate Parameters. This category involves modifications to the low-level PDDL domain by abstracting away one or more action/predicate parameters. Typically, this includes the removal of a PDDL type, which often results in the corresponding deletion or modification of predicates and actions that utilize this type, for instance dropping parameters of this type.

Example 3. Consider the following low-level domain which models hotel bookings. A hotel room is characterized by room type (e.g., single or double), and a room view (e.g., ocean view or garden view). It is possible to modify a room's type, e.g., convert a single room to a double room.

```
(define (domain travelArrange02_LL)
...
(:action book_hotel
  :parameters (?h - hotel ?r - room ?rv - r_view ?rt -
    ↪ r_type)
  :precondition (and (available_room_hotel ?r ?h)
                     (roomType ?r ?rt)
                     (roomView ?r ?rv))
  :effect (and (booked_hotel ?rt ?h ?rv)
               (not (available_room_hotel ?r ?h))))
(:action  change_RoomType
   :parameters (?r - room ?rt1 ?rt2 - r_type)
   :precondition (and (roomType ?r ?rt1)
                      (difRoomType ?rt1 ?rt2)
   :effect (and (roomType ?r ?rt2)
                (not (roomType ?r ?rt1))))
```

Suppose that the purpose of abstraction is defined as "provide an abstraction of the concrete domain such that the high-level domain must not include information about room view". Then we expect the LLM to abstract over the room view, and generate the following high-level domain, where the type r_view has been removed, and predicates that utilize r_view have been either adjusted or removed:

```
(define (domain travelArrange02_HL)
...
(:action book_hotel
    :parameters (?h - hotel ?r - room ?rt - r_type)
    :precondition (and (available_room_hotel ?r ?h)
                       (roomType ?r ?rt))
    :effect (and (booked_hotel ?rt ?h)
                 (not (available_room_hotel ?r ?h))))
(:action change_RoomType
    :parameters (?r - room ?rt1 ?rt2 - r_type)
    :precondition (and (roomType ?r ?rt1)
                       (difRoomType ?rt1 ?rt2))
    :effect (and (roomType ?r ?rt2)
                 (not (roomType ?r ?rt1))))
```

If instead we ask the LLM to abstract over the room type, then in addition to removing r_type and adjusting/removing the associated predicates, we also expect the LLM to eliminate the action $change_RoomType$. The problem instance can then be adapted accordingly. ■

This category includes 10 examples with domains modeling diverse scenarios such as email composition, flight reservation, library, technical report writing, and travel (hotel) arrangements. The number of actions defined within these domains ranges from 1 to 5.

Abstraction of Alternative Sequences of Concrete Actions. In this category, the low-level domain contains multiple action sequences that provide alternative pathways to achieving a subgoal, and we require each such alternative sequence of actions to be abstracted into a single action.

Example 4. Consider a low-level domain which models alternative ways for holding a workshop at a campus: either a lecture hall is scheduled and the workshop is offered on campus, or web conferencing software is installed and the workshop is held online. Now suppose that the purpose of abstraction instructs the LLM to "abstract over delivering a workshop session". A high-level action which satisfies this purpose of abstraction may use the parameter type $teachingPlatform$ which abstracts over the disjunction of types $lectureHall$ and $webConferenceSoftware$, and a predicate $teachingCompleted$ (as an effect) that abstracts over the disjunction of the conjunction of predicates $lectureHallScheduled$ and $lecturedOnCampus$ when teaching platform is $lectureHall$, and the conjunction of predicates $installedVideoConferencing$ and $lecturedOnline$ when teaching platform refers to $webConferenceSoftware$. ■

This category includes 6 examples with domains modeling scenarios such as event planning, travel arrangements, repair robot, beverage preparation, and choosing apparel. The number of actions defined in these domains ranges from 4 to 12.

5 Implementation

5.1 Developing Prompts

Our approach relies on one-shot and zero-shot learning, combined with CoT and role-play. In the chat completions API of OpenAI's GPT-4o, prompts can be created by providing an array of messages that contain instructions for the model. It is possible to assign a different role to each message, and as a result, influence how the model might interpret the input. Here, we consider three roles: *system*, *user*, and *assistant*. A *system prompt* sets the overarching context, behavior, or persona for the responses generated. A *user prompt* contains the specific instructions or queries that a user provides to an LLM to elicit a desired response. An *assistant prompt* represents the LLM's response to user inputs, and is conditioned on the context provided by both the system and user prompts. Assistant prompts also serve as demonstrations of desired outputs when using in-context learning (e.g., providing example completions in one-shot prompts).

Abstraction of Alternative Concrete Actions: Zero-Shot Prompt. The system prompt requires the LLM to assume the role of a PDDL expert and to reason about the task in two stages: abstraction of domain and abstraction of problem instance (see [Banihashemi *et al.*, 2025] for the listings of prompts). In the first stage, it instructs the LLM to consider the purpose of abstraction and reason about generating the high-level domain ontology (types, predicates and actions), by combining or generalizing related elements, with the help of partial examples (e.g., two types 'hotel' and 'airbnb' can be combined into 'accommodation'). Note that inclusion of partial examples is necessary to guide the LLM towards the intended abstraction as our desired abstraction has not been part of any LLM's training data. In the second stage, the LLM is instructed to use the generated high-level domain components to generate the abstract problem instance. The LLM is required to follow a number of rules which include trying to minimize the number of domain ontology elements, using terminology for abstracted domain elements that preserves focus of the domain, complying with the STRIPS fragment of PDDL, and ensuring that the goal of the problem instance remains logically consistent with the abstract domain's purpose. A user's query is represented by a user prompt which includes a brief description of the low-level domain, low-level domain and problem files in PDDL, and the purpose of abstraction. Note that the partial examples in the system prompt are based on a different topic area than the PDDL domains/instances provided in user queries.

Abstraction of Alternative Concrete Actions: One-Shot Prompt. The system prompt is similar to the zero-shot approach but removes the partial examples, and instead the LLM is instructed to learn from the example provided, referred to as $Case1$. The one-shot example is modeled by a pair of user and assistant prompts. The user prompt includes a low-level domain and problem instance in PDDL, purpose of abstraction, and a brief explanation of the domain. In addition to the high-level domain and problem instance in PDDL, the assistant prompt includes a rationale for making choices of abstraction of actions, predicates and types. The user's query (referred to as $Case2$) is modelled as a user prompt like the zero-shot approach above. Note that the topic area in $Case1$ differs from that in $Case2$.

Abstraction of Sequences of Concrete Actions. This category only uses one-shot prompting, as preliminary experiments with zero-shot prompts produced unsatisfactory results. The system prompt directs the LLM through a two-stage reasoning process: domain abstraction followed by problem instance abstraction, guided by Case1 and predefined rules which are mainly similar to the previous category. The LLM is asked to consider the purpose of abstraction and identify sequences of actions that should be merged into high-level actions that achieve the same effect but eliminate unnecessary intermediate steps. The one-shot example and user query are modelled similarly to the one-shot approach in the previous category.

Abstraction of Action/Predicate Parameters. This category only uses a zero-shot prompting approach, as it proved sufficient for generating mostly correct results. The system prompt instructs the LLM to consider the purpose of abstraction and decide whether various domain elements (parame-

ters, types, predicates, actions) must be retained or eliminated in the abstract domain. The user query for this category is defined similar to the previous zero-shot prompt.

Abstraction of Alternative Sequences of Concrete Actions. Here we restrict our analysis to one-shot prompts, as initial experiments with zero-shot prompts produced unsatisfactory outcomes. The system prompt is structured similar to previous categories' system prompt for one-shot learning, however, it is more generic and includes abstraction instructions which combine elements from all previous categories The user query and one-shot demonstration are modelled similar to previous categories.

5.2 Evaluating the Generated Models

We use a hybrid evaluation approach that incorporates both human evaluation and automated tools. Our method first uses VAL [Howey *et al.*, 2004] to check for syntax errors in the generated high-level domain and problem instance, and then calls the Fast Downward planner [Helmert, 2006] to try to generate a plan.

Next, the generated high-level domain and problem instance are reviewed by human domain experts. Each generated high-level domain undergoes a comparative analysis with a sample abstract domain (already created by a knowledge engineer) which represents a sound abstraction of the low-level domain in user's query, and it is part of the solution to the PDAG task. This evaluation considers the correctness of abstract actions, predicates, types, and parameters generated, as well as the removal of actions, predicates, types, and parameters from the high-level domain that are considered unnecessary details wrt the purpose of abstraction. Concrete domain elements that must be retained in the generated abstract domain are also considered. Syntax errors are also noted. High-level problem instances are evaluated similarly.

Human expert evaluation is essential as there may be more than one correct approach to abstract a domain. E.g., in case of "CookFood01" when abstracting over sequence of low-level actions $wash, chop, marinate, grill$ using two high-level abstract actions $prepare$ and $cook$, one solution may abstract sequence of concrete actions $wash, chop$ to action $prepare$, while another solution may consider $wash, chop, marinate$ as refinement of action $prepare$. Moreover, the LLM often generates names for actions/predicates/types that may be different from those in the sample solution provided. So this part of the evaluation would be difficult to automate.

As our aim in this paper is to evaluate the feasibility of using advanced LLMs to generate abstract planning domain and problem files based on an initial input prompt, we do not consider providing corrective feedback from users or validation tools to the LLM to allow it to fix errors and re-generate high-level domains and/or problem files.

5.3 System Architecture

Fig. 1 shows the outline of our system. Initially, a system prompt (and possibly a user/assistant prompt in case of one-shot in-context learning) is provided to the LLM (GPT-4o) which describes the task, rules to follow, expected input and desired output. The user then provides his query which consists of a low-level PDDL domain and its brief description, PDDL problem instance, and the purpose of abstraction. After the LLM processes the query, it generates a response which includes a high-level domain and a high-level problem instance in PDDL as well as a natural language description of the justification for the specific choices of abstraction made.

Our code then extracts the generated high-level PDDL domain and problem instance as .pddl files and sends them to the validation module (which includes VAL and the Fast Downward Planner). The validation results and a plan in case of success, or error messages in case of failure, a summary report, in addition to the generated PDDL domain and problem instance as well as the natural language description of the justification for the specific choices of abstraction are then saved to a datastore. A human evaluator then reviews the stored information and saves the results of analysis to the datastore. Note that we don't *formally verify* that the LLM generated high-level domain/problem instance is a sound abstraction and satisfies the constraints on the mapping associated with the purpose of abstraction; this is left for future work.

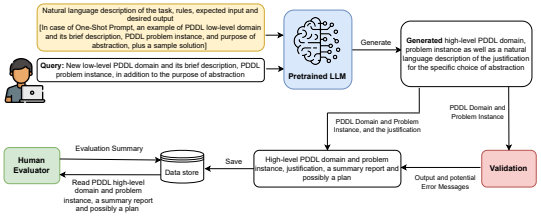


Figure 1: Generating Abstract PDDL Domain and Problem Instance

6 Empirical Evaluation

Each of our benchmark examples within each category was run five times. Tables 1 to 5 present the aggregated results. The table columns show the following metrics: **Changes Needed (CN):** correctness score of predicates/actions/types intended to be abstracted or eliminated as evaluated by human experts (e.g., generalizing *hotel* and *airbnb* to *accommodation*). **CN-SD:** standard deviation of CN scores. **Avoid Unnecessary Changes (AUC):** correctness score of predicates/actions/types/objects that were expected to remain unchanged in the high-level domain and problem instance as evaluated by human domain experts (e.g., keeping the type *room* in the abstract domain and problem instance in Example 1). **HDE, VAL, and FD** indicate if syntax errors were present and whether they were detected by human domain experts (HDE), VAL, or the Fast Downward planner (FD). If the planner fails to generate a plan, we count this as an FD error.

Our experiment results show that GPT-4o can generate almost error-free results for abstraction of action/predicate parameters, but as the difficulty of abstraction increases with abstraction of alternative concrete actions, sequences of concrete actions, and alternative sequences of concrete actions, the quality of generated domains/problem instances declines and the number of syntax errors increases. We observed that

	CN (Avg)	CN-SD	AUC (Avg)	AUC-SD	HDE (Count)	FD (Count)	VAL (Count)
DailyPlan01	75.11%	13.8	100%	0	2	0	2
DailyPlan02	85.07%	5.5	98.46%	3.08	3	3	3
HouseHold01	100%	0	100%	0	0	0	0
ClothesShop01	88.46%	14.13	74.55%	31.18	0	0	0
SoftwareDev01	94.28%	6.29	100%	0	0	0	0
SoftwareDev02	90.95%	4.86	97.71%	2.8	0	0	0
SoftwareDev03	96%	2.42	91.11%	4.44	0	0	0
SoftwareDev04	97.88%	1.73	100%	0	0	0	0

Table 1: Abstraction of Alternative Concrete Actions - Zero-Shot

	CN (Avg)	CN-SD	AUC (Avg)	AUC-SD	HDE (Count)	FD (Count)	VAL (Count)
DailyPlan01	93.19%	2.08	100%	0	2	2	2
DailyPlan02	100%	0	100%	0	0	0	0
HouseHold01	100%	0	100%	0	0	0	0
ClothesShop01	100%	0	100%	0	0	0	0
SoftwareDev01	100%	0	100%	0	0	0	0
SoftwareDev02	87.14%	11.82	68%	27.01	1	1	0
SoftwareDev03	98.12%	2.31	100%	0	2	0	2
SoftwareDev04	100%	0	100%	0	0	0	0

Table 2: Abstraction of Alternative Concrete Actions - One-Shot

the majority of mistakes were in generating new predicates which abstract over concrete predicates. We also noticed that changing the order of action declarations or including additional constructs (actions, types, objects, predicates) which are irrelevant to the goal of the task do not cause additional errors. In general, the LLM was able to name abstracted actions/predicates/types reasonably well; however, in some cases, names for high-level domain constructs were incorrect. E.g., in *DailyPlan01*, the abstraction of alternative concrete actions *orderFood* and *cook* was named *eat*. See [Banihashemi *et al.*, 2025] for a more detailed discussion on evaluation results.

7 Related Work

Several authors have studied using LLMs to generate PDDL specifications given natural language descriptions of the task. [Liu *et al.*, 2023] assumes that a PDDL domain description and a contextual example demonstrating the conversion of a natural language problem within the domain into a PDDL problem are provided by human experts, and the LLM is tasked with generation of the PDDL specification of a problem instance (based on the domain) given a natural language description of the problem. [Xie *et al.*, 2023] studies extracting a planning goal given a natural language instruction, by using zero-shot and few-shot prompting. [Guan *et al.*, 2023] follows a strategy where few-shot prompting is used to generate a PDDL representation of a single action at a time, while dynamically updating a list of predicates, and ultimately repeating the process with all the extracted predicates. [Oswald *et al.*, 2024] also uses in-context learning to generate PDDL domains on an action-by-action basis using context examples from other domains. The query in the input prompt includes the allowed predicates that can be used in the definition of that action and a natural language description of the action. The above approaches do not involve generating abstractions of low-level PDDL domains or problem instances. To address complex sequential decision-making, [Liu *et al.*, 2024] introduces SkillAct, a prompting method that integrates reusable, task-relevant skill descriptions into prompts. Skills abstract high-level behaviors from agent trajectories (observation-action sequences) and are derived from the LLM's embedded world knowledge via prompting. The skills/generated outputs are not formalized in PDDL.

	CN (Avg)	CN-SD	AUC (Avg)	AUC-SD	HDE (Count)	FD (Count)	VAL (Count)
CookFood01	79.2%	6.01	88.61%	13.41	0	0	0
CookFood02	72.4%	7.2	95.45%	9.09	0	0	0
CarManufacturing01	48.04%	9.71	92.68%	14.63	2	2	2
CleanItem02	71.59%	7.65	100%	0	2	1	1
LaptopShop02	65.15%	15.15	83.64%	13.36	0	0	0
DeliveryRobot01	78%	15.86	80.47%	10.36	2	1	2
DeliveryRobot02	86.21%	0	92.45%	0	0	0	0
DeliveryRobot03	100%	0	94.67%	2.67	0	0	0
DeliveryRobot04	92.55%	5.17	89.33%	9.98	1	1	1
RescueRobot02	89.92%	4.96	96.42%	5.22	0	0	0
RescueRobot03	87.5%	5.24	92.54%	5.26	1	1	1

Table 3: Abstraction of Sequences of Concrete Actions - One-Shot

	CN (Avg)	CN-SD	AUC (Avg)	AUC-SD	HDE (Count)	FD (Count)	VAL (Count)
Email01-0	100%	0	100%	0	0	0	0
Email01-1	100%	0	100%	0	0	0	0
Flight01	100%	0	100%	0	0	0	0
Flight02	100%	0	100%	0	0	0	0
Library01-1	100%	0	100%	0	0	0	0
Library01-2	96.92%	6.15	96.10%	7.62	0	0	0
TechReport01	100%	0	100%	0	0	0	0
TechReport02	100%	0	100%	0	0	0	0
Travel02	100%	0	100%	0	0	0	0
Travel03	100%	0	100%	0	0	0	0

Table 4: Abstraction of Action/Predicate Parameters - Zero Shot

	CN (Avg)	CN-SD	AUC (Avg)	AUC-SD	HDE (Count)	FD (Count)	VAL (Count)
BeveragePreparation01	43.92%	27.79	100%	0	9	1	9
ClothesShop02	65.37%	13.59	100%	0	0	0	0
EventPlanning01	56.62%	1.5	96.1%	2.46	13	3	14
RepairRobot04	67.03%	5.21	87.06%	7.8	3	3	3
RoomRedesign01	69.6%	3.32	100%	0	12	3	12
Travel05	62.14%	11.85	100%	0	10	2	11

Table 5: Abstraction of Alternative Sequences of Concrete Actions

8 Conclusion and Future Work

In this paper, we investigated the feasibility of using an LLM (GPT-4o) to generate high-level PDDL domains and problem instances from low-level PDDL representations, guided by a specified abstraction purpose.

In future work, we plan to extend our set of examples and study additional abstraction categories. We will also consider using the more expressive ADL [Pednault, 1989] fragment of PDDL. Building a dataset of planning examples that can be used for fine-tuning an LLM is another avenue for further research. We are also interested in designing other types of prompts that could enhance the generated results. We will also examine extending the current abstraction task to incorporate the generation of a refinement mapping so that one can then check that the output high-level model is a sound abstraction of the low-level model relative to this mapping. A significant area for future work lies in the development of (partially) automated validation mechanisms to ensure the correctness of the abstracted domains and problems generated by LLMs. More work on the formalization of the abstraction purpose and automated mechanisms that check the generated models' adherence to it is also indicated.

Acknowledgements

This work is partially supported by the National Science and Engineering Research Council of Canada and by York University.

References

[Bacchus and Kabanza, 2000] Fahiem Bacchus and Froduald Kabanza. Using temporal logics to express search control knowledge for planning. *Artificial Intelligence*, 116(1-2):123–191, 2000.

[Banihashemi *et al.*, 2017] Bita Banihashemi, Giuseppe De Giacomo, and Yves Lespérance. Abstraction in situation calculus action theories. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pages 1048–1055. AAAI Press, 2017.

[Banihashemi *et al.*, 2024] Bita Banihashemi, Giuseppe De Giacomo, and Yves Lespérance. Abstracting situation calculus action theories. *CoRR*, abs/2410.14712, 2024.

[Banihashemi *et al.*, 2025] Bita Banihashemi, Megh Patel, and Yves Lespérance. Using large language models for abstraction of planning domains - Extended Version. To appear on arXiv, 2025.

[Brown *et al.*, 2020] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020.

[Chen and Bercher, 2021] Dillon Chen and Pascal Bercher. Fully observable nondeterministic HTN planning - formalisation and complexity results. In *Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling, ICAPS 2021*, pages 74–84. AAAI Press, 2021.

[Claßen *et al.*, 2007] Jens Claßen, Patrick Eyerich, Gerhard Lakemeyer, and Bernhard Nebel. Towards an integration of golog and planning. In *IJCAI*, pages 1846–1851, 2007.

[De Giacomo *et al.*, 2000] Giuseppe De Giacomo, Yves Lespérance, and Hector J. Levesque. ConGolog, a concurrent programming language based on the situation calculus. *Artificial Intelligence*, 121(1-2):109–169, 2000.

[Dong *et al.*, 2024] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. A survey on in-context learning. *CoRR*, abs/2301.00234, 2024.

[Dong *et al.*, 2025] Hao Dong, Zheyuan Shi, Hemeng Zeng, and Yongmei Liu. An automatic sound and complete abstraction method for generalized planning with baggable types. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence*, pages 14875–14884. AAAI Press, 2025.

[Erol *et al.*, 1996] Kutluhan Erol, James A. Hendler, and Dana S. Nau. Complexity results for HTN planning. *Annals of Mathematics and Artificial Intelligence*, 18(1):69–93, 1996.

[Fang *et al.*, 2025] Liangda Fang, Xiaoman Wang, Zhang Chen, Kailun Luo, Zhenhe Cui, and Quanlong Guan. A syntactic approach to computing complete and sound abstraction in the situation calculus. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence*, pages 14911–14921. AAAI Press, 2025.

[Fox and Long, 2011] Maria Fox and Derek Long. PDDL2.1: an extension to PDDL for expressing temporal planning domains. *CoRR*, abs/1106.4561, 2011.

[Ghallab *et al.*, 2004] Malik Ghallab, Dana S. Nau, and Paolo Traverso. *Automated planning - theory and practice*. Elsevier, 2004.

[Guan *et al.*, 2023] Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.

[He *et al.*, 2023] Weinan He, Canming Huang, Zhanhao Xiao, and Yongmei Liu. Exploring the capacity of pretrained language models for reasoning about actions and change. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023*, pages 4629–4643. Association for Computational Linguistics, 2023.

[Helmert, 2006] Malte Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.

[Howey *et al.*, 2004] Richard Howey, Derek Long, and Maria Fox. VAL: automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI 2004)*, pages 294–301. IEEE Computer Society, 2004.

[Hurst *et al.*, 2024] Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Madry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, Alex Nichol, Alex Paino, Alex Renzin, Alex Tachard Passos, Alexander Kirillov, Alexi Christakis, Alexis Conneau, Ali Kamali, Allan Jabri, Allison Moyer, Allison Tam, Amadou Crookes, Amin Tootoonchian, Ananya Kumar, Andrea Vallone, Andrej Karpathy, Andrew Braunstein, Andrew Cann, Andrew Codispoti, Andrew Galu, Andrew Kondrich, Andrew Tulloch, Andrey Mishchenko, Angela Baek, Angela Jiang, Antoine Pelisse, Antonia Woodford, Anuj Gosalia, Arka Dhar, Ashley Pantuliano, Avi Nayak, Avital Oliver, Barret Zoph, Behrooz Ghorbani, Ben Leimberger, Ben Rossen, Ben Sokolowsky, Ben Wang, Benjamin Zweig, Beth Hoover, Blake Samic, Bob McGrew,

Bobby Spero, Bogo Giertler, Bowen Cheng, Brad Lightcap, Brandon Walkin, Brendan Quinn, Brian Guarraci, Brian Hsu, Bright Kellogg, Brydon Eastman, Camillo Lugaresi, Carroll L. Wainwright, Cary Bassin, Cary Hudson, Casey Chu, Chad Nelson, Chak Li, Chan Jun Shern, Channing Conger, Charlotte Barette, Chelsea Voss, Chen Ding, Cheng Lu, Chong Zhang, Chris Beaumont, Chris Hallacy, Chris Koch, Christian Gibson, Christina Kim, Christine Choi, Christine McLeavey, Christopher Hesse, Claudia Fischer, Clemens Winter, Coley Czarnecki, Colin Jarvis, Colin Wei, Constantin Koumouzelis, and Dane Sherburn. Gpt-4o system card. *CoRR*, abs/2410.21276, 2024.

[Kong *et al.*, 2024] Aobo Kong, Shiwan Zhao, Hao Chen, Qicheng Li, Yong Qin, Ruiqi Sun, Xin Zhou, Enzhi Wang, and Xiaohang Dong. Better zero-shot reasoning with role-play prompting. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024*, pages 4099–4113. Association for Computational Linguistics, 2024.

[Li, 2023] Yinheng Li. A practical survey on zero-shot prompt design for in-context learning. In *Proceedings of the 14th International Conference on Recent Advances in Natural Language Processing, RANLP 2023*, pages 641–647. INCOMA Ltd., Shoumen, Bulgaria, 2023.

[Liu *et al.*, 2023] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: empowering large language models with optimal planning proficiency. *CoRR*, abs/2304.11477, 2023.

[Liu *et al.*, 2024] Anthony Zhe Liu, Jongwook Choi, Sungryull Sohn, Yao Fu, Jaekyeom Kim, Dong-Ki Kim, Xinhe Wang, Jaewon Yoo, and Honglak Lee. SkillAct: Using skill abstractions improves LLM agents. In *ICML 2024 Workshop on LLMs and Cognition*, 2024.

[Luo *et al.*, 2020] Kailun Luo, Yongmei Liu, Yves Lespérance, and Ziliang Lin. Agent abstraction via forgetting in the situation calculus. In *ECAI 2020 - 24th European Conference on Artificial Intelligence*, volume 325 of *Frontiers in Artificial Intelligence and Applications*, pages 809–816. IOS Press, 2020.

[Luo, 2023] Kailun Luo. Automated verification of propositional agent abstraction for classical planning via CTLK model checking. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023*, pages 6475–6482. AAAI Press, 2023.

[McCarthy and Hayes, 1969] J. McCarthy and P. J. Hayes. Some philosophical problems from the standpoint of artificial intelligence. *Machine Intelligence*, 4:463–502, 1969.

[McDermott *et al.*, 1998] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL – the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998.

[Oswald *et al.*, 2024] James T. Oswald, Kavitha Srinivas, Harsha Kokel, Junkyu Lee, Michael Katz, and Shirin Sohrabi. Large language models as planning domain generators. In *Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling, ICAPS 2024*, pages 423–431. AAAI Press, 2024.

[Pednault, 1989] Edwin P. D. Pednault. ADL: exploring the middle ground between STRIPS and the situation calculus. In *Proceedings of the 1st International Conference on Principles of Knowledge Representation and Reasoning (KR'89)*, pages 324–332. Morgan Kaufmann, 1989.

[Reiter, 2001] Ray Reiter. *Knowledge in Action. Logical Foundations for Specifying and Implementing Dynamical Systems*. The MIT Press, 2001.

[Seegebarth *et al.*, 2012] Bastian Seegebarth, Felix Müller, Bernd Schattenberg, and Susanne Biundo. Making hybrid plans more clear to human users - A formal approach for generating sound explanations. In *Proceedings of the Twenty-Second International Conference on Automated Planning and Scheduling, ICAPS 2012*. AAAI, 2012.

[Sutton *et al.*, 1999] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and Semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.

[Valmeekam *et al.*, 2024] Karthik Valmeekam, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. Planning in strawberry fields: Evaluating and improving the planning and scheduling capabilities of LRM o1. *CoRR*, abs/2410.02162, 2024.

[Wei *et al.*, 2022] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022*, 2022.

[Xie *et al.*, 2023] Yaqi Xie, Chen Yu, Tongyao Zhu, Jinbin Bai, Ze Gong, and Harold Soh. Translating natural language to planning goals with large-language models. *CoRR*, abs/2302.05128, 2023.

[Zhao *et al.*, 2023] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. *CoRR*, abs/2303.18223, 2023.

Initial Models and Serialisability in Abstract Dialectical Frameworks (Extended Abstract)

Lars Bengel , Matthias Thimm
Artificial Intelligence Group, University of Hagen, Germany
{lars.bengel, matthias.thimm}@fernuni-hagen.de

1 Introduction

In this work we consider the *abstract dialectical framework* (ADF) [Brewka *et al.*, 2013], which has emerged as a very powerful generalisation of the *abstract argumentation framework* (AF) [Dung, 1995]. An AF is a directed graph, where the nodes represent abstract arguments and the edges represent attacks among them. In an ADF, the relations between the arguments are more general and are represented by *acceptance conditions*, i. e., (propositional) logical formulae that specify the conditions under which arguments may be accepted. This enables the representation of more complex relationships between arguments beyond simple attacks, such as collective attacks or support relations.

Formal semantics for both AFs and ADFs are given through functions that determine admissible sets of arguments, called extensions [Baroni *et al.*, 2018], or, in the case of ADFs, three-valued models [Brewka *et al.*, 2017]. In particular, the classical admissibility-based semantics of Dung have been generalised to ADFs [Brewka *et al.*, 2013].

As the name suggests, ADFs are inspired by dialectics [Brewka *et al.*, 2013]. An important element of dialectics is procedurality, i. e., the fact that arguments are put forward sequentially and are then followed by counter-arguments [Hage, 2000; Rescher, 1977]. While this aspect is modelled well on the syntactic level in ADFs, on the semantical side this aspect is somewhat lost, just like in the case of AFs [Verheij, 1996]. Consider, for instance, the ADF in Figure 1, where acceptance conditions of arguments are placed right above them (we will provide formal definitions in Section 2). We have that a and b can only be accepted if the other is rejected, meaning they form a sort of atomic conflict that must be resolved. Only after resolving this conflict, for example by accepting a and rejecting b, can we turn to the remaining arguments and evaluate them properly. Now, rejecting b directly implies that c must also be rejected and in turn that we shall accept d afterwards. On the other hand, if we accept b and reject a in the initial conflict, that implies that we accept c and subsequently reject d. If we only consider the resulting admissible model that assigns the respective truth values, we disregard this information about the reasoning process of the argumentation performed to arrive at the conclusion.

An approach to address the above case is *serialisability* for AFs [Thimm, 2022]. It provides a non-deterministic construction scheme for extensions, where initial sets are selected iteratively. An initial set [Xu and Cayrol, 2018] is thereby defined as a non-empty, minimal admissible set, essentially representing a minimal semantical unit of the AF. An extension can be represented by *serialisation sequences*, i. e., sequences of initial sets that represent an order in which the extension can be built. In this work, we characterise initial models for ADFs and generalise the notion of serialisation sequences to ADFs. The full version of the presented work has been published in [Bengel and Thimm, 2025].

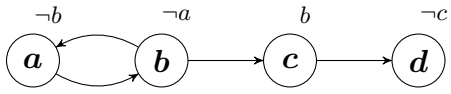


Figure 1: An ADF $\mathcal{D}$, the arguments a and b are in a conflict, while b supports c and the argument d may only be accepted if c is rejected.

2 Background

An *abstract argumentation framework* (AF) is a pair $\mathcal{F} = (\mathcal{A}, \mathcal{R})$ where $\mathcal{A}$ is a finite set of arguments and $\mathcal{R}$ is a relation $\mathcal{R} \subseteq \mathcal{A} \times \mathcal{A}$. For two arguments $a, b \in \mathcal{A}$, the relation $a\mathcal{R}b$ means that argument a *attacks* b. We say that a set $S \subseteq \mathcal{A}$ is *conflict-free* iff for all $a, b \in S$ it is not the case that $(a, b) \in \mathcal{R}$. A set S *defends* an argument $b \in \mathcal{A}$ iff for all a with $(a, b) \in \mathcal{R}$ there is $c \in S$ with $(c, a) \in \mathcal{R}$. Furthermore, a set S is called *admissible* (ad) iff it is conflict-free and S defends all $a \in S$. Non-empty minimal admissible sets have been coined *initial sets* by Xu and Cayrol (2018).

Definition 1. For $\mathcal{F} = (\mathcal{A}, \mathcal{R})$, a set $S \subseteq \mathcal{A}$ with $S \neq \emptyset$ is called an *initial set* if S is admissible and there is no admissible $S' \subsetneq S$ with $S' \neq \emptyset$.

An *abstract dialectical framework* (ADF) is a pair $\mathcal{D} = (\mathcal{A}, \mathcal{C})$ where $\mathcal{A}$ is a set of arguments and $\mathcal{C}$ is a set of propositional formulae $\{\phi_a\}_{a \in \mathcal{A}}$ over $\mathcal{A}$, called *acceptance conditions*.[1] Reasoning in ADFs is performed via three-valued propositional interpretations $v : \mathcal{A} \mapsto \{\mathsf{t}, \mathsf{f}, \mathsf{u}\}$ that satisfy all acceptance conditions called *models*. Under three-valued interpretations a formula is then evaluated inductively as usual.

[1] Note that the original definition of ADFs [Brewka *et al.*, 2013] includes the *links* between arguments explicitly, but here we assume them implicitly given by the acceptance conditions.

We will also write $a \mapsto \mathsf{x}$ instead of $v(a) = \mathsf{x}$ and omit assignments to u for readability.

We consider the following partial order $\leq_i$: $\mathsf{u} <_i \mathsf{t}$, $\mathsf{u} <_i \mathsf{f}$ and no other pair in $<_i$. For two interpretations v_1, v_2 we define $v_1 \leq_i v_2$ iff $v_1(a) \leq_i v_2(a)$ for all $a \in \mathcal{A}$. For two models v_1, v_2, let $v_1 \sqcap v_2$ be the *consensus*, i. e., the model that takes the assignments where both v_1 and v_2 coincide and assigns u otherwise. For some three-valued interpretation v, we define the set of *completions* $[v]_2$ as $[v]_2 = \{v' \mid v \leq_i v', (v')^{-1}(\mathsf{u}) = \emptyset\}$.

For the semantic evaluation of an ADF $\mathcal{D}$ we then define the *characteristic operator* $\Gamma_\mathcal{D}$ which computes for a model v the consensus of all its completions for every $a \in \mathcal{A}$ as

$$\Gamma_\mathcal{D}(v)(a) = \bigsqcap \{v'(\phi_a) \mid v' \in [v]_2\}.$$

A model v is then called *admissible* in $\mathcal{D}$ iff $v \leq_i \Gamma_\mathcal{D}(v)$.

For a given argumentation framework $\mathcal{F} = (\mathcal{A}, \mathcal{R})$, we define the corresponding ADF $\mathcal{D}_\mathcal{F}$ as follows.

Definition 2. Let $\mathcal{F} = (\mathcal{A}, \mathcal{R})$ be an AF. Then, we define the ADF $\mathcal{D}_\mathcal{F} = (\mathcal{A}, \mathcal{C})$, where $\mathcal{A}$ is the same set of arguments and $\mathcal{C}$ is defined as $\{\phi_a = \bigwedge_{b \in a^-} \neg b\}_{a \in \mathcal{A}}$.

3 Characterising Initial Models and Serialisation Sequences in ADFs

We now introduce *initial models* for ADFs and formulate the notion of *serialisation sequences* based on them. First, for some conflict-free set of arguments $S \subseteq \mathcal{A}$ of $\mathcal{F}$ we define the corresponding interpretation v_S of $\mathcal{D}_\mathcal{F}$ via

$$v_S(a) = \begin{cases} \mathsf{t} & \text{if } a \in S \\ \mathsf{f} & \text{if } a \mathcal{R} S \\ \mathsf{u} & \text{otherwise} \end{cases} \quad (1)$$

With that, we now define the *initial models* of an ADF as those models that are admissible and minimal wrt. the information ordering $\leq_i$, excluding the model v_u that assigns u to all arguments.

Definition 3. Let $\mathcal{D} = (\mathcal{A}, \mathcal{C})$ be an ADF. An interpretation $v : \mathcal{A} \mapsto \{\mathsf{t}, \mathsf{f}, \mathsf{u}\}$ is called an *initial model* of $\mathcal{D}$, iff v is admissible with $v \neq v_\mathsf{u}$ and there is no admissible model $v' \neq v_\mathsf{u}$ with $v' <_i v$. $\mathsf{is}(\mathcal{D})$ denotes the initial models of $\mathcal{D}$.

Theorem 1. *Let $\mathcal{F} = (\mathcal{A}, \mathcal{R})$ be an AF and $\mathcal{D}_\mathcal{F}$ is the corresponding ADF. Then $S \subseteq \mathcal{A}$ is an initial set of $\mathcal{F}$ if and only if v_S is an initial model of $\mathcal{D}_\mathcal{F}$.*

Example 1. Consider again the ADF $\mathcal{D}$ in Figure 1. There are two initial models for $\mathcal{D}$: $v_1 = \{a \mapsto \mathbf{t}, b \mapsto \mathbf{f}\}$ and $v_2 = \{a \mapsto \mathbf{f}, b \mapsto \mathbf{t}\}$. Note that for both a and b assigning them to t means that the other must necessarily be assigned f in order to have a valid admissible model. Note also that for the latter initial model v_2, while $v_2(\phi_c) = \mathsf{t}$, due to the minimality c is still assigned u.

We now characterise admissibility for ADFs in terms of serialisation sequences, i. e. sequences of initial models. First, we generalise the notion of the *reduct* in the sense of [Baumann *et al.*, 2020] to ADFs. For some model v, we define the v-reduct $\mathcal{D}^v$ of $\mathcal{D}$ as the ADF where all arguments that are evaluated to t or f by v are removed and their occurrence in the acceptance condition of some other argument is replaced by $\top$ or $\bot$ respectively. The intuition being that the reduct of an ADF $\mathcal{D}$ wrt. some model v represents the part of the ADF that is unresolved by v.

Definition 4. Let $\mathcal{D} = (\mathcal{A}, \mathcal{C})$ be an ADF and $v : \mathcal{A} \mapsto \{\mathsf{t}, \mathsf{f}, \mathsf{u}\}$ is a three-valued interpretation. Then we define the *v-reduct* of $\mathcal{D}$ as the ADF $\mathcal{D}^v = (\mathcal{A}', \{\phi'_a\}_{a \in \mathcal{A}'})$ where

$$\begin{aligned} \mathcal{A}' &= \mathcal{A} \setminus \{a \in \mathcal{A} \mid v(a) \neq \mathsf{u}\}, \\ \mathcal{C}' &= \{\phi'_a\}_{a \in \mathcal{A}'} \end{aligned}$$

with $\mathsf{x} \in \{\mathsf{t}, \mathsf{f}\}$ and $\phi'_a = \phi_a^{[b/\mathsf{x} \,:\, v(b) = \mathsf{x}]}$.

For two non-conflicting models v_1, v_2, let $v_1 \sqcup v_2$ be the union of v_1 and v_2. We now define the concept of the *serialisation sequence* for ADFs as a series of initial models of the respective reducts.

Definition 5. A *serialisation sequence* for $\mathcal{D} = (\mathcal{A}, \mathcal{C})$ is a sequence $\mathcal{Y} = (v_1, \dots v_n)$ with $v_1 \in \mathsf{is}(\mathcal{D})$ and for each $2 \leq i \leq n$ we have that $v_i \in \mathsf{is}(\mathcal{D}^{v_1 \sqcup \dots \sqcup v_{i-1}})$.

Based on the above results, we can then show that the union of all initial models v_i in some serialisation sequence $\mathcal{Y} = (v_1, \dots v_n)$ corresponds directly to an admissible model. In particular, we can characterise the admissible models for ADFs in this way.

Theorem 2. *A serialisation sequence $\mathcal{Y} = (v_1, \dots v_n)$ induces an admissible model $v = v_1 \sqcup \dots \sqcup v_n$ and for every admissible model there is at least one such sequence.*

Example 2. Consider again the ADF $\mathcal{D}$ in Figure 1. Consider for instance the sequence

$$(\{a \mapsto \mathbf{t}, b \mapsto \mathbf{f}\}, \{c \mapsto \mathbf{f}\}, \{d \mapsto \mathbf{t}\}\}).$$

As shown in Example 1, $v_1 = \{a \mapsto \mathbf{t}, b \mapsto \mathbf{f}\}$ is an initial model of $\mathcal{D}$. We then consider the reduct $\mathcal{D}^{v_1}$ with $\phi_c = \bot$ and $\phi_d = \neg c$. It is easy to see that after resolving a and b, the only initial model of the reduct is $v_2 = \{c \mapsto \mathbf{f}\}$. Finally, in the reduct $\mathcal{D}^{v_1 \sqcup v_2}$, we have only $\phi_d = \top$ and thus the initial model $v_3 = \{d \mapsto \mathbf{t}\}$. This matches exactly the intuition described in the introduction and is also the only serialisation sequence for the admissible model $v_1 \sqcup v_2 \sqcup v_3$. Alternatively there is also the unique serialisation sequence

$$(\{a \mapsto \mathbf{f}, b \mapsto \mathbf{t}\}, \{c \mapsto \mathbf{t}\}, \{d \mapsto \mathbf{f}\}\})$$

for the other maximal admissible model of $\mathcal{D}$.

Note that in general there can be multiple serialisation sequences for the same admissible model. By putting restrictions on the serialisation sequences, e. g., maximal length, we can then also characterise other admissibility-based semantics for ADFs.

4 Conclusion

In this work, we considered abstract dialectical frameworks and introduced the notion of initial models. Moreover, we showed that they correspond to initial sets in AFs. Finally, we generalised serialisation sequences to ADFs which allows us to characterise admissibility-based semantics in a sequence-based form. In the full paper, we also investigated the computational complexity of tasks related to initial models.

Acknowledgements

We thank Johannes P. Wallner for some discussions related to the computational complexity of the problems discussed in this paper. The research reported here was partially supported by the Deutsche Forschungsgemeinschaft (grant 550735820).

References

[Baroni *et al.*, 2018] Pietro Baroni, Martin Caminada, and Massimiliano Giacomin. Abstract argumentation frameworks and their semantics. In Pietro Baroni, Dov Gabbay, Massimiliano Giacomin, and Leendert van der Torre, editors, *Handbook of Formal Argumentation*, pages 159–236. College Publications, 2018.

[Baumann *et al.*, 2020] Ringo Baumann, Gerhard Brewka, and Markus Ulbricht. Revisiting the foundations of abstract argumentation - semantics based on weak admissibility and weak defense. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020*, pages 2742–2749. AAAI Press, 2020.

[Bengel and Thimm, 2025] Lars Bengel and Matthias Thimm. Initial models and serialisability in abstract dialectical frameworks. In *Proceedings of the Thirty-fourth International Joint Conference on Artificial Intelligence, IJCAI 2025*. ijcai.org, 2025.

[Brewka *et al.*, 2013] Gerhard Brewka, Hannes Strass, Stefan Ellmauthaler, Johannes Peter Wallner, and Stefan Woltran. Abstract dialectical frameworks revisited. In Francesca Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, 2013*, pages 803–809. IJCAI/AAAI, 2013.

[Brewka *et al.*, 2017] Gerhard Brewka, Stefan Ellmauthaler, Hannes Strass, Johannes Peter Wallner, and Stefan Woltran. Abstract dialectical frameworks. an overview. *Handbook of Formal Argumentation*, 1, 2017.

[Dung, 1995] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–358, 1995.

[Hage, 2000] Jaap Hage. Dialectical models in artificial intelligence and law. *Artificial Intelligence and Law*, 8(2/3):137–172, 2000.

[Rescher, 1977] Nicholas Rescher. *Dialectics: A controversy-oriented approach to the theory of knowledge*. Suny Press, 1977.

[Thimm, 2022] Matthias Thimm. Revisiting initial sets in abstract argumentation. *Argument & Computation*, 13(3):325–360, 2022.

[Verheij, 1996] Bart Verheij. Two approaches to dialectical argumentation: admissible sets and argumentation stages. *Proc. NAIC*, 96:357–368, 1996.

[Xu and Cayrol, 2018] Yuming Xu and Claudette Cayrol. Initial sets in abstract argumentation frameworks. *Journal of Applied Non-Classical Logics*, 28(2-3):260–279, 2018.

Planning with Dynamically Changing Domains

Mikhail Soutchanski[1] , **Yongmei Liu**[2]
[1]Toronto Metropolitan University (TMU), Toronto, Canada
[2]Sun Yat-sen University, Guangzhou, China

Abstract

In classical planning and conformant planning, it is assumed that there are finitely many named objects given in advance, and only they can participate in actions and in fluents. This is the Domain Closure Assumption (DCA). However, there are practical planning problems where the set of objects changes dynamically as actions are performed. We formulate the planning problem in first order logic, assume an initial theory is a finite consistent set of fluent literals, discuss when this guarantees that in every situation there are only finitely many possible actions, impose a finite bound on the length of the plan, and propose to organize search over sequences of actions that are grounded at run-time. We show soundness and completeness of our approach. It can be used to solve the bounded planning problems without DCA that belong to intersection of sequential generalized planning (without sensing actions) and conformant planning, restricted to the case without the disjunction over fluent literals. We discuss a proof-of-the-concept implementation of our planner.

1 Introduction

In a generalization of the British TV show Countdown (numbers round), the task is how to produce the target number out of six given integers using only standard arithmetical operations with the condition that once a number has been used, it is no longer available for subsequent calculations. This is an instance of a planning problem in the case when fluents and actions have parameters that vary over infinite domains, knowledge about an initial state is incomplete, and objects can be created or destroyed during planning. This requires planning over dynamically changing domains. A planning algorithm has to search for a correct sequential plan with respect to several different infinitely sized logical models. It turns out, the foundations for this kind of planning research have been developed years ago, as we explain next.

The real world around us can sometimes be conveniently understood in computational terms as a very large symbolic data base with incomplete knowledge, since a logical reconstruction of this open-world data base is a scalable logical theory that supports computationally tractable (sometimes) reasoning about incomplete knowledge. This perspective emerged in the early years of knowledge representation, for example, see [Reiter, 1980; Reiter, 1986; Lin and Reiter, 1997; Levesque, 1998; Liu and Levesque, 2003], and inspired significant subsequent research, for example, in description logic [Baader *et al.*, 2003]. Surprisingly, it has not been widely explored as much as should be in automated sequential planning with deterministic actions [Ghallab *et al.*, 2004; Geffner and Bonet, 2013].

There are conceptual connections between our research and several popular directions in modern planning. We clarify these connections next before we explain our contribution.

In classical and conformant planning, it is usually assumed that there are finitely many explicitly named objects given in advance, and no objects are created or destroyed: this is the Domain Closure Assumption (DCA) [Reiter, 1980]. Moreover, in conformant planning, incomplete knowledge about an initial state is usually formulated using disjunctions of fluents, but this requires combinatorial case analysis [Hoffmann and Brafman, 2006; Palacios and Geffner, 2009; Grastien and Scala, 2020]. These papers include the case where an initial state is a finite consistent set of fluent *literals*, that is, there are fluents known to be true and fluents known to be false, but these and other papers implicitly require DCA. To the best of our knowledge, the case of planning with incomplete knowledge based on fluent literals without DCA has not been explored in conformant planning.

Moreover, in classical, conformant and numerical planning, it is common to restrict the parameters of fluents and actions to finite domains, despite that there are practical examples of the planning problems that require actions with control parameters that vary over infinite domains (such as fuel, weight); for example, see [Savas *et al.*, 2016].

In the active research area of generalized planning (GP), the planning problem has to be solved for multiple instances at once. To give a few examples, research has ranged from contingent iterative planning (with sensing) [Hu and Levesque, 2010; Srivastava, 2010; Belle and Levesque, 2016], to using abstractions [Bonet and Geffner, 2020; Illanes and McIlraith, 2019], to exploring how generalized planning can be done with existentially quantified goals [Funkquist *et al.*, 2024], or to GP as a heuristic search approach that uses a GP program to solve a set of classical planning instances [Segovia-Aguas *et al.*, 2024]. The latter paper and [Celorrio *et al.*, 2019] review previous work in GP.

We consider the generalized planning problem in first order logic, with an (existentially quantified) goal formula, as a search for a plan from a consistent set of fluent literals and additional axioms, without the DCA, and compute a sequential plan with deterministic actions, when it exists. This plan can be applied to any (infinite) model of an initial incomplete logical theory. This planning problem requires computing action preconditions at run-time. In our setting, heuristic search over state space is infeasible since there are infinitely many models (states), and actions – as well fluents – may have parameters that vary over infinite domains. Therefore, we search for a plan over sequences of instantiated actions, and formulate the

conditions that guarantee, at every planning step, there are only finitely many actions possible. If the length of a plan is bounded, then there are only finitely many sequences of possible action that can be considered. We adapt a version of a domain independent heuristic developed for the FF planner [Hoffmann and Nebel, 2001] to facilitate automated search.

Our approach can be used to solve the bounded planning problems (without DCA) that belong to intersection of sequential generalized planning (with deterministic actions) and conformant planning, restricted to the case without the disjunction over fluent literals. We discuss a proof-of-the-concept implementation of our heuristic planner, and demonstrate that it can solve a few instances without DCA.

2 Preliminaries

2.1 Proper KBs and V

We use a standard first-order (FO) language with equality, a countably infinite set of constants $\{C_1, C_2, \ldots\}$, and no other function symbols. We restrict our attention to standard interpretations, where equality is identity, and there is a bijection between the set of constants and the domain of discourse. This restriction can be captured by a set of axioms $\mathcal{E}$, consisting of the axioms of equality and the set of formulas $\{C_i \neq C_j \mid i \neq j\}$. We use ρ to range over atoms whose arguments are distinct variables. We use e to range over ewffs, that is, quantifier-free formulas with only equalities (i.e. no predicates). We use $\forall\phi$ to denote the universal closure of ϕ. We let θ range over substitutions of all variables by constants, and write $\phi\theta$ as the result of applying θ to ϕ. We use the letter d to range over the constants. We write $\Gamma \models_{\mathcal{E}} \phi$ to denote $\mathcal{E} \cup \Gamma \models \phi$.

Intuitively, a proper KB represents a consistent set of ground literals. It generalizes databases by allowing incomplete knowledge about some of the elements.

Definition 1. A KB Γ is proper if $\mathcal{E} \cup \Gamma$ is consistent and Γ is a finite set of formulas of the form $\forall(e \supset \rho)$ or $\forall(e \supset \neg\rho)$.

Since the logical entailment problem for proper KBs is undecidable, Levesque [Levesque, 1998] proposes a reasoning scheme called V for proper KBs. Given a proper KB and a query, V returns one of three values 0 (known false), 1 (known true), or $\frac{1}{2}$ (unknown) as follows:

1. $V[\Gamma, \rho\theta] = \begin{cases} 1 & \text{if there is a } \forall(e \supset \rho) \text{ in } \Gamma \text{ s.t. } V[\Gamma, e\theta] = 1 \\ 0 & \text{if there is a } \forall(e \supset \neg\rho) \text{ in } \Gamma \text{ s.t. } V[\Gamma, e\theta] = 1 \\ \frac{1}{2} & \text{otherwise} \end{cases}$.
2. $V[\Gamma, (d = d')] = 1$ if d and d' are identical constants, and 0 otherwise;
3. $V[\Gamma, \neg\phi] = 1 - V[\Gamma, \phi]$;
4. $V[\Gamma, (\phi \vee \psi)] = \max\{V[\Gamma, \phi], V[\Gamma, \psi]\}$;
5. $V[\Gamma, \exists x\phi] = \max\{V[\Gamma, \phi^x_d] \mid d \in H_1^+(\Gamma \cup \{\phi\})\}$, where $H_1^+(\Gamma \cup \{\phi\})$ includes all constants from Γ, from ϕ, and one extra constant that represents infinitely many objects not mentioned in $\Gamma \cup \{\phi\}$.

Note that if a query includes m quantifiers, then m extra constants can be required to answer the queries.

V is sound but incomplete. Levesque proposes a certain normal form called $\mathcal{NF}$, and shows that V is complete for queries in $\mathcal{NF}$:

Theorem 2. *[Levesque, 1998] Suppose Γ is proper. Then for every ϕ in $\mathcal{NF}$, $\Gamma \models_{\mathcal{E}} \phi$ iff $V[\Gamma, \phi] = 1$; and $\Gamma \models_{\mathcal{E}} \neg\phi$ iff $V[\Gamma, \phi] = 0$.*

We will not review the definition of $\mathcal{NF}$. The intuition behind $\mathcal{NF}$ is that different parts of a formula must be logically independent; an example of a formula not in $\mathcal{NF}$ is $(p \vee \neg p)$.

2.2 The Situation Calculus

The situation calculus (SC) is a logical approach to representation and reasoning about actions and their effects. It was introduced by John McCarthy, and it was refined by Reiter [Reiter, 2001] who introduced the Basic Action Theories (BAT). Unlike the notion of state that is common in model-based planning, SC relies on situation, namely a sequence of actions, which is a concise symbolic representation and a convenient proxy for the state in the cases when all actions are deterministic [Levesque *et al.*, 1998; Lin, 2008b]. We use variables s, s', s_1, s_2 for situations, variables a, a' for actions, and $\bar{x}, \bar{y}$ for tuples of object variables. The constant S_0 represents the initial situation, and the successor function $do : action \times situation \mapsto situation$, e.g., $do(a, s)$, denotes situation that results from doing action a in previous situation s. The terms σ, σ' denote situation terms, and $A_i(\bar{x})$, or $\alpha, \alpha_1, \alpha_2, \alpha'$, represent action functions and action terms, respectively. The shorthand $do([\alpha_1, \cdots, \alpha_n], S_0))$ represents situation $do(\alpha_n, do(\cdots, do(\alpha_1, S_0)\cdots)$ resulting from execution of actions $\alpha_1, \cdots, \alpha_n$ in S_0. The relation $\sigma \sqsubset \sigma'$ between situations terms σ and σ' means that σ is an initial sub-sequence of σ'. Any predicate symbol $F(\bar{x}, s)$ with exactly one situation argument s and possibly a tuple of object arguments $\bar{x}$ is called a (relational) fluent. Without loss of generality, we consider only relational fluents in this paper. A first order logic (FO) formula $\psi(s)$ is called *uniform in* s if all fluents in ψ mention only situation s as their situation argument, and $\psi(s)$ has no quantifiers over situations.

The basic action theory (BAT) $\mathcal{D}$ is the conjunction of the following classes of axioms $\mathcal{D} = \Sigma \wedge \mathcal{D}_{ss} \wedge \mathcal{D}_{ap} \wedge \mathcal{D}_{una} \wedge \mathcal{D}_{S_0}$. We use examples from the Blocks World (BW) domain [Lin and Reiter, 1997; Liu and Lakemeyer, 2009]. For brevity, all $\bar{x}, a, s$ variables are assumed $\forall$-quantified at front.

$\mathcal{D}_{\mathbf{ap}}$ is a set of action precondition axioms of the form

$$poss(A(\bar{x}), s) \leftrightarrow \Pi_A(\bar{x}, s),$$

where $poss(a, s)$ is a new predicate symbol that means that an action a is possible in situation s, $\Pi_A(\bar{x}, s)$ is a formula uniform in s, and A is an n-ary action function. In most planning benchmarks, the formula Π_A is simply a conjunction of fluent literals and possibly negations of equality. We consider a version of BW, where there are three actions: *move-b-to-b*(x, y, z), move a block x from a block y to another block z, *move-b-to-t*(x, y), move a block x from a block y to the table, *move-t-to-b*(x, z), move a block x from the table to z.

$poss(\textit{move-b-to-b}(x,y,z), s) \leftrightarrow clear(x,s) \wedge clear(z,s) \wedge on(x,y,s) \wedge x \neq z.$
$poss(\textit{move-b-to-t}(x,y), s) \leftrightarrow clear(x,s) \wedge on(x,y,s),$
$poss(\textit{move-t-to-b}(x,z), s) \leftrightarrow ontable(x,s) \wedge clear(x,s) \wedge clear(z,s).$

Let $\mathcal{D}_{\mathbf{ss}}$ be a set of successor state axioms (SSA):
$F(\bar{x}, do(a,s)) \leftrightarrow \gamma^{+}_{F}(\bar{x},a,s) \vee F(\bar{x},s) \wedge \neg\gamma^{-}_{F}(\bar{x},a,s),$
where $\bar{x}$ is a tuple of object arguments of the fluent F, and each of the γ_F's is a disjunction of uniform formulas $[\exists\bar{z}].a = A(\bar{u}) \wedge \phi(\bar{x},\bar{z},s)$,where $A(\bar{u})$ is an action with a tuple $\bar{u}$ of object arguments, $\phi(\bar{x},\bar{z},s)$ is a context condition, and $\bar{z} \subseteq \bar{u}$ are optional object arguments; possibly $\bar{x} \subset \bar{u}$. If $\bar{u}$ in an action function $A(\bar{u})$ does not include any z variables, then there is no optional $\exists\bar{z}$ quantifier. When $\bar{x} \subset \bar{u}$, that is the tuple of action arguments $\bar{u}$ contains all fluent arguments $\bar{x}$, and possibly contains $\bar{z}$, we say that the action $A(\bar{u})$ has a *local effect*. A BAT is called a *local-effect* BAT if all of its actions have only local effects. In a local-effect action theory, each action can change the fluent values only for objects explicitly named as arguments of the action. If (optional) conditions $\phi(\bar{x},\bar{z})$ do not mention s, and have no quantifiers, then SSA is called *context-free*. If there are no context conditions, then SSA is called *strictly context free* (SCF). In BW, we consider fluents $clear(x,s)$, block x has no blocks on top of it, $on(x,y,s)$, block x is on block y in situation s, $ontable(x,s)$, block x is on the table is s. The following SSAs are local-effect (with implicit outside $\forall x, \forall y, \forall a, \forall s$).

$clear(x, do(a,s)) \leftrightarrow$
$\exists y,z(a=\textit{move-b-to-b}(y,x,z)) \vee \exists y(a=\textit{move-b-to-t}(y,x)) \vee clear(x,s) \wedge \neg\exists y,z(a=\textit{move-b-to-b}(y,z,x)) \wedge \neg\exists y(a=\textit{move-t-to-b}(y,x)),$
$on(x,y,do(a,s)) \leftrightarrow$
$\exists z(a=\textit{move-b-to-b}(x,z,y)) \vee \exists y'(a=\textit{move-t-to-b}(x,y')) \vee on(x,y,s) \wedge \neg\exists z(a=\textit{move-b-to-b}(x,y,z)) \wedge \neg\exists y'(a=\textit{move-b-to-t}(x,y')),$
$ontable(x,do(a,s)) \leftrightarrow \exists y(a=\textit{move-b-to-t}(x,y)) \vee ontable(x,s) \wedge \neg\exists y(a=\textit{move-t-to-b}(x,y)).$

$\mathcal{D}_{una}$ is a finite set of unique name axioms (UNA) for actions and named objects. For example,
$\textit{move-b-to-b}(x,y,z) \neq \textit{move-b-to-t}(x',y'),$
$\textit{move-b-to-t}(x,y)=\textit{move-b-to-b}(x',y') \rightarrow x=x' \wedge y=y'.$

$\mathcal{D}_{S_0}$ is a set of FO formulas whose only situation term is S_0. It specifies the initial values of fluents.

Finally, the foundational axioms Σ are generalization of axioms for a single successor function (see Section 3.1 in [Enderton, 2001]) since SC has a family of successor functions $do(\cdot,s)$, and each situation may have multiple successors (omit $\forall s_1, a_1, s_2, a_2, s, a, s'$ for brevity).

$do(a_1,s_1)=do(a_2,s_2) \rightarrow a_1=a_2 \wedge s_1=s_2$

$\neg(s \sqsubset S_0)$

$s \sqsubset do(a,s') \leftrightarrow s \sqsubseteq s'$, where $s \sqsubseteq s' \stackrel{def}{=} (s \sqsubset s' \vee s=s')$

$\forall P.\ \big(P(S_0) \wedge \forall a\forall s(P(s) \rightarrow P(do(a,s)))\big) \rightarrow \forall s(P(s))$

The last second order (SO) axiom limits the sort *situation* to the smallest set containing S_0 that is closed under the application of do to an action and a situation. These axioms say that the set of situations is really a tree: there are no cycles, no merging. Since situations are finite sequences of actions, they can be implemented as lists in PROLOG, e.g., S_0 is like [ ], and $do(A,S)$ adds an action A to the front of a list representing S, i.e. $[A|S]$. Therefore, in PROLOG, situations satisfy Σ, and no SO reasoning is required to plan over situations.

It is often convenient to consider only executable (legal) situations: these are action histories in which it is actually possible to perform the actions one after the other.

$s \prec s' \stackrel{def}{=} s \sqsubset s' \wedge \forall a\forall s^*(s \sqsubset do(a,s^*) \sqsubseteq s' \rightarrow poss(a,s^*))$

where $s \prec s'$ means that s is an initial sub-sequence of s' and all intermediate actions are possible. Subsequently, we use the following abbreviations: $s \preceq s' \stackrel{def}{=} (s \prec s') \vee s=s'$. Also, $executable(s) \stackrel{def}{=} S_0 \leq s$. [Reiter, 2001] formulates

Theorem 3. $executable(s) \leftrightarrow \big((s=S_0) \vee \exists a\exists s'(s=do(a,s') \wedge poss(a,s') \wedge executable(s'))\big)$

Theorem 4. *[Pirri and Reiter, 1999] A basic action theory $\mathcal{D}=\Sigma\wedge\mathcal{D}_{ss}\wedge\mathcal{D}_{ap}\wedge\mathcal{D}_{una}\wedge\mathcal{D}_{S_0}$ is satisfiable iff $\mathcal{D}_{una}\wedge\mathcal{D}_{S_0}$ is satisfiable.*

Thus, Σ are *not* needed to check the satisfiability of $\mathcal{D}$.

There are two main reasoning mechanisms in SC. One of them relies on the regression operator. Another mechanism called progression is responsible for reasoning forward, where after each action α the initial theory $\mathcal{D}_{S_0}$ is updated to a new theory $\mathcal{D}_{S_\alpha}$. In this paper, we focus on progression.

Let α be a ground action, and let S_α denote the situation term $do(\alpha,S_0)$. A progression $\mathcal{D}_{S_\alpha}$ of $\mathcal{D}_{S_0}$ in response to α is a set of sentences uniform in S_α such that for all queries about the future of S_α, $\mathcal{D}$ is equivalent to $(\mathcal{D}-\mathcal{D}_{S_0})\cup\mathcal{D}_{S_\alpha}$. In general, progression $\mathcal{D}_{S_\alpha}$ is SO [Lin and Reiter, 1997]. However, in a special case in the next section, Theorem 8 implies $\mathcal{D}_{S_\alpha}$ is in FO. Therefore, the following property of progression is a key to tractability of the queries about $do(\alpha,S_0)$.

Theorem 5. *[Lin and Reiter, 1997] Let $\mathcal{D}_{S_\alpha}$ be a progression of the initial theory to S_α. For any sentence ϕ uniform in S_α, $\mathcal{D} \models \phi$ iff $\mathcal{D}_{S_\alpha} \models \phi$.*

3 Bounded Proper Planning

In this section, we first formalize the task of bounded proper planning, then we present two examples of it.

In this paper, we consider a special form of proper KBs in the form of a finite set of ground literals, which we call a finite grounded proper (FGP) KB. Let Γ be FGP. For a predicate P, we let KP denote $\{\vec{d} \mid P(\vec{d}) \in \Gamma\}$, the set of tuples where P is known to be true and $K\neg P$ for $\{\vec{d} \mid \neg P(\vec{d}) \in \Gamma\}$, the set of tuples where P is known to be false. Then the V procedure is simplified as follows:

$$V[\Gamma, P(\vec{d})] = \begin{cases} 1 & \text{if } P(\vec{d}) \in \Gamma \\ 0 & \text{if } \neg P(\vec{d}) \in \Gamma \\ \frac{1}{2} & \text{otherwise} \end{cases}.$$

Recall that conjunctive queries are a common class of formulas for retrieving information from a database, e.g., see [Chandra and Merlin, 1977; Abiteboul *et al.*, 1995]. In this paper, we extend them with safe dis-equalities, and have the following definition:

Definition 6. An extended conjunctive query (ECQ) is of the form $\exists \vec{x}\phi(\vec{x}, \vec{y})$, where ϕ is a conjunction of positive literals and safe disequalities, that is, disequalities between variables or variables and constants such that each variable has to appear in at least one positive literal.

A sufficient condition for a formula in negation normal form to be in $\mathcal{NF}$ is that any two literals in the formula are conflict-free, that is, either they have the same polarity, or they use different predicates, or they use different constants at some argument position. Therefore, extended conjunctive queries are in $\mathcal{NF}$. Thus, V is sound and complete for ECQ.

[Liu and Levesque, 2005] proposes a method of progression of proper KBs for local-effect and SCF action theories. In this paper, we define a generalization of SCF action theories, called weakly context-free, and adapt their result to compute progression of finite grounded proper KBs for weakly context-free action theories.

Definition 7. A successor state axiom for fluent $F(\bar{x},s)$ is weakly context-free (WCF) if $\gamma^+_F(\bar{x}, a)$ and $\gamma^-_F(\bar{x}, a)$ are disjunctions of formulas of the form $\exists \bar{z}[a = A(\bar{y}) \wedge \phi(\bar{u})]$, where A is an action function, $\bar{y}$ contains $\bar{z}$, $\bar{u} = \bar{x} - \bar{y}$, and $\phi(\bar{u})$ is the conjunction, for each variable u in $\bar{u}$, of equalities $u = t(\bar{y})$, where $t(\bar{y})$ is a term using external (situation independent) functions. A BAT is WCF if each SSA is.

Here by external functions, we mean situation independent functions such as additions and multiplications.

So for an SCF SSA, the tuples for which the truth value of a fluent changes can be read from the action arguments. But for a WCF SSA, the tuples for which the truth value of a fluent changes can be determined from the action arguments by using external functions. Note that a WCF SSA is not necessarily local-effect, as each fluent argument in $\bar{u}$ is not mentioned in $A(\bar{y})$.

Using unique name axioms and external functions, the instantiation of a WCF SSA on a ground action can be significantly simplified. Suppose that the SSA for fluent $F(\bar{x}, s)$ is WCF. Let $\alpha = A(\bar{C})$ be a ground action. Let $\mu(a) = \exists \bar{z}[a = A'(\bar{y}) \wedge \phi(\bar{u})]$ be a disjunct of $\gamma^*_F(\vec{x}, a)$, where $*$ is $+$ or $-$. If A is different from A', then $\mu(\alpha)$ is equivalent to false. Otherwise, $\mu(\alpha)$ is equivalent to $\exists \bar{z}[\bar{y} = \bar{C} \wedge \phi(\bar{u})]$. Since $\bar{y}$ contains $\bar{z}$ and $\bar{u} = \bar{x} - \bar{y}$, by using external functions, $\mu(\alpha)$ is logically equivalent to $\bar{x} = \bar{d}$, where $\bar{d}$ contains constants computed from external functions. Thus, $\gamma^*_F(\vec{x}, \alpha)$ is equivalent to a formula of the following form: $\bar{x} = \bar{d}_1 \vee \ldots \bar{x} = \bar{d}_n$. We will use $\gamma^*_F(\alpha)$ to denote the set $\{\bar{d}_i \mid i = 1, \ldots, n\}$.

Although WCF is an extension of SCF, the progression of WCF actions can be done similarly to SCF actions except that we use external functions to supply fluent arguments.

Theorem 8. *Let $\mathcal{D}$ be weakly context-free, Γ be FGP, and α be a ground action. Then the progression of Γ wrt α, $\mathcal{P}(\Gamma, \alpha)$, remains FGP, and it can be computed as follows:*

$$KF := KF - \gamma^-_F(\alpha) \cup \gamma^+_F(\alpha),$$
$$K\neg F := K\neg F - \gamma^+_F(\alpha) \cup \gamma^-_F(\alpha).$$

Proof. Follows directly from Theorems 4 and 6 in [Liu and Levesque, 2005]. Note that the set $\gamma^+_F(\alpha)$ includes tuples for which fluent becomes true thanks to α, $\gamma^-_F(\alpha)$ are tuples for which fluent becomes false due to α. $\mathcal{P}(\Gamma, \alpha)$ is in FO. □

We now define the kind of BATs we consider in this paper.

Definition 9. We say a BAT is *proper* if the following holds: (1) The initial KB is a finite grounded proper KB; (2) all action preconditions axioms are quantifier-free extended conjunctive queries; (3) all SSAs are weakly context free.

We now formalize our bounded proper planning (BPP) problem. We use $Length(\sigma)$ for the number of actions in situation σ, i.e., $Length(do([\alpha_1, \cdots, \alpha_N], S_0)) = N$ and $Length(S_0) = 0$.

Definition 10. A bounded proper planning (BPP) problem is a triple $P = (\mathcal{D}, G(s), N)$, where $\mathcal{D}$ is a proper BAT, $G(s)$ is a goal formula, which is an extended conjunctive query uniform in s and without other free variables, and $N \geq 0$ is a bound. A solution to P is a ground σ with length $\leq N$ s.t.

$$\mathcal{D} \models_{\mathcal{E}} executable(\sigma) \wedge G(\sigma). \quad (1)$$

Below we present examples of bounded proper planning.

For many years, British TV has run a popular show Countdown that includes number round. We consider a version of Countdown that is modified for our purposes. The task is to produce the target integer number from a few given positive integers using only standard arithmetical operations. Once two numbers have been used in calculations, they are no longer available, but the new number calculated from those two can be used next. We allow any positive integer number as an initial, an intermediate, or a target number. For simplicity, consider only addition and multiplication.

We encode the modified Countdown problem using fluents $available(c, s)$, means that a counter c is available in situation s, and $value(c, n, s)$, means that a counter c holds the number n in s. Action $add(c_1, v_1, c_2, v_2)$ adds the value v_2 from the counter c_2 to the value v_1 of the counter c_1, with the result stored in the counter c_1, overwriting the previous value stored there, and making c_2 unavailable. Action $mult(c_1, v_1, c_2, v_2)$ represents the multiplication of values v_1, v_2 and storing the result in c_1. In the following preconditions, the right hand sides are quantifier-free ECQ:

$$poss(add(c_1, v_1, c_2, v_2), s) \leftrightarrow$$
$$available(c_1, s) \wedge available(c_2, s) \wedge c_1 \neq c_2 \wedge$$
$$value(c_1, v_1, s) \wedge value(c_2, v_2, s).$$
$$poss(mult(c_1, v_1, c_2, v_2), s) \leftrightarrow$$
$$available(c_1, s) \wedge available(c_2, s) \wedge c_1 \neq c_2 \wedge$$
$$value(c_1, v_1, s) \wedge value(c_2, v_2, s).$$

The successor-state axioms for the fluents are WCF:

$$available(c, do(a, s)) \leftrightarrow$$
$$available(c, s) \wedge \neg\exists c', x, y(a = add(c', x, c, y)) \wedge$$
$$\neg\exists c', x, y(a = mult(c', x, c, y)).$$
$$value(c_1, v, do(a, s)) \leftrightarrow$$
$$\exists c_2, v_1, v_2(a = add(c_1, v_1, c_2, v_2) \wedge v = v_1 + v_2) \vee$$
$$\exists c_2, v_1, v_2(a = mult(c_1, v_1, c_2, v_2) \wedge v = v_1 \times v_2) \vee$$
$$value(c_1, v, s) \wedge \neg\exists c_2, v_2(a = add(c_1, v, c_2, v_2)) \wedge$$
$$\wedge \neg\exists c_2, v_2(a = mult(c_1, v, c_2, v_2))$$

Consider several different initial theories. For clarity, instead of using the constant names such as C_{54}, we simply write 54 to represent this integer number. Similarly, we write 1 (or 2) to represent the first (the second) counter in actions.

Example 1. Let $\mathcal{D}_{S_0}$ be the following proper theory
$\forall c.(c{=}1 \vee c{=}2) \rightarrow available(c, S_0)$
$\forall c, v.(c{=}1 \wedge v = 4 \vee c{=}2 \wedge v = 5) \rightarrow value(c, v, S_0)$

Note that D_{S_0} does not say which counters are not available and if there are any other counters and what their values might be. If the goal formula requires $\exists c(value(c, 20, \sigma))$, then this goal can be reached by performing a single action $mult(1, 4, 2, 5)$, since in every (infinite) model of D_{S_0} this action is possible and leads to the goal state, regardless of the values in the other counters.

Example 2. Let $\mathcal{D}_{S_0}$ be the following proper theory
$\forall c.(c{=}1 \vee c{=}2) \rightarrow available(c, S_0)$
$\forall c, v.(c{=}1 \wedge v{=}4 \vee c{=}2 \wedge v{=}5) \rightarrow \neg value(c, v, S_0)$

If the goal formula requires $\exists c(value(c, 20, \sigma))$, then in the model $\mathcal{M}$ of D_{S_0} where the first counter holds the number 5, and the second counter holds the number 4, this goal can be reached by doing a single action $mult(1, 5, 2, 4)$. However, in another model $\mathcal{M}'$ where both counters have say number 10, this action is not even possible. In this example, there is no action that is possible in all models of $\mathcal{D}_{S_0}$. In particular, actions operating with the numbers, say, in the counters 7 and 8 can be possible in some models, but not in all models.

Note that in the above examples, new objects are created via addition and multiplication, and DCA does not apply.

Example 3. The chopping tree example discussed in the papers about iterative generalized planning [Hu and Levesque, 2010; Srivastava, 2010] provides connections with that area. We use encoding from [Lin, 2008a], since it does not use sensing actions. Fluent $down(s)$ means the tree is down, $size(n, s)$ for a tree means n is the number of chops needed to bring the tree down. Action $chop(m)$ decreases the size of the tree from m to $m-1$. The following are the action precondition axioms and SSAs:

$$poss(chop(m), s) \leftrightarrow size(m, s) \wedge m \neq 0$$
$$size(n, do(a, s)) \leftrightarrow \exists m(a = chop(m) \wedge n = m - 1) \vee size(n, s) \wedge a \neq chop(n)$$
$$down(do(a, s)) \leftrightarrow a = chop(1) \wedge size(1, s) \vee down(s)$$

There are infinitely many different models of $\mathcal{D}_{S_0}$ where the tree has different sizes. Suppose that in $\mathcal{D}_{S_0}$ we have $\forall x(x = 4 \rightarrow size(x, S_0))$. Then, every model of $\mathcal{D}_{S_0}$ includes the fact that the initial size is 4, and a combination of other facts. It is easy to see that the sequence of 4 chop actions cuts the tree down. These actions are consecutively possible wrt all models. However, if $\mathcal{D}_{S_0}$ is empty, i.e., it does not include any statements about fluent $size(x, S_0)$, then this subtle modification has significant consequences. Namely, there is no sequence of actions possible in all models that leads to situation σ where the goal formula $down(\sigma)$ holds.

4 Solving Bounded Proper Planning

Since proper BATs have different models that may include facts about infinitely many unnamed objects, it is not immediately obvious how to search starting from $\mathcal{D}_{S_0}$ for a plan that satisfies a goal formula wrt all models of the BAT $\mathcal{D}$. We propose to search over ground situations, since they represent sequences of instantiated actions. To make this search finitely feasible, we introduce an upper bound (a positive integer) on the number of actions, assuming that if an instance has a plan solving the problem, then its number of actions is less than a provided bound. At each step of planning, we have to find all possible actions, and we have to make sure that there are only finitely many ground actions that are possible wrt all models of the BAT $\mathcal{D}$. (We cannot choose an action that is possible wrt some models, but not possible wrt other models.) Thus, there might be at most finitely many ground situations that we can ever consider, and if a solution exists, then our search will find it. Therefore, the following Lemma 1 is important, since it guarantees that there are at most finitely many successors of each situation node. Recall a BAT mentions finitely many action functions $A(\bar{x})$.

Lemma 1. *Let $\mathcal{D}$ be a proper BAT, σ be a ground situation such that $\mathcal{D} \models_{\mathcal{E}} executable(\sigma)$. For any action function $A(\bar{x})$, there are only finitely many tuples $\vec{d}$ such that $\mathcal{D} \models_{\mathcal{E}} poss(A(\bar{d}), \sigma)$.*

Proof. Let $\mathcal{P}(\mathcal{D}_{S_0}, \sigma)$ be the progression of $\mathcal{D}_{S_0}$ wrt ground actions in σ: it is a FGP theory, see Theorem 8. Let the precondition axiom for $A(\bar{x})$ be $poss(A(\bar{x}), s) \leftrightarrow \Pi_A(\bar{x}, s)$. By Theorem 5, $\mathcal{D} \models_{\mathcal{E}} poss(A(\bar{d}), \sigma)$ iff $\mathcal{P}(\mathcal{D}_{S_0}, \sigma) \models_{\mathcal{E}} \Pi_A(\bar{d}, \sigma)$. Since Π_A is a quantifier-free ECQ, that is, a conjunction of positive literals and safe dis-equalities, $\mathcal{P}(\mathcal{D}_{S_0}, \sigma)$ is equivalent to a finite set of ground literals, and the procedure $V(\cdot, \cdot)$ is complete for ECQ queries, it retrieves only finitely many $\vec{d}$ such that $\mathcal{P}(\mathcal{D}_{S_0}, \sigma) \models_{\mathcal{E}} \Pi_A(\bar{d}, \sigma)$. □

Note this proof does not require the DCA for objects. The fluent $available(\bar{x}, S)$ can be used to track objects destroyed by previous actions, and this fluent can occur in preconditions. Even if objects can be constructed or destroyed, at each planning step the set of named objects remains finite.

The planner has to search over executable sequences of actions. The state space remains implicit, since situations serve as symbolic proxies to states. Namely, each situation uniquely corresponds to fluent literals that comprise a state. Whenever a sequence of i ground actions results in a situation $do([\alpha_1, \cdots, \alpha_i], S_0)$, to find the next action the planner must check among the actions $A_1(\bar{x_1}), \ldots, A_k(\bar{x_k})$ for which of the values of their object arguments these actions are possible.Since this computation is done at run-time, but not before search starts, our planner is lifted. Note that in proper BAT, checking if a conjunctive $G(\sigma)$ holds for a ground σ is easy.

The second key observation is that an efficient planner needs control that helps select for each situation the most promising possible action to do. This control can be provided by an A* algorithm that relies on a domain independent heuristic function inspired by [Hoffmann and Nebel, 2001].

The main advantage of this design is that the frontier stored in a priority queue consists of situations and their f-values computed as the sum of situation length (cost g) and a heuristic estimate h, i.e., for each ground σ, $f(\sigma) = g(\sigma) + h(\sigma)$.

Algorithm 1 A^* search over situation tree to find a plan

Input: $(\mathcal{D}, G, N)$ - a bounded proper planning problem
Input: H - Heuristic function $H(\mathcal{D},G,d,S_n,St)$
Output: S that satisfies (1) ▷ Plan is the list of actions in S

```
1: procedure PLAN(D, G, N, H, S)
2:     PriorityQueue ← ∅                                  ▷ Initialize PQ
3:     S_0.Val ← (N + 1)
4:     PriorityQueue.insert(S_0, S_0.Val)
5:     Init ← InitialState(D_{S_0})                       ▷ Initialize state
6:     while not PriorityQueue.empty() do
7:         S ← PriorityQueue.remove()
8:         Now ← Progress(Init, S)                        ▷ Current state
9:         if Satisfy(Now, G) then
10:            return S                                   ▷ Found list of actions
11:        end if
12:        Acts ← FindAllPossibleActions(Now)
13:        if Acts == ∅ then
14:            continue                                   ▷ No actions are possible in S
15:        end if
16:        for A_i ∈ Acts do
17:            S_n ← do(A_i, S)                           ▷ S_n is next situation
18:            St ← Progress(Now, A_i)                    ▷ Next state
19:            if Length(S_n) ≥ N then
20:                continue                               ▷ S_n exceeds upper bound
21:            else d ← N − Length(S_n)                   ▷ d is depth bound
22:            end if                                     ▷ /*Each action has cost 1*/
23:            S_n.Val ← Length(S_n) + H(D,G,d,S_n,St)
24:            PriorityQueue.insert(S_n, S_n.Val)
25:        end for
26:    end while
27:    return False                                       ▷ No plan for bound N
28: end procedure
```

In Algorithm 1, Line 7, the algorithm extracts the next most promising situation S from the frontier. Then, on Line 8, it computes progression Now of the initial state using the actions mentioned in S. On Line 9, there is a check for whether the goal formula G is satisfied in the current state Now. If it is, then S is returned as a plan. If not, then on Line 12, the algorithm finds all actions that are possible from the current state using the precondition axioms. If there are no actions possible from Now, then the algorithm proceeds to the next situation from the frontier. Otherwise, for each possible ground action A_i, it constructs the next situation $S_n = do(A_i, S)$, and if its length does not exceed the upper bound N, it computes the positive integer number d on Line 21 as $N - Length(S_n)$. This bound d is provided as an input to the heuristic function $H(\mathcal{D}, G, d, S_n, St)$ that does limited look-ahead up to depth d from St to evaluate situation S_n. On Line 24, S_n and its f-value $S_n.Val$ are inserted into the frontier, and then search continues until the algorithm finds a plan, or it explores all situations with at most N actions. The **for**-loop, Lines 16-24, makes sure that all possible successors of S are constructed, evaluated and inserted into the frontier. This is to guarantee completeness of Algorithm 1.

Theorem 11. *Algorithm 1 is sound and complete for BPP.*

Proof. By Lemma 1, there are only finitely many executable situations with length $\leq N$. The algorithm enumerates them in the non-decreasing order of the f values. Thus all of them will be enumerated and checked for satisfaction of the goal in Line 9. By Theorem 5, $\mathcal{D} \models_{\mathcal{E}} G(do(\sigma, S_0))$ iff $\Gamma \models_{\mathcal{E}} G$, where $\Gamma = \mathcal{P}(\mathcal{D}_{S_0}, \sigma)$. By soundness and completeness of V for ECQs, $V[\Gamma, G] = 1$ iff $\Gamma \models_{\mathcal{E}} G$. So the algorithm is sound and complete. Obviously, it terminates. □

Computing the heuristic function is done in two stages, with usual delete relaxation, i.e., negative effects of actions are ignored. (We do not include the pseudo-code of this algorithm due to lack of space, but the readers can find it in [Soutchanski and Young, 2023]. This algorithm follows [Hoffmann and Nebel, 2001].) First, a planning graph is built from the current state, layer-by-layer until the goal is satisfied or the state layer stops changing. Supporting actions are then found for the goal literals, going backwards through the graph. We skip the details of how $Reachability(\mathcal{D}, Goal, PG)$ is computed, since this is a minor improvement of the FF heuristic from Hoffmann and Nebel. The interested readers can find these details in [Soutchanski and Young, 2023]. A minor adaptation of the proof in Hoffmann and Nebel leads to

Theorem 12. *Suppose the state layer stops changing and the goal is not satisfied. Then the original planning problem is not solvable.*

5 Implementation and Experiments

We have developed a straightforward implementation of Algorithms 1 in PROLOG, following a logic programming approach from [Reiter, 2001] and [Levesque, 2012] with a few important differences. Reiter and Levesque represented the initial theory as a collection of atomic statements for fluents that are true, but since PROLOG has a built-in CWA and DCA this is too restrictive for our purposes. Our implementation of proper BATs considers fluents as terms, and collects initial proper KB literals into two lists of nodes, one node per fluent name, where each node includes a finite list of tuples. The first list of nodes represents those fluents which are initially known as true, and the second list of nodes represents those fluents which are initially known as false. Both lists can be empty for some fluents, if the initial proper KB does not include any literals for those fluents. When we compute progression, these two lists of nodes are updated accordingly to what are the effects of each action. Thus, we can work with infinite logical models by using two finite lists of tuples.

Our planner, domain files and problem instances have been loaded, compiled and run within ECLiPSe Constraint Logic Programming System, Ver7.0 #63 (x86_64_linux), released on April 24, 2022. We run it on desktop computer equipped with 11th Genuine Intel(R) Core(TM) i7-11700K CPU (3.60GHz) using only a single thread. PROLOG never exceeded 512M of memory limit allocated by default. The program has a minimal memory footprint since we only stored short situations in our priority queue. We could not compare our program with alternative implementations, since we are not aware about other planners that can solve our benchmarks.

First, we tried to solve a few instances of the **Countdown** benchmark that was implemented according to a proper BAT specified above. The program could easily solve the instances with up to 3 counters. When we experimented with different instances, the planner took less than 1 sec to find a solution with 2 actions. For the instances that involved 4 counters, the planner could find a sequence of 3 actions within 5 min. When the bound N was small, the planner could find a solution quickly, but otherwise, it was taking a long time. For the instances that had more than 4 counters, the planner could not find a solution within 10 min. To understand the issue, assume there are C counters, and notice that the first counter may have up to $2(C-1)$ new values after doing all additions or multiplications once with the numbers in other counters. These new values can participate in operations with other new elements in other counters, and so on. The number of objects destroyed or created by actions grows fast, and this increases the complexity of finding a solution.

Finally, we have developed a new benchmark **Mixers** designed to illustrate planning with a dynamically changing domain. Mixers is a significant modification of the well-known Logistics benchmark. In Mixers, there are vehicles that can move between locations. In addition, there are ingredients and compounds of different types. They can be loaded into vehicles, or they can be unloaded under the usual preconditions for these actions in Logistics. However, we added an extra condition that a vehicle must be empty to ensure that only one object can be loaded at a time. Imagine any powders or liquids as ingredients, and think about mixing them to produce a compound of a new type. Each type is a positive integer. Initially available ingredients or intermediary available compounds can be mixed according to given recipes. After mixing, the starting ingredients are no longer available, but a compound produced becomes available for subsequent actions. Each mix action has a unique ID that corresponds to a recipe specifying the name and type of a compound produced. The new type is computed according to a specific numerical function. In our PROLOG implementation, we make sure that a new compound is named with a new string computed as concatenation of the strings that name the starting ingredients. This is to make sure that new domain elements are never mentioned initially, and the names of new elements are produced at planning time. Note the objects can be created or destroyed by the mix actions, but without doing the actions, one cannot specify the objects in advance without extensive engineering of the domain. Therefore, this benchmark reflects well the purposes of planning with dynamically changing objects and without DCA. Some computational experiments conducted with instances demonstrated that Mixers is challenging. We used small values of N. We easily solved the instances with 3 recipes, two locations and 3 or 4 ingredients that required a few move actions and 1 mix actions. The plans with 4 or 5 actions completed in less than 1 sec. However, the plan that required doing two different mix actions at the same location, was computed in about 10min. Any more complex instances could not be solved within 10min limit. This benchmark is challenging because the available objects change quickly when several mix actions are required.

As a summary of our observations, we propose the hypothesis that planning with dynamically changing objects is a computationally intensive process, since the planner has to deal with the increasing number of combinatorial possibilities. We consider our planner as a proof-of-the-concept implementation, and anticipate significant work is required to design an efficient program that can plan without DCA.

6 Related work

The previous research on generalized planning focused on iterative plans, or programs that support the search for a plan that works at once for a set of classical planning problems. Our work considers sequential planning without the DCA. In addition, we solve the planning problem for goals that are conjunctive queries with $\exists$-quantifiers over object variables, but except for [Francès and Geffner, 2016; Funkquist *et al.*, 2024], most previous work considered only conjunctions of ground fluents as goal formulas. Note since we ground actions at run-time in preconditions, checking whether resulting ground situation satisfies a goal query is easy in proper BATs.

Note that in contrast to [De Giacomo *et al.*, 2016], we do not require that the number of objects where fluent can hold is bounded for all s. Informally, boundedness of the set of named objects that may ever be considered by our planner becomes the consequence of working with a proper BAT and imposing the upper bound on the number of actions.

Lifted classical planning has been previously explored, e.g., see [Corrêa *et al.*, 2021; Ståhlberg, 2023]. Our work is different since we focus on planning without DCA.

[Soutchanski and Young, 2023] proposed a lifted deductive planner based on the situation calculus (SC), but their implementation required both DCA and CWA.

[Corrêa *et al.*, 2024] consider an extension of classical planning (the domain of objects is *finite*), with *complete knowledge*, but their semantics relies on an unusual object assignment. Our BPP is more general.

To the best of our knowledge, there are no other heuristic planners that can solve problems without the DCA given *incomplete* initial theory. All previously developed conformant planners require DCA. The planner in [Hoffmann and Brafman, 2006] was actually inspired by SC, and it does search over sequences of actions, but it works only at a propositional level. [Finzi *et al.*, 2000] does open-world planning, but they require DCA, see details in [Reiter, 2001].

7 Conclusion

To the best of our knowledge, our planner is the first heuristic planner that does not require DCA, can use actions with parameters ranging over infinite domains, works with incomplete knowledge, and can solve the planning problems with goal formulas that have $\exists$-quantifiers over objects. When a plan exists, it includes ground actions that are possible in every (infinite) model of a proper BAT. Future work may consider the case when identity of some objects is not known, and for this reason $\mathcal{D}_{S_0}$ may include $\exists$-quantifiers over objects; they can be replaced with Skolem constants. This case was discussed for proper KBs in the paper [De Giacomo *et al.*, 2011]. However, there are no heuristic planners that support unknown individuals represented with Skolem constants.

References

[Abiteboul *et al.*, 1995] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. 1995.

[Baader *et al.*, 2003] Franz Baader, Diego Calvanese, Deborah McGuinness, Daniele Nardi, and Peter F. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications*. 2003.

[Belle and Levesque, 2016] Vaishak Belle and Hector J. Levesque. Foundations for generalized planning in unbounded stochastic domains. In *Principles of Knowledge Representation and Reasoning: Proceedings of the 15th Intern. Conf., KR 2016*, pages 380–389, 2016.

[Bonet and Geffner, 2020] Blai Bonet and Hector Geffner. Qualitative numeric planning: Reductions and complexity. *J. Artif. Intell. Res.*, 69:923–961, 2020.

[Celorrio *et al.*, 2019] Sergio Jiménez Celorrio, Javier Segovia-Aguas, and Anders Jonsson. A review of generalized planning. *Knowl. Eng. Rev.*, 34:e5, 2019.

[Chandra and Merlin, 1977] Ashok Chandra and Philip Merlin. Optimal implementation of conjunctive queries in relational data bases. In *9th Annual ACM Symposium on Theory of Computing (STOC)*, pages 77–90, 1977.

[Corrêa *et al.*, 2021] Augusto B. Corrêa, Guillem Francès, Florian Pommerening, and Malte Helmert. Delete-relaxation heuristics for lifted classical planning. In *Proceedings of the 31st ICAPS-2021*, pages 94–102, 2021.

[Corrêa *et al.*, 2024] Augusto B. Corrêa, Giuseppe De Giacomo, Malte Helmert, and Sasha Rubin. Planning with object creation. In *34th International Conference on Automated Planning and Scheduling, ICAPS 2024*, pages 104–113, 2024.

[De Giacomo *et al.*, 2011] Giuseppe De Giacomo, Yves Lespérance, and Hector J. Levesque. Efficient reasoning in proper knowledge bases with unknown individuals. In *22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 827–832, 2011.

[De Giacomo *et al.*, 2016] Giuseppe De Giacomo, Yves Lespérance, and Fabio Patrizi. Bounded situation calculus action theories. *Artif. Intell.*, 237:172–203, 2016.

[Enderton, 2001] Herbert Enderton. *A Mathematical Introduction to Logic*. Harcourt Press, 2nd edit., 2001.

[Finzi *et al.*, 2000] Alberto Finzi, Fiora Pirri, and Ray Reiter. Open world planning in the situation calculus. In *Proceedings of the 7th AAAI*, pages 754–760, 2000.

[Francès and Geffner, 2016] Guillem Francès and Hector Geffner. $\exists$-strips: Existential quantification in planning and constraint satisfaction. In Subbarao Kambhampati, editor, *25th International Joint Conference on Artificial Intelligence, IJCAI-2016*, pages 3082–3088, 2016.

[Funkquist *et al.*, 2024] Martin Funkquist, Simon Ståhlberg, and Hector Geffner. Learning to ground existentially quantified goals. In *21st Intern. Conf. on Principles of Knowledge Represent. and Reasoning*, pages 856 – 866, 2024.

[Geffner and Bonet, 2013] Hector Geffner and Blai Bonet. *A Concise Introduction to Models and Methods for Automated Planning*. Morgan & Claypool Publ., 2013.

[Ghallab *et al.*, 2004] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.

[Grastien and Scala, 2020] Alban Grastien and Enrico Scala. CPCES: A planning framework to solve conformant planning problems through a counterexample guided refinement. *Artif. Intell.*, 284:103271, 2020.

[Hoffmann and Brafman, 2006] Jörg Hoffmann and Ronen Brafman. Conformant planning via heuristic forward search: A new approach. *Artificial Intelligence*, 170(6–7):507–541, 2006.

[Hoffmann and Nebel, 2001] Jörg Hoffmann and Bernhard Nebel. The FF System: Fast Plan Generation Through Heuristic Search. *J. Artif. Intell. Res.*, 14:253–302, 2001.

[Hu and Levesque, 2010] Yuxiao Hu and Hector Levesque. A correctness result for reasoning about one-dimensional planning problems. In *12th Intern. Conf on Principles of Knowledge Representation and Reasoning*, 2010.

[Illanes and McIlraith, 2019] León Illanes and Sheila A. McIlraith. Generalized planning via abstraction: Arbitrary numbers of objects. In *The 33d AAAI Conference on Artificial Intelligence, AAAI 2019*, pages 7610–7618, 2019.

[Levesque *et al.*, 1998] H.J. Levesque, F. Pirri, and R. Reiter. Foundations for the situation calculus. *Linköping Electronic Articles in Computer and Information Science. Available at: http://www.ep.liu.se/ea/cis/1998/018/*, vol. 3, N 18, 1998.

[Levesque, 1998] Hector J. Levesque. A Completeness Result for Reasoning with Incomplete First-Order KBs. In *KR-1998*, pages 14–23, 1998.

[Levesque, 2012] Hector J. Levesque. *Thinking as Computation: A First Course*. MIT Press, 2012.

[Lin and Reiter, 1997] Fangzhen Lin and Raymond Reiter. How to Progress a Database. *Artificial Intelligence*, 92:131–167, 1997.

[Lin, 2008a] Fangzhen Lin. Proving goal achievability. In *11th Intern Conf. Principles of Knowledge Representation and Reasoning*, pages 621–628, 2008.

[Lin, 2008b] Fangzhen Lin. Situation calculus. In *Handbook of Knowledge Representation*, volume 3 of *Foundations of Artificial Intelligence*, pages 649–669. Elsevier, 2008.

[Liu and Lakemeyer, 2009] Yongmei Liu and Gerhard Lakemeyer. On First-Order Definability and Computability of Progression for Local-Effect Actions and Beyond. In *21st IJCAI-2009*, pages 860–866, 2009.

[Liu and Levesque, 2003] Yongmei Liu and Hector J. Levesque. A tractability result for reasoning with incomplete first-order knowledge bases. In *IJCAI-03, Proceedings of the 18th International Joint Conference on Artificial Intelligence*, pages 83–88, 2003.

[Liu and Levesque, 2005] Yongmei Liu and Hector J. Levesque. Tractable reasoning with incomplete first-order knowledge in dynamic systems with context-dependent actions. In *19th International Joint Conference on Artificial Intelligence, IJCAI-2005*, pages 522–527, 2005.

[Palacios and Geffner, 2009] Héctor Palacios and Hector Geffner. Compiling uncertainty away in conformant planning problems with bounded width. *J. Artif. Intell. Res.*, 35:623–675, 2009.

[Pirri and Reiter, 1999] Fiora Pirri and Ray Reiter. Some contributions to the metatheory of the situation calculus. *Journal of the ACM (JACM)*, 46(3):325–361, 1999.

[Reiter, 1980] Raymond Reiter. Equality and Domain Closure in First-Order DBs. *J. ACM*, 27(2):235–249, 1980.

[Reiter, 1986] Raymond Reiter. A sound and sometimes complete query evaluation algorithm for relational databases with null values. *J. ACM*, 33(2):349–370, 1986.

[Reiter, 2001] Raymond Reiter. *Knowledge in Action. Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT, http://cognet.mit.edu/book/knowledge-action, 2001.

[Savas *et al.*, 2016] Emre Savas, Maria Fox, Derek Long, and Daniele Magazzeni. Planning using actions with control parameters. In *22nd European Conference on Artificial Intelligence ECAI*, pages 1185–1193, 2016.

[Segovia-Aguas *et al.*, 2024] Javier Segovia-Aguas, Sergio Jiménez Celorrio, and Anders Jonsson. Generalized planning as heuristic search: A new planning search-space that leverages pointers over objects. *Artif. Intell.*, 330:104097, 2024.

[Soutchanski and Young, 2023] Mikhail Soutchanski and Ryan Young. Planning as theorem proving with heuristics. In *Proceedings of the the Italian Workshop on Planning and Scheduling, co-located with AIxIA-2023*, volume 3585 of *CEUR Workshop Proceedings*, 2023.

[Srivastava, 2010] Siddharth Srivastava. *Foundations and Applications of Generalized Planning*. PhD thesis, Department of Computer Science, University of Massachusetts Amherst, 2010.

[Ståhlberg, 2023] Simon Ståhlberg. Lifted successor generation by maximum clique enumeration. In *ECAI 2023 - 26th European Conference on Artificial Intelligence*, pages 2194–2201, 2023.

On Independence and SCC-Recursiveness in Assumption-Based Argumentation (Extended Abstract)

Lydia Blümel[1] , **Anna Rapberger**[2] , **Matthias Thimm**[1] and **Francesca Toni**[2]
[1]Artificial Intelligence Group, University of Hagen
[2]Imperial College London
{lydia.bluemel, matthias.thimm}@fernuni-hagen.de, {a.rapberger,ft}@imperial.ac.uk

1 Introducing Independence to ABA

Determining conditional (in)dependence between variables is an important concern in AI. The ability to recognize and manage (in)dependence is crucial in symbolic reasoning at large [Darwiche and Pearl, 1994; Darwiche, 1997; Lang *et al.*, 2002]. In the remainder of this extended abstract we give an overview on our work [Blümel *et al.*, 2025], where we study conditional independence in Assumption-based Argumentation.

Assumption-based argumentation (ABA) [Bondarenko *et al.*, 1997; Čyras *et al.*, 2018] is a well-known form of computational argumentation, whose building blocks are assumptions (defeasible elements) and inference rules. ABA is widely applicable, e.g., in healthcare [Cyras *et al.*, 2021b], to provide explanations [Cyras *et al.*, 2021a], for causal discovery [Russo *et al.*, 2024], and planning [Fan, 2018]. In all these settings, a good understanding of independence between various components in ABA frameworks (ABAFs) is crucial. We illustrate the main idea of our approach with the following example.

Example 1.1. *Alice, Bob, and Carol plan a tandem trip; we consider assumptions* a *(*Alice cycles*),* b *(*Bob cycles*), and* c *(*Carol cycles*) for each of our protagonists. Naturally, only two of them cycle at the same time, that is, not all our assumptions can be true at once. We capture the relations between our assumptions with inference rules; e. g., "if Bob and Carol cycle then Alice does not" is captured by the rule (*$\overline{a} \leftarrow b, c$*), here,* $\overline{a}$ *is the contrary of* a*. Crucially, if the* weather is bad *(*d*) nobody can cycle. On the bright side, then there is no need to* water the plants *(*w*) outside. Also, Alice thinks about* bringing her book *(*k*), but if she cycles, it might be too heavy.*

In situations where certain assumptions cannot be true at the same time, ABA is a suitable reasoning tool. Based on a deductive system $(\mathcal{L}, \mathcal{R})$, where $\mathcal{L}$ is a set of sentences and $\mathcal{R}$ is a set of inference rules, an ABA framework (ABAF) is a tuple $D = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$, where $\mathcal{A}$ is a set of assumptions we make and from which we infer according to the rules $\mathcal{R}$, and $^- : \mathcal{A} \to \mathcal{L}$ a contrary function, which allows us to derive attacks on assumptions.

Example 1.2. *We formalise Example 1.1 as ABAF* D *with assumptions* $\{a, b, c, d, w, k\}$*, their contraries, and inference rules,* ($\overline{v} \leftarrow d$)*,* ($\overline{d} \leftarrow v$) *for* $v \in \{a, b, c\}$*, and*

$$\overline{a} \leftarrow b, c \quad \overline{b} \leftarrow a, c \quad \overline{c} \leftarrow a, c \quad \overline{k} \leftarrow a \quad \overline{w} \leftarrow d$$

In order to conduct defeasible reasoning with an ABAF $D = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$ we consider *arguments* that can be built by applying rules to assumptions. More precisely, we consider the fact, that some $p \in \mathcal{L}$ can be derived from a set of assumptions $A \subseteq \mathcal{A}$ according to the rules $\mathcal{R}$ an *(ABA) argument*, and denote this by $A \vdash p$. Furthermore, if p is the contrary $\overline{b}$ of some assumption $b \in \mathcal{A}$ we say the set A *attacks* any set $B \subseteq \mathcal{A}$ with $b \in B$, thus introducing attacks between sets of assumptions. A set of assumptions A is conflict-free if it does not attack itself; admissible if it is conflict-free and defends itself, i.e., attacks all of its attackers. On top of this attack structure, semantics for ABAFs can be defined using labellings, which assign one of the three labels $\mathtt{in}, \mathtt{out}$, or $\mathtt{und}$ to each assumption. A *labelling-based semantics* σ assigns to each ABAF D a set of σ-labellings $\Lambda_\sigma(D)$.

For our example, we recall the labelling-based version of preferred semantics [Schulz and Toni, 2017]. A labelling on some ABAF D is preferred, if sets of $\mathtt{in}$-labeled assumptions only attack $\mathtt{out}$-labeled assumptions(conflict-free), every set attacking an $\mathtt{in}$-labeled assumption contains an $\mathtt{out}$-labeled assumption (defense) and the set of $\mathtt{in}$-labeled assumptions is maximal among all labellings satisfying these two conditions (maximality).

Example 1.3. *The ABAF* D *from Example 1.2 has four preferred labellings.*

pr-lab	*a*	*b*	*c*	*d*	*w*	*k*
λ_1	in	in	out	out	in	out
λ_2	in	out	in	out	in	out
λ_3	out	in	in	out	in	in
λ_4	out	out	out	in	out	in

Having defined the setting, we are now ready to formulate the topic of our investigation: *under what conditions are two (sets of) assumptions independent from each other, relative to a (potentially empty) set of assumptions that is considered prior knowledge?*

Motivated by the work [Rienstra *et al.*, 2020] on conditional independence in abstract argumentation, we identify (in)dependencies between sets of assumptions in ABA through the compatibility of partial labellings on them. For $A \subseteq \mathcal{A}$, we let $\lambda|_A : A \to \{\mathtt{in}, \mathtt{out}, \mathtt{und}\}$ denote the restriction of some labelling λ to a partial labelling on A.

Definition 1.4. *Let $D = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$ be an ABAF, σ be a semantics, and let $A, B, C \subseteq \mathcal{A}$ be disjoint sets of assumptions. Then A is σ-independent of B, given C in D, written $A \perp_\sigma B \mid C$ iff, for all labellings $\lambda_1, \lambda_2 \in \Lambda_\sigma(D)$, it holds that if $\lambda_1|_C = \lambda_2|_C$ then there is some labelling $\lambda_3 \in \Lambda_\sigma(D)$ s.t. $\lambda_3|_A = \lambda_1|_A$,$\lambda_3|_B = \lambda_2|_B$, and $\lambda_3|_C = \lambda_1|_C = \lambda_2|_C$.*

Intuitively, a set of assumptions A is dependent on another set B under a semantics σ, if knowing the labelling on B excludes some possible σ-labellings on A. Indeed, this labelling-based independence notion allows us to derive the expected (in)dependencies for our Example.

Example 1.5. *Let us find out if Alice and Bob influence each other's cycling activities: knowing that Alice cycles does not give us any information about Bob, Alice could cycle with either Bob or Carol; similarly, if Alice does not cycle we do not know whether Bob cycles; either Bob and Carol cycle together or the weather could have ruined their trip. In contrast, a and b depend on each other, given c: knowing that Carol cycles implies that one of the other two cycles as well, and knowing she does not cycle implies either both or none of the others cycle. And indeed we can infer:*

- *The assumptions a and b are σ-independent wrt. $\emptyset$ for $\sigma = pr$. For this, we identify the labels which are individually assigned to a and b under σ, i.e., a and b can both be labeled* `out` *at the same time (λ_4).*
- *When conditioning on $\{c\}$, the assumptions a and b are dependent under pr semantics. If $\lambda(c) =$* `in`*, then a and b can be individually* `in`*, but not together.*

2 Overview of results

In [Blümel *et al.*, 2025] we focus on a well-studied fragment of ABA whereby assumptions cannot be inferred (known as *flat* ABA) and where assumptions, their contraries and any sentences in inference rules are atomic. We provide complexity results for deciding independence in both ABA and abstract argumentation frameworks [Rienstra *et al.*, 2020] and propose an SCC-recursive schema for ABA semantics alongside sound polynomial time checks for independence between sets of assumptions in ABAFs. Below, we outline our main contributions.

Independence in ABA We introduce conditional independence to ABA as a means to analyze the relations between the acceptance of disjoint sets of assumptions. To be more precise, we say two sets of assumptions A and B are independent, given a third set C, if, once we know the labellings of the assumptions in C (`in`, `out`, `und`), then the possible labellings for A are no longer influenced by those for B. This notion, captured by Definition 1.4, allows us to identify when reasoning about one part (e.g., A) can be done independently of another (e.g., B). We show our definition behaves as expected and satisfies the semi-graphoid axioms. The complexity of deciding independence, however, turns out to be rather high.

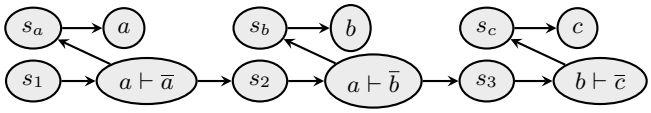
Proposition 2.1. *Let $\mathcal{C}$ denote the class of AFs/flat ABAFs. Deciding σ-independence in $\mathcal{C}$ is Π_2^P-complete for $\sigma \in \{co, st\}$ and Π_3^P-complete for $\sigma = pr$.*

We also show a correspondence to conditional independence for AFs [Rienstra *et al.*, 2020] and settle the complexity of the corresponding decision problems for those.

SCC-recursiveness for ABA To alleviate the high computational complexity we exploit the structure of ABA and explore *SCC-recursiveness* for ABAFs. SCC-recursiveness is well-studied in the realm of abstract argumentation [Baroni *et al.*, 2005; Dvorák *et al.*, 2024]; semantics that satisfy this property can be processed locally, along the *strongly connected components (SCCs)* of a graph. We construct an SCC-recursive scheme for ABAFs around the defeasible rules, which are the core of reasoning in ABA, and provide SCC-recursive characterizations of the standard semantics.

SCC-based Independency Checks We first examine whether we can exploit the close connection between AFs and ABAFs to use the polynomial-time check for independence within AFs proposed by [Rienstra *et al.*, 2020] for ABAFs. The method utilizes the SCC-structure of the induced AF to facilitate a check via d-separation which requires a DAG (directed acyclic graph). For that we use the d-graph of the AF, which has a tree-structure and contains a node for each SCC of the AF, having the contained arguments as children, from which in turn other SCC-nodes are reached. The approach is illustrated in the following example.

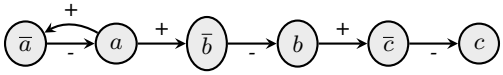
Example 2.2. *Consider an ABAF D with assumptions a, b, c and rules $(\overline{a} \leftarrow a), (\overline{b} \leftarrow a), (\overline{c} \leftarrow b)$. All assumptions of the d-graph G_{F_D} of AF F_D for the ABAF D are in terminal SCCs.*



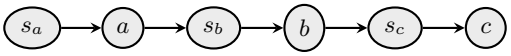
In G_{F_D}, we cannot say if a and c are independent, given b, since they are connected by a collider-free path not containing b.

As the example shows, the AF-SCCs that contain assumption arguments are often either terminal or initial SCCs. For a sensitive independence check we require an SCC-structure which is informative enough. We propose an alternative check using directly the SCC-structure of the ABAF. The SCC-Decomposition of an ABAF is conducted via the dependency graph P_D [Rapberger *et al.*, 2022]. We can now compute the d-graph G_D with respect to said dependency graph and check for independencies using the d-separation criterion. As the example shows, the second approach indeed allows insights on the (in)dependencies between assumptions the AF-based approach cannot procure.

Example 2.3. *Consider again the ABAF D from Example 2.2. It produces the following dependency graph P_D:*



Each assumption has its own SCC in P_D, thus, G_D wrt. P_D is a simple chain. In contrast to Example 2.2, b separates a and c in G_D. So, from G_D we can conclude $a \perp_\sigma c \mid b$.



Acknowledgements

The research reported here was partially supported by the Deutsche Forschungsgemeinschaft (grant 550735820), by ERC under the EU's Horizon 2020 research and innovation programme (grant agreement no. 101020934, ADIX), and by J.P. Morgan and the Royal Academy of Engineering under the Research Chairs and Senior Research Fellowships scheme (grant agreement no. RCSRF2021\11\45).

References

[Baroni *et al.*, 2005] Pietro Baroni, Massimiliano Giacomin, and Giovanni Guida. SCC-recursiveness: a general schema for argumentation semantics. *Artif. Intell.*, 168(1-2):162–210, 2005.

[Blümel *et al.*, 2025] Lydia Blümel, Anna Rapberger, Matthias Thimm, and Francesca Toni. On Independence and SCC-Recursiveness in Assumption-Based Argumentation. In *Proceedings of the Thirty-fourth International Joint Conference on Artificial Intelligence (IJCAI'25)*, 2025.

[Bondarenko *et al.*, 1997] Andrei Bondarenko, Phan Minh Dung, Robert A. Kowalski, and Francesca Toni. An abstract, argumentation-theoretic approach to default reasoning. *Artif. Intell.*, 93:63–101, 1997.

[Čyras *et al.*, 2018] Kristijonas Čyras, Xiuyi Fan, Claudia Schulz, and Francesca Toni. Assumption-based argumentation: Disputes, explanations, preferences. In *Handbook of Formal Argumentation*, chapter 7, pages 365–408. College Publications, 2018.

[Cyras *et al.*, 2021a] Kristijonas Cyras, Quentin Heinrich, and Francesca Toni. Computational complexity of flat and generic assumption-based argumentation, with and without probabilities. *Artif. Intell.*, 293:103449, 2021.

[Cyras *et al.*, 2021b] Kristijonas Cyras, Tiago Oliveira, Amin Karamlou, and Francesca Toni. Assumption-based argumentation with preferences and goals for patient-centric reasoning with interacting clinical guidelines. *Argument Comput.*, 12(2):149–189, 2021.

[Darwiche and Pearl, 1994] Adnan Darwiche and Judea Pearl. Symbolic causal networks. In *Proceedings of the 12th National Conference on Artificial Intelligence, Seattle (AAAI 1994)*, pages 238–244. AAAI Press / The MIT Press, 1994.

[Darwiche, 1997] Adnan Darwiche. A logical notion of conditional independence: properties and applications. *Artif. Intell.*, 97(1):45–82, 1997.

[Dvorák *et al.*, 2024] Wolfgang Dvorák, Matthias König, Markus Ulbricht, and Stefan Woltran. Principles and their computational consequences for argumentation frameworks with collective attacks. *J. Artif. Intell. Res.*, 79:69–136, 2024.

[Fan, 2018] Xiuyi Fan. On generating explainable plans with assumption-based argumentation. In *Principles and Practice of Multi-Agent Systems (PRIMA 2018)*, volume 11224 of *LNCS*, pages 344–361. Springer, 2018.

[Lang *et al.*, 2002] Jérôme Lang, Paolo Liberatore, and Pierre Marquis. Conditional independence in propositional logic. *Artif. Intell.*, 141(1/2):79–121, 2002.

[Rapberger *et al.*, 2022] Anna Rapberger, Markus Ulbricht, and Johannes Peter Wallner. Argumentation frameworks induced by assumption-based argumentation: Relating size and complexity. In *Proceedings of the 20th International Workshop on Non-Monotonic Reasoning (NMR 2022)*, volume 3197 of *CEUR Workshop Proceedings*, pages 92–103. CEUR-WS.org, 2022.

[Rienstra *et al.*, 2020] Tjitze Rienstra, Matthias Thimm, Kristian Kersting, and Xiaoting Shao. Independence and d-separation in abstract argumentation. In *Proceedings of the 17th International Conference on Principles of Knowledge Representation and Reasoning (KR 2020)*, pages 713–722, 2020.

[Russo *et al.*, 2024] Fabrizio Russo, Anna Rapberger, and Francesca Toni. Argumentative causal discovery. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR 2024)*, 2024.

[Schulz and Toni, 2017] Claudia Schulz and Francesca Toni. Labellings for assumption-based and abstract argumentation. *Int. J. Approx. Reason.*, 84:110–149, 2017.

MOOSE: Satisficing and Optimal Generalised Planning via Goal Regression

Dillon Z. Chen[1,2,3], **Till Hofmann**[4], **Toryn Q. Klassen**[1,3], **Sheila A. McIlraith**[1,3]
[1]Vector Institute, Toronto, Canada [2]LAAS-CNRS, University of Toulouse, France
[3]University of Toronto, Canada [4]RWTH Aachen University, Germany

Abstract

Generalised Planning refers to the task of synthesising programs that can solve families of related planning problems. We propose a novel generalised planner MOOSE which employs powerful, decades-old ideas from the knowledge representation and planning communities: regression rewriting, relaxation, and database algorithms. We formalise and describe a new class of generalised problems for which our approach can learn sound and complete generalised plans. MOOSE is also the first learning approach for provably optimal planning across problems with arbitrarily many objects. Experimental results show that MOOSE solves more problems than state-of-the-art planners over classical, numeric, and optimal planning settings on the tested benchmarks. Notably, MOOSE uses less than 1GB of memory to synthesise and instantiate generalised plans across all experiments.

1 Introduction

Generalised Planning (GP) aims to compute generalised plans: programmatic plans that can solve families of related planning problems. A grand goal of GP is to overcome the scalability issues of general-purpose planners which are sound and complete for decidable planning problems, but in turn are not able to leverage patterns in and solve similarly structured problems more efficiently. Indeed, several planning domains exist that are computationally easy to solve and exhibit satisficing policies, such as variants of the package delivery domain [Helmert, 2003]. UPS™ delivered over 20 million packages every day across over 200 countries and territories in 2024 [UPS, 2025]. However, state-of-the-art planners struggle to scale up to a simplified version of the delivery problem with 100 packages [Taitler *et al.*, 2024].

A GP problem typically comprises a planning domain $\mathbb{D}$ representing an action theory, and a set of planning problems $\mathbb{P}$. Figure 1 illustrates the two main modules characteristic of many approaches for GP. The first module (green) takes as input an action theory $\mathbb{D}$ and a set of example problems $\mathbb{P}_{\text{train}}$ drawn from $\mathbb{P}$ in order to synthesise a generalised plan for solving problems in $\mathbb{P}$. The second module (blue) instantiates the generalised plan on a given input problem sharing the same action theory in order to provide a goal-achieving plan for that problem. Problems that are input into the second module may exhibit any number of objects.

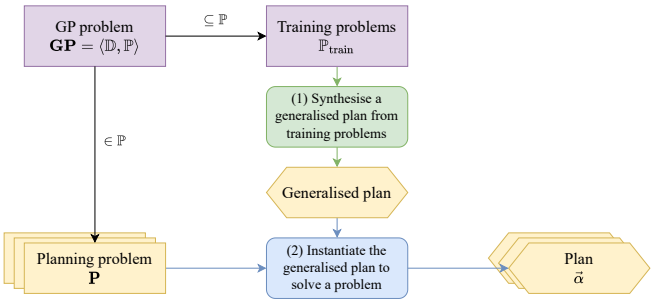


Figure 1: A common GP setup consisting of an action theory $\mathbb{D}$ and set of planning problems $\mathbb{P}$ satisfying the action theory. A generalised planner consists of two modules: (1) synthesis and (2) instantiation. See text for more details.

Srivastava *et al.* [2011a] introduced several metrics for measuring the effectiveness of a generalised planner, including the size of the set of planning domains an approach can solve (*expressiveness*), the time and data it takes to synthesise a generalised plan (*synthesis cost*) for a given domain, and the cost it takes to instantiate and execute the generalised plan for new planning problems (*execution cost*) of a given domain. Generalised planners aim to amortise the cost of synthesising a generalised plan against the cost of solving each planning problem individually. However, recent planning competition results also show that current generalised planners are still outperformed by traditional planners on standard planning domains [Taitler *et al.*, 2024], suggesting much room for improvement across all three GP metrics.

In this paper, we introduce MOOSE, a new generalised planner which draws upon insights and long-standing ideas from the knowledge representation and reasoning (KRR) and planning communities to advance the state of the art in GP. To capture expressive policies we employ *regression* rewriting, a preimage computation of a state under a sequence of actions [Fikes *et al.*, 1972; Waldinger, 1977; Reiter, 1991]. We use regression to identify sufficient and minimal state conditions required for generating lifted, goal-achieving rules [Muise *et al.*, 2012; Illanes and McIlraith, 2019] from our training plans. To lower synthesis costs, we *relax* the

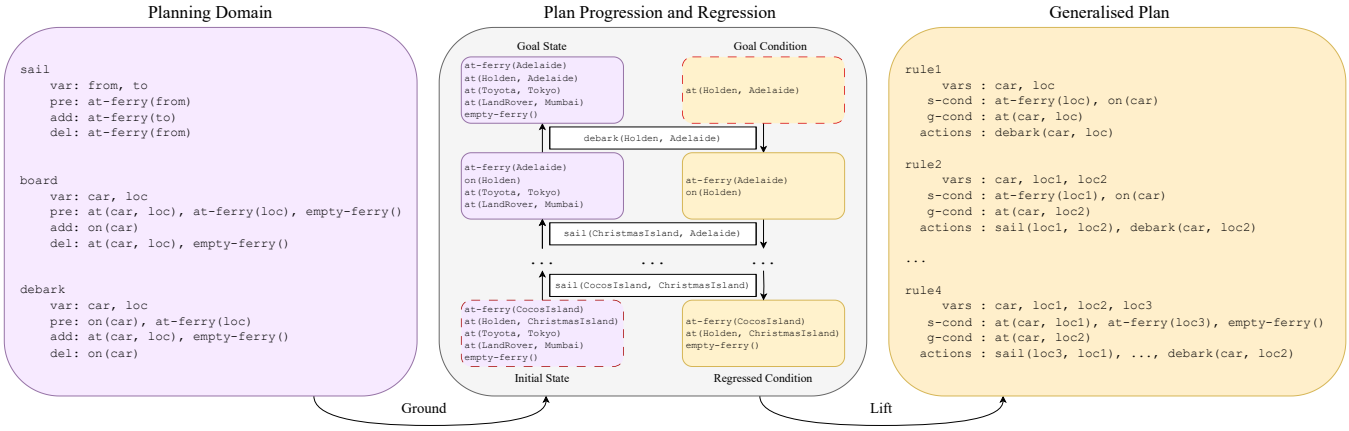


Figure 2: Left: the Ferry planning domain consisting of three PDDL action schemata for a ferry to sail, board a car, and debark a car. Middle: progression of the initial state of a planning problem (purple) via the plan in the center (white), and the regression of the goal condition (yellow) via the same plan. Right: the generalised plan created by lifting the plan actions, regressed states, and the goal condition.

goal by splitting conjunctive goals into singleton goals, under the observation that lifted policies are agnostic to the counting and unique identification of objects. Relaxing the policy synthesis problem parallels the planning technique of relaxing the problem to derive tractable and powerful heuristics for solving planning problems [Hoffmann and Nebel, 2001; Edelkamp, 2002; Helmert and Domshlak, 2009]. Lastly, to optimise execution costs, we leverage *database* algorithms in order to query and execute lifted policy rules efficiently.

We also theoretically formalise the conditions under which MOOSE learns sound and complete generalised plans as well as pruning rules which preserve optimal solutions when used in search. This is done by providing a refined classification of planning domains via goal decoupling, and an equivalence of planning problems via object isomorphism. We then conduct experiments on Easy-to-Solve, Hard-to-Optimise (ESHO) planning domains, P-time solvable and NP-hard to solve optimally domains. Experimental results show that MOOSE solves 33/61/23% more planning problems than state-of-the-art planners for classical/numeric/optimal planning, respectively. We summarily list our contributions as follows.

- We introduce MOOSE, a generalised planner for synthesising and executing generalised plans, by making use of regression rewriting, relaxation, and database algorithms.
- We formalise the conditions under which MOOSE is sound and complete for satisficing and optimal planning.
- We conduct experiments and demonstrate the effectiveness of MOOSE in satisficing, optimal and numeric planning settings relative to state-of-the-art planners.

2 Planning Background and Notation

We adopt standard notation and terminology for representing planning problems via the Planning Domain Definition Language (PDDL) [McDermott *et al.*, 1998; Haslum *et al.*, 2019]. A planning problem is represented by two components: a domain, consisting of lifted predicates and action schemata describing the action theory, and a problem specification, consisting of a finite set of objects, an initial state, and a goal condition. We begin with some general notation.

Mathematical Notation Let $\mathbb{N}/\mathbb{N}_0$ denote the natural numbers excluding/including 0, $\vec{\alpha}$ denote an ordered sequence of items, $\vec{\alpha}_i$ denote the ith element of $\vec{\alpha}$, $\vec{\alpha}_{[i:]}$ all elements from the ith element onwards in $\vec{\alpha}$ inclusive, and $|s|$ the size of a set or length of a sequence s.

Definition 1 (Planning Domain)**.** A planning domain is a tuple $\mathbb{D} = \langle \mathcal{P}, \mathcal{C}, \mathcal{A} \rangle$, where $\mathcal{P}$ is a set of predicates, $\mathcal{C}$ a set of constant objects, and $\mathcal{A}$ a set of action schemata. A predicate $p \in \mathcal{P}$ has a set of argument terms $x_1, \ldots, x_{n_p}$ where $n_p \in \mathbb{N}_0$ depends on p. An action schema $a \in \mathcal{A}$ is a tuple

$$a = \langle \mathit{var}(a), \mathit{pre}(a), \mathit{add}(a), \mathit{del}(a) \rangle$$

where $\mathit{var}(a)$ is a set of parameter variables, and preconditions $\mathit{pre}(a)$, add $\mathit{add}(a)$ and delete $\mathit{del}(a)$ effects are finite sets of predicates from $\mathcal{P}$ with arguments instantiated with variables or objects from $\mathit{var}(a) \cup \mathcal{C}$.

Definition 2 (Planning Problem)**.** A planning problem is a tuple $\mathbf{P} = \langle \mathbb{D}, s_0, g, \mathcal{O} \rangle$ where $\mathbb{D} = \langle \mathcal{P}, \mathcal{C}, \mathcal{A} \rangle$ is a planning domain, s_0 the initial state, g the goal condition, and $\mathcal{O} \supseteq \mathcal{C}$ a finite set of objects. A (ground) atom is a predicate whose argument terms are all instantiated with objects. A state in a planning problem is a set of atoms and operate under the closed world assumption: any atom not in a state is presumed false. The initial state s_0 and goal condition g are both sets of atoms. A state s is a goal state if $g \subseteq s$. We say that the problem $\mathbf{P}$ belongs to the domain $\mathbb{D}$.

A (ground) action is an action schema a where each parameter term is instantiated with an object, denoted $a(o_1, \ldots, o_n)$ with $o_1, \ldots, o_n \in \mathcal{O}$. An action a is applicable in a state s if $\mathit{pre}(a) \subseteq s$, in which case we define the successor

$$\mathrm{succ}(s, a) = (s \setminus \mathit{del}(a)) \cup \mathit{add}(a). \tag{1}$$

Otherwise, a is not applicable in s and $\mathrm{succ}(s, a) = \bot$. A plan for a planning problem is a finite sequence of actions $\vec{\alpha} = a_1, \ldots, a_n$ where $s_i = \mathrm{succ}(s_{i-1}, a_i) \neq \bot$ for $i = 1, \ldots, n$ and s_n is a goal state. We overload the notation of successor for sequences of actions with $\mathrm{succ}(s, \vec{\alpha}) = s_n$ as if $s = s_0$. The length of a plan $\vec{\alpha}$ is the number of actions it

contains. A problem $\mathbf{P}$ is solvable if a plan exists for $\mathbf{P}$. Satisficing (resp. optimal) planning refers to the task of finding any plan (resp. any plan with the lowest length) for $\mathbf{P}$.

Notational Shorthands We now introduce a few additional notational shorthands which will be useful for later. Let $\mathbf{P}[\omega]$ denote the ω component of a problem $\mathbf{P}$; e.g., $\mathbf{P}[s_0]$ is the initial state of $\mathbf{P}$. Furthermore, given a state s and set of atoms g' for a problem $\mathbf{P}$, let $\mathbf{P}_s = \langle \mathbb{D}, s, \mathbf{P}[g], \mathbf{P}[\mathcal{O}] \rangle$ denote the same problem with the initial state replaced with s, and $\mathbf{P}_{s,g'} = \langle \mathbb{D}, s, g', \mathbf{P}[\mathcal{O}] \rangle$ denote the same problem with the initial state and goal replaced with s and g'.

Goal Regression Goal regression refers to the act of computing the preimage of a goal under a sequence of actions via regression rewriting [Fikes *et al.*, 1972; Waldinger, 1977; Reiter, 1991; Reiter, 2001]. It is a powerful technique for computing the minimal set of goal relevant atoms. It has been used for heuristic synthesis [Bonet and Geffner, 2001; Scala *et al.*, 2016], plan monitoring [Fritz and McIlraith, 2007], policy synthesis for lifted Markov decision processes [Gretton and Thiébaux, 2004; Sanner and Boutilier, 2006; Sanner and Boutilier, 2009], nondeterministic planning [Muise *et al.*, 2012; Muise *et al.*, 2024], symbolic search [Pang and Holte, 2011; Alcázar *et al.*, 2013; Torralba, 2015; Speck *et al.*, 2025], numeric planning [Illanes and McIlraith, 2017], generating macro-actions [Hofmann *et al.*, 2020], and GP [Illanes and McIlraith, 2019; Yang *et al.*, 2022].

Definition 3 (STRIPS Regression [Rintanen, 2008])**.** A set of atoms g is regressable over an action a if $\mathit{add}(a) \cap g \neq \emptyset$ and $\mathit{del}(a) \cap g = \emptyset$, in which case we define the regression

$$\mathrm{regr}(g, a) = (g \setminus \mathit{add}(a)) \cup \mathit{pre}(a). \tag{2}$$

Otherwise, g is not regressable over a and $\mathrm{regr}(g, a) = \bot$.

3 Generalised Planning via Goal Regression

We first introduce the generalised planning (GP) problem as a set of planning problems sharing the same domain following from [Levesque, 2005; Srivastava *et al.*, 2008; Bonet *et al.*, 2009; Srivastava *et al.*, 2011a; Hu and De Giacomo, 2011; Celorrio *et al.*, 2019; Segovia-Aguas *et al.*, 2024].

Definition 4 (Generalised Planning Problem)**.** A generalised planning (GP) problem is a tuple of a domain and a possibly infinite set of solvable planning problems $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ where all planning problems $\mathbf{P} \in \mathbb{P}$ belong to $\mathbb{D}$.

A generalised plan π for a GP problem $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ is a programmatic plan that can be instantiated on any planning problem $\mathbf{P} \in \mathbb{P}$ to return a plan $\pi(\mathbf{P}) = \vec{\alpha}$ for $\mathbf{P}$. We call a generalised plan π a **solution** if for every $\mathbf{P} \in \mathbb{P}$, the plan $\pi(\mathbf{P})$ is a valid solution for $\mathbf{P}$. Examples of programmatic plans may include (memoryless) finite state controllers [Bonet *et al.*, 2009; Bonet *et al.*, 2010; Hu and De Giacomo, 2011; Aguas *et al.*, 2018], policies derived from lifted rules [Srivastava *et al.*, 2011b; Illanes and McIlraith, 2019; Francès *et al.*, 2021] and general-purpose programs [Levesque, 2005; Srivastava *et al.*, 2008; Segovia-Aguas *et al.*, 2024; Silver *et al.*, 2024]. Figure 1 illustrates the pipeline with which one synthesises generalised plans. In practice, the generalised plan is synthesised using a representative subset of the problems in $\mathbb{P}$, i.e., $\mathbb{P}_{\text{train}} = \{\mathbf{P}^{(1)}, \ldots, \mathbf{P}^{(n)}\} \subseteq \mathbb{P}$. We will later discuss the selection of this subset and the implications it has for the completeness of the resultant generalised plan of our approach.

MOOSE Program We name our generalised plans as MOOSE programs. MOOSE programs are found in a two-step process as described in Section 3.1: (a) decompose the set of training problems $\mathbb{P}_{\text{train}}$ into smaller problems constituting singleton goal conditions and generate optimal plans for each in order, and (b) apply goal regression from the singleton goals using the order of the optimal plans found in (a) to generate a set of lifted rules. MOOSE programs can be instantiated into a plan for a problem by deriving an action from the rule set at every state until the goal is reached (Section 3.2), or used to guide search for optimal planning (Section 3.3).

MOOSE programs are sets of lifted rules which indicate an action or macro action to execute, conditioned on a partial state and a goal that has not yet been achieved. A distinct feature of such rules is that the antecedent of a single rule compactly captures a set of states. Lifted rules can then be grounded on states if their antecedent condition is satisfied. Our rules are similar to existing lifted rules [Khardon, 1999; Illanes and McIlraith, 2019; Yang *et al.*, 2022] with the extension that we may now have macro actions in rule heads. Furthermore, each rule has an associated precedence value which determines its execution priority as is common in logic programming. Figure 2 illustrates the synthesis procedure and structure of MOOSE programs.

Definition 5 (MOOSE Rule)**.** Let $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ be a GP problem. A MOOSE rule r is a tuple

$$r = \langle \mathit{var}(r), \mathit{stateCond}(r), \mathit{goalCond}(r), \mathit{actions}(r) \rangle$$

where $\mathit{var}(r)$ is a finite set of free variables, $\mathit{stateCond}(r)$ and $\mathit{goalCond}(r)$ are finite sets of predicates instantiated with terms in $\mathit{var}(r)$, and $\mathit{actions}(r)$ is a finite sequence of action schemata instantiated with terms in $\mathit{var}(r)$.

Definition 6 (Grounding)**.** Let r be a MOOSE rule, $\mathbf{P}$ be a problem and s a state in the state space of $\mathbf{P}$. A grounding of r in s is an assignment of objects to variables $f : \mathit{var}(r) \to \mathbf{P}[\mathcal{O}]$ such that $\mathit{stateCond}(r)|_f \subseteq s$ and $\mathit{goalCond}(r)|_f \subseteq (\mathbf{P}[g] \setminus s)$, the set of goal atoms not yet achieved. The $|_f$ notation denotes replacing every occurrence of a free variable term with the corresponding object in f. In the case that a grounding f exists, we define the nondeterministic function

$$\mathrm{grounding}(r, s, \mathbf{P}[g]) = \mathit{actions}(r)|_f, \tag{3}$$

where f is some grounding. Otherwise, if no grounding exists then $\mathrm{grounding}(r, s, \mathbf{P}[g]) = \bot$.

Definition 7 (MOOSE Program)**.** A MOOSE program π is a set of MOOSE rules $\mathcal{R}$ and a function $\mathcal{R} \to \mathbb{N}$ representing a precedence ranking on the rules for execution.

As to be described later, if several rules are applicable in a given state, the rule with the lowest precedence ranking with ties broken arbitrarily is chosen for execution. Relatedly, Yang *et al.* [2022] specify a total order on policy rules, whereas MOOSE specifies more relaxed partial order. Next, we define lifting of a ground plan and set of atoms to a set of

Algorithm 1: MOOSE Program Synthesis

Input: Training problems $\mathbb{P}_{\text{train}} = \mathbf{P}^{(1)}, \ldots, \mathbf{P}^{(n_t)}$, and number of goal permutations $n_p \in \mathbb{N}$ (default: 3).
Output: MOOSE program π.

1 $\pi \leftarrow \emptyset$
2 **for** $i = 1, \ldots, n_t$ **do**
3 $\quad n_g \leftarrow |\mathbf{P}^{(i)}[g]|$
4 $\quad$ **for** $j = 1, \ldots, \min(n_p, n_g!)$ **do**
5 $\quad\quad s \leftarrow \mathbf{P}^{(i)}[s_0]$
6 $\quad\quad \vec{g} \leftarrow \text{newPermutation}(\mathbf{P}^{(i)}[g])$
7 $\quad\quad$ **for** $k = 1, \ldots, n_g$ **do**
8 $\quad\quad\quad g' \leftarrow \{\vec{g}_k\}$
9 $\quad\quad\quad \vec{\alpha} \leftarrow \text{optimalPlan}(\mathbf{P}^{(i)}_{s,g'})$
10 $\quad\quad\quad$ **if** $\vec{\alpha} = \bot$ **then continue**
11 $\quad\quad\quad \pi \leftarrow \pi \cup \text{extractRules}(\vec{\alpha}, g')$ `// Alg. 2`
12 $\quad\quad\quad s \leftarrow \text{succ}(s, \vec{\alpha})$ `// Eqn. (1) for plans`
13 **return** π

Algorithm 2: Rule Extraction Routine

Input: Sequence of actions $\vec{\alpha}$ and set of atoms g.
Output: MOOSE rules with precedence values π.

1 $\pi \leftarrow \emptyset; s \leftarrow g$
2 **for** $i = |\vec{\alpha}|, \ldots, 1$ **do**
3 $\quad s \leftarrow \text{regr}(s, \vec{\alpha}_i)$ `// Eqn. (2)`
4 $\quad r \leftarrow \text{lift}(s, g, \vec{\alpha}_{[i:]})$ `// Eqn. (4)`
5 $\quad \pi \leftarrow \pi \cup \{(r, |\vec{\alpha}| - i + 1)\}$
6 **return** π

quantified actions and predicates. Lifting will be used in the synthesis module to generate reusable rules.

Definition 8 (Lifting)**.** Let s and g be finite sets of ground atoms and $\vec{\alpha} = a_1, \ldots, a_m$ a sequence of ground actions. Then let $o_1, \ldots, o_q$ be the union of all objects from the atoms and actions that are not in $\mathcal{C}$. Next we define the set of free variable terms $\mathit{var} = \{v_1, \ldots, v_q\}$ and lift each action and atom by replacing each constant o_i with its corresponding free variable v_i in *var*. We denote

$$\text{lift}(s, g, \vec{\alpha}) = \langle \mathit{var}, s', g', \vec{\alpha}' \rangle \tag{4}$$

where $\vec{\alpha}'$ is the sequence of ground actions lifted by variables in *var*, and similarly for s' and g' the sets of lifted atoms.

3.1 Synthesising MOOSE Programs

Algorithm 1 summaries the main MOOSE program synthesis procedure. The input is a set of unlabelled training problems and a number n_p representing the effort spent on extracting information from a single problem. The main idea is that the problem is relaxed by decoupling the goals and greedily solving them optimally and in order. Each resulting plan is used to regress the corresponding singleton goal, and the regressed goal and plan is then lifted into a lifted macro action rule.

The main algorithm gradually builds an empty rule set (Line 1) by iterating over all training problems (Line 2) and the specified number n_p of goal orderings (Line 4). For each goal ordering and training problem, MOOSE finds plans via an optimal planner (Line 9) with the singleton goals (Line 8)

Algorithm 3: MOOSE Program Instantiation

Input: A planning problem $\mathbf{P}$ and MOOSE program π.
Output: A plan $\vec{\alpha}$ and success or failure status.

1 $s \leftarrow \mathbf{P}[s_0]$
2 $\vec{\alpha} \leftarrow []$ `// empty sequence`
3 **while** $\mathbf{P}[g] \not\subseteq s$ **do**
4 $\quad \vec{\beta} \leftarrow \bot$
5 $\quad$ **for** $r \in \pi$ *in ascending precedence values* **do**
6 $\quad\quad \vec{\beta} \leftarrow \text{grounding}(r, s, \mathbf{P}[g])$ `// Eqn. (3)`
7 $\quad\quad$ **if** $\vec{\beta} \neq \bot$ **then break**
8 $\quad$ **if** $\vec{\beta} = \bot$ *or detected cycle* **then return** $\vec{\alpha}$, failure
9 $\quad \vec{\alpha} \leftarrow \vec{\alpha}; \vec{\beta}$ `// sequence concatenation`
10 $\quad s \leftarrow \text{succ}(s, \vec{\beta})$
11 **return** $\vec{\alpha}$, success

in order (Line 7), while progressing the current state along the way (Line 12). If a plan exists, then we extract rules from the plan and add them to the incumbent plan (Line 11) as we will describe in more detail in the next paragraph. Otherwise if no plan exists, i.e. if the problem is unsolvable with the current state and singleton goal pair, no rules are extracted and the state is not progressed (Line 10).

Algorithm 2 describes how to extract lifted rules from a plan trace and set of goal atoms. It begins by initialising the to-be-regressed state s by the goal (Line 1). Next, it regresses s in reverse order of the plan $\vec{\alpha}$ (Lines 2 to 3) and then lifts (Line 4) the corresponding regressed state s, goal g, and suffix of the plan into a rule r. Then we compute the precedence value of the rule to be the cost-to-go from the partial state s to the goal g with respect to the input plan suffix and append it to the incumbent plan (Line 5).

3.2 Satisficing Planning via Policy Execution

A learned MOOSE program can be used for satisficing planning by repeatedly choosing and executing a rule to progress the initial state to a goal state. Algorithm 3 summaries the execution procedure for an input planning problem and MOOSE program. Summarily, each iteration of the algorithm's main loop queries the set of rules in order of ascending precedence values until a rule associated with goals not yet achieved can be grounded (Lines 5 to 7), from which the corresponding macro action is added to the incumbent plan and applied to the current state (Lines 9 to 10). The loop breaks once the goal is reached (Line 3), no actions can be queried from the set of rules, or a cycle is encountered (Line 8).

3.3 Optimal Planning via Action Pruning

Optimal planning can be performed via a synthesised MOOSE program by extending the corresponding planning problem with MOOSE rules. The rules, ignoring precedence values, are encoded into PDDL axioms [Thiébaux *et al.*, 2005] representing search control for optimal planners that support axioms. Theorem 22 later formalises conditions under which encodings of MOOSE programs preserve optimal solutions.

We now extend a given GP domain $\mathbb{D} = \langle \mathcal{P}, \mathcal{C}, \mathcal{A} \rangle$ with a MOOSE program π. Firstly, we add predicates p_g and p_{goalCond} for each $p \in \mathcal{P}$, representing goals in a planning

problem and goals that not been achieved in the state, respectively. Then each state in a problem $\mathbf{P}$ is extended with atoms $p_g(\vec{o})$ for each goal atom $p(\vec{o}) \in \mathbf{P}[\mathcal{G}]$ following Martín and Geffner [2004]. For each predicate p we introduce the axiom

$$p_{goalCond}(\vec{x}) \leftarrow p_g(\vec{x}) \wedge \neg p(\vec{x})$$

for computing unachieved goals. Secondly, for each action schema $a \in \mathcal{A}$ we add a new predicate a_π to $\mathcal{P}$ and $pre(a)$. Then for each MOOSE rule $\langle \vec{x}, stateCond, goalCond, \vec{\alpha} \rangle$, we introduce an axiom

$$(\vec{\alpha}_1)_\pi(\vec{x}) \leftarrow \bigwedge_{p(\vec{y}) \in stateCond} p(\vec{y}) \wedge \bigwedge_{p(\vec{y}) \in goalCond} p_{goalCond}(\vec{y})$$

for restricting the application of an action with the MOOSE rule condition. In this way, the axioms restrict the set of applicable actions at any ground state to the first action of each macro action that the MOOSE rules would generate, and hence prune the entire search space. Differently to previous works that prune the search space with lifted rules [Bacchus and Kabanza, 2000; Yoon *et al.*, 2008; Krajnanský *et al.*, 2014], our approach does not require writing new solvers but making use of existing planners that support PDDL axioms.

4 Soundness and Completeness Conditions

In this section, we provide theoretical results concerning the soundness and completeness of MOOSE programs for both satisficing and optimal planning (Theorems 19 and 22). Proofs of all statements are provided in Appendix B. The idea is that under assumptions on the complexity of a GP problem $\langle \mathbb{D}, \mathbb{P} \rangle$ and given sufficient training problems, Algorithm 1 synthesises generalised plans that are sound and complete for solving problems in $\mathbb{P}$, and furthermore finds optimal plans when MOOSE rules are used for search as described in Section 3.3. We begin by classifying planning domains based on the separability of goals, refining the class of Easy-to-Solve, Hard-to-Optimise (ESHO) planning domains, the class of P-time solvable and NP-hard to solve optimally domains.

Goal Independence Early works in planning worked under the assumption that conjunctive goals can be split apart into their individual components and achieved independently. The Sussman [1973] anomaly illustrates a simple Blocksworld example for how this was not true in general, giving rise to algorithms which aim to achieve goals simultaneously [Waldinger, 1977] and to provably complete algorithms in the current planning age. Regardless, most planning domains are P-time solvable and furthermore exhibit goals that can be achieved independently from one another. In this section, we formalise two variants of goal independence (TGI and SGI) and provide their computational complexity.

Definition 9 (True Goal Independence)**.** A planning problem $\mathbf{P}$ exhibits true goal independence (TGI) if for all orderings $\vec{g}$ of goal atoms in $\mathbf{P}[g]$, the following greedy algorithm is sound and complete: (1) set $s = \mathbf{P}[s_0]$ and then (2) iterate over goal atoms $\vec{g}_i$ in $\vec{g}$ in order by (2a) finding any optimal plan $\vec{\alpha}^{(i)}$ from s to a goal state containing $\vec{g}_i$ and (2b) progressing s via $\vec{\alpha}^{(i)}$. We say that $\mathbf{P}$ exhibits polynomial TGI (pTGI) if step (2a) can run in polynomial time. Lastly, we say that $\mathbf{P}$ exhibits TGI with respect to $C \in \mathbb{N}$, denoted TGI$_C$, if all optimal plans in step (2a) have plan length bounded by C.

Definition 10 (Sequential Goal Independence)**.** A planning problem $\mathbf{P}$ exhibits sequential goal independence (SGI) if there exists an ordering $\vec{g}$ of goals $\mathbf{P}[g]$ such that the greedy algorithm operating on $\vec{g}$ is sound and complete. Similarly, we say that $\mathbf{P}$ exhibits polynomial SGI (pSGI) if step (2a) in the greedy algorithm runs in polynomial time.

Next, we say that a GP problem $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ exhibits TGI if every problem in $\mathbb{P}$ exhibits TGI, and analogously for SGI. We then let PLANSAT($\mathbf{GP}$) denote the computational problem of deciding if a plan exists for a problem in $\mathbb{P}$. The following proposition shows that without the polynomial time constraint of step (2a) of the aforementioned greedy algorithm, TGI and SGI do not make planning any easier.

Proposition 11. PLANSAT*(*$\mathbf{GP}$*) of a GP problem* $\mathbf{GP}$ *exhibiting TGI is* PSPACE*-complete.*

Corollary 12. PLANSAT*(*$\mathbf{GP}$*) of a GP problem* $\mathbf{GP}$ *exhibiting SGI is* PSPACE*-complete.*

Once we add the polynomial time constraint of step (2a), both pTGI and pSGI become easier. However, only pTGI becomes tractable while pSGI becomes NP-complete.

Proposition 13. PLANSAT*(*$\mathbf{GP}$*) of a GP problem* $\mathbf{GP}$ *exhibiting pTGI is in* PTIME*.*

Corollary 14. *Let* $C \in \mathbb{N}$*.* PLANSAT*(*$\mathbf{GP}$*) of a GP problem* $\mathbf{GP}$ *exhibiting TGI*$_C$ *is in* PTIME*.*

Proposition 15. PLANSAT*(*$\mathbf{GP}$*) of a GP problem* $\mathbf{GP}$ *exhibiting pSGI is* NP*-complete.*

Planning Problem Equivalence Before we state the assumptions required for MOOSE to synthesise sound and complete generalised plans, we define the notion of equivalence for (lifted) problems. We define equivalence via bijection between objects, in contrast to work by Sievers *et al.* [2019] which reduce problems to graph automorphisms.

Definition 16 (Equivalence Relation)**.** Given a GP problem $\langle \mathbb{D}, \mathbb{P} \rangle$, we define a relation $\sim_U$ on planning problems in $\mathbb{P}$ by $\mathbf{P}_1 \sim_U \mathbf{P}_2$ if there exists a bijective mapping $f : \mathbf{P}_1[\mathcal{O}] \to \mathbf{P}_2[\mathcal{O}]$ such that $f(c) = c$ for $c \in \mathcal{C}$, $F(\mathbf{P}_1[s_0]) = \mathbf{P}_2[s_0]$ and $F(\mathbf{P}_1[g]) = \mathbf{P}_2[g]$ where $F(s) := \{p(f(o_1), \ldots, f(o_n)) \mid p(o_1, \ldots, o_n) \in s\}$.

Indeed the defined relation is an equivalence relation and furthermore defines a natural notion of equivalence for planning problems, where reflexivity, symmetry and transitivity follows from bijective functions in the definition of $\sim_U$.

Proposition 17. *The relation* $\sim_U$ *on planning problems of any given GP problem is an equivalence relation.*

Proposition 18. *Suppose* $\mathbf{P}_1 \sim_U \mathbf{P}_2$ *and let* $f : \mathbf{P}_1[\mathcal{O}] \to \mathbf{P}_2[\mathcal{O}]$ *be the bijective mapping satisfying the definition of* $\sim_U$*. Then a sequence of actions* $a_1, \ldots, a_n$ *is a plan for* $\mathbf{P}_1$ *if and only if* $a'_1, \ldots, a'_n$ *is a plan for* $\mathbf{P}_2$*, where* a'_i *is defined by* $a'_i = a(f(o_1), \ldots, f(o_n))$ *if* $a_i = a(o_1, \ldots, o_n)$ *for some* $a \in \mathcal{A}$ *and* $o_1, \ldots, o_n \in \mathbf{P}_1[\mathcal{O}]$*.*

Soundness and Completeness of MOOSE Now we state and prove the main theorems of the section. The main idea of the statement is that given enough training data MOOSE can construct a database of rules for TGI$_C$ problems which can

solve all possible problems with singleton goals. By assuming a bound in the definition of TGI_C of plan lengths, this database has a finite size.

Theorem 19 (MOOSE is sound and complete for PLANSAT)**.** *Let $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ be a GP problem exhibiting TGI_C. There exists a set of training problems $\mathbb{P}_{train}$ where Algorithm 3 using the plan Π synthesised from Algorithm 1 with $\mathbb{P}_{train}$ is sound and complete for $\mathbb{P}$ for satisficing planning.*

The bound on the size of training problems in the proof of Theorem 19 is exponential in the domain size. This bound may be tight and unavoidable given that it has been shown that GP under the QNP [Srivastava *et al.*, 2011b] framework is provably equivalent to FOND planning [Bonet and Geffner, 2020] which is known to be EXPTIME-complete. A fruitful next step is to develop a sound and complete learning algorithm to learns to generate and select what training data is required, possibly given implicitly in the input GP problem [Srivastava *et al.*, 2011a; Grundke *et al.*, 2024].

To prove optimality we require an additional assumption which we see is no harder than SGI.

Definition 20 (Optimal Goal Independence)**.** A planning problem exhibits Optimal Goal Independence (OGI) if there *exists* an ordering $\vec{g}$ of goals $\mathbf{P}[g]$ such that the algorithm described in Definition 9 is optimally sound and complete with the change that it is now nondeterministic and step (2a') is changed to "*guess* an optimal plan $\vec{\alpha}^{(i)}$ such that the concatenation of plans is optimal".

Proposition 21. *Let $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ be a GP problem exhibiting OGI. The computational problem* PLANOPT*$(\mathbf{GP})$ of deciding if there exists a plan for any problem in $\mathbb{P}$ with length less than an input k is* NP*-complete.*

Now we state the main theorem for optimal planning. We also provide a counterexample to the theorem for when the OGI assumption is dropped in Example 1 in the Appendix.

Theorem 22 (MOOSE is sound and complete for PLANOPT)**.** *Let $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ be a GP problem exhibiting TGI_C and OGI. There exists a set of training problems $\mathbb{P}_{train}$ where an optimal planner run on the transformation in Section 3.3 via the generalised plan Π learned from Algorithm 1 with $\mathbb{P}_{train}$ is sound and complete for $\mathbb{P}$ for optimal planning.*

5 Experiments

In this section we aim to understand how MOOSE places in performance in comparison to state-of-the-art planners. We first describe the experimental setup as follows.

Implementation We implement MOOSE from scratch in Python, but make use of the following tools: the `pddl` [Favorito *et al.*, 2025] parser for parsing PDDL problems; the `SQLite` [Hipp, 2020] database system for grounding in Algorithm 3 as planning states can be viewed as databases and rules as queries [Corrêa *et al.*, 2020; Corrêa and De Giacomo, 2024]; (NUMERIC) FAST DOWNWARD's implementation of A* with the (Numeric) LM-cut heuristic [Helmert and Domshlak, 2009; Kuroiwa *et al.*, 2022] for generating (numeric) optimal plans in Line 9 of Algorithm 1; and SYMK [Speck *et al.*, 2019; Speck *et al.*, 2025] for optimal planning with MOOSE as described in Section 3.3.

Additional Extensions We introduce an optimisation for generalised plan instantiation which tries to fire the previous successfully fired rule first during Line 9 of Algorithm 3. Furthermore, we introduce a validation and refinement procedure for handling domains which do not exhibit the TGI_C assumption in Appendix C. We also extend our approach to handle numeric planning problems formalised in Appendix A.

Benchmarks For classical planning, we make use of the training and testing problems of the 2023 International Planning Competition (IPC) [Taitler *et al.*, 2024] Learning Track for the Childsnack, Ferry, Miconic, Rover, Satellite and Transport domains, and also introduce new training and testing problems for the Barman and Logistics domains. For numeric planning, we make use of the training and testing problems from benchmarks by [Chen and Thiébaux, 2024a] for Childsnack, Ferry, Miconic, and Transport, and introduce new problems for the PDDL Minecraft domain [Benyamin *et al.*, 2024], as well as a new planning domain 'Adultsnack' which extends Childsnack to an arbitrary number of dietary restrictions. We modified Rover and Transport to remove their path-finding component. All domains are ESHO, P-time to solve but NP-hard to solve optimally, with the exception of Adultsnack and Childsnack which are also P-time to solve optimally. Detailed distributions of training and testing problem object sizes are provided in Appendix D.

Baselines For classical planning, we compare against LAMA [Richter and Westphal, 2010], the state-of-the-art standalone satisficing planner in the 2023 IPC Learning Track, and the sketch learner (SLEARN) [Drexler *et al.*, 2022] with width in $\{0, 1, 2\}$. In our results, we take the best SLEARN configuration for every problem. For numeric planning, we compare against the multi-queue (M$(3h\|3n)$) [Chen and Thiébaux, 2024b] and the multi-repetition relaxed plan heuristic with jumping actions (MRP+HJ) [Scala *et al.*, 2020] configurations of ENHSP. For optimal planning, we compare against blind A* search, A* search with the LM-cut heuristic, SCORPION [Seipp *et al.*, 2020], the state-of-the-art standalone optimal planner according to the 2023 IPC Optimal Track, and SYMK [Speck *et al.*, 2025].

Resources For synthesis, satisficing, and optimal planning, we gave MOOSE and all baselines an 8GB memory and 1800s runtime limit, and ran them on a cluster with Intel Xeon Platinum 8274 CPUs (3.20GHz). An exception is that SLEARN was given *more* compute for synthesis (32GB memory and 12 hours for each width configuration). For all experiments, we set the only hyperparameter in the MOOSE learning procedure in Algorithm 1 to be $n_p = 3$.

Results We first note that MOOSE completed synthesis in the given synthesis budgets while SLEARN failed to learn domain knowledge for 3 domains despite having access to substantially more compute and multiple configurations. Figure 3 summarises synthesis metrics and planner performance and we refer to Appendix E for more detailed results.

Satisficing Numeric Planning MOOSE solves all 540 numeric testing problems. Figure 3 shows that each problem is solved in under 100 seconds. In comparison, state-of-the-art, numeric planners solve fewer than two thirds of the bench-

	Time (s)		Memory (GB)	
	MOOSE	SLEARN	MOOSE	SLEARN
Ba	380	–	< 1	–
Ch	113	5593	< 1	2
Fe	17	15	< 1	< 1
Lo	92	–	< 1	–
Mi	18	16	< 1	< 1
Ro	667	–	< 1	–
Sa	530	5964	< 1	6
Tr	23	30	< 1	< 1

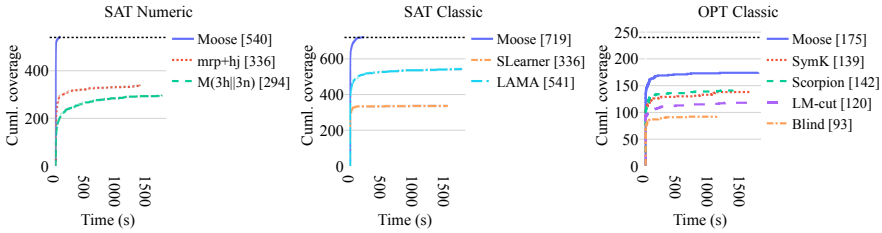


Figure 3: Left: synthesis time and memory usage (↓) of MOOSE and SLEARN. We present the lowest synthesis metrics over multiple SLEARN configurations, while MOOSE has only a single configuration. Cells marked – indicate that synthesis failed with the given budgets. Right: cumulative coverage (↑) of planners over time for various planning settings. The dotted black line indicates the total number of problems.

mark problems with the 1800s limit. MOOSE achieves the highest quality plans for Adultsnack and Childsnack, while plan quality is similar to the baselines for all other domains.

Satisficing Classical Planning MOOSE solves 719 out of 720 classical testing problems, outperforming all tested baselines. The generalised planner SLEARN manages to learn domain knowledge for 5 out of 8 domains but only solves all problems from one domain (Miconic). From Figure 27 in the Appendix, MOOSE has better/worse plans than LAMA on 5/3 domains (Barman, Childsnack, Ferry, Rover and Transport)/ (Logistics, Miconic and Satellite). The single failure was due to the rules not returning an action for a (rare) Logistics state not captured in the training or elsewhere in the testing set. However, when rules are queried in descending instead of ascending order of precedence values in Line 5 of Algorithm 3, the state is avoided and this problem is solved.

Optimal Classical Planning MOOSE solves 175 out of 240 classical testing problems optimally. MOOSE achieves the best (tied) performance for 5 domains out of 8, while the next best optimal planner, Scorpion, achieves the best (tied) performance for 4 domains. We also note that MOOSE matches or outperforms the base planner SYMK on all domains. This fact suggests that the reduction in search space from encoding learned policies via axioms outweighs the cost of evaluating such axioms. Although it is not guaranteed that the transformation from policies learned from finite training data preserves optimality, the plans output by MOOSE are empirically optimal in comparison to plans output by other optimal planners. We also note that MOOSE learns provably optimal *policies* for Num-Adultsnack and (Num-)Childsnack.

6 Related Work

Gretton and Thiébaux [2004] employed regression rewriting for GP in the context of lifted MDPs. Lifted regression was used to generate relevant features for building decision-tree policies via inductive logic programming. Their approach handles optimal, probabilistic planning although the horizons of testing problems were bounded by those seen in the training set. LOOM [Illanes and McIlraith, 2019] is the work that is most closely related to MOOSE theory-wise. LOOM automatically synthesises an abstraction from a single planning problem of a GP problem via bagging equivalent objects [Fuentetaja and de la Rosa, 2016; Riddle *et al.*, 2016; Dong *et al.*, 2025] into a nondeterministic, quantified problem. The quantified problem is then solved with an extension of the PRP planner [Muise *et al.*, 2012] to synthesise generalisable policies via regression which satisfy a policy termination test [Srivastava *et al.*, 2011b]. MOOSE takes inspiration from the powerful regression rewriting technique employed in these works, but differs in the methodology. MOOSE takes a *bottom-up* approach of performing ground regression from example plans to generate ground condition-action pairs that are then lifted into rules. Gretton and Thiébaux takes a *top-down* approach of performing lifted regression to generate relevant lifted features, and LOOM uses ground regression on a top-down abstraction of the GP problem for synthesising generalised policies. Furthermore, MOOSE implicitly generalises over goals expressed by Skolem functions, whereas LOOM is restricted to Skolem constants (nullary Skolem functions) due to the dependency on bagging.

More generally, MOOSE lies in the class of generalised planners that synthesise generalised plans by sampling from training problems. Examples of recent work includes PG3 [Yang *et al.*, 2022] which performs heuristic search over a space of generalised policies [Segovia-Aguas *et al.*, 2021; Segovia-Aguas *et al.*, 2024] and also uses goal regression for handling 'missed' states. Toyer *et al.* [2018; 2020] introduced ASNets, the first deep learning approach for GP, which learns generalised policies with graph neural networks and training data. Chen *et al.* [2024] introduced WL-GOOSE which synthesised generalised plans as heuristic functions via statistical graph learning. Silver *et al.* [2024] employed language models for synthesising Python code as generalised plans. Celorrio *et al.* [2019] surveys more prior GP work which synthesise generalised plans from training problems.

7 Conclusion

We introduce a new generalised planner, MOOSE, for both satisficing and optimal planning based on the KRR concept of goal regression. We formally classify and define the classes of planning domains and problems for which MOOSE is sound and complete. Experimental results show that our approach significantly advances the state of the art for classical, numeric, and optimal (generalised) planning for a substantial class of planning domains. MOOSE is also memory efficient for both generalised plan synthesis and instantiation, taking less than 1GB of memory to perform synthesis and instantiation across all experiments. Future work involves handling domains requiring transitive closure computations and assumptions weaker than those introduced in the our theory.

Acknowledgements

The first two authors carried out this work while at the Vector Institute, where the first author was working as a Research Intern. We gratefully acknowledge funding from the Natural Sciences and Engineering Research Council of Canada (NSERC) and the Canada CIFAR AI Chairs Program. The second author was supported by an RWTH IRS Theodore von Kármán Fellowship under the grant ID G:(DE-82)ZUK2-TvK-GSO101 and the Excellence Strategy of the Federal Government and the Länder, Germany. Resources used in preparing this research were provided, in part, by the Province of Ontario, the Government of Canada through CIFAR, and companies sponsoring the Vector Institute.

References

[Aguas *et al.*, 2018] Javier Segovia Aguas, Sergio Jiménez, and Anders Jonsson. Computing hierarchical finite state controllers with classical planning. *J. Artif. Intell. Res.*, 62:755–797, 2018.

[Alcázar *et al.*, 2013] Vidal Alcázar, Daniel Borrajo, Susana Fernández, and Raquel Fuentetaja. Revisiting regression in planning. In *IJCAI*, 2013.

[Bacchus and Kabanza, 2000] Fahiem Bacchus and Froduald Kabanza. Using temporal logics to express search control knowledge for planning. *Artif. Intell.*, 116:123–191, 2000.

[Benyamin *et al.*, 2024] Yarin Benyamin, Argaman Mordoch, Shahaf Shperberg, and Roni Stern. Solving minecraft tasks via model learning. In *PRL Workshop Series - Bridging the Gap Between AI Planning and Reinforcement Learning*, 2024.

[Bonet and Geffner, 2001] Blai Bonet and Hector Geffner. Planning as heuristic search. *Artif. Intell.*, 129:5–33, 2001.

[Bonet and Geffner, 2020] Blai Bonet and Hector Geffner. Qualitative numeric planning: Reductions and complexity. *J. Artif. Intell. Res.*, 69:923–961, 2020.

[Bonet *et al.*, 2009] Blai Bonet, Héctor Palacios, and Hector Geffner. Automatic derivation of memoryless policies and finite-state controllers using classical planners. In *ICAPS*, 2009.

[Bonet *et al.*, 2010] Blai Bonet, Héctor Palacios, and Hector Geffner. Automatic derivation of finite-state machines for behavior control. In *AAAI*, 2010.

[Bylander, 1994] Tom Bylander. The computational complexity of propositional STRIPS planning. *Artif. Intell.*, 69(1-2):165–204, 1994.

[Celorrio *et al.*, 2019] Sergio Jiménez Celorrio, Javier Segovia-Aguas, and Anders Jonsson. A review of generalized planning. *Knowl. Eng. Rev.*, 34:e5, 2019.

[Chen and Thiébaux, 2024a] Dillon Z. Chen and Sylvie Thiébaux. Graph learning for numeric planning. In *NeurIPS*, 2024.

[Chen and Thiébaux, 2024b] Dillon Z. Chen and Sylvie Thiébaux. Novelty heuristics, multi-queue search, and portfolios for numeric planning. In *SOCS*, 2024.

[Chen *et al.*, 2024] Dillon Z. Chen, Felipe Trevizan, and Sylvie Thiébaux. Return to tradition: Learning reliable heuristics with classical machine learning. In *ICAPS*, 2024.

[Corrêa and De Giacomo, 2024] Augusto B. Corrêa and Giuseppe De Giacomo. Lifted planning: Recent advances in planning using first-order representations. In *IJCAI*, 2024.

[Corrêa *et al.*, 2020] Augusto B. Corrêa, Florian Pommerening, Malte Helmert, and Guillem Francès. Lifted successor generation using query optimization techniques. In *ICAPS*, 2020.

[Dong *et al.*, 2025] Hao Dong, Zheyuan Shi, Hemeng Zeng, and Yongmei Liu. An automatic sound and complete abstraction method for generalized planning with baggable types. *AAAI*, 39(14):14875–14884, 2025.

[Drexler *et al.*, 2022] Dominik Drexler, Jendrik Seipp, and Hector Geffner. Learning sketches for decomposing planning problems into subproblems of bounded width. In *ICAPS*, 2022. Code accessed from commit `7a7ea6` in https://github.com/drexlerd/sketch-learner.

[Edelkamp, 2002] Stefan Edelkamp. Symbolic pattern databases in heuristic search planning. In *AIPS*, 2002.

[Erol *et al.*, 1995] Kutluhan Erol, Dana S. Nau, and V. S. Subrahmanian. Complexity, decidability and undecidability results for domain-independent planning. *Artif. Intell.*, 76:75–88, 1995.

[Favorito *et al.*, 2025] Marco Favorito, Francesco Fuggitti, and Christian Muise. pddl, 2025. Accessed from https://github.com/ai-Planning/pddl.

[Fikes *et al.*, 1972] Richard Fikes, Peter E. Hart, and Nils J. Nilsson. Learning and executing generalized robot plans. *Artif. Intell.*, 3(1-3):251–288, 1972.

[Fox and Long, 2003] Maria Fox and Derek Long. PDDL2.1: an extension to PDDL for expressing temporal planning domains. *J. Artif. Intell. Res.*, 20, 2003.

[Francès *et al.*, 2021] Guillem Francès, Blai Bonet, and Hector Geffner. Learning general planning policies from small examples without supervision. In *AAAI*, 2021.

[Fritz and McIlraith, 2007] Christian Fritz and Sheila A. McIlraith. Monitoring plan optimality during execution. In *ICAPS*, 2007.

[Fuentetaja and de la Rosa, 2016] Raquel Fuentetaja and Tomás de la Rosa. Compiling irrelevant objects to counters. special case of creation planning. *AI Commun.*, 29(3):435–467, 2016.

[Garey and Johnson, 1979] M. R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[Gretton and Thiébaux, 2004] Charles Gretton and Sylvie Thiébaux. Exploiting first-order regression in inductive policy selection. In *UAI*, 2004.

[Grundke *et al.*, 2024] Claudia Grundke, Gabriele Röger, and Malte Helmert. Formal representations of classical planning domains. In *ICAPS*, 2024.

[Haslum *et al.*, 2019] Patrik Haslum, Nir Lipovetzky, Daniele Magazzeni, and Christian Muise. *An Introduction to the Planning Domain Definition Language*. Morgan & Claypool Publishers, 2019.

[Hearn and Demaine, 2005] Robert A. Hearn and Erik D. Demaine. Pspace-completeness of sliding-block puzzles and other problems through the nondeterministic constraint logic model of computation. *Theor. Comput. Sci.*, 343(1-2):72–96, 2005.

[Helmert and Domshlak, 2009] Malte Helmert and Carmel Domshlak. Landmarks, critical paths and abstractions: What's the difference anyway? In *ICAPS*, 2009.

[Helmert, 2002] Malte Helmert. Decidability and undecidability results for planning with numerical state variables. In *AIPS*, 2002.

[Helmert, 2003] Malte Helmert. Complexity results for standard benchmark domains in planning. *Artif. Intell.*, 143(2):219–262, 2003.

[Hipp, 2020] Dwayne Richard Hipp. Sqlite, 2020. Accessed from https://www.sqlite.org/index.html.

[Hoffmann and Nebel, 2001] Jörg Hoffmann and Bernhard Nebel. The FF planning system: Fast plan generation through heuristic search. *J. Artif. Intell. Res.*, 14:253–302, 2001.

[Hofmann *et al.*, 2020] Till Hofmann, Tim Niemueller, and Gerhard Lakemeyer. Macro operator synthesis for ADL domains. In *ECAI*, 2020.

[Hu and De Giacomo, 2011] Yuxiao Hu and Giuseppe De Giacomo. Generalized planning: Synthesizing plans that work for multiple environments. In *IJCAI*, 2011.

[Illanes and McIlraith, 2017] Leon Illanes and Sheila A. McIlraith. Numeric planning via abstraction and policy guided search. In *IJCAI*, 2017.

[Illanes and McIlraith, 2019] León Illanes and Sheila A. McIlraith. Generalized planning via abstraction: Arbitrary numbers of objects. In *AAAI*, 2019.

[Khardon, 1999] Roni Khardon. Learning action strategies for planning domains. *Artif. Intell.*, 113:125–148, 1999.

[Krajnanský *et al.*, 2014] Michal Krajnanský, Jörg Hoffmann, Olivier Buffet, and Alan Fern. Learning pruning rules for heuristic search planning. In *ECAI*, 2014.

[Kuroiwa *et al.*, 2022] Ryo Kuroiwa, Alexander Shleyfman, Chiara Piacentini, Margarita P. Castro, and J. Christopher Beck. The LM-cut heuristic family for optimal numeric planning with simple conditions. *J. Artif. Intell. Res.*, 75:1477–1548, 2022.

[Levesque, 2005] H. J. Levesque. Planning with loops. In *IJCAI*, 2005.

[Martín and Geffner, 2004] Mario Martín and Hector Geffner. Learning generalized policies from planning examples using concept languages. *Appl. Intell.*, 20:9–19, 2004.

[McDermott *et al.*, 1998] Drew McDermott, Malik Ghallab, Adele E. Howe, Craig A. Knoblock, Ashwin Ram, Manuela M. Veloso, Daniel S. Weld, and David E. Wilkins. PDDL-the planning domain definition language. Technical report, 1998.

[Muise *et al.*, 2012] Christian J. Muise, Sheila A. McIlraith, and J. Christopher Beck. Improved non-deterministic planning by exploiting state relevance. In *ICAPS*, 2012.

[Muise *et al.*, 2024] Christian Muise, Sheila A. McIlraith, and J. Christopher Beck. PRP rebooted: Advancing the state of the art in FOND planning. In *AAAI*, 2024.

[Pang and Holte, 2011] Bo Pang and Robert C. Holte. State-set search. In *SOCS*, 2011.

[Reiter, 1991] Raymond Reiter. The frame problem in the situation calculus: A simple solution (sometimes) and a completeness result for goal regression. In *Artificial and Mathematical Theory of Computation*. Academic Press, 1991.

[Reiter, 2001] Raymond Reiter. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT Press, 2001.

[Richter and Westphal, 2010] Silvia Richter and Matthias Westphal. The LAMA planner: Guiding cost-based anytime planning with landmarks. *J. Artif. Intell. Res.*, 39:127–177, 2010.

[Riddle *et al.*, 2016] Pat Riddle, Jordan Douglas, Mike Barley, and Santiago Franco. Improving performance by reformulating PDDL into a bagged representation. In *Proceedings of the 8th Workshop on Heuristic Search for Domain-independent Planning*, 2016.

[Rintanen, 2008] Jussi Rintanen. Regression for classical and nondeterministic planning. In *ECAI*, 2008.

[Sanner and Boutilier, 2006] Scott Sanner and Craig Boutilier. Practical linear value-approximation techniques for first-order MDPs. In *UAI*, 2006.

[Sanner and Boutilier, 2009] Scott Sanner and Craig Boutilier. Practical solution techniques for first-order MDPs. *Artif. Intell.*, 173(5-6):748–788, 2009.

[Scala *et al.*, 2016] Enrico Scala, Patrik Haslum, and Sylvie Thiébaux. Heuristics for numeric planning via subgoaling. In *IJCAI*, 2016.

[Scala *et al.*, 2020] Enrico Scala, Alessandro Saetti, Ivan Serina, and Alfonso Emilio Gerevini. Search-guidance mechanisms for numeric planning through subgoaling relaxation. In *ICAPS*, 2020.

[Segovia-Aguas *et al.*, 2021] Javier Segovia-Aguas, Sergio Jiménez, and Anders Jonsson. Generalized planning as heuristic search. In *ICAPS*, 2021.

[Segovia-Aguas *et al.*, 2024] Javier Segovia-Aguas, Sergio Jiménez Celorrio, and Anders Jonsson. Generalized planning as heuristic search: A new planning search-space that leverages pointers over objects. *Artif. Intell.*, 330:104097, 2024.

[Seipp *et al.*, 2020] Jendrik Seipp, Thomas Keller, and Malte Helmert. Saturated cost partitioning for optimal classical planning. *J. Artif. Intell. Res.*, 67:129–167, 2020.

[Sievers *et al.*, 2019] Silvan Sievers, Gabriele Röger, Martin Wehrle, and Michael Katz. Theoretical foundations for structural symmetries of lifted PDDL tasks. In *ICAPS*, 2019.

[Silver *et al.*, 2024] Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B. Tenenbaum, Leslie Kaelbling, and Michael Katz. Generalized planning in PDDL domains with pretrained large language models. In *AAAI*, 2024.

[Speck *et al.*, 2019] David Speck, Florian Geißer, Robert Mattmüller, and Álvaro Torralba. Symbolic planning with axioms. In *ICAPS*, 2019.

[Speck *et al.*, 2025] David Speck, Jendrik Seipp, and Álvaro Torralba. Symbolic search for cost-optimal planning with expressive model extensions. *J. Artif. Intell. Res.*, 82:1349–1405, 2025.

[Srivastava *et al.*, 2008] Siddharth Srivastava, Neil Immerman, and Shlomo Zilberstein. Learning generalized plans using abstract counting. In *AAAI*, 2008.

[Srivastava *et al.*, 2011a] Siddharth Srivastava, Neil Immerman, and Shlomo Zilberstein. A new representation and associated algorithms for generalized planning. *Artif. Intell.*, 175(2):615–647, 2011.

[Srivastava *et al.*, 2011b] Siddharth Srivastava, Shlomo Zilberstein, Neil Immerman, and Hector Geffner. Qualitative numeric planning. In *AAAI*, 2011.

[Sussman, 1973] Gerald Jay Sussman. *A Computational Model of Skill Acquisition*. PhD thesis, MIT, 1973.

[Taitler *et al.*, 2024] Ayal Taitler, Ron Alford, Joan Espasa, Gregor Behnke, Daniel Fiser, Michael Gimelfarb, Florian Pommerening, Scott Sanner, Enrico Scala, Dominik Schreiber, Javier Segovia-Aguas, and Jendrik Seipp. The 2023 international planning competition. *AI Mag.*, 45:280–296, 2024.

[Thiébaux *et al.*, 2005] Sylvie Thiébaux, Jörg Hoffmann, and Bernhard Nebel. In defense of PDDL axioms. *Artif. Intell.*, 168:38–69, 2005.

[Torralba, 2015] Álvaro Torralba. *Symbolic Search and Abstraction Heuristics for Cost-Optimal Planning*. PhD thesis, Universidad Carlos III de Madrid, 2015.

[Toyer *et al.*, 2018] Sam Toyer, Felipe W. Trevizan, Sylvie Thiébaux, and Lexing Xie. Action schema networks: Generalised policies with deep learning. In *AAAI*, 2018.

[Toyer *et al.*, 2020] Sam Toyer, Sylvie Thiébaux, Felipe Trevizan, and Lexing Xie. Asnets: Deep learning for generalised planning. *J. Artif. Intell. Res.*, 68:1–68, 2020.

[UPS, 2025] UPS. Global reporting initiative. Accessed from https://about.ups.com/content/dam/upsstories/images/our-impact/reporting/2024-UPS-GRI-Report.pdf, 2025.

[Waldinger, 1977] Richard J. Waldinger. Achieving several goals simultaneously. *Machine Intelligence*, 8:94–136, 1977.

[Yang *et al.*, 2022] Ryan Yang, Tom Silver, Aidan Curtis, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. PG3: policy-guided planning for generalized policy generation. In *IJCAI*, 2022.

[Yoon *et al.*, 2008] Sung Wook Yoon, Alan Fern, and Robert Givan. Learning control knowledge for forward search planning. *J. Mach. Learn. Res.*, 9:683–718, 2008.

Appendix

Table of Contents

A Numeric Planning Definitions **11**

B Proofs for Section 4 **12**

C Validation and Refinement of MOOSE Rules **13**

D Training and Testing Task Size Distributions **15**
D.1 Classical Planning 15
D.2 Numeric Planning 16

E Additional Experimental Results **17**
E.1 Coverage Tables 17
E.2 Satisficing Numeric Planning 17
E.3 Satisficing Classical Planning 19
E.4 Optimal Classical Planning 21
E.5 MOOSE vs. LAMA on Plan Quality 23

A Numeric Planning Definitions

We can extend the definition of a planning problem in Definition 2 to handle numeric variables, numeric conditions and numeric effects by making use of the fragment of PDDL 2.1 [Fox and Long, 2003] excluding durative actions. In this paper, we consider the fragment of numeric planning where numeric conditions are restricted to involve one numeric variable per formula with comparisons in $\{\geq, >, =\}$, and numeric action effects to additions by a constant value. This fragment is very expressive as it is undecidable by reduction from an abacus program [Helmert, 2002, Theorem 12].

Definition 23 (Abacus Program Numeric Planning Problem)**.** A (lifted) numeric planning problem is a tuple $\Pi = \langle \mathcal{P}, \mathcal{F}, \mathcal{O}, \mathcal{A}, s_0, g \rangle$ where $\mathcal{P}$ is a set of lifted predicates, $\mathcal{F}$ is a set of lifted (numeric) functions, $\mathcal{O}$ is a set of objects, $\mathcal{A}$ is a set of action schemata, s_0 is the initial state, and g the goal condition. Predicates are defined as in Definition 2 and functions are similar where each function $f \in \mathcal{F}$ has a set of argument terms $x_1, \ldots, x_{n_f}$ where $n_f \in \mathbb{N}_0$ depends on f. A propositional variable is a predicate whose argument terms are all instantiated with objects, and has a range in $\{\top, \bot\}$. A numeric variable is a function whose argument terms are all instantiated with objects, and has a range in $\mathbb{R}$. We let N_p/N_n denote the set of propositional/numeric variables for Π given by all possible instantiations of predicates/functions. A state is a value assignment to all N_p and N_n.

A literal is a predicate p or its negation $\neg p$. A propositional condition is a positive (resp. negative) literal $x = \top$ (resp. $\bot$), and a numeric condition has the form $f \trianglerighteq c$ where f is a function, $c \in \mathbb{R}$ and $\trianglerighteq \in \{\geq, >, =\}$. A state s satisfies a set of conditions (i.e. a set of propositional and numeric conditions) if each condition in the set evaluates to true given the values of the state variables in s. The goal g is a set of conditions.

An action schema $a \in \mathcal{A}$ is a tuple $\langle \mathit{var}(a), \mathit{pre}(a), \mathit{add}(a), \mathit{del}(a), \mathit{num_eff}(a) \rangle$ where $\mathit{var}(a)$ is a set of parameter terms, the preconditions $\mathit{pre}(a)$ is a set of conditions, $\mathit{add}(a)$ and $\mathit{del}(a)$ are add and delete lists of predicates as in the classical case, $\mathit{num_eff}(a)$ is a set of numeric conditions of the form $f(x_1, \ldots, x_{n_f}) = f(x_1, \ldots, x_{n_f}) + c$ for $c \in \mathbb{R}$ with argument terms instantiated with terms or objects from $\mathit{var}(a) \cup \mathcal{O}$. An action a is applicable in a state s if s satisfies $\mathit{pre}(a)$. In this case, its successor $\mathrm{succ}(s, a)$ is the state where the effects $\mathit{num_eff}(a)$ are applied to the numeric variables in s, and propositional variables are modified in the same way as in the classical case. If a is not applicable in s, we have $\mathrm{succ}(s, a) = \bot$. The definition of plan is the same as in the classical case.

Next, we extend the definition of logical regression for classical planning in Definition 3 to handle numeric conditions and effects for the class of abacus program numeric problems.

Definition 24 (Abacus Program Numeric Planning Regression)**.** A set of conditions g is regressable over an action a if the classical conditions are regressable over a as in Definition 3, with no restriction for numeric conditions. In this case, we define the regression of g under a by transforming numeric conditions $f \trianglerighteq c$ to $f \trianglerighteq c - v$ if there is an effect of a of the form $f = f + v$, and if there exists any precondition $\xi = (f \trianglerighteq c)$ in a, the associated numeric condition corresponding to f in g is transformed to ξ.

B Proofs for Section 4

Proof Sketch for Proposition 11. Membership follows from PSPACE-completeness of planning for fixed domains [Bylander, 1994; Erol *et al.*, 1995], and hardness by reduction from the Rush Hour problem which has singleton goals and has been shown to be PSPACE-hard [Hearn and Demaine, 2005]. □

Proof Sketch for Corollary 12. Membership again follows from PSPACE-completeness of planning for fixed domains. Hardness follows from Proposition 11 as TGI is a special case of SGI. □

Proof Sketch for Proposition 13. This follows by noting that the greedy algorithm described in the definition of TGI runs in polynomial time under the assumption that step (2a) can run in polynomial time. □

Proof Sketch for Corollary 14. TGI_C of a GP problem implies pTGI as optimal search in step (2a) of the greedy TGI algorithm runs in polynomial time with exponent C but C is a constant. □

Proof Sketch for Proposition 15. For NP membership, one guesses the correct ordering of goals after which running the greedy algorithm is in polynomial time. For NP-hardness, we reduce from the Hamiltonian path problem which is NP-complete [Garey and Johnson, 1979, p. 60]. The Hamiltonian path problem asks to find a path on a graph that visits every vertex exactly once. To encode this as a planning domain, one would require goals of the form *visited*(x) and actions which traverse a graph but make each vertex untraversable once it has been visited, such as by deleting initially true *clear*(x) atoms. Then the Hamiltonian path problem is equivalent to finding an correct ordering of goals representing (adjacent) vertices to visit. □

Proof Sketch for Proposition 17. Reflexivity, symmetry and transitivity follows from the usage of bijective functions in the definition of $\sim_U$. □

Proof Sketch for Proposition 18. The statement follows by definitions and bijectiveness of f. □

Proof Sketch for Theorem 19. Let $\mathbf{GP} = \langle \mathbb{D}, \mathbb{P} \rangle$ be a GP problem exhibiting TGI_C as in the statement assumption, with bound $C \in \mathbb{N}$ from Definition 9. To show completeness, it suffices to prove that there exists a finite number of MOOSE rules π, representing all possible optimal plans for singleton goals in step (2a) of the definition of TGI in Definition 9, which when executed with Algorithm 3 can solve any problem in $\mathbf{GP}$ with singleton goals. Then by the TGI assumption, any arbitrary problem $\mathbf{P}$ in $\mathbb{P}$ can be solved because execution of π would achieve the goals $\mathbf{P}[g]$ in a monotonic fashion. Soundness follows by the fact that application of rules lifted from regression are sound and thus, the execution of Algorithm 3 is sound.

Now, note that the set of possible singleton goals in $\mathbf{GP}$ is finite modulo the equivalence relation $\sim_U$ restricted to sets of atoms, as there are a finite number of predicates and instantiations of nonequivalent objects. Furthermore by the TGI_C assumption any optimal plan for any possible singleton goal g has plan length less than C. Thus the set of all possibles sequences of actions that can be regressed from g is bounded by $\sum_{k=0}^{C}(|\mathcal{A}| \cdot (kN' + M')^N)^k$ where

- $N = \max_{a \in \mathcal{A}}(|var(a)|)$ is the maximum arity of schemata,
- M is the maximum arity of predicates,
- $N' = N + |\mathcal{C}|$, and
- $M' = M + |\mathcal{C}|$.

Note that by Proposition 18, it suffices to count the equivalence class of plans under $\sim_U$. This is because modulo $\sim_U$ there are at most $(kN' + M')^N$ possible instantiations of an action where there are at most $kN' + M'$ possible objects across all actions in a length-k plan and the singleton goal under $\sim_U$. By a similar argument, there are $|\mathcal{P}| \cdot M'^M$ possible singleton goal instantiations, and hence it takes a finite number of up to $n = \sum_{k=0}^{C}(|\mathcal{A}| \cdot (kN' + M')^N)^k \cdot (|\mathcal{P}| \cdot M'^M)$ different problems to synthesise a general policy for $\mathbf{GP}$. □

Proof Sketch for Proposition 21. NP membership follows by definition of OGI, more specifically the nondeterministic algorithm defined within. NP-hardness follows by the NP-hardness of SGI which can be viewed as a general case of OGI. □

Proof Sketch for Theorem 22. Completeness follows from the completeness of rules as discussed in Theorem 19. To show soundness for optimal planning, we note that given enough training problems, bounded by $n = \sum_{k=0}^{C}(|\mathcal{A}| \cdot (kN' + M')^N)^k \cdot (|\mathcal{P}| \cdot M'^M)$ from the previous theorem, learned rules do not throw away any optimal actions at every state. Then the proof follows from the definition of OGI. □

Example 1 (Theorem 22 counterexample without the OGI assumption)**.** *A counterexample to the previous theorem for when the OGI assumption is dropped occurs if we can find a case where achieving a singleton goal for a TGI problem suboptimally is necessary to achieve optimality for the whole problem. In the planning problem illustrated by the state space in Figure 4 with goal $\{g_1, g_2\}$, an optimal plan to either g_1 or g_2 has plan length 2 and the greedy algorithm in Definition 9 of TGI returns a plan of length 4. However, the optimal plan has length 3.*

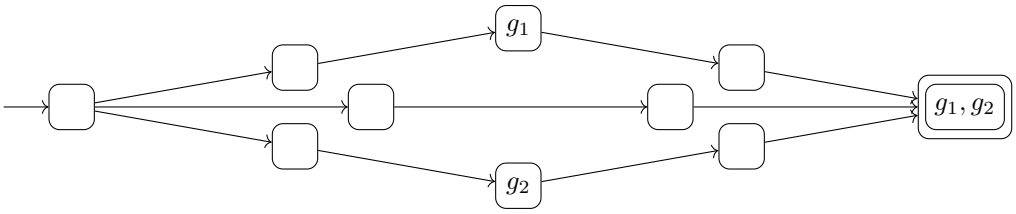


Figure 4: A planning problem illustrating the necessity of the OGI assumption for learning provably optimal policies in Theorem 22.

Algorithm 4: Validation and Refinement Routine

Input: MOOSE program π, and training problems $\mathbb{P}_{\text{train}} = \mathbf{P}^{(1)}, \ldots, \mathbf{P}^{(n_t)}$.
Output: MOOSE program π and *execution* mode for π.

1 $execution \leftarrow$ greedy
2 **do**
3 **for** $i = 1, \ldots, n_t$ **do**
4 $\vec{\alpha}, status \leftarrow \text{exec}(\mathbf{P}^{(i)}, \pi)$ `// Alg. 3`
5 **if** $status =$ success **then continue**
6 **if** *reason for failure* = π *caused a deadend* **then**
7 $r \leftarrow$ rule that caused the deadend
8 $\pi.r.precedence \leftarrow \pi.r.precedence + (0, 1)$
9 $execution \leftarrow$ conservative
10 **else if** *reason for failure* = π *ran into a cycle* **then**
11 $r \leftarrow$ last executed rule
12 $\pi.r.precedence \leftarrow \pi.r.precedence + (-1, 1)$
13 $execution \leftarrow$ conservative
14 **else if** *reason for failure* = π *derived no rule* **then**
15 $\pi \leftarrow \pi \cup \text{learn}(\{\mathbf{P}^{(i)}_{\text{succ}(\mathbf{P}^{(i)}[s_0], \vec{\alpha})}\})$
16 **while** π *has changed*
17 **return** π, *execution*

C Validation and Refinement of MOOSE Rules

It may be the case that MOOSE generates rules successfully in Algorithm 1 but does not solve problems from a domain if it does not exhibit the TGI assumption, i.e. if there exists an ordering of goals that must not be ignored. In order to handle such cases, we introduce a validation and refinement procedure for the synthesis of MOOSE programs, as well as different execution modes for MOOSE programs. First, we modify the type of the precedence ranking function from $\mathcal{R} \to \mathbb{N}$ to $\mathcal{R} \to \mathbb{N} \times \mathbb{N}$. Line 5 in Algorithm 2 is then changed to $\pi \leftarrow \pi \cup \{(r, (|\vec{\alpha}| - i + 1, 1))\}$.

Now, the execution of a MOOSE program π may fail due to a number of reasons. The validation and refinement procedure displayed in Algorithm 4 aims to resolve some failures by iterating over all training problems and trying to execute the generalised plan (Line 4) and checking for success (Line 5). If the generalised plan fails, it is caused by one of the following reasons: the generalised plan caused a deadend, in which case the precedence of the rule which caused the deadend is lowered (Line 8); the generalised plan ran into a cycle, in which case the rule which caused the cycle is strictly lowered (Line 12); or the generalised plan has not encountered a solvable state before, in which case learning is done on the new state (Line 15). In the former two cases, the generalised plan is switched to a 'conservative mode' as it has learned that some actions are dangerous if executed greedily. Otherwise, the generalised plan is kept in a 'greedy mode' which means that it uses Algorithm 3 for instantiation. Although the generalised plan is not always guaranteed to stabilise, it does so for all domains used in the experiments.

Algorithm 5 shows how one instantiates a MOOSE program when the synthesis procedure has determined that the plan be executed in a conservative mode. The algorithm is equivalent to Algorithm 3 with two additions highlighted in blue. Rules are now queried in order of descending precedence values (Line 5) and furthermore, only the first action of each rule head is executed (Line 9).

Algorithm 5: MOOSE Conservative Plan Instantiation

Input: A planning problem $\mathbf{P}$ and MOOSE program π.
Output: A plan $\vec{\alpha}$ and success or failure status.

$s \leftarrow \mathbf{P}[s_0]$
$\vec{\alpha} \leftarrow []$ `// empty sequence`
while $\mathbf{P}[g] \not\subseteq s$ **do**
  $\vec{\beta} \leftarrow \perp$
  for $r \in \pi$ *in descending precedence values* **do**
    $\vec{\beta} \leftarrow \text{grounding}(r, s, \mathbf{P}[g])$ `// Eqn.` (3)
    if $\vec{\beta} \neq \perp$ **then break**
  if $\vec{\beta} = \perp$ *or detected cycle* **then return** $\vec{\alpha}$, failure
  $\vec{\beta} \leftarrow [\vec{\beta}_0]$
  $\vec{\alpha} \leftarrow \vec{\alpha}; \vec{\beta}$ `// sequence concatenation`
  $s \leftarrow \text{succ}(s, \vec{\beta})$
return $\vec{\alpha}$, success

D Training and Testing Task Size Distributions

D.1 Classical Planning

Domain	Object Types	Train Min	Train Max	Test Min	Test Max
Barman	Σ	16	27	21	853
	cocktail	3	7	4	393
	dispenser	3	3	3	30
	hand	2	2	2	2
	ingredient	3	3	3	30
	level	3	3	3	3
	shaker	1	1	1	1
	shot	1	8	5	394
Childsnack	Σ	6	32	20	1326
	bread-portion	1	6	4	292
	child	1	6	4	292
	content-portion	1	6	4	292
	place	1	3	3	3
	sandwich	1	9	4	437
	tray	1	2	1	10
Ferry	Σ	3	8	7	1461
	car	1	2	2	974
	location	2	6	5	487
Logistics	Σ	29	29	10	1260
	airplane	3	3	1	64
	city	3	3	1	64
	location	15	15	2	960
	package	5	5	5	108
	truck	3	3	1	64
Miconic	Σ	3	11	5	681
	floor	2	7	4	196
	passenger	1	4	1	485
Rover	Σ	10	36	12	596
	camera	1	4	1	99
	lander	1	1	1	1
	mode	3	3	3	3
	objective	1	10	1	236
	rover	1	4	1	30
	store	1	4	1	30
	waypoint	2	10	4	197
Satellite	Σ	5	43	11	402
	direction	2	10	4	98
	instrument	1	20	3	195
	mode	1	3	1	10
	satellite	1	10	3	99
Transport	Σ	6	17	12	354
	location	2	7	5	99
	package	1	4	1	194
	size	2	3	3	11
	vehicle	1	3	3	50

Table 1: Object distribution of training and testing splits for *classical* planning.

D.2 Numeric Planning

Domain	Objects	Train Min	Train Max	Test Min	Test Max
Num-adultsnack	Σ	8	39	43	2644
	at_kitchen_bread	1	10	12	876
	at_kitchen_content	1	10	12	876
	food	3	3	3	3
	hungry	1	10	12	876
	place	1	3	3	3
	tray	1	3	1	10
Num-childsnack	Σ	5	10	16	889
	at_kitchen_bread	1	2	4	292
	at_kitchen_content	1	2	4	292
	hungry	1	2	4	292
	place	1	2	3	3
	tray	1	2	1	10
Num-ferry	Σ	7	9	11	1465
	car	1	2	2	974
	ferry-capacity	4	4	4	4
	location	2	3	5	487
Num-miconic	Σ	7	15	9	685
	floor	2	7	4	196
	lift-capacity	4	4	4	4
	passenger	1	4	1	485
Num-minecraft	Σ	5	5	15	2100
	cell	4	4	11	1800
	pogo_sticks_to_make	1	1	4	300
Num-transport	Σ	5	14	13	605
	capacity	1	4	4	262
	location	2	4	5	99
	package	1	4	1	194
	vehicle	1	2	3	50

Table 2: Object distribution of training and testing splits for *numeric* planning.

E Additional Experimental Results

E.1 Coverage Tables

Domain	M(3h‖3n)	MRP+HJ	MOOSE
Num-Adultsnack	32	23	**90**
Num-Childsnack	64	49	**90**
Num-Ferry	60	61	**90**
Num-Miconic	63	71	**90**
Num-Minecraft	30	68	**90**
Num-Transport	45	64	**90**
∑ (540)	294	336	**540**

Domain	LAMA	SLEARN-0	SLEARN-1	SLEARN-2	MOOSE
Barman	49	0	0	0	**90**
Childsnack	35	0	66	0	**90**
Ferry	69	67	67	60	**90**
Logistics	77	0	0	0	**89**
Miconic	**90**	**90**	**90**	**90**	**90**
Rovers	66	0	0	0	**90**
Satellite	89	0	47	48	**90**
Transport	66	0	63	46	**90**
∑ (720)	541	157	333	244	**719**

Domain	Blind	LMCUT	SCORPION	SYMK	MOOSE
Barman	0	0	0	12	**26**
Childsnack	9	9	9	15	**17**
Ferry	10	18	17	18	**30**
Logistics	8	15	**22**	10	16
Miconic	**30**	**30**	**30**	**30**	**30**
Rovers	15	17	18	**20**	**20**
Satellite	12	22	**26**	21	21
Transport	9	9	**20**	13	15
∑ (240)	93	120	142	139	**175**

Table 3: Planning coverage for satisficing numeric planning (left), satisficing classical planning (middle), and optimal classical planning (right). Domains have 90/30 problems each for SAT/OPT planning.

E.2 Satisficing Numeric Planning

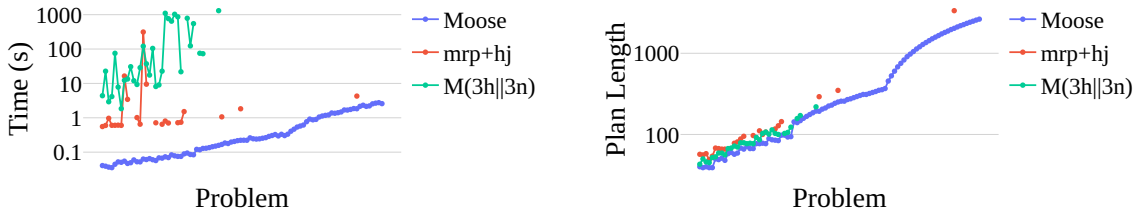


Figure 5: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Numeric Adultsnack**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

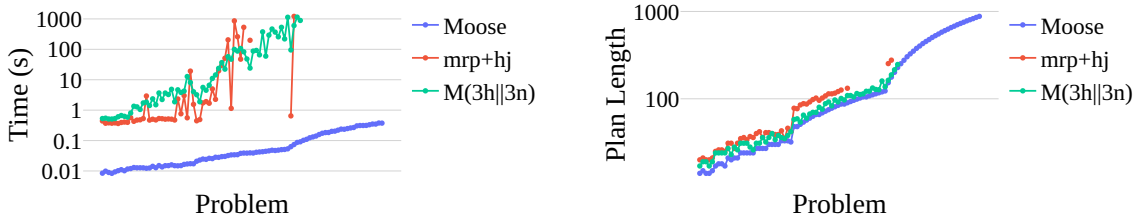


Figure 6: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Numeric Childsnack**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

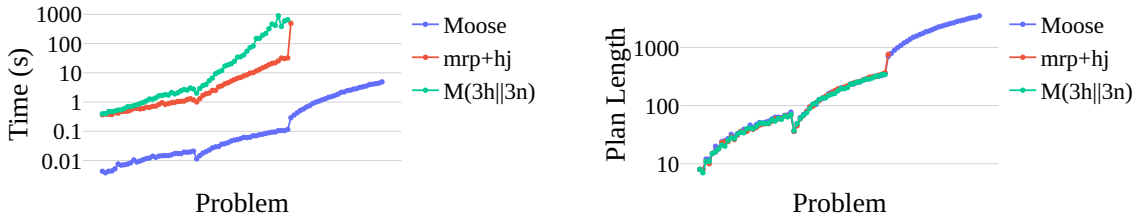


Figure 7: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Numeric Ferry**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

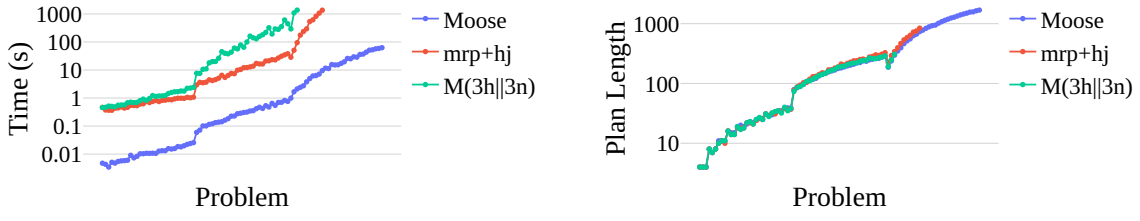


Figure 8: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Numeric Miconic**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

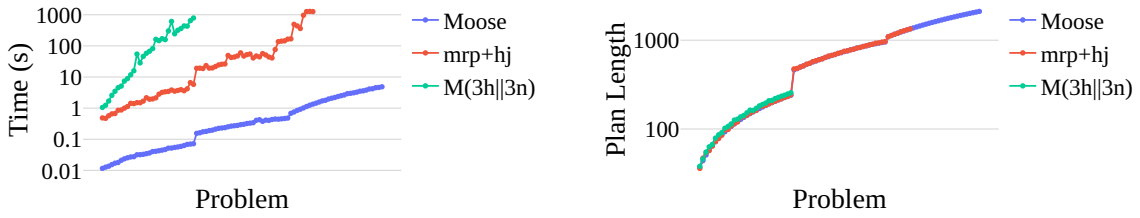


Figure 9: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Numeric Minecraft**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

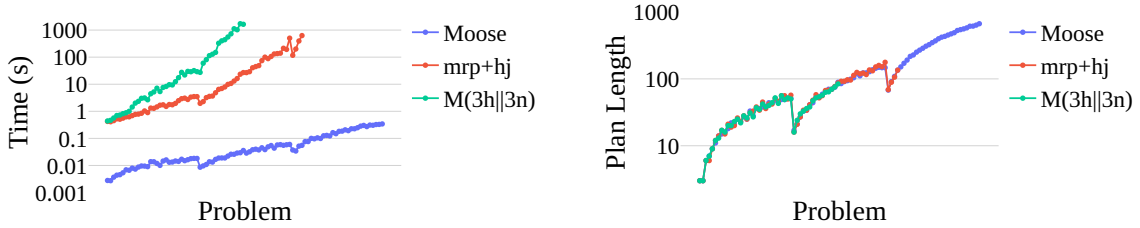


Figure 10: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Numeric Transport**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

E.3 Satisficing Classical Planning

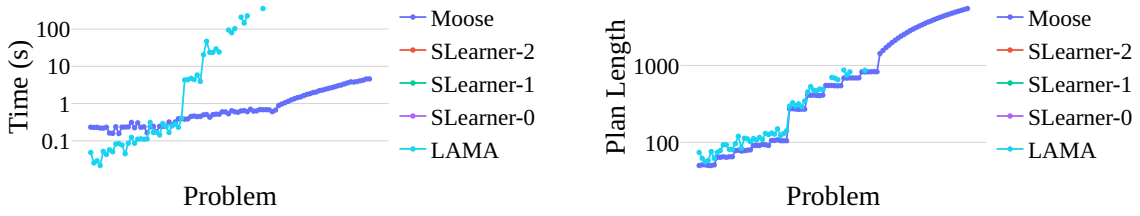


Figure 11: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Barman**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

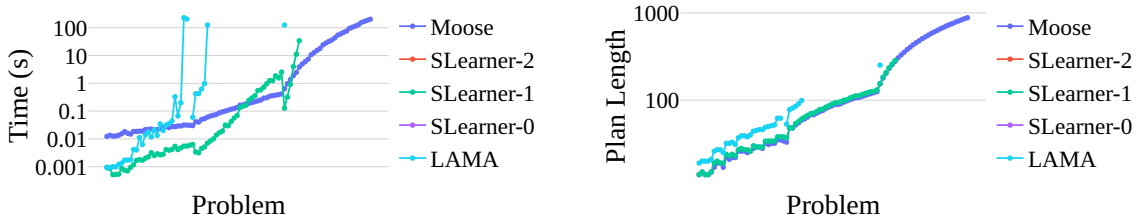


Figure 12: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Childsnack**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

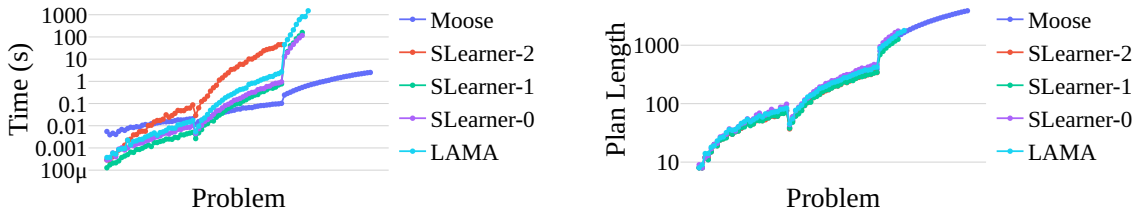


Figure 13: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Ferry**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

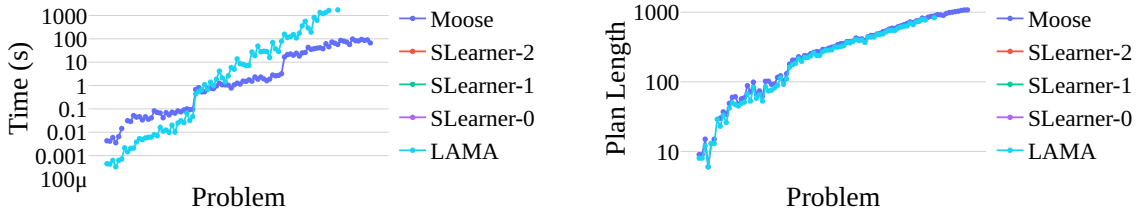


Figure 14: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Logistics**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

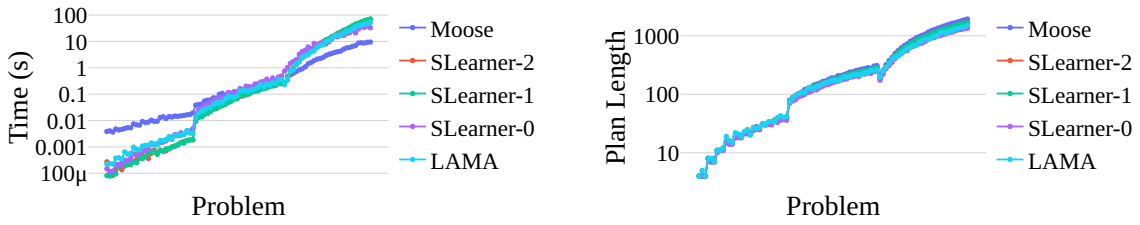


Figure 15: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Miconic**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

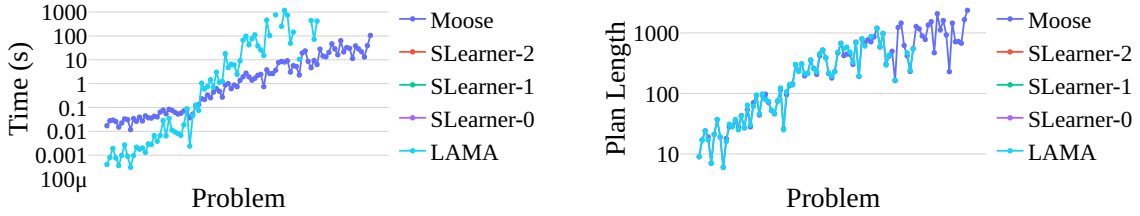


Figure 16: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Rover**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

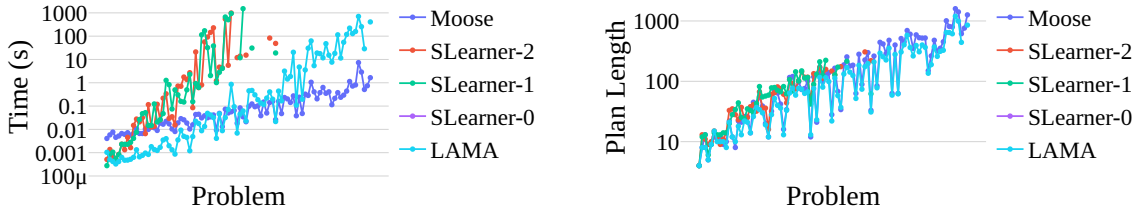


Figure 17: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Satellite**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

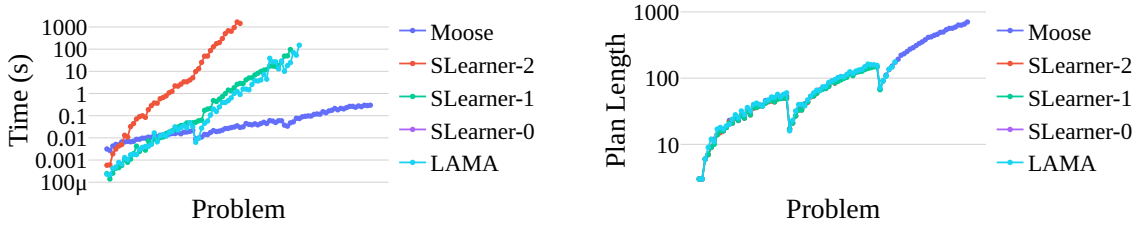


Figure 18: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Transport**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime and plan length.

E.4 Optimal Classical Planning

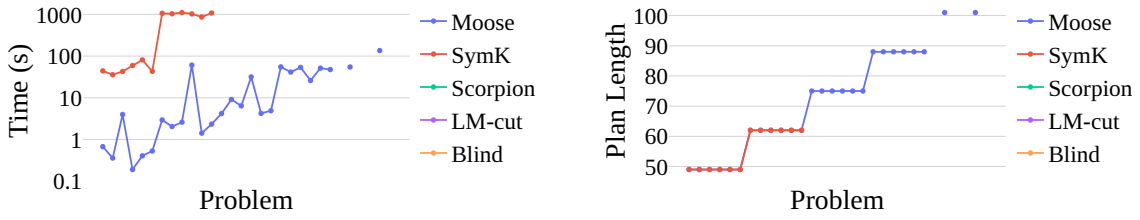


Figure 19: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Barman**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

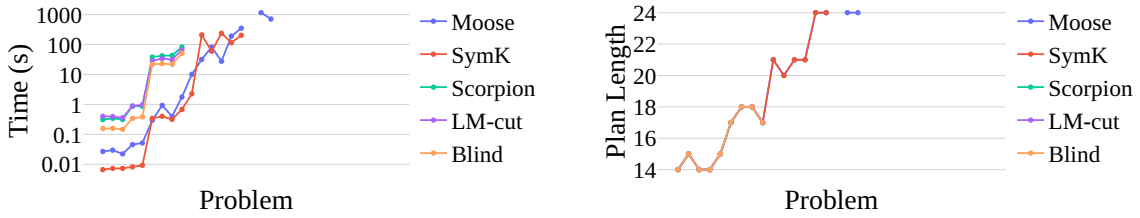


Figure 20: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Childsnack**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

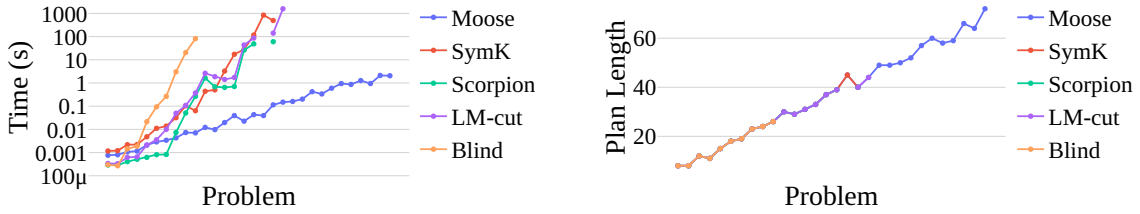


Figure 21: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Ferry**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

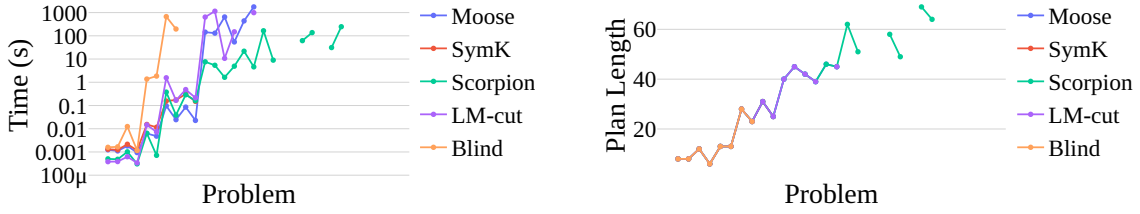


Figure 22: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Logistics**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

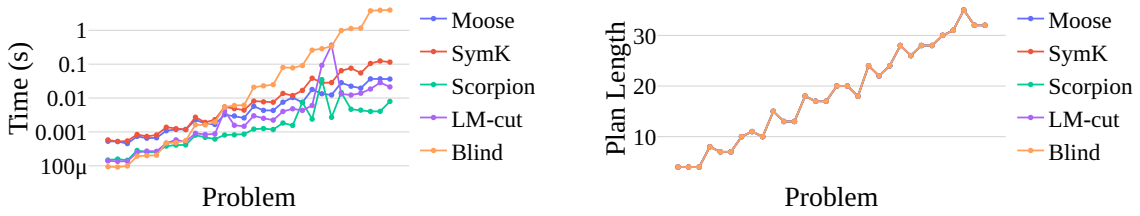


Figure 23: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Miconic**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

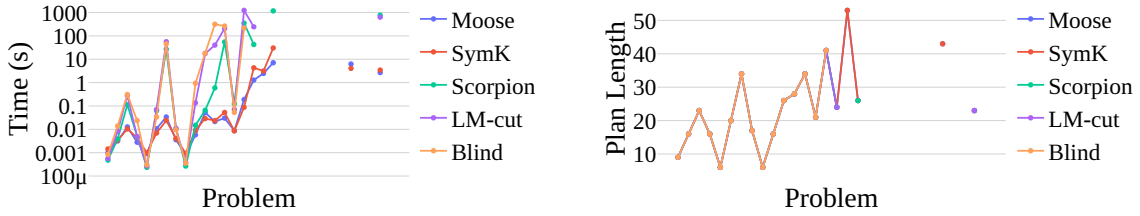


Figure 24: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Rover**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

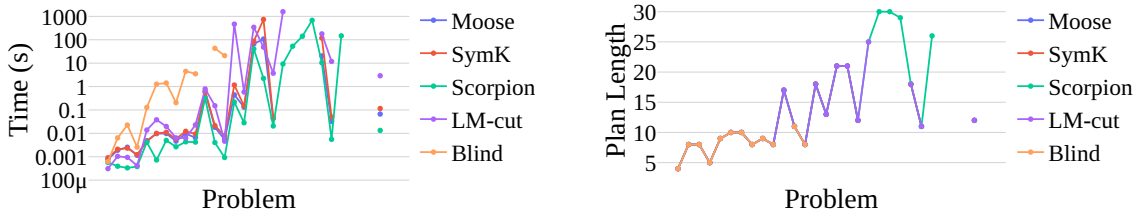


Figure 25: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Satellite**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

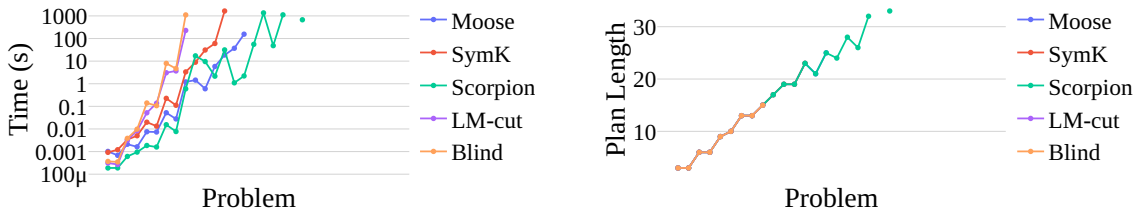


Figure 26: Line plots for time in seconds (left) and plan length (right) of planners across solved problems for **Transport**. Planning problem difficulty increases across the x-axis, and lower y-axis values are better (↓). Note the log scale of the y-axis for runtime.

E.5 MOOSE vs. LAMA on Plan Quality

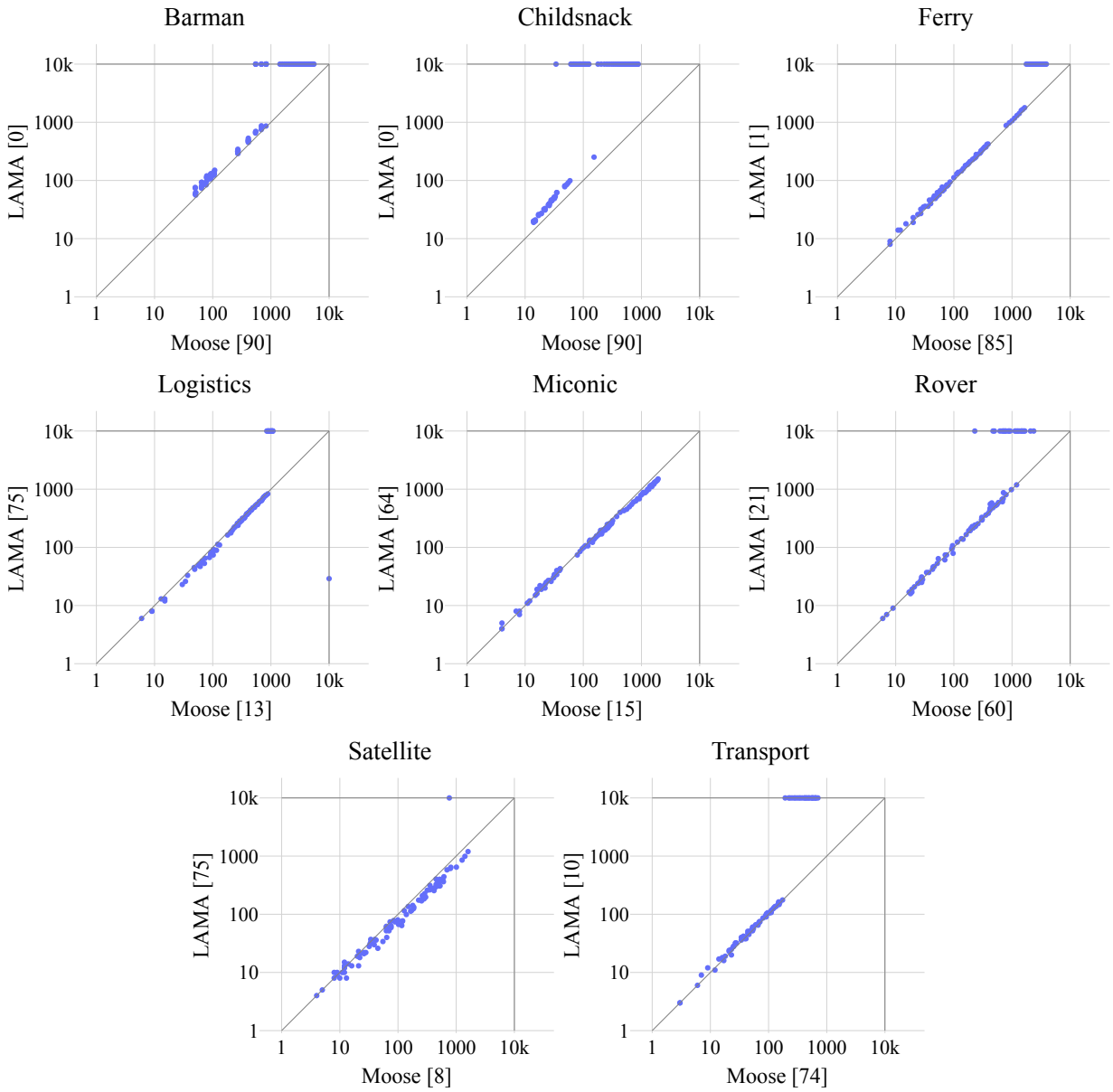


Figure 27: Plot comparisons of expanded nodes of LAMA (y-axis) and MOOSE (x-axis) for different classical planning domains. A point (x, y) represents the metric of the models indicated on the x and y axis on the domain. The number in the brackets next to each model indicates how many planning problems the model returned a higher quality plan than the model on the other axis. Points on the top left (resp. bottom right) triangle favour MOOSE (resp. LAMA).

A Fine-Grained Complexity View on Propositional Abduction — Algorithms and Lower Bounds

Extended Abstract*

Victor Lagerkvist[1] , **Mohamed Maizia**[1,2] , **Johannes Schmidt**[2]
[1]Department of Computer and Information Science, Linköping University, Sweden
[2]Department of Computer Science and Informatics, Jönköping University, Sweden
victor.lagerkvist@liu.se, mohamed.maizia@ju.se, johannes.schmidt@ju.se

The *Boolean satisfiability* problem is a well-known NP-complete problem. Due to the rapid advance of SAT solvers, many combinatorial problems are today solved by reducing to SAT, which can then be solved with off-the-shelf solvers. SAT fundamentally encodes a form of *monotonic* reasoning in the sense that conclusions remain valid regardless if new information is added. However, this turned out to be inappropriate for practical modeling, and thus *non-monotonic* logics emerged. Here, one is able to retract a statement if new data is added which violates the previous conclusion. One of the best known examples of non-monotonic reasoning is *abductive* reasoning where we are interested in finding explanations. More formally, an instance of the abduction problem is given by the triple (KB, H, M), where KB is the knowledge base, H the hypotheses, and M the manifestation. The question is whether there exists an explanation, that is, a set $E \subseteq H$ such that 1) $\mathrm{KB} \cup E$ is consistent, and 2) $\mathrm{KB} \cup E \models M$. Abduction has many practical applications, e.g., scientific discovery [Inoue *et al.*, 2009], network security [Alberti *et al.*, 2005], computational biology [Ray *et al.*, 2006], medical diagnosis [Obeid *et al.*, 2019], and XAI [Ignatiev, 2020; Racharak and Tojo, 2021]. The latter application could be especially important due to the continued emergence of AI in new and surprising applications, which need to be made GDPR compliant [Sovrano *et al.*, 2020]. The incitement for solving abduction fast, even when it is classically intractable, thus seems highly practically motivated.

Can non-monotonic reasoning be performed as efficiently as monotonic reasoning, or are there fundamental differences between the two? Classical complexity theory says that the two problems are different: SAT is NP-complete, while most forms of non-monotonic reasoning, including *propositional* abduction, are generally Σ_2^P-complete. However, modern complexity theory typically tells a different story, where classical hardness results do not imply that the problems are hopelessly intractable, but rather that different algorithmic schemes should be applied. For SAT, there is a healthy amount of theoretical research complementing the advances of SAT solvers, and k-SAT for every k can be solved substantially faster than 2^n (where n is the number of variables) via the resolution-based *PPSZ* algorithm [Paturi *et al.*, 2005]. There is a complementary theory of lower bounds where the central conjecture is that 3-SAT is not solvable in $2^{o(n)}$ time (*exponential-time hypothesis* (ETH) [Impagliazzo and Paturi, 2001]) and the *strong exponential-time hypothesis* (SETH) which implies that SAT with unrestricted clause length (CNF-SAT) cannot be solved in c^n time for any $c < 2$.

In contrast, the precise exponential time complexity of abduction is currently a blind spot, and *no* improved algorithms are known for the intractable cases. We thus issue a systematic attack on the complexity of abduction with a particular focus on the natural complexity parameter n, the number of variables in the knowledge base, sometimes supplemented by $|H|$ or $|M|$, the number elements in the hypothesis H or manifestation M. To obtain general results we primarily consider the setup where we are given a set of relations Γ (a *constraint language*) where the knowledge base of an instance is provided by a Γ-formula. We write $\mathrm{ABD}(\Gamma)$ for this problem and additionally also consider the variant where an explanation only consists of positive literals (P-$\mathrm{ABD}(\Gamma)$) since these two variants exhibit interesting differences. The classical complexity of abduction is either in P, NP-complete, coNP-complete, or Σ_2^P-complete [Nordh and Zanuttini, 2008], and the main question now is for which intractable Γ it is possible to beat exhaustive search. According to Cygan et al., tools to precisely analyze the exponential time complexity of NP-complete problems are in its infancy [2016]. For problems at higher levels of the polynomial hierarchy the situation is even more dire. Are algorithmic approaches for problems in NP still usable? Are the tools to obtain lower bounds still usable? Why are no sharp upper bounds known for problems in non-monotonic reasoning, and are these problems fundamentally different from e.g. satisfiability problems?

We successfully answer many of these questions. First, we show why enumerating all possible subsets of the hypothesis gives a bound 2^n for ABD and the (surprisingly bad) 3^n bound for P-ABD. Hence, any notion of improvement should be measured against 2^n for ABD and 3^n for P-ABD. Generally improving the factor $2^{|H|}$ (which may equal 2^n) seems difficult but we do manage this for languages Γ where all possible models of the knowledge base can be enumerated in c^n time, for some $c \leq 2$, which we call *sparsely enumerable* languages. We succeed with this for both $\mathrm{ABD}(\Gamma)$

*A full version has been accepted at IJCAI 2025.

and P-ABD(Γ), and while the algorithms for the two different cases share ideas, the details differ in intricate ways. It should be remarked that both algorithms solve the substantially more general problem of enumerating *all* (maximal) explanations which may open up further, e.g., probabilistic, applications for abduction. The enumeration algorithms in addition to exponential time also need exponential memory, but we manage to improve the naive 3^n bound for P-ABD(Γ) to 2^n with only polynomial memory. The sparsely enumerable property is strong: it fails even for 2-SAT and it is a priori not clear if it is ever true for intractable languages. Despite this, we manage to describe three properties implying sparse enumerability. This captures relations definable by equations $x_1 + \ldots + x_k = q \pmod p$ (EQUATIONS^k). The problem(s) (P-)ABD(EQUATIONS) is Σ_2^P-complete and is, to the best of our knowledge, the first example of beating exhaustive search for a Σ_2^P-complete problem (under n). This yields improved algorithms for Σ_2^P-complete P-ABD(XSAT) (*exact satisfiability*) and NP-complete P-ABD(AFF$^{(\leq k)}$) (arity bounded equations over GF(2)).

(Type) Class	Classical complexity	Improved
EQUATIONS^k ($k \geq 1$)	Σ_2^P-C	Yes
XSAT	Σ_2^P-C	$O^*(2^{\frac{n}{2}})$
(P) AFF^k ($k \geq 1$)	NP-C	Yes
(M) k-CNF$^+$ ($k \geq 1$)	NP-C	Yes
(P) k-CNF$^-$ $\cup$ IMP ($k \geq 1$)	NP-C	Yes
(P) finite 1-valid	coNP-C	Yes

Table 1: Upper bounds for P-ABD and ABD.

(Type) Class	Assumption	Bound
(M) 2-CNF$^+$	ETH	$(\frac{\|H\|}{\|M\|})^{o(\|M\|)}$
(P) 2-CNF$^+$ $\cup$ IMP	ETH	$(\frac{\|H\|}{\|M\|})^{o(\|M\|)}$
(M) k-CNF ($k \geq 4$)	SETH	2^n
(P) k-CNF ($k \geq 4$)	SETH	1.4142^n
CNF$^-$ $\cup$ IMP, Horn	SETH	1.2599^n
(M) CNF$^+$, DualHorn	SETH	1.2599^n

Table 2: Lower bounds for P-ABD and ABD.

We also consider more restricted types of abduction problems with a particular focus on (P-)ABD(k-CNF$^+$) where k-CNF$^+$ contains all positive clauses of arity k. Here, the problems are only NP-complete, in which case circumventing the $2^{|H|}$ barrier appears easier. For these, and similar problems, we construct an improved algorithm based on a novel reduction to a problem SIMPLESAT^p which can be solved by branching. For coNP, only P-ABD(Γ) becomes relevant, and we prove a simple but general improvement whenever a finite Γ is invariant under a constant Boolean operation.

We further prove lower bounds under (S)ETH for missing intractable cases. Let IMP $= \{\{(0,0),(0,1),(1,1)\}\}$. Under the ETH, we first prove that ABD(2-CNF$^+$) and ABD(2-CNF$^-$ $\cup$ IMP) cannot be solved in time $(\frac{|H|}{|M|})^{o(|M|)}$ under ETH, which asymptotically matches exhaustive search. For classical cases like k-CNF ($k \geq 4$) and NAE-k-SAT ($k \geq 5$) we establish sharp lower bounds of the form 2^n for ABD and 1.4142^n for P-ABD under the SETH. For (P-)ABD(CNF$^-$ $\cup$ IMP), we rule out improvements to 1.2599^n, $1.4142^{|H|}$, $2^{|M|}$, or $(|H|/|M|)^{|M|}$ under SETH. This transfers to Horn for (P-)ABD and to DualHorn for ABD. For ABD(2-CNF), we prove that sharp lower bounds under the SETH are unlikely unless NP $\subseteq$ P/Poly, leaving its precise fine-grained complexity as interesting open question.

The results are summarized in Table 1 and Table 2 where (P), respectively (M), indicates that the result only holds for P-ABD, respectively ABD. Thus, put together, we obtain a rather precise picture of the fine-grained complexity of ABD(Γ) and P-ABD(Γ) for almost all classical intractable languages Γ. Notably, we have proven that even Σ_2^P-complete problems can admit improved algorithms with respect to n, and that the barrier of exhaustively enumerating all possible explanations can be broken.

Concluding Remarks

We proved that propositional abduction, for many non-trivial cases *do* admit improvements over exhaustive search. We find it particularly interesting that even Σ_2^P-complete problems fall under the scope of our methods. Might it even be the case that Σ_2^P is not such an imposing barrier as classical complexity theory tells us? Nevertheless, despite many positive and negative results, there are still open cases remaining and many interesting directions for future research.

Faster Enumeration? We proved that finite subsets of AFF and EQUATIONS are susceptible to enumeration. It is easy to see that Pol(AFF) contains the *Maltsev* operation $x - y + z \pmod 2$, while EQUATIONS is exactly the set of symmetric relations invariant under a *partial Maltsev* operation [Lagerkvist and Wahlström, 2022]. Is this a coincidence, or could universal algebra be applied even further? For example, one can prove that if a language is *not* preserved by partial Maltsev, then it can not be sparsely enumerable. Extending this further, if one allows e.g. a polynomial-time preprocessing, could it even be the case that a Boolean (possibly non-symmetric) language is sparsely enumerable if and only if it is invariant under partial Maltsev?

2- and 3-CNF. While (P-)ABD(4-CNF) is unlikely to admit improved algorithms, (P-)ABD(k-CNF) for $k \leq 3$ is wide open. These languages are not sparsely enumerable, so the enumeration algorithms are not applicable. Yet, it appears highly challenging to prove sharp lower bounds for them (we have shown that CNF-SAT does not admit an LV-reduction to (P-)ABD(2-CNF) unless NP $\subseteq$ P/Poly).

Other Parameters? Related to the above question one could more generally ask when (P-)ABD(Γ) admits an improved algorithm with complexity parameter m (number of clauses/constraints), which we observe in general can be much larger than n. Do any of the algorithmic results carry over, and can lower bounds be obtained? For the related quantified Boolean formula problems, Williams [2002] constructed an $O(1.709^m)$ time algorithm, so one could be cautiously optimistic about analyzing abduction with m.

References

[Alberti *et al.*, 2005] Marco Alberti, Federico Chesani, Marco Gavanelli, Evelina Lamma, Paola Mello, and Paolo Torroni. Security protocols verification in abductive logic programming: A case study. In *Engineering Societies in the Agents World VI, 6th Internat. Workshop, ESAW'05*, pages 106–124, 2005.

[Cygan *et al.*, 2016] Marek Cygan, Holger Dell, Daniel Lokshtanov, Dániel Marx, and Jesper Nederlof et al. On problems as hard as CNF-SAT. *ACM Trans. Algorithms*, 12(3):41:1–41:24, 2016.

[Ignatiev, 2020] Alexey Ignatiev. Towards trustable explainable AI. In Christian Bessiere, editor, *Proc. 29th Internat. Joint Conf. on Artificial Intelligence (IJCAI'20)*, pages 5154–5158. ijcai.org, 2020.

[Impagliazzo and Paturi, 2001] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-SAT. *J. Comput. Syst. Sci*, 62(2):367 – 375, 2001.

[Inoue *et al.*, 2009] Katsumi Inoue, Taisuke Sato, Masakazu Ishihata, Yoshitaka Kameya, and Hidetomo Nabeshima. Evaluating abductive hypotheses using an EM algorithm on bdds. In *Proc. 21st Internat. Joint Conf. on Artificial Intelligence (IJCAI'09)*, pages 810–815, 2009.

[Lagerkvist and Wahlström, 2022] Victor Lagerkvist and Magnus Wahlström. The (coarse) fine-grained structure of NP-hard SAT and CSP problems. *ACM Trans. Comput. Theory.*, 14(1), 2022.

[Nordh and Zanuttini, 2008] Gustav Nordh and Bruno Zanuttini. What makes propositional abduction tractable. *Artif. Intell.*, 172(10):1245–1284, 2008.

[Obeid *et al.*, 2019] Mariam Obeid, Zeinab Obeid, Asma Moubaiddin, and Nadim Obeid. Using description logic and abox abduction to capture medical diagnosis. In *Proc. 32nd Internat. Conf. on Industrial, Engineering and Other Applications of Applied Intelligent Systems, IEA/AIE 2019*, pages 376–388, 2019.

[Paturi *et al.*, 2005] Ramamohan Paturi, Pavel Pudlák, Michael E. Saks, and Francis Zane. An improved exponential-time algorithm for *k*-sat. *J. ACM*, 52(3):337–364, 2005.

[Racharak and Tojo, 2021] Teeradaj Racharak and Satoshi Tojo. On explanation of propositional logic-based argumentation system. In *Proc. 13th Internat. Conf. on Agents and Artificial Intelligence (ICAART'21)*, pages 323–332, 2021.

[Ray *et al.*, 2006] Oliver Ray, Athos Antoniades, Antonis C. Kakas, and Ioannis Demetriades. Abductive logic programming in the clinical management of HIV/AIDS. In *Proc. 17th European Conf. on Artificial Intelligence*, pages 437–441, 2006.

[Sovrano *et al.*, 2020] Francesco Sovrano, Fabio Vitali, and Monica Palmirani. Modelling GDPR-compliant explanations for trustworthy AI. In *Proc. 9th Internat. Conf. on Electronic Government and the Information Systems Perspective (EGOVIS'20)*, pages 219–233, 2020.

[Williams, 2002] Ryan Williams. Algorithms for quantified boolean formulas. In *Proc. 13th Annual ACM-SIAM Symp. on Discrete Algorithms (SODA'02)*, page 299–307, 2002.

A Unifying Framework for Semiring-Based Constraint Logic Programming With Negation

Jeroen Spaans[1] and **Jesse Heyninck**[2]
[1]Open Universiteit, The Netherlands
[1,2]University of Cape Town, South Africa
{jeroen.spaans, jesse.heyninck}@ou.nl

Authors' Note

In this document you will find an extended abstract of a full paper accepted at IJCAI 2025. A complete copy of the work, including proofs and appendices, will be made freely available on arXiv prior to the event.

1 Extended Abstract

Many problems require the consideration of constraints; examples of such problems can be relatively simple, like sudokus and similar puzzles, or more complex, like the real-world applications of resource allocation and automated planning and scheduling. Classically, a problem as such may be formulated as a Constraint Satisfaction Problem (CSP) and solved using Constraint Logic Programming (CLP). We investigate an extension of CLP capable of handling semiring-based constraints with negation.

Consider an informal example constraint logic program, which describes the allocation of a limited number of working hours to two different tasks.

```
1  % We define two tasks,
2  % taking 6 and 4 hours to complete respectively.
3  task(t1, 6).
4  task(t2, 4).
5  % A task is completed if enough time is scheduled.
6  completed(Task, HoursScheduled) :-
7     task(Task, HoursRequired),
8     HoursRequired ≤ HoursScheduled.
9  % We set a time limit of eight hours.
10 inTimeLimit(Hours1, Hours2) :-
11    Hours1 + Hours2 ≤ 8.
12 % A schedule is evaluated by the degree to which both tasks
13 % are completed and the total allotted time does not exceed
14 % the time available.
15 schedule(HoursTask1, HoursTask2) :-
16    completed(t1, HoursTask1),
17    completed(t2, HoursTask2),
18    inTimeLimit(HoursTask1, HoursTask2).
```

Using classical answer set programming semantics to evaluate this program with the goal `schedule(HoursTask1, HoursTask2)` returns *false*. Intuitively, this happens because no schedule can complete the two tasks—totalling ten hours of work—in less than eight hours. Knowing that the two tasks cannot both be completed in the time available, we may wish to optimize their partial completion instead. One way to do this is to replace *false* and *true* with values in $[0, 1]$ (where 1 represents complete truth and 0 complete falsity), replace *or* with $\max$, replace *and* with $\min$, and replace `HoursRequired ≤ HoursScheduled` with `HoursScheduled / HoursRequired`.

Our example demonstrated that CLP is limited to strict satisfaction or violation of constraints, and that we require an alteration of this framework to solve problems of constraint optimization. The same holds—for example—for problems involving fuzziness, uncertainty, or probability.

Bistarelli et al. [Bistarelli, 2004; Bistarelli *et al.*, 1997; Bistarelli *et al.*, 2001] proposed Semiring-based Constraint Logic Programming, a generalization of CLP replacing the Boolean evaluation domain and the associated logical *and* and *or* connectives with semirings—algebraic structures consisting of a set equipped with an additive operator for disjunction and a multiplicative operator for conjunction—much like we did in our example. Since, many related formalisms have likewise been extended to the semiring setting [Belle and Raedt, 2020; Eiter and Kiesel, 2020a; Eiter and Kiesel, 2020b; Green *et al.*, 2007; Khamis *et al.*, 2024; Kimmig *et al.*, 2011; Kimmig *et al.*, 2017] to certain success. Each of these works makes some assumptions about the semirings used, but what exactly those assumptions are and how they relate is left implicit or has not been studied. Herein lies the first major contribution of this work; we perform an analysis of the various families of semirings in relation to semiring-based semantics for CLP, paying special attention to the orderings each family gives rise to.

While some of the above-mentioned works permit negation, most do not, and a general analysis of negation in the semiring setting is so far absent from the literature. Herein lies the second major contribution of this work; a semiring-agnostic form of negation—based on negation in Gödel logics [Gödel, 1932], and also used in the semiring-based formalism of Eiter and Kiesel [Eiter and Kiesel, 2020b]—is proposed and the effects of its addition on the semantics of semiring-based constraint logic programming are studied. The addition of negation gives us the expressive power to, for example, make the completion of a task in our scheduling problem contingent on the absence of blocking factors by replacing lines 5 through 8 of our example program with the following.

```
% A task is completed if enough time is scheduled
% and its completion is not blocked.
completed(Task, HoursScheduled) :-
   task(Task, HoursRequired),
   HoursRequired ≤ HoursScheduled,
   not blocked(Task).
```

Notably, and as is to be expected, the addition of negation leads to nonmonotonicity of the immediate consequence operator. To work around this problem we capture the new negation-permitting formalism in Approximation Fixpoint Theory [Denecker *et al.*, 2001], endowing it with AFT's various semantics like Kripke-Kleene, Well-founded, and Stable, which generalize the semantics of normal logic programs.

Concretely, the contributions of this work come in five parts: first, a novel notion of model—considering all contributing clauses at once, and specific to the semiring-based setting—is introduced and compared to the traditional notion of model which considers each clause's satisfaction separately. The minimal model semantics based on these notions of model are then compared with the least fixpoint semantics based on an immediate consequence operator. Next, we investigate a generalized method for deriving orderings of semiring elements needed to define models and least fixpoints—but also needed in the later application of approximation fixpoint theory—and find it to be a generalization of the method studied by Bistarelli et al. 2001. Then, we study a generalized notion of negation appearing at various points in the literature as applied to our semiring-based framework. Finally, we apply approximation fixpoint theory to our immediate consequence operator—made nonmonotonic by the addition of negation—to define Kripke-Kleene, and well-founded and other stable semantics, studying both ultimate approximation and a novel approximator.

This work studies semiring-based semantics for constraint logic programming with negation, generalizing the approaches of [Bistarelli *et al.*, 2001] and [Khamis *et al.*, 2023]. Computational complexity, implementations, and applying AFT-based notions such as stratification [Vennekens *et al.*, 2004], conditional independence [Heyninck, 2024] and non-determinism [Heyninck *et al.*, 2024] are left as future work.

References

[Belle and Raedt, 2020] Vaishak Belle and Luc De Raedt. Semiring programming: A semantic framework for generalized sum product problems. *Int. J. Approx. Reason.*, 126:181–201, 2020.

[Bistarelli *et al.*, 1997] Stefano Bistarelli, Ugo Montanari, and Francesca Rossi. Semiring-based constraint satisfaction and optimization. *J. ACM*, 44(2):201–236, 1997.

[Bistarelli *et al.*, 2001] Stefano Bistarelli, Ugo Montanari, and Francesca Rossi. Semiring-based contstraint logic programming: Syntax and semantics. *ACM Trans. Program. Lang. Syst.*, 23(1):1–29, 2001.

[Bistarelli, 2004] Stefano Bistarelli. *Semirings for Soft Constraint Solving and Programming*, volume 2962 of *Lecture Notes in Computer Science*. Springer, 2004.

[Denecker *et al.*, 2001] Marc Denecker, Victor Marek, and Miros Truszczynski. Approximations, Stable Operators, Well-Founded Fixpoints And Applications In Nonmonotonic Reasoning. *Logic-based Artificial Intelligence*, December 2001.

[Eiter and Kiesel, 2020a] Thomas Eiter and Rafael Kiesel. ASP(AC): Answer Set Programming with Algebraic Constraints. *Theory Pract. Log. Program.*, 20(6):895–910, 2020.

[Eiter and Kiesel, 2020b] Thomas Eiter and Rafael Kiesel. Weighted LARS for Quantitative Stream Reasoning. In *ECAI 2020 - 24th European Conference on Artificial Intelligence, 29 August-8 September 2020, Santiago de Compostela, Spain, August 29 - September 8, 2020 - Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020)*, pages 729–736, 2020.

[Green *et al.*, 2007] Todd J. Green, Gregory Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the Twenty-Sixth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 11-13, 2007, Beijing, China*, pages 31–40, 2007.

[Gödel, 1932] Kurt Gödel. Zum intuitionistischen aussagenkalkul. *Anzeiger Akademie der Wissenschaften Wien, mathematisch-naturwissenschaftliche Klasse*, 69:65–66, 1932.

[Heyninck *et al.*, 2024] Jesse Heyninck, Ofer Arieli, and Bart Bogaerts. Non-deterministic approximation fixpoint theory and its application in disjunctive logic programming. *Artificial Intelligence*, 331:104110, 2024.

[Heyninck, 2024] Jesse Heyninck. An algebraic notion of conditional independence, and its application to knowledge representation (full version). *arXiv preprint arXiv:2412.13712*, 2024.

[Khamis *et al.*, 2023] Mahmoud Abo Khamis, Hung Q. Ngo, Reinhard Pichler, Dan Suciu, and Yisu Remy Wang. Convergence of Datalog over (Pre-) Semirings. *SIGMOD Rec.*, 52(1):75–82, 2023.

[Khamis *et al.*, 2024] Mahmoud Abo Khamis, Hung Q. Ngo, Reinhard Pichler, Dan Suciu, and Yisu Remy Wang. Convergence of datalog over (Pre-) Semirings. *J. ACM*, 71(2):8:1–8:55, April 2024.

[Kimmig *et al.*, 2011] Angelika Kimmig, Guy Van den Broeck, and Luc De Raedt. An Algebraic Prolog for Reasoning about Possible Worlds. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2011, San Francisco, California, USA, August 7-11, 2011*, pages 209–214, 2011.

[Kimmig *et al.*, 2017] Angelika Kimmig, Guy Van den Broeck, and Luc De Raedt. Algebraic model counting. *J. Appl. Log.*, 22:46–62, 2017.

[Vennekens *et al.*, 2004] Joost Vennekens, David Gilis, and Marc Denecker. Splitting an operator: Algebraic modularity results for logics with fixpoint semantics. *CoRR*, cs.AI/0405002, 2004.

Facets in Argumentation: A Formal Approach to Argument Significance

Yasir Mahmood[1] , **Jonathan Persson**[2] , **Victor Lagerkvist**[2] , **Nicolas Fröhlich**[3] , **Arne Meier**[3] , **Markus Hecher**[4] , **Johannes K. Fichte**[2]

[1]DICE group, Paderborn University, Germany
[2]Linköping University, Sweden
[3]Institut für Theoretische Informatik, Leibniz Universität Hannover, Germany
[4]Artois University, CRIL,CNRS Lens, France

Introduction

Abstract argumentation [Dung, 1995; Bench-Capon and Dunne, 2007] is a formalism for modeling and evaluating arguments and its reasoning problems has many applications in artificial intelligence (AI) [Amgoud and Prade, 2009; Rago *et al.*, 2018]. The semantics is based on sets of arguments (called *extensions*) satisfying certain conditions regarding their mutual relationship, such as being stable (every argument is either accepted, or attacked by an accepted argument) or admissible [Dung, 1995]. To compute such extensions of a framework, various practical solvers for decision and reasoning tasks [Egly *et al.*, 2008; Niskanen and Järvisalo, 2020; Thimm *et al.*, 2021; Alviano, 2021] compete biennially in the ICCMA competition [Thimm *et al.*, 2024]. Qualitative reasoning tasks (e.g., extension computation, credulous or skeptical acceptance) are computationally efficient [Dvořák, 2012], but offer limited insight into argument preferences. To address this, enumeration, counting, and quantitative reasoning have been explored and classified [Fichte *et al.*, 2023; Fichte *et al.*, 2024], enabling probabilistic analysis. While enumeration is feasible with few extensions, many semantics produce a large number of them. Nonetheless, users may wish to explore extension spaces further—for example, by restricting or diversifying extensions, identifying resilient arguments, performing sanity checks, evaluating LLM-generated frameworks, or deriving explanations. Assessing the significance of individual arguments within a framework is central. Existing approaches compute significance using quantitative measures over extensions that contain specific arguments or support particular claims. This typically involves counting extensions—an inherently costly task [Fichte *et al.*, 2024]. Example 1 illustrates the challenges of comparing argument significance across an extension space.

Example 1. *Consider the AF F, depicted in Figure 1. Intuitively, a preference for savory flavors excludes choices like maple syrup, just as sweet opposes burrito. Maple syrup or small portions are unexpected at Taco Bell. Burritos, though can be made at home, doing so requires expensive ingredients—avoided by making either small portion or opting for Taco Bell. Stable extensions of F are: $\{W, M, P\}$, $\{S, B, P\}$, $\{S, B, T\}$. Here, it is not immediate to compare the significance of accepting/rejecting certain arguments to each other.*

In this paper, we propose a notion of facets defined as arguments accepted in some extension (credulous) but not by all extensions (skeptical). Facets quantify the uncertainty of arguments in extensions, providing a measure of their indeterminacy within the framework. Furthermore, they can be utilized to evaluate the significance of specific arguments. Example 2 provides a brief intuition.

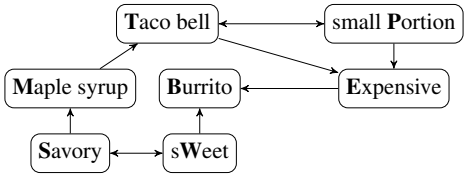


Figure 1: An example argumentation framework.

Example 2. *Reconsider Example 1 where six of the seven arguments are facets under stable semantics, the only exception being E. To compare the relative significance, consider stable extensions of F rejecting the argument "s**W**eet". This includes $\{S, B, P\}$ and $\{S, B, T\}$, thus leaving us with two facets P and T. In contrast,* accepting *"s**W**eet" gives a single stable extension $\{W, M, P\}$ and hence no facets. Accordingly, accepting W eliminates any uncertainty, and we consider accepting "s**W**eet" as more significant than rejecting it.*

Contributions. We introduce facets to abstract argumentation as a reasoning tool for significance and filtering extensions. Furthermore, we present a comprehensive complexity analysis (Table 1) for various qualitative and quantitative problems involving facets. Finally, we present experiments that demonstrate the feasibility of our framework and evaluate our implementation on instances of the ICCMA competition.

Related Work. Facets were initially proposed for answer-set programming (ASP) by Alrabbaa *et al.* [2018] as a tool to navigate large solution spaces and their complexity has been systematically classified [Rusovac *et al.*, 2024]. Note that ASP-navigation is based on forbidding or enforcing atoms in a program using integrity constraints. In contrast, argumentation facets enable approving or disapproving arguments, while not necessarily removing the extensions entirely leading to a natural notion of significance of an argument. The computational complexity of credulous and skeptical reasoning is well studied [Dvořák, 2012]. Here, we ask for the concrete complexity of facets and whether it provides a theoretical benefit

Problems/σ	σ_1	σ_2	σ_3
IsFacet$_\sigma$	**P**	**NP**	$\boldsymbol{\Sigma}_2^{\mathbf{P}}$
Facets$_\sigma^{\geq k}$	**P**	**NP**	$\boldsymbol{\Sigma}_2^{\mathbf{P}}$
Facets$_\sigma^{\leq k}$	**P**	co**NP**	$\boldsymbol{\Pi}_2^{\mathbf{P}}$
Facets$_\sigma^{=k}$	**P**	**DP**	$\in \mathbf{DP}_2$
$\mathbb{S}_\sigma[F, \ell]$	"$\in \mathbf{P}$"	"$\in \boldsymbol{\Delta}_2^{\mathbf{P}}$"	"$\in \boldsymbol{\Delta}_3^{\mathbf{P}}$"

Table 1: A complexity overview for semantics $\sigma_1 \in \{\mathsf{conf}, \mathsf{naiv}\}$, $\sigma_2 \in \{\mathsf{adm}, \mathsf{stab}, \mathsf{comp}\}$, and $\sigma_3 \in \{\mathsf{pref}, \mathsf{semiSt}, \mathsf{stag}\}$. IsFacet$_\sigma$ asks whether a given argument is a facet; Facets$_\sigma^{\cdot k}$ asks whether there are at least ($\geq k$), at most ($\leq k$), or exactly ($= k$) facets. $\mathbb{S}_\sigma[F, \ell]$ asks what the significance of approving ($\ell = a$) or disapproving ($\ell = \bar{a}$) of a σ-facet a in AF F is. "$\in \boldsymbol{\Delta}_i^{\mathbf{P}}$" slightly abuses notation meaning that it can be computed by a deterministic polynomial-time Turing machine with access to a $\boldsymbol{\Sigma}_{i-1}^{\mathbf{P}}$ oracle.

over projected counting and enumerating extensions.

Facet Reasoning

We assuming basic familiarity with mostly studied argumentation semantics, denoted henceforth by σ. Given an AF F, $\sigma(F)$ denotes the set of all σ-extensions of F. Moreover, $\mathsf{C}_\sigma = \bigcup_{E \in \sigma(F)} E$ denotes credulously, and $\mathsf{S}_\sigma = \bigcap_{E \in \sigma(F)} E$ denotes skeptically accepted arguments of F under σ. A σ-facet is an argument which is accepted in some, but not all σ-extensions. Formally, an argument a is a *σ-facet* if $a \in \mathsf{C}_\sigma \setminus \mathsf{S}_\sigma$. Given an AF F and semantics σ, $\mathcal{F}_\sigma(F)$ denotes the set of all σ-facets in F. We consider the following reasoning problems: The problem IsFacet$_\sigma$ asks, given an AF $F = (A, R)$ and an argument $a \in A$, is a a σ-facet in F? The problems Facets$_\sigma^{=k}$, Facets$_\sigma^{\geq k}$ and Facets$_\sigma^{\leq k}$ has an integer k as additional input, and ask whether an input $F = (A, R)$ has exactly, at least, or at most k σ-facets, respectively.

Significance via Facets Our notion of significance adopts a decision-driven perspective. We define significance of arguments in terms of the influence of a decision to eliminate the degree of freedom (on choices of remaining arguments), which actually complements counting/conditional probability. Observe that, counting approaches assess the plausibility of arguments in terms of their likelihood of being accepted (global perspective on all extensions). However, we measure how much the acceptance of an argument decreases freedom (or increases the significance of the decision). Intuitively, a higher significance score indicates that a specific decision does have a huge influence on the remaining facets. Furthermore, the acceptance or rejection of arguments across extensions reflects inherent uncertainty in an AF, with the number of facets indicating its degree. For an argument a denote by $\bar{a}$ the complement/negation of a. E.g., an argument is approved (a) versus not approved ($\bar{a}$). A facet $\ell \in \{a, \bar{a}\}$ can be seen as the *uncertainty* regarding a, since a can either be included in, or excluded from extensions. Let F be an AF and σ be a semantics. For an argument a, let $\sigma^a(F)$ denotes the σ-extensions in F *approving* the argument a defined as $\sigma^a(F) = \{E \in \sigma(F) \mid a \in E\}$. Likewise, $\sigma^{\bar{a}}(F) = \{E \in \sigma(F) \mid a \notin E\}$ represents σ-extensions in F *disapproving* a. Now, let C_σ^a (resp., S_σ^a) be the arguments in some (all) $E \in \sigma^a(F)$ and $\mathcal{F}_\sigma^a(F)$ denotes the σ-facets by considering extensions in $\sigma^a(F)$ (i.e., $\mathsf{C}_\sigma^a \setminus \mathsf{S}_\sigma^a$).

Approving a facet $\ell \in \{a, \bar{a}\}$ reduces the uncertainty regarding the remaining arguments in F by restricting the extensions space to sets (not) containing a. Further, approving ℓ can render a facet argument $b \in A$ non-facet. This holds since, either $b \in E$ for each $E \in \sigma^\ell(F)$ but $b \notin E$ for each $E \in \sigma(F)$, or $b \notin E$ for any $E \in \sigma^\ell(F)$ but $b \in E$ for some $E \in \sigma(F)$. Intuitively, we say that the uncertainty of such an argument b has been *resolved* by approving ℓ.

Let σ be a semantics, $a \in A$ be a σ-facet and $\ell \in \{a, \bar{a}\}$. The observation that "ℓ reduces the uncertainty among remaining arguments" leads to the notion of *significance* of ℓ under semantics σ. For an AF F, we define: $\mathbb{S}_\sigma[F, \ell] := |\mathcal{F}_\sigma(F)| - \left|\mathcal{F}_\sigma^\ell(F)\right| / |\mathcal{F}_\sigma(F)|$. Intuitively, approving an argument a is less significant if many uncertain arguments (facets) remain in $\mathcal{F}_\sigma^a(F)$. Similarly, disapproving a (and thus approving $\bar{a}$) is less significant if many facets remain in $\mathcal{F}_\sigma^{\bar{a}}(F)$.

$\ell \in$	$\{W, M, T, \bar{S}, \bar{B}, \bar{P}\}$	$\{S, B, \bar{W}, \bar{M}\}$	$\{P, \bar{T}\}$
$\left\lvert\mathcal{F}_{\mathsf{stab}}^\ell(F)\right\rvert$	0	2	4
$\mathbb{S}_{\mathsf{stab}}[F, \ell]$	1	$\frac{2}{3}$	$\frac{1}{3}$

Table 2: Argument significance for the AF from Example 3.

Example 3 (Arguments Significance)**.** *Reconsider the AF F from Example 1 with stable extensions* $\mathsf{stab}(F) = \{\{W, M, P\}, \{S, B, P\}, \{S, B, T\}\}$*. While Example 2 gave an intuition of significance, Table 2 presents precise values for each argument. As outlined, the argument W has score* 1*, and is thus more significant than $\bar{W}$ (score* $2/3$*).*

Conclusion.

We defined a new perspective on exploring significance of arguments in extensions of an abstract argumentation framework. We present a comprehensive complexity analysis for deciding whether an argument is a facet (IsFacet) and deciding whether an argumentation framework has at least k (Facets$_\sigma^{\geq k}$), at most k (Facets$_\sigma^{\leq k}$), and exactly k facets (Facets$_\sigma^{=k}$). We establish that the complexity ranges between **P** and $\mathbf{DP}_2$, including tight lower bounds for most cases (see Table 1). While our primary focus lies on establishing a comprehensive complexity picture, our implementation allows computing the number of facets practically for concrete abstract argumentation frameworks building on top of existing solvers. Thereby, we demonstrate that we can practically compute facets as well and our new notions are ready to use.

For future work, we plan to investigate techniques whether significance originating from facets can be extended to arguments depending on each other and notions of fairness in argumentation frameworks. Moreover, we aim to explore facets in three-valued variants of complete and preferred semantics [Wu *et al.*, 2010], which distinguish rejected arguments by whether they are undefended or attacked by accepted arguments. From a practical perspective, we believe that it would be interesting to integrate facet-based reasoning and significance computation into modern SAT-based argumentation solvers.

Acknowledgments

The work has received funding from the Austrian Science Fund (FWF), grants J 4656 and P 32830, the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation), grants TRR 318/1 2021 – 438445824 and ME 4279/3-1 (511769688), the European Union's Horizon Europe research and innovation programme within project ENEXA (101070305), the Society for Research Funding in Lower Austria (GFF, Gesellschaft für Forschungsförderung NÖ), grant ExzF-0004, the Swedish research council under grant VR-2022-03214, as well as the Vienna Science and Technology Fund (WWTF), grant ICT19-065, and by the Ministry of Culture and Science of North Rhine-Westphalia (MKW NRW) within project WHALE (LFN 1-04) funded under the Lamarr Fellow Network programme, and ELLIIT funded by the Swedish government.

References

[Alrabbaa *et al.*, 2018] Christian Alrabbaa, Sebastian Rudolph, and Lukas Schweizer. Faceted answer-set navigation. In *RuleML+RR'18*, pages 211–225. Springer Verlag, 2018.

[Alviano, 2021] Mario Alviano. The pyglaf argumentation reasoner (ICCMA2021). *CoRR*, abs/2109.03162, 2021.

[Amgoud and Prade, 2009] Leila Amgoud and Henri Prade. Using arguments for making and explaining decisions. *Artificial Intelligence*, 173(3-4):413–436, 2009.

[Bench-Capon and Dunne, 2007] Trevor J.M. Bench-Capon and Paul E. Dunne. Argumentation in artificial intelligence. *Artificial Intelligence*, 171(10–15):619–641, 2007.

[Dung, 1995] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.

[Dvořák, 2012] Wolfgang Dvořák. *Computational aspects of abstract argumentation*. PhD thesis, TU Wien, 2012.

[Egly *et al.*, 2008] Uwe Egly, Sarah Alice Gaggl, and Stefan Woltran. Aspartix: Implementing argumentation frameworks using answer-set programming. In Maria Garcia de la Banda and Enrico Pontelli, editors, *Logic Programming*, pages 734–738, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

[Fichte *et al.*, 2023] Johannes K. Fichte, Markus Hecher, Yasir Mahmood, and Arne Meier. Quantitative reasoning and structural complexity for claim-centric argumentation. In *IJCAI'23*, pages 3212–3220. ijcai.org, 2023.

[Fichte *et al.*, 2024] Johannes K. Fichte, Markus Hecher, and Arne Meier. Counting complexity for reasoning in abstract argumentation. *JAIR*, 80, 2024.

[Niskanen and Järvisalo, 2020] Andreas Niskanen and Matti Järvisalo. μ-toksia: An Efficient Abstract Argumentation Reasoner. In *KR'20*, pages 800–804, 9 2020.

[Rago *et al.*, 2018] Antonio Rago, Oana Cocarascu, and Francesca Toni. Argumentation-based recommendations: Fantastic explanations and how to find them. In *IJCAI'18*, pages 1949–1955. IJCAI Organization, 2018.

[Rusovac *et al.*, 2024] Dominik Rusovac, Markus Hecher, Martin Gebser, Sarah Alice Gaggl, and Johannes K. Fichte. Navigating and querying answer sets: How hard is it really and why? In Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco, editors, *KR'24*, 2024.

[Thimm *et al.*, 2021] Matthias Thimm, Federico Cerutti, and Mauro Vallati. Fudge: A light-weight solver for abstract argumentation based on SAT reductions. *CoRR*, abs/2109.03106, 2021.

[Thimm *et al.*, 2024] Matthias Thimm, Jean-Guy Mailly, and Johannes P. Wallner. International competitions on computational models of argumentation. https://argumentationcompetition.org/2025/index.html, 2024.

[Wu *et al.*, 2010] Yining Wu, Martin Caminada, and Mikotaj Podlaszewski. A labelling-based justification status of arguments. *Studies in Logic*, 3(4):12–29, 2010.

Inconsistency Handling in DatalogMTL (Extended Abstract)

Meghyn Bienvenu[1] , **Camille Bourgaux**[2] , **Atefe Khodadaditaghanaki**[3]
[1]Université de Bordeaux, CNRS, Bordeaux INP, LaBRI, UMR 5800, Talence, France
[2]DI ENS, ENS, CNRS, PSL University & Inria, Paris, France
[3]University of Paderborn, Paderborn, Germany
meghyn.bienvenu@u-bordeaux.fr, camille.bourgaux@ens.fr, atefe@mail.uni-paderborn.de

Abstract

This extended abstract summarizes our IJCAI'25 paper [Bienvenu *et al.*, 2025] on inconsistency handling in DatalogMTL, an extension of Datalog with metric temporal operators.

Introduction There has been significant recent interest in formalisms for reasoning over temporal data [Artale *et al.*, 2017]. Since its introduction by Brandt *et al.* [2017; 2018], the DatalogMTL language, which extends Datalog [Abiteboul *et al.*, 1995] with operators from metric temporal logic (MTL) [Koymans, 1990], has risen to prominence. In DatalogMTL, facts are annotated by *time intervals* on which they are valid (e.g., $R(a,b)@[1,5]$) and rules may use atoms involving metric temporal logic operators like $\boxplus_{[t_1,t_2]}\ \varphi$ and $\diamondplus_{[t_1,t_2]}\ \varphi$ (which hold at time t if φ holds at all, resp. some, timepoints in the interval $[t+t_1, t+t_2]$) to express dependencies between such facts (e.g., $\boxplus_{[0,2]}\ Q \leftarrow \diamondplus_{[3,3]}\ P$ states that if P holds at time $t-3$, Q holds from t to $t+2$). The complexity of reasoning in DatalogMTL has been investigated for various fragments and extensions and for different semantics (continuous vs pointwise, rational vs integer timeline) [Brandt *et al.*, 2018; Walega *et al.*, 2019; Ryzhikov *et al.*, 2019; Walega *et al.*, 2020b; Walega *et al.*, 2023a; Walega *et al.*, 2024]. Moreover, there are also several implemented reasoning systems for (fragments of) DatalogMTL [Kalayci *et al.*, 2019; Wang *et al.*, 2022; Wang *et al.*, 2024; Bellomarini *et al.*, 2022; Walega *et al.*, 2023b; Ivliev *et al.*, 2024].

One important issue that has yet to be addressed is how to handle the case where the temporal dataset is inconsistent with the DatalogMTL program. Indeed, it is widely acknowledged that real-world data typically contains many erroneous or inaccurate facts, and this is true in particular for temporal sensor data, due to faulty sensors. In such cases, classical logical semantics is rendered useless, as every query is entailed from a contradiction. A prominent approach to obtain meaningful information from an atemporal dataset that is inconsistent w.r.t. a logical theory (e.g., an ontology or a set of database integrity constraints) is to use *inconsistency-tolerant semantics* based on *subset repairs*, which are maximal subsets of the dataset consistent with the theory [Bertossi, 2019; Bienvenu, 2020]. The *consistent query answering (CQA)* approach considers that a (Boolean) query is true if it holds w.r.t. every repair [Arenas *et al.*, 1999; Lembo *et al.*, 2010]. Other natural semantics have also been proposed, such as the *brave* semantics, under which a query is true if it holds w.r.t. at least one repair [Bienvenu and Rosati, 2013], and the *intersection* semantics which evaluates queries w.r.t. the intersection of all repairs [Lembo *et al.*, 2010]. It is also useful to consider the minimal subsets of the dataset that are inconsistent with the theory, called *conflicts*, to explain the inconsistency to a user or help with debugging.

It is natural to extend these notions to the temporal setting. First work in this direction was undertaken by Bourgaux *et al.* [2019], who considered queries with linear temporal logic (LTL) operators, an atemporal DL-Lite ontology, and a sequence of datasets stating what holds at different timepoints. In that work, however, it was clear how to transfer definitions from the atemporal setting, and the main concerns were complexity and algorithms. By contrast, in DatalogMTL, facts are annotated with time intervals, which may contain exponentially or even infinitely many timepoints (if the timeline is dense or $\infty/-\infty$ can be used as interval endpoints). One can therefore imagine multiple different ways of minimally repairing an inconsistent dataset. For example, if a dataset states that P is true from 0 to 4 and Q from 2 to 6 ($P@[0,4]$, $Q@[2,6]$), and a rule states that P and Q cannot hold at the same time ($\bot \leftarrow P \wedge Q$), one can regain consistency by removing one of the two facts, adjusting their intervals, or treating intervals as sets of points and conserving as much information as possible. Similarly, there can be multiple ways of defining conflicts to identify minimal parts of the dataset responsible for inconsistency.

Repairs and Conflicts in DatalogMTL Our first contribution is to explore how the basic notions of repair and conflict, which are well studied in the atemporal setting, can be suitably adapted to DatalogMTL. We first define three different notions of repair of a dataset $\mathcal{D}$ w.r.t. a DatalogMTL program Π. We omit the formal definitions, which are somewhat tedious, as we must put datasets into a normal form and consider different ways of manipulating and comparing sets of DatalogMTL facts. Instead we give the main intuitions:

- Subset or s-repairs: we view $\mathcal{D}$ as a set of facts and delete a minimal subset to regain consistency with Π
- Pointwise or p-repairs: we view $\mathcal{D}$ as the possibly infinite set of the punctual facts it represents,

$$\{R(\vec{a})@[t,t] \mid t \in [t_1,t_2], R(\vec{a})@[t_1,t_2] \in \mathcal{D}\},$$

then minimally remove punctual facts until consistency with Π is achieved

- Interval-based or i-repairs: we consider datasets obtained from $\mathcal{D}$ by replacing each $R(\vec{a})@[t_1, t_2] \in \mathcal{D}$ by a fact $R(\vec{a})@[t'_1, t'_2]$ whose interval $[t'_1, t'_2]$ is included in $[t_1, t_2]$, or by nothing (we retain the option to delete a fact entirely), then compare such datasets w.r.t. how much information they retain, selecting the maximal ones consistent with Π

While p-repairs achieve the maximum preservation of information, an oft-desired property, they can lead to a single original fact being replaced by (possibly infinitely) many component facts, so repairs might be much larger in size than the original dataset. Both s- and i-repairs guarantee, by definition, that the number of facts does not increase, with i-repairs striking a nice balance between preserving information and respecting the structure of the original dataset. In the same way, we can define s-conflicts, p-conflicts, and i-conflicts of an inconsistent DatalogMTL knowledge base. Furthermore, we can use the new notions of repair to transfer existing definitions of repair-based semantics to DatalogMTL, yielding x-brave, x-CQA, and x-intersection semantics for $x \in \{s, p, i\}$.

We study the formal properties of these notions. While we show that s-repairs and s-conflicts possess similar properties to their atemporal analogs, p- and i-conflicts and repairs behave rather differently. In particular, we show that p- and i-conflicts and repairs are not guaranteed to exist. Even when they do, p-repairs and p-conflicts might contain infinitely many facts, and some datasets might only give rise to p-repairs and p-conflicts of infinite size. Moreover, for both $x = i$ and $x = p$, there can be infinitely many x-repairs / x-conflicts. One way to circumvent these negative results is to adopt the $\mathbb{Z}$ timeline and restrict datasets to only use *bounded intervals* (i.e., finite integers as endpoints).

Data Complexity Analysis Our second contribution is a data complexity analysis of the main computational tasks: recognizing x-conflicts and x-repairs, generating a single x-conflict or x-repair, and testing query entailment under the x-brave, x-CQA, and x-intersection semantics. For this initial study, we focus on the two cases where x-repairs are guaranteed to exist: (i) $x = s$, and (ii) bounded datasets over $\mathbb{Z}$.

We recall that in DatalogMTL, consistency checking and query entailment are PSPACE-complete w.r.t. data complexity [Walega *et al.*, 2019], and PSPACE-completeness holds for many fragments (such as core and linear) [Walega *et al.*, 2020b] as well as for DatalogMTL over $\mathbb{Z}$ [Walega *et al.*, 2020a]. We also consider some *tractable fragments* for which these tasks can be performed in PTIME w.r.t. data complexity: $\text{Datalog}_{\text{nr}}$MTL, $\text{DatalogMTL}^{\diamondsuit}_{\mathsf{core}}$, and $\text{DatalogMTL}^{\diamondsuit}_{\mathsf{lin}}$ (over $\mathbb{Q}$ or $\mathbb{Z}$) and propositional DatalogMTL over $\mathbb{Z}$ [Brandt *et al.*, 2018; Walega *et al.*, 2020b; Walega *et al.*, 2020a].

We briefly summarize our results concerning s-repairs and s-conflicts. For arbitrary DatalogMTL programs, we obtain PSPACE upper bounds for all tasks (and PSPACE-completeness for the decision problems) by adapting known procedures for reasoning with subset repairs and conflicts in the atemporal setting. If we consider tractable DatalogMTL fragments, then we can show that the s-repair and s-conflict recognition are in PTIME, and it is also in PTIME to generate a single s-repair or s-conflict. We can use the PTIME upper bounds on recognizing s-repairs to obtain (co)NP upper bounds for query entailment under s-brave, s-CQA, and s-intersection semantics for tractable DatalogMTL fragments. Moreover, we provide matching lower bounds for $\text{Datalog}_{\text{nr}}$MTL and $\text{DatalogMTL}^{\diamondsuit}_{\mathsf{lin}}$ (as well as for $\text{DatalogMTL}^{\diamondsuit}_{\mathsf{core}}$ in the case of the s-CQA semantics), which hold even for bounded datasets and $\mathbb{T} = \mathbb{Z}$.

The hardness results for $\text{Datalog}_{\text{nr}}$MTL are somewhat surprising in view of the AC^0 data complexity and FO$<$-rewritability of query entailment in $\text{Datalog}_{\text{nr}}$MTL, as a result from [Bienvenu and Rosati, 2013] shows how to transfer FO-rewritability results from classical to brave and intersection semantics. However, the latter result relies upon the fact that in the setting of atemporal ontologies, the existence of a rewriting guarantees a data-independent bound on the size of minimal inconsistent subsets and minimal query-entailing subsets. This property fails to hold in $\text{Datalog}_{\text{nr}}$MTL.

In $\text{DatalogMTL}^{\diamondsuit}_{\mathsf{core}}$, by contrast, Walega et al. [2020b; 2020a] have shown that every minimal Π-inconsistent subset contains at most two facts, and query entailment can be traced back to a single fact. This is the key to showing that query entailment under s-brave and s-intersection semantics are in PTIME for $\text{DatalogMTL}^{\diamondsuit}_{\mathsf{core}}$. For propositional DatalogMTL, we even get tractability for s-CQA semantics – notable in view of the notorious intractability of CQA semantics even in restricted atemporal settings.

Let us also briefly summarize our results for bounded-interval datasets over $\mathbb{Z}$. For general DatalogMTL programs, we obtain PSPACE upper bounds for all tasks concerning i-repairs and i-conflicts. We further show that when we consider tractable fragments, one can tractably recognize or generate an i-conflict, using binary search to identify optimal endpoints. The situation for pointwise notions is starkly different as even in this restricted setting, a single p-conflict or p-repair may be exponentially large.

Discussion and Future Work We initiated an investigation into how classical atemporal notions of repairs and conflicts can be suitably defined for inconsistent DatalogMTL knowledge bases, proposing three distinct approaches which differ on how the temporal intervals of facts can be modified. Our results show that s-repairs and s-conflicts retain desirable properties from the atemporal setting, which is why we made them the focus of our complexity analysis. In future work, we plan to analyze in more detail the complexity of reasoning with i-repairs and i-conflicts in the bounded-interval $\mathbb{Z}$ setting, as we believe the i-notions provide a nice tradeoff between the simplicity of the s-notions and preservation of information of the p-notions. Our results also raise several interesting theoretical questions, such as the decidability of i- and p-repair / conflict existence in unrestricted settings. It would also be worth exploring quantitative variants of our notions, e.g. to take into account how much interval endpoints have been adjusted in an i-repair. A more practical direction is to devise practical SAT- or SMT-based algorithms for the identified (co)NP cases, as has been done in some atemporal settings, cf. [Bienvenu and Bourgaux, 2022].

Acknowledgments

This work was supported by the ANR AI Chair INTENDED (ANR-19-CHIA-0014) and the ANR PRAIRIE 3IA Institute (ANR-19-P3IA-0001).

References

[Abiteboul *et al.*, 1995] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995.

[Arenas *et al.*, 1999] Marcelo Arenas, Leopoldo E. Bertossi, and Jan Chomicki. Consistent query answers in inconsistent databases. In *Proceedings of PODS*, 1999.

[Artale *et al.*, 2017] Alessandro Artale, Roman Kontchakov, Alisa Kovtunova, Vladislav Ryzhikov, Frank Wolter, and Michael Zakharyaschev. Ontology-mediated query answering over temporal data: A survey (invited talk). In *Proceedings of TIME*, 2017.

[Bellomarini *et al.*, 2022] Luigi Bellomarini, Livia Blasi, Markus Nissl, and Emanuel Sallinger. The temporal Vadalog system. In *Proceedings of RuleML+RR*, 2022.

[Bertossi, 2019] Leopoldo E. Bertossi. Database repairs and consistent query answering: Origins and further developments. In *Proceedings of PODS*, 2019.

[Bienvenu and Bourgaux, 2022] Meghyn Bienvenu and Camille Bourgaux. Querying inconsistent prioritized data with ORBITS: algorithms, implementation, and experiments. In *Proceedings of KR*, 2022.

[Bienvenu and Rosati, 2013] Meghyn Bienvenu and Riccardo Rosati. Tractable approximations of consistent query answering for robust ontology-based data access. In *Proceedings of IJCAI*, 2013.

[Bienvenu *et al.*, 2025] Meghyn Bienvenu, Camille Bourgaux, and Atefe Khodadaditaghanaki. Inconsistency handling in DatalogMTL, 2025. arxiv.org/abs/2505.10394 [cs.LO].

[Bienvenu, 2020] Meghyn Bienvenu. A short survey on inconsistency handling in ontology-mediated query answering. *Künstliche Intelligenz*, 34(4):443–451, 2020.

[Bourgaux *et al.*, 2019] Camille Bourgaux, Patrick Koopmann, and Anni-Yasmin Turhan. Ontology-mediated query answering over temporal and inconsistent data. *Semantic Web*, 10(3):475–521, 2019.

[Brandt *et al.*, 2017] Sebastian Brandt, Elem Güzel Kalayci, Roman Kontchakov, Vladislav Ryzhikov, Guohui Xiao, and Michael Zakharyaschev. Ontology-based data access with a horn fragment of metric temporal logic. In *Proceedings of AAAI*, 2017.

[Brandt *et al.*, 2018] Sebastian Brandt, Elem Güzel Kalayci, Vladislav Ryzhikov, Guohui Xiao, and Michael Zakharyaschev. Querying log data with metric temporal logic. *J. Artif. Intell. Res.*, 62:829–877, 2018.

[Ivliev *et al.*, 2024] Alex Ivliev, Lukas Gerlach, Simon Meusel, Jakob Steinberg, and Markus Krötzsch. Nemo: Your friendly and versatile rule reasoning toolkit. In *Proceedings of KR*, 2024.

[Kalayci *et al.*, 2019] Elem Güzel Kalayci, Sebastian Brandt, Diego Calvanese, Vladislav Ryzhikov, Guohui Xiao, and Michael Zakharyaschev. Ontology-based access to temporal data with Ontop: A framework proposal. *Int. J. Appl. Math. Comput. Sci.*, 29(1):17–30, 2019.

[Koymans, 1990] Ron Koymans. Specifying real-time properties with metric temporal logic. *Real Time Syst.*, 2(4):255–299, 1990.

[Lembo *et al.*, 2010] Domenico Lembo, Maurizio Lenzerini, Riccardo Rosati, Marco Ruzzi, and Domenico Fabio Savo. Inconsistency-tolerant semantics for description logics. In *Proceedings of RR*, 2010.

[Ryzhikov *et al.*, 2019] Vladislav Ryzhikov, Przemyslaw Andrzej Walega, and Michael Zakharyaschev. Data complexity and rewritability of ontology-mediated queries in metric temporal logic under the event-based semantics. In *Proceedings of IJCAI*, 2019.

[Walega *et al.*, 2019] Przemyslaw Andrzej Walega, Bernardo Cuenca Grau, Mark Kaminski, and Egor V. Kostylev. DatalogMTL: Computational complexity and expressive power. In *Proceedings of IJCAI*, 2019.

[Walega *et al.*, 2020a] Przemyslaw Andrzej Walega, Bernardo Cuenca Grau, Mark Kaminski, and Egor V. Kostylev. DatalogMTL over the integer timeline. In *Proceedings of KR*, 2020.

[Walega *et al.*, 2020b] Przemyslaw Andrzej Walega, Bernardo Cuenca Grau, Mark Kaminski, and Egor V. Kostylev. Tractable fragments of datalog with metric temporal operators. In *Proceedings of IJCAI*, 2020.

[Walega *et al.*, 2023a] Przemyslaw Andrzej Walega, Mark Kaminski, Dingmin Wang, and Bernardo Cuenca Grau. Stream reasoning with DatalogMTL. *J. Web Semant.*, 76:100776, 2023.

[Walega *et al.*, 2023b] Przemyslaw Andrzej Walega, Michal Zawidzki, Dingmin Wang, and Bernardo Cuenca Grau. Materialisation-based reasoning in DatalogMTL with bounded intervals. In *Proceedings of AAAI*, 2023.

[Walega *et al.*, 2024] Przemyslaw Andrzej Walega, David J. Tena Cucala, Bernardo Cuenca Grau, and Egor V. Kostylev. The stable model semantics of datalog with metric temporal operators. *Theory Pract. Log. Program.*, 24(1):22–56, 2024.

[Wang *et al.*, 2022] Dingmin Wang, Pan Hu, Przemyslaw Andrzej Walega, and Bernardo Cuenca Grau. MeTeoR: Practical reasoning in datalog with metric temporal operators. In *Proceedings of AAAI*, 2022.

[Wang *et al.*, 2024] Dingmin Wang, Przemyslaw Andrzej Walega, Pan Hu, and Bernardo Cuenca Grau. Practical reasoning in DatalogMTL. *CoRR*, abs/2401.02869, 2024.

A Logic for General Attention Using Edge-Conditioned Event Models

Paper accepted at IJCAI 2025

Gaia Belardinelli , Thomas Bolander , Sebastian Watzl
Stanford University, Technical University of Denmark, University of Oslo
gaiabel@stanford.edu, tobo@dtu.dk, sebastian.watzl@ifikk.uio.no

Attention is the crucial ability of the mind to select and prioritize specific subsets of available information [Watzl, 2017]. Research in psychology suggests that agents who do not pay attention to something do not update their beliefs and do not learn about it [Simons and Chabris, 1999]. Hence, while the ability to focus is a strength, as it helps agents ignore irrelevant information, restricting which information is learned can also introduce biases [Johnson, 2024]: An agent with limited attention may never learn about certain news or what certain individuals have to say. Researchers have begun to investigate the significance of attention-driven biases in both humans and AI [Johnson, 2024; Munton, 2023]. AI systems that can robustly reason about attention could potentially detect such attentional biases and correct them.

Earlier work introduced dynamic epistemic logic models (DEL models) of attention, capturing the effects of attention on agents' beliefs. However, the first proposal only modeled a notion of all-or-nothing attention [Bolander *et al.*, 2016], later extended to apply to atomic propositions [Belardinelli and Bolander, 2023]. While the latter of these can treat cases where a member of a hiring committee systematically attends only to, say, the research parts of applicants' CVs, it cannot represent an AI system that directs its attention to whether a committee member has such an attentional bias. It can also not represent agents whose attention is biased toward specific agents, e.g. paying attention to whether candidate a has achieved some result, but not whether b achieved the same [Munton, 2023; Smith and Archer, 2020]. A main obstacle to richer notions is that models for attention get highly complex, even for all-or-nothing attention, and that such models grow at least exponentially large. To obtain a fully general (and still manageable) theory of attention several technical and conceptual innovations were necessary.

In this work, we present the needed technical and conceptual innovations as well as the fully general logic of attention, where agents can attend to any arbitrary formulas. Attention is modelled as a modality that restricts which parts of an event an agent learns. An agent who is not attending to a formula is unable to distinguish an event containing that formula from one that does not. To formalize this, we adopt the framework of DEL, in particular its edge-conditioned version [Bolander, 2018]. We generalize edge-conditioned event models to have both source and target conditions. This simple modification allows to use source conditions to encode information about agents' current attention, while the target conditions encode what agents learn about their own attention. This representation brings both a conceptually cleaner and more tractable reformulation of earlier event models for propositional attention [Belardinelli and Bolander, 2023]. Indeed, in the main paper of this extended abstract [Belardinelli *et al.*, 2025], we show that while edge-conditioned DEL models are as expressive as standard event models [Baltag *et al.*, 1998], they are exponentially more succinct than them. Additionally, we provide a sound and complete axiomatization of edge-conditioned event models, show that they are as expressive as generalized arrow updates [Kooi and Renne, 2011], and as succinct as them. Finally, largely due to the gained simplicity of the new formalization, we managed to generalize the entire framework along another axis: We generalize the framework to handle attention to arbitrary formulas, not only propositional ones.

In this extended abstract, we focus on presenting edge-conditioned DEL and the logic of general attention.

Edge-conditioned DEL We use Ag to denote a finite set of *agents* and P to denote a finite set of *propositional atoms*. The *language of dynamic epistemic logic* $\mathcal{L}$ is given by the grammar: $\varphi ::= \top \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid B_a\varphi \mid [\mathcal{C}]\varphi$, with $p \in P$, $a \in Ag$, and $\mathcal{C}$ an edge-conditioned event model (defined later). $B_a\varphi$ reads "agent a believes φ", and $[\mathcal{C}]\varphi$ reads "after $\mathcal{C}$ happens, φ is the case".

We now introduce DEL with edge-conditions. In the main paper [Belardinelli *et al.*, 2025], we also introduce standard DEL and *event models for propositional attention* [Belardinelli and Bolander, 2023], here omitted for brevity.

Definition 1 (Kripke model)**.** *A* Kripke model *for $\mathcal{L}$ is a tuple $\mathcal{M} = (W, R, V)$ where $W \neq \emptyset$ is a finite set of* worlds*, $R : Ag \to \mathcal{P}(W^2)$ assigns an* accessibility relation *R_a to each agent $a \in Ag$, and $V : W \to \mathcal{P}(P)$ is a* valuation function*. For $w \in W$, $(\mathcal{M}, w)$ is a* pointed Kripke model*.*

Definition 2 (Edge-conditioned event models)**.** *An* edge-conditioned event model *for $\mathcal{L}$ is $\mathcal{C} = (E, Q, pre)$ where $E \neq \emptyset$ is a finite set of* events*, $Q : Ag \to \mathcal{P}(E \times \mathcal{L} \times E \times \mathcal{L})$ assigns to each agent a set of quadruples (e, φ, f, ψ) (abbreviated $(e{:}\varphi, f{:}\psi)$), and $pre : E \to \mathcal{L}$. For $(e{:}\varphi, f{:}\psi) \in Q_a$, we call $(e{:}\varphi, f{:}\psi)$ a* conditioned edge*, where φ is the* source condition *(at e) and ψ is the* target condition *(at f). For $e \in E$, $(\mathcal{C}, e)$ is a* pointed edge-conditioned event model*.*

Intuitively, a Kripke model represents an epistemic state, while a event model represents an epistemic action/event happening. The idea behind edge-conditioned event models is to make the edges of event models, capturing accessible worlds, conditional on formulas. In edge-conditioned event models, the events that an agent can access are modeled by quadruples: $(e{:}\varphi, f{:}\psi) \in Q_a$ means that f is accessible from e by a under the condition that φ is the case at the source e and ψ is the case at target f. The *product update operator*, defined next, expresses how an epistemic state is updated as the consequence of an epistemic event.

Definition 3 (Edge-conditioned product update)**.** *Let $\mathcal{M} = (W, R, V)$ be a Kripke model and $\mathcal{C} = (E, Q, pre)$ an edge-conditioned event model, both for $\mathcal{L}$. The* product update *of $\mathcal{M}$ with $\mathcal{C}$ is $\mathcal{M} \otimes \mathcal{C} = (W', R', V')$ where $W' = \{(w, e) \in W \times E \colon (\mathcal{M}, w) \models pre(e)\}$, $V'((w, e)) = \{p \in At(\mathcal{L}) \colon p \in V(w)\}$, and $R'_a = \{((w, e), (v, f)) \in (W')^2 \colon (w, v) \in R_a$ and $\exists \varphi, \psi \in \mathcal{L}$ s.t. $(e{:}\varphi, f{:}\psi) \in Q_a, (\mathcal{M}, w) \models \varphi$ and $(\mathcal{M}, v) \models \psi\}$. If $(\mathcal{M}, w) \models pre(e)$, the* product update *of $(\mathcal{M}, w)$ with $(\mathcal{C}, e)$ is the pointed Kripke model $(\mathcal{M}, w) \otimes (\mathcal{C}, e) = (\mathcal{M} \otimes \mathcal{C}, (w, e))$.*

Definition 4 (Satisfaction)**.** *Let $(\mathcal{M}, w) = ((W, R, V), w)$ be a pointed Kripke model for $\mathcal{L}$. Satisfaction of $\mathcal{L}$-formulas in $(\mathcal{M}, w)$ is given by the following clauses extended with the standard clauses for propositional logic:*

$$
\begin{array}{lll}
(\mathcal{M}, w) \models p & \textit{iff} & p \in V(w), \textit{ where } p \in At(\mathcal{L}) \\
(\mathcal{M}, w) \models B_a\varphi & \textit{iff} & (\mathcal{M}, v) \models \varphi \textit{ for all } (w, v) \in R_a \\
(\mathcal{M}, w) \models [\mathcal{C}]\varphi & \textit{iff} & \textit{if } \mathcal{C} \textit{ is applicable in } (\mathcal{M}, w) \textit{ then} \\
 & & (\mathcal{M}, w) \otimes \mathcal{C} \models \varphi.
\end{array}
$$

In the main paper [Belardinelli *et al.*, 2025], we provide an axiomatization for edge-conditioned DEL that we show to be sound and complete. Moreover, we recast *event models for propositional attention* [Belardinelli and Bolander, 2023] in the edge-conditioned format, and we show that the result is equally expressive, but exponentially more succinct. Moreover, we compare the classes of standard DEL event models and edge-conditioned event models, showing the two equally expressive. We show the same for generalized arrow updates.

A logic for general attention We now apply edge-conditioned event models to provide a formalism for attention to, and revelation of, arbitrary formulas. The *language of general attention* $\mathcal{L}_{\text{GA}}$ is $\mathcal{L}$ extended with the clause $\varphi ::= A_a\varphi$, where $a \in Ag$ and A_a is a new modal operator. The formula $A_a\varphi$ reads "agent a pays attention to φ". Following Belardinelli and Bolander [2023], we understand attention as being directed to truthful revelations.

A logic for general attention allows to formalize many more scenarios than the logic of propositional attention, namely all those where agents attend to more complex stimuli than just conjunctions of literals. For example, we may have $A_a((p \vee q) \to r)$, meaning that agent a is paying attention to the conditional $(p \vee q) \to r$ which might e.g. be representing a mathematical theorem attended to. Agents may also pay attention to the utterances of specific agents only (or combinations of specific agents and topics). Say that agent a only pays attention to what agent b, but not agent c, says about p. In DEL, the truthful and public announcement of a formula φ by an agent i can be represented by the singleton event model where the actual event has precondition $B_i\varphi$ [van Ditmarsch, 2023]. Such an announcement makes all agents know that i believes φ. Then, to formalise the mentioned attention situation, we could use the formula $A_aB_bp \wedge A_aB_b\neg p \wedge \neg A_aB_cp \wedge \neg A_aB_c\neg p$: if agent b truthfully announces the (believed) truth-value of p, then agent a receives that announcement, but if agent c does the same, a receives nothing. We interpret $\mathcal{L}_{\text{GA}}$ in attention models.

Definition 5 (Attention model)**.** *An* attention model *is a tuple $\mathcal{M} = (W, R, V, \mathcal{A})$ where (W, R, V) is a Kripke model for $\mathcal{L}_{GA}$ and $\mathcal{A} : Ag \times W \to \mathcal{P}(\mathcal{L}_{GA})$ is an* attention function*. For an* actual world *w, $(\mathcal{M}, w)$ is a* pointed attention model*.*

The set of formulas that agent a is paying attention to at world w is the *attention set* $\mathcal{A}(a, w)$, also denoted $\mathcal{A}_a(w)$. The truth of $\mathcal{L}_{\text{GA}}$ formulas is defined by extending the above semantic clauses with: $(\mathcal{M}, w) \models A_a\varphi$ iff $\varphi \in \mathcal{A}_a(w)$. This setting has clear similarities with the logic for general awareness [Fagin and Halpern, 1988]. This does not mean that our framework reduces to a formalism for awareness, as the crucial aspect of attention separating it from awareness is that attention determines what agents learn, as we will see, which awareness does not. We do not place any universal restriction on agents' attention sets, e.g. we may have that $\varphi \wedge \psi \in \mathcal{A}_a(w)$ but $\psi \wedge \varphi \notin \mathcal{A}_a(w)$. Some applications may however require to impose a range of closure properties on $\mathcal{A}_a$, called *attention principles* (see the main paper [Belardinelli *et al.*, 2025] for relevant examples).

Event models for general attention Suppose given $\Gamma \subseteq \mathcal{L}_{\text{GA}}$: the formulas that are revealed (or announced) by the occurring event. Every agent learns the subset of Γ that they are paying attention to: if $\psi \in \Gamma \cap \mathcal{A}_a(w)$, then agent a learns ψ at world w. In the definition below, id_E is the identity function on E, i.e. $id_E(e) = e$ for all $e \in E$.

Definition 6 (Event model for general attention $\mathcal{R}(\Gamma)$)**.** *Let $\Gamma \subseteq \mathcal{L}_{GA}$ be a set of* revealed formulas*. The* event model for general attention *representing the revelation of Γ is the pointed edge-conditioned event model $\mathcal{R}(\Gamma) = ((E, Q, id_E), \bigwedge \Gamma)$ for $\mathcal{L}_{GA}$ defined by:*

$$
\begin{aligned}
&E = \{\textstyle\bigwedge S \colon S \subseteq \Gamma\}, \\
&Q_a = \{(\textstyle\bigwedge S \colon \bigwedge\limits_{\varphi \in T} A_a\varphi \wedge \bigwedge\limits_{\varphi \in S \setminus T} \neg A_a\varphi, \bigwedge T \colon \bigwedge\limits_{\varphi \in T} A_a\varphi) \colon \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad T \subseteq S \subseteq \Gamma\}.
\end{aligned}
$$

This event model contains, for each subset S of the revealed Γ, an event $\bigwedge S$: the subset that an agent may learn by paying attention. The intuition behind the tuple $(\bigwedge S \colon \bigwedge_{\varphi \in T} A_a\varphi \wedge \bigwedge_{\varphi \in S \setminus T} \neg A_a\varphi, \bigwedge T \colon \bigwedge_{\varphi \in T} A_a\varphi)$ is that if $\bigwedge S$ is revealed at an event and agent a pays attention only to the subset T of S, then at that event agent a believes that only $\bigwedge T$ was revealed (as in all events accessible from $\bigwedge S$, $\bigwedge T$ holds), and a also learns that she was paying attention to T (as in all events accessible from $\bigwedge S$, $\bigwedge_{\varphi \in T} A_a\varphi$ holds).

The logic of general attention may be used, for example by an AI agent, to reason about and discover the attentional biases of other agents, explained in more detail in the main paper [Belardinelli *et al.*, 2025].

References

[Baltag *et al.*, 1998] Alexandru Baltag, Lawrence S. Moss, and Slawomir Solecki. The logic of public announcements and common knowledge and private suspicions. In Itzhak Gilboa, editor, *Proceedings of the 7th Conference on Theoretical Aspects of Rationality and Knowledge (TARK-98)*, pages 43–56. Morgan Kaufmann, 1998.

[Belardinelli and Bolander, 2023] Gaia Belardinelli and Thomas Bolander. Attention! Dynamic Epistemic Logic Models of (In)Attentive Agents. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, AAMAS '23, page 391–399, Richland, SC, 2023. International Foundation for Autonomous Agents and Multiagent Systems.

[Belardinelli *et al.*, 2025] Gaia Belardinelli, Thomas Bolander, and Sebastian Watzl. A Logic for General Attention Using Edge-Conditioned Event Models. In *Proceedings of the 2025 International Joint Conference on Artificial Intelligence (IJCAI)*, IJCAI '25, 2025.

[Bolander *et al.*, 2016] Thomas Bolander, Hans van Ditmarsch, Andreas Herzig, Emiliano Lorini, Pere Pardo, and François Schwarzentruber. Announcements to Attentive Agents. *Journal of Logic, Language and Information*, 25:1–35, 2016.

[Bolander, 2018] Thomas Bolander. Seeing is believing: Formalising false-belief tasks in dynamic epistemic logic. In *Outstanding Contributions to Logic*, number 12, pages 207–236. Springer, 2018.

[Fagin and Halpern, 1988] R. Fagin and J. Halpern. Belief, awareness, and limited reasoning. *Artificial Intelligence*, 34:39–76, 1988.

[Johnson, 2024] Gabbrielle M Johnson. Varieties of bias. *Philosophy Compass*, 19(7):e13011, 2024.

[Kooi and Renne, 2011] Barteld Kooi and Bryan Renne. Generalized arrow update logic. In *Proceedings of the 13th Conference on Theoretical Aspects of Rationality and Knowledge*, pages 205–211. ACM, 2011.

[Munton, 2023] Jessie Munton. Prejudice as the misattribution of salience. *Analytic Philosophy*, 64(1):1–19, 2023.

[Simons and Chabris, 1999] Daniel J. Simons and Christopher F. Chabris. Gorillas in our midst: Sustained inattentional blindness for dynamic events. *Perception: Visual perception*, 28(9):1059–1074, 1999.

[Smith and Archer, 2020] Leonie Smith and Alfred Archer. Epistemic injustice and the attention economy. *Ethical Theory and Moral Practice*, 23(5):777–795, 2020.

[van Ditmarsch, 2023] Hans van Ditmarsch. To be announced. *Information and Computation*, 292:105026, 2023.

[Watzl, 2017] Sebastian Watzl. *Structuring mind: The nature of attention and how it shapes consciousness*. Oxford University Press, 2017.

Generalized Safe Conditional Syntax Splitting of Belief Bases (Extended Abstract)

Lars-Phillip Spiegel[1] , **Jonas Haldimann**[2,3] , **Jesse Heyninck**[4,3] , **Gabriele Kern-Isberner**[5] , **Christoph Beierle**[1]
[1]FernUniversität in Hagen, Hagen, Germany
[2]TU Wien, Vienna, Austria
[3]University of Cape Town and CAIR, Cape Town, South Africa
[4]Open Universiteit, 6419 AT Heerlen, the Netherlands
[5]TU Dortmund University, Dortmund, Germany
{lars-phillip.spiegel,christoph.beierle}@fernuni-hagen.de, jonas@haldimann.de, jesse.heyninck@gmail.com, gabriele.kern-isberner@cs.tu-dortmund.de

Abstract

Splitting techniques in knowledge representation help focus on relevant parts of a belief base and reduce the complexity of reasoning generally. We propose a generalization of safe conditional syntax splittings that broadens the applicability of splitting postulates for inductive inference from conditional belief bases. In contrast to safe conditional syntax splitting, our generalized notion supports syntax splittings of a belief base Δ where the subbases of Δ may share atoms and nontrivial conditionals.

1 Introduction

For epistemic reasoning, both from a cognitive point of view and from the point of view of effective implementations, it is often vital to focus on the relevant parts, and leave aside facts and knowledge irrelevant for the question at hand, thus enabling local reasoning [Pearl, 1988]. This is the basic motivation underlying the concept of syntax splitting [Parikh, 1999; Peppas *et al.*, 2015; Kern-Isberner and Brewka, 2017], and of the related idea of minimum irrelevance [Weydert, 1998]. Under the motto "syntax splitting = relevance + independence", respecting syntax splitting was formalized for inductive inference from conditional belief bases [Kern-Isberner *et al.*, 2020], taking splittings over a belief base Δ into account where the subbases Δ_1, Δ_2 are given over disjoint subsignatures of Δ. This condition is a severe restriction in practice because full disjointness is often not the case. The concept of conditional syntax splitting [Heyninck *et al.*, 2023] is an approach to overcome this restriction by allowing Δ_1 and Δ_2 to overlap syntactically. A safety condition ensures that semantic (conditional) independence holds given the joint atoms, enabling local reasoning within the subbases. It has been shown that the postulate of conditional independence (CInd) for safe conditional splittings precisely characterizes avoiding the drowning effect [Pearl, 1990; Benferhat *et al.*, 1993], yielding the first formal definition of the notorious drowning problem that had been described before only by specific examples [Heyninck *et al.*, 2023]. However, it has been shown recently that the safety condition given in [Heyninck *et al.*, 2023] requires that every conditional in the intersection of Δ_1 and Δ_2 is a trivial self-fulfilling conditional, that cannot be falsified [Beierle *et al.*, 2024]. For avoiding this restriction and broadening appliction possibilites of syntax splitting, we develop a generalization of safe conditional syntax splittings. This generalization allows us to greatly increase both the amount of splittings and the amount of belief bases where splittings can be exploited for inductive reasoning. We characterize the splittings useful for inductive inference by identifying the subclass of genuine splittings, separating them from the class of simple splittings that have no benefits for inductive inference. This is an extended abstract of the paper accepted for IJCAI2025 [Spiegel *et al.*, 2025].

2 Formal Basics

We consider a finitely generated propositional language $\mathcal{L}$ over a signature Σ with atoms $a, b, c, \ldots$ and with formulas $A, B, C, \ldots$ As models of formulas we will use the set Ω of *possible worlds* over $\mathcal{L}$. We will use ω both for the model and the corresponding conjunction of all positive or negated atoms. For subsets Σ_i of Σ, let $\mathcal{L}(\Sigma_i)$ denote the propositional language defined by Σ_i, with associated set of interpretations $\Omega(\Sigma_i)$. With ω^i we denote the reduct of ω to Σ_i [Delgrande, 2017]. Conditionals $(B|A)$ are meant to express plausible, yet defeasible rules "If A then plausibly B". $(B|A)$ is *verified* by ω if $\omega \models AB$ and *falsified* by ω if $\omega \models A\overline{B}$. A conditional $(B|A)$ is called *self-fulfilling*, or *trivial*, if $A \models B$, i.e., there is no world that falsifies it. A belief base Δ is a finite set of conditionals, and we focus on (strongly) consistent belief bases in the sense of [Pearl, 1990; Goldszmidt and Pearl, 1996]. A semantic framework for interpreting conditionals are *ordinal conditional functions (OCFs)* $\kappa : \Omega \to \mathbb{N} \cup \{\infty\}$ with $\kappa^{-1}(0) \neq \emptyset$, also called *ranking functions* [Spohn, 1988]. Intuitively, less plausible worlds are assigned higher numbers. Formulas are assigned the rank of their most plausible models, i.e. $\kappa(A) := \min\{\kappa(\omega) \mid \omega \models A\}$. A conditional $(B|A)$ is *accepted* by κ written as $\kappa \models (B|A)$, iff $\kappa(AB) < \kappa(A\overline{B})$. A belief base Δ is accepted by κ written $\kappa \models \Delta$ iff κ accepts all its conditionals.

The nonmonotonic inference relation $\mid\!\sim_\kappa$ induced by κ is

$$A \mid\!\sim_\kappa B \quad \text{iff} \quad A \equiv \bot \text{ or } \kappa(AB) < \kappa(A\overline{B}). \tag{1}$$

To formalize inductive inference from belief bases, [Kern-Isberner *et al.*, 2020] introduced the notion of an *inductive inference operator* which is a mapping $\mathbf{C}$ that assigns to each belief base $\Delta \subseteq (\mathcal{L} \mid \mathcal{L})$ an inference relation $\mid\!\sim_\Delta$ on $\mathcal{L}$, i.e., $\mathbf{C} : \Delta \mapsto \mid\!\sim_\Delta$, such that Direct Inference (DI): If $(B|A) \in \Delta$ then $A \mid\!\sim_\Delta B$, and Trivial Vacuity (TV): $A \mid\!\sim_\emptyset B$ implies $A \models B$ are satisfied.

3 Generalized Safe Conditional Syntax Splitting

We introduce the notion of *generalized safe* conditional syntax splitting as a generalization of safe conditional syntax splitting [Heyninck *et al.*, 2023] that allows us to greatly increase the number of splittings exploitable for inductive inference, and to also increases the amount of belief bases that admit such a splitting at all.

A belief base Δ can be *split into subbases* Δ_1, Δ_2 *conditional on a subsignature* Σ_3, if there are $\Sigma_1, \Sigma_2 \subseteq \Sigma$ such that $\Delta_i = \Delta \cap (\mathcal{L}(\Sigma_i \cup \Sigma_3) \mid \mathcal{L}(\Sigma_i \cup \Sigma_3))$ for $i \in \{1, 2\}$ and $\{\Sigma_1, \Sigma_2, \Sigma_3\}$ is a partition of Σ. This is denoted as

$$\Delta = \Delta_1 \bigcup_{\Sigma_1,\Sigma_2} \Delta_2 \mid \Sigma_3. \tag{2}$$

We denote the intersection of Δ_1 and Δ_2 with $\Delta_3 = \Delta_1 \cap \Delta_2$.

Definition 1. *A belief base* $\Delta = \Delta_1 \bigcup_{\Sigma_1,\Sigma_2} \Delta_2 \mid \Sigma_3$ *can be* generalized safely split into subbases Δ_1, Δ_2 conditional on a subsignature Σ_3*, writing*

$$\Delta = \Delta_1 \bigcup^{\text{gs}}_{\Sigma_1,\Sigma_2} \Delta_2 \mid \Sigma_3 \tag{3}$$

if the following generalized safety property *holds for* $i, i' \in \{1, 2\}, i \neq i'$*:*

$$\begin{aligned} &\textit{for every } \omega^i\omega^3 \in \Omega(\Sigma_i \cup \Sigma_3), \textit{ there is } \omega^{i'} \in \Omega(\Sigma_{i'}) \\ &\textit{s.t. } \omega^i\omega^3\omega^{i'} \not\models \bigvee_{(F|E)\in\Delta_{i'}\setminus\Delta_3} E \wedge \neg F. \end{aligned} \tag{4}$$

The generalized safety condition demands, in essence, that no complete conjunction over Σ_3 may force the falsification of a conditional in $\Delta \setminus \Delta_3$ when considering Σ as a whole. The deciding difference between safe splittings from [Heyninck *et al.*, 2023] and our generalization is that the original safety property demands that no conditional in Δ as a whole can be falsified by a complete conjunction over Σ_3. This begets the side effect that Δ_3 contains only self-fulfilling conditionals, i.e., Δ_1 and Δ_2 may not share "meaningful" conditionals [Beierle *et al.*, 2024] which our generalization avoids.

For inductive inference operators we introduce variants of the postulates (CRel) and (CInd) from [Heyninck *et al.*, 2023] by adapting them to our generalized notion of safety.

Definition 2 (adapted from [Heyninck *et al.*, 2023])**.** *For any* $\Delta = \Delta_1 \bigcup^{\text{gs}}_{\Sigma_1,\Sigma_2} \Delta_2 \mid \Sigma_3$*, for* $i, i' \in \{1, 2\}$ *with* $i \neq i'$ *and any* $A, B \in \mathcal{L}(\Sigma_i)$*,* $D \in \mathcal{L}(\Sigma_{i'})$ *and a complete conjunction* $E \in \mathcal{L}(\Sigma_3)$*, such that* $DE \not\mid\!\sim_\Delta \bot$*, an inductive inference operator* $\mathbf{C}$ *satisfies*

(CRel^{g}) *if* $AE \mid\!\sim_\Delta B$ *iff* $AE \mid\!\sim_{\Delta_i} B$.

(CInd^{g}) *if* $AE \mid\!\sim_\Delta B$ *iff* $AED \mid\!\sim_\Delta B$.

(CSynSplit^{g}) *if it satisfies (CRel^{g}) and (CInd^{g}).*

For governing inductive inference, we are therefore interested in (generalized) safe conditional syntax splittings. However there exists safe conditional syntax splittings where Δ_1 is a subset of Δ_2 or vice versa. In these cases the postulates (CInd^{g}) and (CRel^{g}) loose their meaning. We identify splittings that are meaningful with respect to inductive inference as so-called *genuine splittings*.

Definition 3. *Let* Δ *be a belief base over a signature* Σ*. A conditional syntax splitting* $\Delta = \Delta_1 \bigcup_{\Sigma_1,\Sigma_2} \Delta_2 \mid \Sigma_3$ *of* Δ *is called* genuine*, if* $\Delta_1 \not\subseteq \Delta_2$ *and* $\Delta_2 \not\subseteq \Delta_1$

We illustrate the importance of identifying genuine splittings and our generalization of the safety property.

Example 4 (Δ^{rain})**.** *We consider the belief base* $\Delta^{rain} = \{(\overline{s}|r), (\overline{r}|s), (b|sr), (o|s\overline{r}), (\overline{o}|r), (u|or)\}$*. One of the possible conditionals syntax splittings of* Δ^{rain} *is*

$$\begin{aligned} \Delta^{rain} = &\{(\overline{s}|r), (\overline{r}|s), (b|sr)\} \\ &\bigcup^{\text{gs}}_{\{b\},\{o,u\}} \{(\overline{s}|r), (\overline{r}|s), (o|s\overline{r}), (\overline{o}|r), (u|or)\} \mid \{s, r\} \end{aligned} \tag{5}$$

which, however, is not safe. But the splitting (5) *is both generalized safe and genuine.* Δ^{rain} *has a total of 37 conditional syntax splittings, out of which 32 are generalized safe splittings, but only 16 are safe splittings. Only 5 of the 37 splittings are genuine. For this belief base all genuine splittings are generalized safe, while no safe splitting is genuine.*

4 Inductive Inference Respecting Generalized Safe Conditional Syntax Splitting

A c-representation is a special kind of ranking function, that assigns penalty points to worlds based on the conditionals they falsify [Kern-Isberner, 2001; Kern-Isberner, 2004]. c-Inference [Beierle *et al.*, 2018; Beierle *et al.*, 2021] is the inductive inference $\mid\!\sim^{c\text{-}sk}_\Delta$ obtained by taking all c-representations into account, i.e., $A \mid\!\sim^{c\text{-}sk}_\Delta B$ iff $A \mid\!\sim_\kappa B$ for all c-representations κ of Δ. We show:

Proposition 5. *c-Inference satisfies (CRel^{g}) and (CInd^{g}) and thus (CSynSplit^{g}).*

A selection strategy σ assigns to each belief base a unique c-representation [Beierle and Kern-Isberner, 2021], thus yielding an inductive inference operator via $\mathbf{C}^{c\text{-}rep}_\sigma : \Delta \mapsto \kappa_{\sigma(\Delta)}$ where $\mid\!\sim_{\kappa_{\sigma(\Delta)}}$ is obtained via Equation (1). In general $\mathbf{C}^{c\text{-}rep}_\sigma$ does not satisfy (CSynSplit^{g}). But we introduce the postulate (IP-CSP^{g}) for selection strategies, demanding that, for any $\Delta = \Delta_1 \bigcup^{\text{gs}}_{\Sigma_1,\Sigma_2} \Delta_2 \mid \Sigma_3$, the selection strategy σ assigns the same impact to the conditionals of Δ_i in Δ as in Δ_i, i.e., $\sigma(\Delta_i) = \sigma(\Delta)|_{\Delta_i}$. We show:

Proposition 6. *Let* σ *be a selection strategy that satisfies (IP-CSP^{g}). Then* $\mathbf{C}^{c\text{-}rep}_\sigma$ *satisfies (CRel^{g}) and (CInd^{g}) and thus (CSynSplit^{g}).*

Also system W [Komo and Beierle, 2020; Komo and Beierle, 2022] satisfies (CSynSplit^{g}), while system Z [Pearl, 1990] only satisfies (CRel^{g}), but not (CInd^{g}) and thus not (CSynSplit^{g}).

Acknowledgments

This work was supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - 512363537, grant BE 1700/12-1 awarded to Christoph Beierle. Lars-Phillip Spiegel was supported by this grant.

This work was supported by the Austrian Science Fund (FWF) projects P30873, PIN8884924, and the FWF and netidee SCIENCE project T1349-N. Jonas Haldimann was supported by these grants.

Jesse Heyninck was partially supported by the project NGF.1609.241.010 of the research programme NGF AiNed XS Europa 2024-1 which is (partly) financed by the Dutch Research Council (NWO).

References

[Beierle and Kern-Isberner, 2021] Christoph Beierle and Gabriele Kern-Isberner. Selection strategies for inductive reasoning from conditional belief bases and for belief change respecting the principle of conditional preservation. In E. Bell and F. Keshtkar, editors, *Proceedings of the 34th International Florida Artificial Intelligence Research Society Conference (FLAIRS-34)*, 2021.

[Beierle *et al.*, 2018] Christoph Beierle, Christian Eichhorn, Gabriele Kern-Isberner, and Steven Kutsch. Properties of skeptical c-inference for conditional knowledge bases and its realization as a constraint satisfaction problem. *Ann. Math. Artif. Intell.*, 83(3-4):247–275, 2018.

[Beierle *et al.*, 2021] Christoph Beierle, Christian Eichhorn, Gabriele Kern-Isberner, and Steven Kutsch. Properties and interrelationships of skeptical, weakly skeptical, and credulous inference induced by classes of minimal models. *Artificial Intelligence*, 297:103489, August 2021.

[Beierle *et al.*, 2024] Christoph Beierle, Lars-Phillip Spiegel, Jonas Haldimann, Marco Wilhelm, Jesse Heyninck, and Gabriele Kern-Isberner. Conditional splittings of belief bases and nonmonotonic inference with c-representations. In Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco, editors, *Principles of Knowledge Representation and Reasoning: Proceedings of the 21st International Conference, KR 2024*, pages 106–116, 2024.

[Benferhat *et al.*, 1993] S. Benferhat, D. Dubois, and H. Prade. Argumentative inference in uncertain and inconsistent knowledge bases. In D. Heckerman and E. H. Mamdani, editors, *Proceedings Ninth Annual Conference on Uncertainty in Artificial Intelligence, UAI-93*, pages 411–419. Morgan Kaufmann, 1993.

[Delgrande, 2017] James P. Delgrande. A knowledge level account of forgetting. *J. Artif. Intell. Res.*, 60:1165–1213, 2017.

[Goldszmidt and Pearl, 1996] M. Goldszmidt and J. Pearl. Qualitative probabilities for default reasoning, belief revision, and causal modeling. *Artificial Intelligence*, 84:57–112, 1996.

[Heyninck *et al.*, 2023] Jesse Heyninck, Gabriele Kern-Isberner, Thomas Meyer, Jonas Philipp Haldimann, and Christoph Beierle. Conditional syntax splitting for non-monotonic inference operators. In Brian Williams, Yiling Chen, and Jennifer Neville, editors, *Proceedings of the 37th AAAI Conference on Artificial Intelligence*, volume 37, pages 6416–6424, 2023.

[Kern-Isberner and Brewka, 2017] Gabriele Kern-Isberner and Gerhard Brewka. Strong syntax splitting for iterated belief revision. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 1131–1137, 2017.

[Kern-Isberner *et al.*, 2020] Gabriele Kern-Isberner, Christoph Beierle, and Gerhard Brewka. Syntax splitting = relevance + independence: New postulates for nonmonotonic reasoning from conditional belief bases. In Diego Calvanese, Esra Erdem, and Michael Thielscher, editors, *Principles of Knowledge Representation and Reasoning: Proceedings of the 17th International Conference, KR 2020*, pages 560–571. IJCAI Organization, 2020.

[Kern-Isberner, 2001] Gabriele Kern-Isberner. *Conditionals in Nonmonotonic Reasoning and Belief Revision – Considering Conditionals as Agents*. Number 2087 in Lecture Notes in Computer Science. Springer Science+Business Media, Berlin, DE, 2001.

[Kern-Isberner, 2004] Gabriele Kern-Isberner. A thorough axiomatization of a principle of conditional preservation in belief revision. *Ann. Math. Artif. Intell.*, 40(1-2):127–164, 2004.

[Komo and Beierle, 2020] Christian Komo and Christoph Beierle. Nonmonotonic inferences with qualitative conditionals based on preferred structures on worlds. In Ute Schmid, Franziska Klügl, and Diedrich Wolter, editors, *KI 2020: Advances in Artificial Intelligence - 43rd German Conference on AI, Bamberg, Germany, September 21-25, 2020, Proceedings*, volume 12325 of *LNCS*, pages 102–115. Springer, 2020.

[Komo and Beierle, 2022] Christian Komo and Christoph Beierle. Nonmonotonic reasoning from conditional knowledge bases with system W. *Ann. Math. Artif. Intell.*, 90(1):107–144, 2022.

[Parikh, 1999] Rohit Parikh. Beliefs, belief revision, and splitting languages. *Logic, Language, and Computation*, 2:266–278, 1999.

[Pearl, 1988] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[Pearl, 1990] Judea Pearl. System Z: A natural ordering of defaults with tractable applications to nonmonotonic reasoning. In *Proc. of the 3rd Conf. on Theoretical Aspects of Reasoning About Knowledge (TARK'1990)*, pages 121–135, San Francisco, CA, USA, 1990. Morgan Kaufmann Publ. Inc.

[Peppas *et al.*, 2015] Pavlos Peppas, Mary-Anne Williams, Samir Chopra, and Norman Y. Foo. Relevance in belief revision. *Artificial Intelligence*, 229((1-2)):126–138, 2015.

[Spiegel *et al.*, 2025] Lars-Phillip Spiegel, Jonas Haldimann, Jesse Heyninck, Gabriele Kern-Isberner, and Christoph Beierle. Generalized safe conditional syntax splitting of belief bases. In *34th International Joint Conference on Artificial Intelligence, IJCAI 2025, Proceedings*. ijcai.org, 2025.

[Spohn, 1988] W. Spohn. Ordinal conditional functions: a dynamic theory of epistemic states. In W.L. Harper and B. Skyrms, editors, *Causation in Decision, Belief Change, and Statistics, II*, pages 105–134. Kluwer Academic Publishers, 1988.

[Weydert, 1998] E. Weydert. System JZ - How to build a canonical ranking model of a default knowledge base. In A.G. Cohn, L.K. Schubert, and S.C. Shapiro, editors, *Proc. of the Sixth International Conference on Principles of Knowledge Representation and Reasoning (KR'98)*, pages 190–201. Morgan Kaufmann, 1998.

On Temporal ASP with Eager Unfoldable Operators (Extended Abstract)

Thomas Eiter[1] , **Davide Soldà**[1]
[1]TUWien, Vienna, Austria
{thomas.eiter, davide.solda}@tuwien.ac.at

Abstract

Temporal Equilibrium Logic (TEL) extends Answer Set Programming (ASP) with linear-time temporal operators (LTL), enabling reasoning about dynamic systems. However, TEL enforces strong minimization criteria that may preclude intuitive models. Liveness formulas, for instance, tend to fail to have infinite equilibrium models, as TEL minimization postpones satisfaction forever. We address this limitation by introducing eager temporal operators (eager Until, eager Release, etc.), and present non-disjunctive temporal programs (NDTP) as a framework for modeling dependencies, inertia, and non-determinism. Deciding satisfiability of NDTPs is possible in exponential time and in fact PSPACE-complete.

Answer Set Programming (ASP) [Brewka *et al.*, 2011; Lifschitz, 2019] has been widely applied in dynamic environments, including applications in planning, multi-agent systems, and reasoning about evolving domains [Falkner *et al.*, 2018]. ASP semantics is particularly well-suited for such settings due to its capability of expressing transitive closure, addressing the frame problem (via inertia rules), and providing both default and strong negation, which enables rich and flexible modeling possibilities. Temporal Equilibrium Logic (TEL) [Aguado *et al.*, 2023] builds on this foundation by bridging the gap between the expressive power of linear-time temporal logic (LTL) [Pnueli, 1977] operators and the stable semantics of ASP. By integrating these paradigms, TEL provides a unified framework for reasoning about dynamic systems while preserving the non-monotonic reasoning capabilities and desirable properties of ASP.

Our main contributions, described in detail and supplemented with examples and discussion in [Eiter and Soldà, 2025], are briefly summarized as follows.

- We introduce new versions of the temporal operators that unfold eagerly. While they are indifferent in LTL, they suspend minimization and thus allow for stable models in scenarios where such models are intuitively expected. Notably, the new versions are expressible in TEL and the resulting extension of TEL has the same complexity, i.e., TEL-satisfiability is EXPSPACE-complete [Bozzelli and Pearce, 2015].
- We define $NDTP$ as a rule-based fragment of the new language that, on the one hand, overcomes the problem of strong minimization and, on the other hand, has considerably lower complexity. The former is achieved by using eager temporal operators in rule heads and the latter by disallowing the disjunction, which quickly leads to EXPSPACE-hardness [Šimkus, 2010]. We show that $NDTP$ programs have benign properties, e.g. deciding TEL-satisfiability is in feasible in exponential time and model checking in polynomial time.
- Based on $NDTP$, we define a tight version of temporal programs (TTP). By generalizing results for ordinary tight logic programs [Erdem and Lifschitz, 2003], we show that the stable models of TTP programs are obtained as the LTL-models of their temporal Clark's completion that we define. As a result, we obtain a polynomial encoding of TTP into LTL. Furthermore, we show that tightness of temporal programs can be decided efficiently in nlogspace.

In this extended abstract, we concentrate on TEL-satisfiability, for which PSPACE-hardness is a lower bound by well-known results on temporal logic programs, cf. also [Šimkus, 2010]. A PSPACE upper bound was considered plausible in [Eiter and Soldà, 2025]. This in fact turns out to apply, closing the gap and establishing PSPACE-completeness. $NDTP$ is thus a rich modeling language for dynamic domains of the same complexity as classical LTL. Note that this complexity results can be applied also to the deontic extension of TEL defined in [Soldà *et al.*, 2025].

1 Preliminaries

Both TEL and THT [Aguado *et al.*, 2023] share the same syntax as LTL. Here we introduce the grammar

$$F ::= \top \mid \bot \mid p \mid F \vartriangle F \mid \circ F \mid \bullet F \mid F \mathsf{O} F \qquad (1)$$

where $p \in \mathcal{P}$ for a finite set $\mathcal{P}$ of propositional atoms, $\vartriangle \in \{\wedge, \vee, \rightarrow\}$, and $\mathsf{O} \in \{\mathbb{U}, \mathbb{R}, \mathbb{S}, \mathbb{T}\}$. Negation is defined as $\neg\phi := \phi \rightarrow \bot$. As usual, $\Box$ (globally) is defined by $\Box\phi := \bot\,\mathbb{R}\,\phi$; $\Diamond$ (eventually) by $\Diamond\phi := \top\,\mathbb{U}\,\phi$; $\blacksquare$ (historically) by $\blacksquare\phi := \bot\,\mathbb{T}\,\phi$; $\blacklozenge$ (once) by $\blacklozenge\phi := \top\,\mathbb{S}\,\phi$; $\hat{\circ}$ (weak next) by $\hat{\circ}\phi := \circ\phi \vee \neg\circ\top$; and $\hat{\bullet}$ (weak previous) by $\hat{\bullet}\phi := \bullet\phi \vee \neg\bullet\top$. For any unary operator u, we let $\mathsf{x}^0\phi$ denote ϕ and $\mathsf{x}^{i+1}\phi$ denote $\mathsf{u}\mathsf{u}^i\phi$, for $i \geq 0$.

The semantics of THT is defined via THT-traces (simply traces, if unambiguous), which are finite or infinite sequences $\langle \mathbf{H}, \mathbf{T} \rangle$ of pairs $\langle H_i, T_i \rangle$, where $H_i \subseteq T_i \subseteq \mathcal{P}$ for each $0 \leq i <$

λ, where λ can be either in $\mathbb{N}$ or ω. Both $\mathbf{H}$ and $\mathbf{T}$ are traces as usual (LTL-traces), i.e., sequences $\mathbf{H} = H_0, H_1, \ldots$ resp. $\mathbf{T} = T_0, T_1, \ldots$ of sets of atoms. Given a THT-trace $\mathbf{I}$ (or an LTL-trace $\mathbf{T}$), we denote its length by $\lambda_{\mathbf{I}}$ (resp. $\lambda_{\mathbf{T}}$).

Definition 1 (THT-Satisfaction)**.** *Satisfaction of a THT formula by a THT-trace $\mathbf{I} = \langle \mathbf{H}, \mathbf{T} \rangle$ at time k, where $0 \leq k$ is integer, is inductively defined as follows:*

1. $\mathbf{I}, k \not\models \bot$ *and* $\mathbf{I}, k \not\models \top$
2. $\mathbf{I}, k \models p$ *if* $p \in H_k$*, for any atom* $p \in \mathcal{P}$
3. $\mathbf{I}, k \models \phi \vee \psi$ *if* $\mathbf{I}, k \models \phi$ *or* $\mathbf{I}, k \models \psi$
4. $\mathbf{I}, k \models \phi \wedge \psi$ *if* $\mathbf{I}, k \models \phi$ *and* $\mathbf{I}, k \models \psi$
5. $\mathbf{I}, k \models \phi \rightarrow \psi$ *if* $\begin{cases} \langle \mathbf{T}, \mathbf{T} \rangle, k \not\models \phi \text{ or } \langle \mathbf{T}, \mathbf{T} \rangle, k \models \psi, \text{ and} \\ \mathbf{I}, k \not\models \phi \text{ or } \mathbf{I}, k \models \psi \end{cases}$
6. $\mathbf{I}, k \models \circ\, \phi$ *if* $k + 1 < \lambda$ *and* $\mathbf{I}, k + 1 \models \phi$
7. $\mathbf{I}, k \models \phi\, \mathbb{U}\, \psi$ *if there is* $j \geq k$ *s.t.* $\mathbf{I}, j \models \psi$*, and for all* $j' \in [k, j)$*,* $\mathbf{I}, j' \models \phi$
8. $\mathbf{I}, k \models \phi\, \mathbb{R}\, \psi$ *if for all* $j \geq k$ *s.t.* $\mathbf{I}, j \not\models \psi$*, there exists* $j' \in [k, j)$*,* $\mathbf{I}, j' \models \phi$
9. $\mathbf{I}, k \models \bullet\, \phi$ *if* $\mathbf{I}, k - 1 \models \phi$ *and* $k > 0$
10. $\mathbf{I}, k \models \phi\, \mathbb{S}\, \psi$ *if there is* $j \leq k$ *s.t.* $\mathbf{I}, j \models \psi$*, and for all* $j' \in (j, k)$*,* $\mathbf{I}, j' \models \phi$
11. $\mathbf{I}, k \models \phi\, \mathbb{T}\, \psi$ *if for all* $j \geq k$ *s.t.* $\mathbf{I}, j \not\models \psi$*, there exists* $j' \in [k, j)$*,* $\mathbf{I}, j' \models \phi$*,*

TEL semantics is now as follows [Aguado *et al.*, 2023].

Definition 2. *A trace $\mathbf{T}$ is a stable (equilibrium, TEL) model of formula ϕ if (i) $\mathbf{T} \models \phi$, i.e., $\mathbf{T}$ is an LTL model of ϕ, and (ii) no $\mathbf{H} \neq \mathbf{T}$ exists s.t. $\langle \mathbf{H}, \mathbf{T} \rangle \models \phi$.*

2 PSPACE-membership for $NDTP$

We propose eager variants of $\mathbb{U}_e$, $\mathbb{R}_e$, $\mathbb{S}_e$, and $\mathbb{T}_e$, which unfold deterministically once a $\mathbf{T}$-trace is fixed.

Definition 3. *The eager variant O_e of the operator $\mathsf{O} \in \{\mathbb{U}, \mathbb{R}, \mathbb{S}, \mathbb{T}\}$ is as follows. For any THT-trace $\mathbf{I} = \langle \mathbf{H}, \mathbf{T} \rangle$ and time point $k \geq 0$,*

12. $\mathbf{I}, k \models \phi\, \mathbb{U}_e\, \psi$ *if there exists some* $j \geq k$ *s.t.* $\mathbf{I}, j \models \psi$*, and for all* $j' \in [k, j)$*,* $\mathbf{I}, j' \models \phi$ *and* $\mathbf{T}, j' \not\models \psi$*;*
13. $\mathbf{I}, k \models \phi\, \mathbb{R}_e\, \psi$ *if for all* $j \geq k$ *s.t. (a)* $\mathbf{I}, j \not\models \psi$ *or (b)* $\mathbf{T}, j \models \phi$ *and*$\mathbf{I}, j \not\models \phi$*, some* $j' \in [k, j)$ *exists s.t.* $\mathbf{I}, j' \models \phi$*;*
14. $\mathbf{I}, k \models \phi\, \mathbb{S}_e\, \psi$ *if there is some* $j \leq k$ *s.t.* $\mathbf{I}, j \models \psi$*, and for all* $j' \in (j, k)$*,* $\mathbf{I}, j' \models \phi$ *and* $\mathbf{T}, j' \not\models \psi$*;*
15. $\mathbf{I}, k \models \phi\, \mathbb{T}_e\, \psi$ *if for all* $j \geq k$ *s.t. (a)* $\mathbf{I}, j \not\models \psi$ *or (b)* $\mathbf{T}, j \models \psi$ *and* $\mathbf{I}, j \not\models \psi$*, some* $j' \in [k, j)$ *exists s.t.* $\mathbf{I}, j' \models \phi$*.*

As usual, we can derive further operators such as: $\Box_e$ (eager globally) by $\Box_e \phi := \bot\, \mathbb{R}_e\, \phi$; $\Diamond_e$ (eager eventually) by $\Diamond_e \phi := \top\, \mathbb{U}_e\, \phi$; $\blacksquare_e$ (eager historically) by $\blacksquare_e \phi := \bot\, \mathbb{T}_e\, \phi$; etc.

We introduce $NDTP$s, nondisjunctive temporal programs, a rule-based fragment extending the class considered in [Erdem and Lifschitz, 2003] to the temporal case by incorporating TEL modalities in the body and allowing arbitrary nesting of eager unfoldable operators in the head.

Definition 4. *A $NDTP$ program π consists of sets*

(i) $init(\pi)$ of initial rules of the form $r : \psi \rightarrow \phi$, where ϕ is either $\bot$ or a head formula *from the grammar*

$$\begin{aligned} \phi &::= \eta[\phi] \mid \phi\, \mathsf{O}_e\, \phi \text{ for } \mathsf{O}_e \in \{\mathbb{U}_e, \mathbb{R}_e, \mathbb{S}_e, \mathbb{T}_e\} \\ \psi &::= \psi_1 \mid \psi_2 \\ \psi_1 &::= \eta[\psi_1] \mid \psi_1 \vee \psi_1 \mid \psi_1\, \mathsf{O}\, \psi_1 \text{ for } \mathsf{O} \in \{\mathbb{U}, \mathbb{R}, \mathbb{S}, \mathbb{T}\} \\ \psi_2 &::= \eta[\psi_2] \mid \neg\gamma \mid \psi_2 \vee \psi_2 \\ \eta[\mu] &::= \top \mid p \mid \circ\eta[\mu] \mid \hat{\circ}\eta[\mu] \mid \bullet\eta[\mu] \mid \hat{\bullet}\eta[\mu] \mid \\ &\quad \eta[\mu] \wedge \eta[\mu] \mid \mu \end{aligned}$$

where $p \in \mathcal{P}$ and γ is an arbitrary TEL formula, $\eta[\mu]$ is parameterized by a grammar symbol, and

(ii) $dyn(\pi)$ of dynamic rules $\Box r$, where r is an initial rule.

The $NDTP$ fragment disallows: (i) nesting of implications outside the scope of a negation, (ii) negation within temporal unfoldable operators, and (iii) disjunctions in the head. It permits (i) eager operators in the head, and (ii) arbitrary formulas under negation. As for deciding TEL-satisfiability of $NDTP$ programs, we have the following result.

Theorem 1. *Deciding whether an $NDTP$ program π is TEL-satisfiable is in PSPACE.*

Proof. (Sketch) The proof proceeds by encoding the program π into a Büchi automaton using well-established LTL-to-automata techniques by adapting the *on-the-fly* automata construction for LTL as discussed in [Vardi, 2005] and adding a condition to guarantee the absence of loops.

The LTL-automata construction we build on our proof is the synchronous product of (1) the *local*-automaton and (2) the *fulfillment*-automaton. More in detail, (1) enforces the local consistency of the evaluation of an LTL-formula along the paths and the handling of the unfolding of the temporal operators; (2) enforces that, whenever an $\alpha\mathbb{U}\beta$-formula is regarded to be true in a state, the fulfillment condition (realization of β) will eventually happen and not infinitely postponed.

We can guarantee the absence of self-supported sets of atoms by guessing the truth-value of the sub-formulas at each node and disregarding the nodes where the guess does not match the set of formulas obtainable via iterative immediate consequence until a fixed-point is reached. To derive an atom p, we may have to make assumptions on future states, e.g. if p at the i-th node can be derived by $\circ q \rightarrow p$; in this case, we employ a labeling schema and label q in the next with index-i, and, recursively, each future-assumption with the minimal index among the states where it is supporting an atom. We further require for acceptance of a path in the Büchi automaton that for each such index, some state exists where such an index does not appear. The latter ensures that foundedness in the derivation of atoms along the trace is eventually met. □

The argument can be adapted to finite-traces by requiring no propagation of indexes at the last state. Notably, the stronger condition that some state has no index-label is not made. This is because all future assumptions might eventually be justified, but through different patterns for different subformulas. A simple case of such a case is

$$\Box((p_r \mathbb{U}_e q_r) \wedge (p_s \mathbb{U}_e q_s) \wedge (\circ q_r \rightarrow r) \wedge (\circ q_s \rightarrow s))$$

which admits a temporal equilibrium trace where s, p_s and q_r hold at even states, and r, p_r and q_s hold at odd states.

Ethical Statement

There are no ethical issues.

Acknowledgments

The project leading to this application has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 101034440.

Furthermore, this research was funded in whole or in part by the Vienna Science and Technology Fund (WWTF) project ICT22-023, the Austrian Science Fund (FWF) project 10.55776/COE12, and the Bilateral Artificial Intelligence 10.55776/COE12.

References

[Aguado *et al.*, 2023] Felicidad Aguado, Pedro Cabalar, Martín Diéguez, Gilberto Pérez, Torsten Schaub, Anna Schuhmann, and Concepción Vidal. Linear-time temporal answer set programming. *Theory and Practice of Logic Programming*, 23(1):2–56, 2023.

[Alviano *et al.*, 2016] Mario Alviano, Carmine Dodaro, et al. Completion of disjunctive logic programs. In *IJCAI*, volume 16, pages 886–892, 2016.

[Bozzelli and Pearce, 2015] Laura Bozzelli and David Pearce. On the complexity of temporal equilibrium logic. In *2015 30th Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 645–656. IEEE, 2015.

[Brewka *et al.*, 2011] Gerhard Brewka, Thomas Eiter, and Miroslaw Truszczynski. Answer set programming at a glance. *Commun. ACM*, 54(12):92–103, 2011.

[Cabalar, 2010] Pedro Cabalar. A normal form for linear temporal equilibrium logic. In *European Workshop on Logics in Artificial Intelligence*, pages 64–76. Springer, 2010.

[Eiter and Soldà, 2025] Thomas Eiter and Davide Soldà. On temporal asp with eager unfoldable operators. In James Kwok, editor, *Proceedings of the 34th International Joint Conference on Artificial Intelligence (IJCAI-25), Montreal, Canada August 16-22, 2025*. ijcai.org, 2025. To appear.

[Erdem and Lifschitz, 2003] Esra Erdem and Vladimir Lifschitz. Tight logic programs. *Theory and Practice of Logic Programming*, 3(4-5):499–518, 2003.

[Falkner *et al.*, 2018] Andreas Falkner, Gerhard Friedrich, Konstantin Schekotihin, Richard Taupe, and Erich C Teppan. Industrial applications of answer set programming. *KI-Künstliche Intelligenz*, 32(2):165–176, 2018.

[Lifschitz, 2019] Vladimir Lifschitz. *Answer set programming*, volume 3. Springer Heidelberg, 2019.

[Pnueli, 1977] Amir Pnueli. The temporal logic of programs. In *18th annual symposium on foundations of computer science (sfcs 1977)*, pages 46–57. ieee, 1977.

[Richard Büchi, 1966] J. Richard Büchi. Symposium on decision problems: On a decision method in restricted second order arithmetic. In Ernest Nagel, Patrick Suppes, and Alfred Tarski, editors, *Logic, Methodology and Philosophy of Science*, volume 44 of *Studies in Logic and the Foundations of Mathematics*, pages 1–11. Elsevier, 1966.

[Šimkus, 2010] Mantas Šimkus. *Nonmonotonic logic programs with function symbols*. PhD thesis, Technische Universität Wien, 2010.

[Sistla and Clarke, 1985] A. Prasad Sistla and Edmund M. Clarke. The complexity of propositional linear temporal logics. *J. ACM*, 32(3):733–749, 1985.

[Soldà *et al.*, 2025] Davide Soldà, Pedro Cabalar, Agata Ciabattoni, and Emery Neufeld. Tackling temporal deontic challenges with equilibrium logic. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, AAMAS '25, page 1950–1958, Richland, SC, 2025. International Foundation for Autonomous Agents and Multiagent Systems.

[Vardi, 2005] Moshe Y Vardi. An automata-theoretic approach to linear temporal logic. *Logics for concurrency: structure versus automata*, pages 238–266, 2005.

[Wolper *et al.*, 1983] Pierre Wolper, Moshe Y. Vardi, and A. Prasad Sistla. Reasoning about infinite computation paths. In *24th Annual Symposium on Foundations of Computer Science (sfcs 1983)*, pages 185–194, 1983.